

AT32F402/405 Firmware BSP&Pack

Introduction

This application note is written to give a brief description of how to use AT32F402/405 BSP (Board Support Package) and install AT32 pack.

Contents

1	Overview	39
2	How to install Pack	40
2.1	IAR Pack installation.....	40
2.2	Keil_v5 Pack installation	42
2.3	Keil_v4 Pack installation	44
2.4	Segger Pack installation	46
3	Flash algorithm file	50
3.1	How to use Keil algorithm file	50
3.2	How to use IAR algorithm files.....	52
4	BSP introduction.....	56
4.1	Quick start	56
4.1.1	Template project.....	56
4.1.2	BSP macro definitions	57
4.2	BSP specifications.....	59
4.2.1	List of abbreviations for peripherals	59
4.2.2	Naming rules	59
4.2.3	Encoding rules.....	60
4.3	BSP structure	62
4.3.1	BSP folder structure	62
4.3.2	BSP function library structure.....	63
4.3.3	Initialization and configuration for peripherals	65
4.3.4	Peripheral functions format description.....	65
5	AT32F402/405 peripheral library functions	66
5.1	HICK automatic clock calibration (ACC)	66
5.1.1	acc_calibration_mode_enable function.....	67
5.1.2	acc_step_set function	67
5.1.3	acc_interrupt_enable function	68

5.1.4	acc_hicktrim_get function.....	69
5.1.5	acc_hickcal_get function	69
5.1.6	acc_write_c1 function.....	70
5.1.7	acc_write_c2 function.....	70
5.1.8	acc_write_c3 function.....	71
5.1.9	acc_read_c1 function	71
5.1.10	acc_read_c2 function	72
5.1.11	acc_read_c3 function	72
5.1.12	acc_flag_get function	73
5.1.13	acc_interrupt_flag_get function.....	73
5.1.14	acc_flag_clear function	74
5.2	Analog-to-digital converter (ADC).....	75
5.2.1	adc_reset function	77
5.2.2	adc_enable function	77
5.2.3	adc_base_default_para_init function	78
5.2.4	adc_base_config function	78
5.2.5	adc_clock_div_set.....	80
5.2.6	adc_dma_mode_enable function.....	81
5.2.7	adc_interrupt_enable function.....	81
5.2.8	adc_calibration_init function.....	82
5.2.9	adc_calibration_init_status_get function	82
5.2.10	adc_calibration_start function	83
5.2.11	adc_calibration_status_get function.....	83
5.2.12	adc_voltage_monitor_enable function	84
5.2.13	adc_voltage_monitor_threshold_value_set function	85
5.2.14	adc_voltage_monitor_single_channel_select function	85
5.2.15	adc_ordinary_channel_set function	86
5.2.16	adc_preempt_channel_length_set function	87
5.2.17	adc_preempt_channel_set function.....	87
5.2.18	adc_ordinary_conversion_trigger_set function.....	88
5.2.19	adc_preempt_conversion_trigger_set function.....	89
5.2.20	adc_preempt_offset_value_set function	90
5.2.21	adc_ordinary_part_count_set function.....	90
5.2.22	adc_ordinary_part_mode_enable function	91
5.2.23	adc_preempt_part_mode_enable function	91

5.2.24	adc_preempt_auto_mode_enable function	92
5.2.25	adc_tempsensor_vintrv_enable.....	93
5.2.26	adc_ordinary_software_trigger_enable function.....	93
5.2.27	adc_ordinary_software_trigger_status_get function.....	94
5.2.28	adc_preempt_software_trigger_enable function.....	94
5.2.29	adc_preempt_software_trigger_status_get function.....	95
5.2.30	adc_ordinary_conversion_data_get function.....	95
5.2.31	adc_preempt_conversion_data_get function.....	96
5.2.32	adc_flag_get function	96
5.2.33	adc_interrupt_flag_get function.....	97
5.2.34	adc_flag_clear function	98
5.2.35	adc_ordinary_oversample_enable function	98
5.2.36	adc_preempt_oversample_enable function.....	99
5.2.37	adc_oversample_ratio_shift_set function	99
5.2.38	adc_ordinary_oversample_trig_enable function.....	100
5.2.39	adc_ordinary_oversample_restart_set function.....	101
5.3	Controller area network (CAN)	102
5.3.1	can_reset function.....	104
5.3.2	can_baudrate_default_para_init function.....	104
5.3.3	can_baudrate_set function.....	105
5.3.4	can_default_para_init function	106
5.3.5	can_base_init function	106
5.3.6	can_filter_default_para_init function	108
5.3.7	can_filter_init function	108
5.3.8	can_debug_transmission_prohibit function	110
5.3.9	can_ttc_mode_enable function	110
5.3.10	can_message_transmit function	111
5.3.11	can_transmit_status_get function	113
5.3.12	can_transmit_cancel function	114
5.3.13	can_message_receive function	114
5.3.14	can_receive_fifo_release function	116
5.3.15	can_receive_message_pending_get function	117
5.3.16	can_operating_mode_set function.....	117
5.3.17	can_doze_mode_enter function.....	118
5.3.18	can_doze_mode_exit function	119

5.3.19	can_error_type_record_get function	119
5.3.20	can_receive_error_counter_get function	120
5.3.21	can_transmit_error_counter_get function	120
5.3.22	can_interrupt_enable function.....	121
5.3.23	can_flag_get function	122
5.3.24	can_interrupt_flag_get function.....	123
5.3.25	can_flag_clear function	124
5.4	CRC calculation unit (CRC).....	125
5.4.1	crc_data_reset function.....	126
5.4.2	crc_one_word_calculate function.....	126
5.4.3	crc_block_calculate function	127
5.4.4	crc_data_get function.....	127
5.4.5	crc_common_data_set function	128
5.4.6	crc_common_data_get function.....	128
5.4.7	crc_init_data_set function	129
5.4.8	crc_reverse_input_data_set function.....	129
5.4.9	crc_reverse_output_data_set function.....	130
5.4.10	crc_poly_value _set function.....	130
5.4.11	crc_poly_value _get function.....	131
5.4.12	crc_poly_size _set function	131
5.4.13	crc_poly_size _get function.....	132
5.5	Clock and reset management (CRM)	133
5.5.1	crm_reset function.....	135
5.5.2	crm_lxt_bypass function.....	135
5.5.3	crm_hext_bypass function	136
5.5.4	crm_flag_get function.....	136
5.5.5	crm_interrupt_flag_get function	137
5.5.6	crm_hext_stable_wait function.....	138
5.5.7	crm_hick_clock_trimming_set function	138
5.5.8	crm_hick_clock_calibration_set function	139
5.5.9	crm_periph_clock_enable	139
5.5.10	crm_periph_reset function.....	140
5.5.11	crm_periph_lowpower_mode_enable function	140
5.5.12	crm_clock_source_enable function.....	141

5.5.13	crm_flag_clear function	142
5.5.14	crm_ertc_clock_select function	143
5.5.15	crm_ertc_clock_enable function	143
5.5.16	crm_ahb_div_set function	144
5.5.17	crm_apb1_div_set function	144
5.5.18	crm_apb2_div_set function	145
5.5.19	crm_hext_sclk_div_set function	146
5.5.20	crm_hick_sclk_div_set function	146
5.5.21	crm_clock_failure_detection_enable function	147
5.5.22	crm_battery_powered_domain_reset function	147
5.5.23	crm_auto_step_mode_enable function	148
5.5.24	crm_i2sf5_clock_select function	148
5.5.25	crm_hick_sclk_frequency_select function	149
5.5.26	crm_usb_clock_source_select function	150
5.5.27	crm_usb_phy12_clock_select function	150
5.5.28	crm_pllu_output_set function	151
5.5.29	crm_pll_config function	151
5.5.30	crm_pllu_div_set function	152
5.5.31	crm_sysclk_switch function	153
5.5.32	crm_sysclk_switch_status_get function	153
5.5.33	crm_clocks_freq_get function	154
5.5.34	crm_clock_out_set function	155
5.5.35	crm_clkout_div_set function	155
5.5.36	crm_interrupt_enable function	156
5.5.37	crm_pll_parameter_calculate function	157
5.6	Debug	158
5.6.1	debug_device_id_get function	158
5.6.2	debug_low_power_mode_set function	159
5.6.3	debug_apb1_periph_mode_set function	159
5.6.4	debug_apb2_periph_mode_set function	160
5.7	DMA controller	161
5.7.1	dma_default_para_init function	163
5.7.2	dma_init function	164
5.7.3	dma_reset function	166

5.7.4	dma_data_number_set function	166
5.7.5	dma_data_number_get function	167
5.7.6	dma_interrupt_enable function	167
5.7.7	dma_channel_enable function	168
5.7.8	dma_flag_get function	168
5.7.9	dma_flag_clear function	170
5.7.10	dma_flexible_config function	172
5.7.11	dmamux_enable function	174
5.7.12	dmamux_init	175
5.7.13	dmamux_sync_default_para_init function	175
5.7.14	dmamux_sync_config function	176
5.7.15	dmamux_generator_default_para_init function	178
5.7.16	dmamux_generator_config function	178
5.7.17	dmamux_sync_interrupt_enable function	180
5.7.18	dmamux_generator_interrupt_enable function	181
5.7.19	dmamux_sync_flag_get function	181
5.7.20	dmamux_sync_flag_clear function	182
5.7.21	dmamux_generator_flag_get function	183
5.7.22	dmamux_generator_flag_clear function	184
5.8	Real-time clock (ERTC)	185
5.8.1	ertc_num_to_bcd function	187
5.8.2	ertc_bcd_to_num function	188
5.8.3	ertc_write_protect_enable function	188
5.8.4	ertc_write_protect_disable function	189
5.8.5	ertc_wait_update function	189
5.8.6	ertc_wait_flag function	190
5.8.7	ertc_init_mode_enter function	191
5.8.8	ertc_init_mode_exit function	191
5.8.9	ertc_reset function	192
5.8.10	ertc_divider_set function	192
5.8.11	ertc_hour_mode_set function	193
5.8.12	ertc_date_set function	193
5.8.13	ertc_time_set function	194
5.8.14	ertc_calendar_get function	194
5.8.15	ertc_sub_second_get function	195

5.8.16	ertc_alarm_mask_set function	196
5.8.17	ertc_alarm_week_date_select function	197
5.8.18	ertc_alarm_set function	198
5.8.19	ertc_alarm_sub_second_set function	199
5.8.20	ertc_alarm_enable function	200
5.8.21	ertc_alarm_get function	200
5.8.22	ertc_alarm_sub_second_get function	202
5.8.23	ertc_wakeup_clock_set function	202
5.8.24	ertc_wakeup_counter_set function	203
5.8.25	ertc_wakeup_counter_get function	203
5.8.26	ertc_wakeup_enable function	204
5.8.27	ertc_smooth_calibration_config function	204
5.8.28	ertc_cal_output_select function	205
5.8.29	ertc_cal_output_enable function	205
5.8.30	ertc_time_adjust function	206
5.8.31	ertc_daylight_set function	206
5.8.32	ertc_daylight_bpr_get function	207
5.8.33	ertc_refer_clock_detect_enable function	207
5.8.34	ertc_direct_read_enable function	208
5.8.35	ertc_output_set function	208
5.8.36	ertc_timestamp_pin_select function	209
5.8.37	ertc_timestamp_valid_edge_set function	210
5.8.38	ertc_timestamp_enable function	210
5.8.39	ertc_timestamp_get function	211
5.8.40	ertc_timestamp_sub_second_get function	212
5.8.41	ertc_tamper_1_pin_select function	212
5.8.42	ertc_tamper_pull_up_enable function	213
5.8.43	ertc_tamper_precharge_set function	213
5.8.44	ertc_tamper_filter_set function	214
5.8.45	ertc_tamper_detect_freq_set function	214
5.8.46	ertc_tamper_valid_edge_set function	215
5.8.47	ertc_tamper_timestamp_enable function	216
5.8.48	ertc_tamper_enable function	216
5.8.49	ertc_interrupt_enable function	217
5.8.50	ertc_interrupt_get function	217

5.8.51	ertc_flag_get function	218
5.8.52	ertc_interrupt_flag_get function	219
5.8.53	ertc_flag_clear function	219
5.8.54	ertc_bpr_data_write function	220
5.8.55	ertc_bpr_data_read function	221
5.9	External interrupt/event controller (EXINT)	222
5.9.1	exint_reset function	223
5.9.2	exint_default_para_init function	223
5.9.3	exint_init function	224
5.9.4	exint_flag_clear function	225
5.9.5	exint_flag_get function	225
5.9.6	exint_interrupt_flag_get function	226
5.9.7	exint_software_interrupt_event_generate function	226
5.9.8	exint_interrupt_enable function	227
5.9.9	exint_event_enable function	227
5.10	Flash memory controller (FLASH)	228
5.10.1	flash_flag_get function	230
5.10.2	flash_flag_clear function	230
5.10.3	flash_operation_status_get function	231
5.10.4	flash_operation_wait_for function	231
5.10.5	flash_unlock function	232
5.10.6	flash_lock function	232
5.10.7	flash_sector_erase function	233
5.10.8	flash_internal_all_erase function	233
5.10.9	flash_user_system_data_erase function	234
5.10.10	flash_word_program function	234
5.10.11	flash_halfword_program function	235
5.10.12	flash_byte_program function	236
5.10.13	flash_user_system_data_program function	237
5.10.14	flash_epp_set function	237
5.10.15	flash_epp_status_get function	238
5.10.16	flash_fap_enable function	239
5.10.17	flash_fap_status_get function	239
5.10.18	flash_fap_high_level_enable	240

5.10.19	flash_fap_high_level_status_get.....	240
5.10.20	flash_ssb_set function.....	241
5.10.21	flash_ssb_status_get function.....	242
5.10.22	flash_interrupt_enable function.....	242
5.10.23	flash_slib_enable function.....	243
5.10.24	flash_slib_disable function	243
5.10.25	flash_slib_state_get function.....	244
5.10.26	flash_slib_start_sector_get function.....	244
5.10.27	flash_slib_inststart_sector_get function.....	245
5.10.28	flash_slib_end_sector_get function.....	245
5.10.29	flash_crc_calibrate function.....	246
5.10.30	flash_boot_memory_extension_mode_enable	246
5.10.31	flash_extension_memory_slib_enable.....	247
5.10.32	flash_extension_memory_slib_state_get.....	247
5.10.33	flash_em_slib_inststart_sector_get.....	248
5.11	General-purpose I/Os and multiplexed I/Os (GPIO/IOMUX).....	249
5.11.1	gpio_reset function	250
5.11.2	gpio_init function	250
5.11.3	gpio_default_para_init function	252
5.11.4	gpio_input_data_bit_read function.....	253
5.11.5	gpio_input_data_read function.....	253
5.11.6	gpio_output_data_bit_read function.....	254
5.11.7	gpio_output_data_read function	254
5.11.8	gpio_bits_set function	255
5.11.9	gpio_bits_reset function	255
5.11.10	gpio_bits_write function.....	256
5.11.11	gpio_bits_write function.....	256
5.11.12	gpio_port_write function	257
5.11.13	gpio_pin_wp_config function.....	257
5.11.14	gpio_pins_huge_driven_config function	258
5.11.15	gpio_pin_mux_config function	258
5.12	I2C interfaces	260
5.12.1	i2c_reset function	262
5.12.2	i2c_init function.....	263

5.12.3	i2c_own_address1_set function.....	263
5.12.4	i2c_own_address2_set function.....	264
5.12.5	i2c_own_address2_enable function.....	265
5.12.6	i2c_smbus_enable function.....	265
5.12.7	i2c_enable function	266
5.12.8	i2c_clock_stretch_enable function	266
5.12.9	i2c_ack_enable function.....	267
5.12.10	i2c_addr10_mode_enable function.....	267
5.12.11	i2c_transfer_addr_set function.....	268
5.12.12	i2c_transfer_addr_get function	268
5.12.13	i2c_transfer_dir_set function	269
5.12.14	i2c_transfer_dir_get function.....	269
5.12.15	i2c_matched_addr_get function.....	270
5.12.16	i2c_auto_stop_enable function	270
5.12.17	i2c_reload_enable function	271
5.12.18	i2c_cnt_set function.....	271
5.12.19	i2c_addr10_header_enable function.....	272
5.12.20	i2c_general_call_enable function.....	272
5.12.21	i2c_smbus_alert_set function	273
5.12.22	i2c_slave_data_ctrl_enable function.....	273
5.12.23	i2c_pec_calculate_enable function	274
5.12.24	i2c_pec_transmit_enable function	274
5.12.25	i2c_pec_value_get function.....	275
5.12.26	i2c_timeout_set function	275
5.12.27	i2c_timeout_detcet_set function	276
5.12.28	i2c_timeout_enable function	276
5.12.29	i2c_ext_timeout_set function.....	277
5.12.30	i2c_ext_timeout_enable function	277
5.12.31	i2c_interrupt_enable function.....	278
5.12.32	i2c_interrupt_get function.....	279
5.12.33	i2c_dma_enable function	280
5.12.34	i2c_transmit_set function	280
5.12.35	i2c_start_generate function.....	281
5.12.36	i2c_stop_generate function	282
5.12.37	i2c_data_send function	282

5.12.38 i2c_data_receive function	283
5.12.39 i2c_flag_get function	283
5.12.40 i2c_interrupt_flag_get function	284
5.12.41 i2c_flag_clear function	285
5.12.42 i2c_wakeup_enable function	286
5.12.43 i2c_analog_filter_enable function	286
5.12.44 i2c_config function	287
5.12.45 i2c_lowlevel_init function	289
5.12.46 i2c_wait_end function	290
5.12.47 i2c_wait_flag function	291
5.12.48 i2c_master_transmit function	292
5.12.49 i2c_master_receive function	293
5.12.50 i2c_slave_transmit function	294
5.12.51 i2c_slave_receive function	294
5.12.52 i2c_master_transmit_int function	295
5.12.53 i2c_master_receive_int function	296
5.12.54 i2c_slave_transmit_int function	297
5.12.55 i2c_slave_receive_int function	297
5.12.56 i2c_master_transmit_dma function	298
5.12.57 i2c_master_receive_dma function	299
5.12.58 i2c_slave_transmit_dma function	299
5.12.59 i2c_slave_receive_dma function	300
5.12.60 i2c_smbus_master_transmit function	301
5.12.61 i2c_smbus_master_receive function	302
5.12.62 i2c_smbus_slave_transmit function	303
5.12.63 i2c_smbus_slave_receive function	304
5.12.64 i2c_memory_write function	304
5.12.65 i2c_memory_write_int function	305
5.12.66 i2c_memory_write_dma function	306
5.12.67 i2c_memory_read function	307
5.12.68 i2c_memory_read_int function	308
5.12.69 i2c_memory_read_dma function	309
5.12.70 i2c_evt_irq_handler function	310
5.12.71 i2c_err_irq_handler function	310
5.12.72 i2c_dma_tx_irq_handler function	311

5.12.73	i2c_dma_rx_irq_handler function.....	311
5.13	Nested vectored interrupt controller (NVIC).....	312
5.13.1	nvic_system_reset function.....	313
5.13.2	nvic_irq_enable function	313
5.13.3	nvic_irq_disable function.....	314
5.13.4	nvic_priority_group_config function	314
5.13.5	nvic_vector_table_set function.....	315
5.13.6	nvic_lowpower_mode_config function	316
5.14	Power controller (PWC).....	317
5.14.1	pwc_reset function	318
5.14.2	pwc_battery_powered_domain_access function	318
5.14.3	pwc_pvm_level_select function	319
5.14.4	pwc_power_voltage_monitor_enable function.....	319
5.14.5	pwc_wakeup_pin_enable function	320
5.14.6	pwc_flag_clear function.....	320
5.14.7	pwc_flag_get function	321
5.14.8	pwc_sleep_mode_enter function	321
5.14.9	pwc_deep_sleep_mode_enter function	322
5.14.10	pwc_voltage_regulate_set function.....	322
5.14.11	pwc_standby_mode_enter function	323
5.15	System configuration controller (SCFG)	324
5.15.1	scfg_reset function	325
5.15.2	scfg_infrared_config function	325
5.15.3	scfg_mem_map_get function.....	326
5.15.4	scfg_i2s_full_duplex_config function	326
5.15.5	scfg_pvm_lock_enable function.....	327
5.15.6	scfg_sram_operr_status_get function.....	327
5.15.7	scfg_sram_operr_lock_enable function	328
5.15.8	scfg_lockup_enable function.....	328
5.15.9	scfg_exint_line_config function	329
5.15.10	scfg_pins_ultra_driven_enable function	330
5.16	Qud-SPI interface (QSPI)	331
5.16.1	qspi_encryption_enable function	332
5.16.2	qspi_sck_mode_set function.....	333

5.16.3	qspi_clk_division_set function.....	333
5.16.4	qspi_xip_cache_bypass_set function	334
5.16.5	qspi_interrupt_enable function.....	334
5.16.6	qspi_interupt_flag_get function	335
5.16.7	qspi_flag_get function	335
5.16.8	qspi_flag_clear function	336
5.16.9	qspi_dma_rx_threshold_set function	336
5.16.10	qspi_dma_tx_threshold_set function	337
5.16.11	qspi_dma_enable function	337
5.16.12	qspi_busy_config function.....	338
5.16.13	qspi_xip_enable function.....	338
5.16.14	qspi_cmd_operation_kick function.....	339
5.16.15	qspi_xip_init function.....	341
5.16.16	qspi_byte_read function	343
5.16.17	qspi_half_word_read function	343
5.16.18	qspi_word_read function	344
5.16.19	qspi_word_write function.....	344
5.16.20	qspi_half_word_write function.....	345
5.16.21	qspi_byte_write function.....	345
5.17	Serial peripheral interface (SPI)/ I ² S	346
5.17.1	spi_i2s_reset function	347
5.17.2	spi_default_para_init function	347
5.17.3	spi_init function.....	348
5.17.4	spi_ti_mode_enable function	350
5.17.5	spi_crc_next_transmit function	350
5.17.6	spi_crc_polynomial_set function	351
5.17.7	spi_crc_polynomial_get function.....	351
5.17.8	spi_crc_enable function	352
5.17.9	spi_crc_value_get function.....	352
5.17.10	spi_hardware_cs_output_enable function	353
5.17.11	spi_software_cs_internal_level_set function	353
5.17.12	spi_frame_bit_num_set function	354
5.17.13	spi_half_duplex_direction_set function.....	354
5.17.14	spi_enable function	355
5.17.15	i2s_default_para_init function	355

5.17.16	i2s_init function.....	356
5.17.17	i2s_enable function	358
5.17.18	spi_i2s_interrupt_enable function	358
5.17.19	spi_i2s_dma_transmitter_enable function	359
5.17.20	spi_i2s_dma_receiver_enable function.....	359
5.17.21	spi_i2s_data_transmit function	360
5.17.22	spi_i2s_data_receive function.....	360
5.17.23	spi_i2s_flag_get function.....	361
5.17.24	spi_i2s_interrupt_flag_get function	362
5.17.25	spi_i2s_flag_clear function.....	363
5.18	Full-duplex I ² S interface I ² SF.....	364
5.18.1	i2sf_full_duplex_mode_enable function.....	365
5.18.2	i2sf_pcm_sample_clock_set function	365
5.19	SysTick.....	366
5.19.1	systick_clock_source_config function	366
5.19.2	SysTick_Config function.....	367
5.20	TMR	368
5.20.1	tmr_reset function.....	370
5.20.2	tmr_counter_enable function.....	371
5.20.3	tmr_output_default_para_init function.....	371
5.20.4	tmr_input_default_para_init function.....	372
5.20.5	tmr_brkdt_default_para_init function.....	372
5.20.6	tmr_base_init function	373
5.20.7	tmr_clock_source_div_set function.....	374
5.20.8	tmr_cnt_dir_set function	374
5.20.9	tmr_repetition_counter_set function.....	375
5.20.10	tmr_counter_value_set function.....	375
5.20.11	tmr_counter_value_get function.....	376
5.20.12	tmr_div_value_set function	376
5.20.13	tmr_div_value_get function	377
5.20.14	tmr_output_channel_config function	377
5.20.15	tmr_output_channel_mode_select function	379
5.20.16	tmr_period_value_set function.....	380
5.20.17	tmr_period_value_get function.....	380

5.20.18 tmr_channel_value_set function	381
5.20.19 tmr_channel_value_get function	382
5.20.20 tmr_period_buffer_enable function	382
5.20.21 tmr_output_channel_buffer_enable function	383
5.20.22 tmr_output_channel_immediately_set function	384
5.20.23 tmr_output_channel_switch_set function.....	385
5.20.24 tmr_one_cycle_mode_enable function	385
5.20.25 tmr_32_bit_function_enable function	386
5.20.26 tmr_overflow_request_source_set function	386
5.20.27 tmr_overflow_event_disable function.....	387
5.20.28 tmr_input_channel_init function	387
5.20.29 tmr_channel_enable function.....	389
5.20.30 tmr_input_channel_filter_set function	390
5.20.31 tmr_pwm_input_config function	390
5.20.32 tmr_channel1_input_select function	391
5.20.33 tmr_input_channel_divider_set function	392
5.20.34 tmr_primary_mode_select function.....	393
5.20.35 tmr_sub_mode_select function	394
5.20.36 tmr_channel_dma_select function	395
5.20.37 tmr_hall_select function	395
5.20.38 tmr_channel_buffer_enable function.....	396
5.20.39 tmr_trgout2_enable function	396
5.20.40 tmr_trigger_input_select function.....	397
5.20.41 tmr_sub_sync_mode_set function	397
5.20.42 tmr_dma_request_enable function	398
5.20.43 tmr_interrupt_enable function	399
5.20.44 tmr_interrupt_flag_get function	400
5.20.45 tmr_flag_get function.....	401
5.20.46 tmr_flag_clear function.....	402
5.20.47 tmr_event_sw_trigger function.....	402
5.20.48 tmr_output_enable function.....	403
5.20.49 tmr_internal_clock_set function	403
5.20.50 tmr_output_channel_polarity_set function	404
5.20.51 tmr_external_clock_config function.....	405
5.20.52 tmr_external_clock_mode1_config function	406

5.20.53	tmr_external_clock_mode2_config function	407
5.20.54	tmr_encoder_mode_config function.....	408
5.20.55	tmr_force_output_set function	409
5.20.56	tmr_dma_control_config function.....	410
5.20.57	tmr_brkdt_config function.....	411
5.20.58	tmr_iremap_config function.....	413
5.21	Universal synchronous/asynchronous receiver/transmitter (USART)	414
5.21.1	usart_reset function.....	415
5.21.2	usart_init function	416
5.21.3	usart_parity_selection_config function.....	417
5.21.4	usart_enable function.....	417
5.21.5	usart_transmitter_enable function.....	418
5.21.6	usart_receiver_enable function.....	418
5.21.7	usart_clock_config function.....	419
5.21.8	usart_clock_enable function.....	420
5.21.9	usart_interrupt_enable function	420
5.21.10	usart_dma_transmitter_enable function.....	421
5.21.11	usart_dma_receiver_enable function.....	421
5.21.12	usart_wakeup_id_set function	422
5.21.13	usart_wakeup_mode_set function	422
5.21.14	usart_receiver_mute_enable function.....	423
5.21.15	usart_break_bit_num_set function.....	423
5.21.16	usart_lin_mode_enable function	424
5.21.17	usart_data_transmit function.....	424
5.21.18	usart_data_receive function.....	425
5.21.19	usart_break_send function.....	425
5.21.20	usart_smartcard_guard_time_set function	426
5.21.21	usart_irda_smartcard_division_set function	426
5.21.22	usart_smartcard_mode_enable function	427
5.21.23	usart_smartcard_nack_set function.....	427
5.21.24	usart_single_line_halfduplex_select function	428
5.21.25	usart_irda_mode_enable function.....	428
5.21.26	usart_irda_low_power_enable function	429
5.21.27	usart_hardware_flow_control_set function	429
5.21.28	usart_flag_get function.....	430

5.21.29	usart_interrupt_flag_get function	431
5.21.30	usart_flag_clear function	432
5.21.31	usart_rs485_delay_time_config function	433
5.21.32	usart_transmit_receive_pin_swap function.....	433
5.21.33	usart_id_bit_num_set function	434
5.21.34	usart_de_polarity_reverse function.....	434
5.21.35	usart_rs485_mode_enable function.....	435
5.21.36	usart_msb_transmit_first_enable function	435
5.21.37	usart_dt_polarity_reverse function.....	436
5.21.38	usart_transmit_pin_polarity_reverse function	436
5.21.39	usart_receive_pin_polarity_reverse function	437
5.21.40	usart_receiver_timeout_detection_enable function	437
5.21.41	usart_receiver_timeout_value_set function	438
5.22	Watchdog timer (WDT)	439
5.22.1	wdt_enable function	440
5.22.2	wdt_counter_reload function	440
5.22.3	wdt_reload_value_set function	441
5.22.4	wdt_divider_set function.....	441
5.22.5	wdt_register_write_enable function	442
5.22.6	wdt_flag_get function	442
5.22.7	wdt_window_counter_set function.....	443
5.23	Window watchdog timer (WWDT).....	444
5.23.1	wwdt_reset function.....	445
5.23.2	wwdt_divider_set function	445
5.23.3	wwdt_enable function.....	446
5.23.4	wwdt_interrupt_enable function	446
5.23.5	wwdt_counter_set function.....	447
5.23.6	wwdt_window_counter_set function	447
5.23.7	wwdt_flag_get function.....	448
5.23.8	wwdt_interrupt_flag_get function	448
5.23.9	wwdt_flag_clear function.....	449

6 Precautions 450

6.1	Device model replacement	450
-----	--------------------------------	-----

6.1.1	KEIL environment.....	450
6.1.2	IAR environment.....	451
6.2	Unable to identify IC by JLink software in Keil	453
6.3	How to change HEXT crystal.....	455
7	Revision history.....	458

List of tables

Table 1. Summary of macro definitions.....	57
Table 2. List of abbreviations for peripherals.....	59
Table 3. Summary of BSP function library files.....	64
Table 4. Function format description for peripherals.....	65
Table 5. Summary of ACC registers.....	66
Table 6. Summary of ACC library functions.....	66
Table 7. acc_calibration_mode_enable function.....	67
Table 8. acc_step_set function.....	67
Table 9. acc_interrupt_enable function.....	68
Table 10. acc_hicktrim_get function.....	69
Table 11. acc_hickcal_get function.....	69
Table 12. acc_write_c1 function.....	70
Table 13. acc_write_c2 function.....	70
Table 14. acc_write_c3 function.....	71
Table 15. acc_read_c1 function.....	71
Table 16. acc_read_c2 function.....	72
Table 17. acc_read_c3 function.....	72
Table 18. acc_flag_get function.....	73
Table 19. acc_interrupt_flag_get function.....	73
Table 20. acc_flag_clear function.....	74
Table 21. Summary of ADC registers.....	75
Table 22. Summary of ADC library functions.....	76
Table 23. adc_reset function.....	77
Table 24. adc_enable function.....	77
Table 25. adc_base_default_para_init function.....	78
Table 26. adc_base_config function.....	78
Table 27. adc_clock_div_set function.....	80
Table 28. adc_dma_mode_enable function.....	81
Table 29. adc_interrupt_enable function.....	81
Table 30. adc_calibration_init function.....	82
Table 31. adc_calibration_init_status_get function.....	82
Table 32. adc_calibration_start function.....	83
Table 33. adc_calibration_status_get function.....	83

Table 34. adc_voltage_monitor_enable function	84
Table 35. adc_voltage_monitor_threshold_value_set function.....	85
Table 36. adc_voltage_monitor_single_channel_select function	85
Table 37. adc_ordinary_channel_set function.....	86
Table 38. adc_preempt_channel_length_set function.....	87
Table 39. adc_preempt_channel_set function.....	87
Table 40. adc_ordinary_conversion_trigger_set function	88
Table 41. adc_preempt_conversion_trigger_set function.....	89
Table 42. adc_preempt_offset_value_set function	90
Table 43. adc_ordinary_part_count_set function.....	90
Table 44. adc_ordinary_part_mode_enable function	91
Table 45. adc_preempt_part_mode_enable function.....	91
Table 46. adc_preempt_auto_mode_enable function.....	92
Table 47. adc_temperSENSOR_vintrv_enable function.....	93
Table 48. adc_ordinary_software_trigger_enable function.....	93
Table 49. adc_ordinary_software_trigger_status_get function	94
Table 50. adc_preempt_software_trigger_enable function.....	94
Table 51. adc_preempt_software_trigger_status_get function	95
Table 52. adc_ordinary_conversion_data_get function	95
Table 53. adc_preempt_conversion_data_get function	96
Table 54. adc_flag_get function	96
Table 55. adc_interrupt_flag_get function	97
Table 56. adc_flag_clear function	98
Table 57. adc_ordinary_oversample_enable function	98
Table 58. adc_preempt_oversample_enable function	99
Table 59. adc_oversample_ratio_shift_set function	99
Table 60. adc_ordinary_oversample_trig_enable function	100
Table 61. adc_ordinary_oversample_restart_set function.....	101
Table 62. Summary of CAN registers.....	102
Table 63. Summary of CAN library functions	103
Table 64. can_reset function.....	104
Table 65. can_baudrate_default_para_init function.....	104
Table 66. can_baudrate_set function.....	105
Table 67. can_default_para_init function	106
Table 68. can_base_init function.....	106

Table 69. can_filter_default_para_init function	108
Table 70. can_filter_init function	108
Table 71. can_debug_transmission_prohibit function	110
Table 72. can_ttc_mode_enable function.....	110
Table 73. can_message_transmit function	111
Table 74. can_transmit_status_get function.....	113
Table 75. can_transmit_cancel function.....	114
Table 76. can_message_receive function	114
Table 77. can_receive_fifo_release function	116
Table 78. can_receive_message_pending_get function	117
Table 79. can_operating_mode_set function.....	117
Table 80. can_doze_mode_enter function	118
Table 81. can_doze_mode_exit function	119
Table 82. can_error_type_record_get function	119
Table 83. can_receive_error_counter_get function	120
Table 84. can_transmit_error_counter_get function	120
Table 85. can_interrupt_enable function.....	121
Table 86. can_flag_get function	122
Table 87. can_interrupt_flag_get function	123
Table 88. can_flag_clear function	124
Table 89. Summary of CRC registers.....	125
Table 90. Summary of CRC library functions	125
Table 91. crc_data_reset function.....	126
Table 92. crc_one_word_calculate function	126
Table 93. crc_block_calculate function	127
Table 94. crc_data_get function	127
Table 95. crc_common_data_set function	128
Table 96. crc_common_data_get function.....	128
Table 97. crc_init_data_set function.....	129
Table 98. crc_reverse_input_data_set function	129
Table 99. crc_reverse_output_data_set function	130
Table 100. crc_poly_value_set function	130
Table 101. crc_poly_value_get function.....	131
Table 102. crc_poly_size_set function	131
Table 103. crc_poly_size_get function	132

Table 104. Summary of CRM registers	133
Table 105. Summary of CRM library functions	134
Table 106. <code>crm_reset</code> function	135
Table 107. <code>crm_lxt_bypass</code> function	135
Table 108. <code>crm_hext_bypass</code> function	136
Table 109. <code>crm_flag_get</code> function	136
Table 110. <code>crm_interrupt_flag_get</code> function	137
Table 111. <code>crm_hext_stable_wait</code> function	138
Table 112. <code>crm_hick_clock_trimming_set</code> function.....	138
Table 113. <code>crm_hick_clock_calibration_set</code> function.....	139
Table 114. <code>crm_periph_clock_enable</code> function	139
Table 115. <code>crm_periph_reset</code> function	140
Table 116. <code>crm_periph_lowpower_mode_enable</code> function.....	140
Table 117. <code>crm_clock_source_enable</code> function	141
Table 118. <code>crm_flag_clear</code> function	142
Table 119. <code>crm_ertc_clock_select</code> function.....	143
Table 120. <code>crm_ertc_clock_enable</code> function	143
Table 121. <code>crm_ahb_div_set</code> function.....	144
Table 122. <code>crm_apb1_div_set</code> function	144
Table 123. <code>crm_apb2_div_set</code> function.....	145
Table 124. <code>crm_hext_sclk_div_set</code> function.....	146
Table 125. <code>crm_hick_sclk_div_set</code> function.....	146
Table 126. <code>crm_clock_failure_detection_enable</code> function.....	147
Table 127. <code>crm_battery_powered_domain_reset</code>	147
Table 128. <code>crm_battery_powered_domain_reset</code>	148
Table 129. <code>crm_i2sf5_clock_select</code>	148
Table 130. <code>crm_hick_sclk_frequency_select</code>	149
Table 131. <code>crm_usb_clock_source_select</code>	150
Table 132. <code>crm_usb_phy12_clock_select</code>	150
Table 133. <code>crm_pllu_output_set</code>	151
Table 134. <code>crm_pll_config</code> function.....	151
Table 135. <code>crm_pllu_div_set</code> function.....	152
Table 136. <code>crm_sysclk_switch</code> function	153
Table 137. <code>crm_sysclk_switch_status_get</code> function	153
Table 138. <code>crm_clocks_freq_get</code> function	154

Table 139. <code>crm_clock_out_set</code> function.....	155
Table 140. <code>crm_clkout_div_set</code> function.....	155
Table 141. <code>crm_interrupt_enable</code> function	156
Table 142. <code>crm_pll_parameter_calculate</code> function	157
Table 143. Summary of DEBUG registers	158
Table 144. Summary of DEBUG library functions	158
Table 145. <code>debug_device_id_get</code> function.....	158
Table 146. <code>debug_low_power_mode_set</code> function.....	159
Table 147. <code>debug_apb1_periph_mode_set</code> function	159
Table 148. <code>debug_apb2_periph_mode_set</code> function	160
Table 149. Summary of DMA registers	161
Table 150. Summary of DMA library functions	162
Table 151. <code>dma_default_para_init</code> function	163
Table 152. <code>dma_init_struct</code> default values	163
Table 153. <code>dma_init</code> function.....	164
Table 154. <code>dma_reset</code> function	166
Table 155. <code>dma_data_number_set</code> function.....	166
Table 156. <code>dma_data_number_get</code> function.....	167
Table 157. <code>dma_interrupt_enable</code> function.....	167
Table 158. <code>dma_channel_enable</code> function.....	168
Table 159. <code>dma_flag_get</code> function	168
Table 160. <code>dma_flag_clear</code> function	170
Table 161. <code>dma_flexible_config</code> function.....	172
Table 162. DMAMUX channel request source ID	173
Table 163. <code>dmamux_enable</code> function	174
Table 164. <code>dmamux_init</code> function	175
Table 165. <code>dmamux_sync_default_para_init</code> function	175
Table 166. <code>dmamux_sync_init_struct</code> default values	176
Table 167. <code>dmamux_sync_config</code> function	176
Table 168. <code>dmamux_generator_default_para_init</code> function.....	178
Table 169. <code>dmamux_gen_init_struct</code> default values.....	178
Table 170. <code>dmamux_generator_config</code> function.....	178
Table 171. <code>dmamux_sync_interrupt_enable</code> function.....	180
Table 172. <code>dmamux_generator_interrupt_enable</code> function	181
Table 173. <code>dmamux_sync_flag_get</code> function	181

Table 174. dmamux_sync_flag_clear function	182
Table 175. dmamux_generator_flag_get function.....	183
Table 176. dmamux_generator_flag_clear function.....	184
Table 177. Summary of ERTC registers.....	185
Table 178. Summary of ERTC library functions	186
Table 179. ertc_num_to_bcd function	187
Table 180. ertc_bcd_to_num function	188
Table 181. ertc_write_protect_enable function	188
Table 182. ertc_write_protect_disable function	189
Table 183. ertc_wait_update function.....	189
Table 184. ertc_wait_flag function	190
Table 185. ertc_init_mode_enter function.....	191
Table 186. ertc_init_mode_exit function	191
Table 187. ertc_reset function	192
Table 188. ertc_divider_set function.....	192
Table 189. ertc_hour_mode_set function.....	193
Table 190. ertc_date_set function	193
Table 191. ertc_time_set function	194
Table 192. ertc_calendar_get function	194
Table 193. ertc_sub_second_get function	195
Table 194. ertc_alarm_mask_set function	196
Table 195. ertc_alarm_week_date_select function	197
Table 196. ertc_alarm_set function.....	198
Table 197. ertc_alarm_sub_second_set function.....	199
Table 198. ertc_alarm_enable function.....	200
Table 199. ertc_alarm_get function.....	200
Table 200. ertc_alarm_sub_second_get function	202
Table 201. ertc_wakeup_clock_set function.....	202
Table 202. ertc_wakeup_counter_set function.....	203
Table 203. ertc_wakeup_counter_get function.....	203
Table 204. ertc_wakeup_enable function	204
Table 205. ertc_smooth_calibration_config function	204
Table 206. ertc_cal_output_select function	205
Table 207. ertc_cal_output_enable function.....	205
Table 208. ertc_time_adjust function.....	206

Table 209. ertc_daylight_set function.....	206
Table 210. ertc_daylight_bpr_get function	207
Table 211. ertc_refer_clock_detect_enable function	207
Table 212. ertc_direct_read_enable function.....	208
Table 213. ertc_output_set function	208
Table 214. ertc_timestamp_pin_select function.....	209
Table 215. ertc_timestamp_valid_edge_set function	210
Table 216. ertc_timestamp_enable function	210
Table 217. ertc_timestamp_get function	211
Table 218. ertc_timestamp_sub_second_get function.....	212
Table 219. ertc_tamper_1_pin_select function.....	212
Table 220. ertc_tamper_pull_up_enable function	213
Table 221. ertc_tamper_precharge_set function.....	213
Table 222. ertc_tamper_filter_set function	214
Table 223. ertc_tamper_detect_freq_set function.....	214
Table 224. ertc_tamper_valid_edge_set function.....	215
Table 225. ertc_tamper_timestamp_enable function.....	216
Table 226. ertc_tamper_enable function	216
Table 227. ertc_interrupt_enable function	217
Table 228. ertc_interrupt_get function	217
Table 229. ertc_flag_get function.....	218
Table 230. ertc_interrupt_flag_get function.....	219
Table 231. ertc_flag_clear function.....	219
Table 232. ertc_bpr_data_write function	220
Table 233. ertc_bpr_data_read function.....	221
Table 234. Summary of EXINT registers.....	222
Table 235. Summary of EXINT library functions.....	222
Table 236. exint_reset function	223
Table 237. exint_default_para_init function.....	223
Table 238. exint_init function.....	224
Table 239. exint_flag_clear function	225
Table 240. exint_flag_get function	225
Table 241. exint_interrupt_flag_get function.....	226
Table 242. exint_software_interrupt_event_generate function	226
Table 243. exint_interrupt_enable function	227

Table 244. exint_event_enable function	227
Table 245. Summary of FLASH registers	228
Table 246. Summary of FLASH library functions.....	229
Table 247. flash_flag_get function	230
Table 248. flash_flag_clear function	230
Table 249. flash_operation_status_get function	231
Table 250. flash_operation_wait_for function	231
Table 251. flash_unlock function	232
Table 252. flash_lock function.....	232
Table 253. flash_sector_erase function.....	233
Table 254. flash_internal_all_erase function	233
Table 255. flash_user_system_data_erase function.....	234
Table 256. flash_word_program function.....	234
Table 257. flash_halfword_program function	235
Table 258. flash_byte_program function.....	236
Table 259. flash_user_system_data_program function	237
Table 260. flash_epp_set function	237
Table 261. flash_epp_status_get function	238
Table 262. flash_fap_enable function	239
Table 263. flash_fap_status_get function	239
Table 264. flash_fap_high_level_enable function	240
Table 265. flash_fap_high_level_status_get function	240
Table 266. flash_ssb_set function	241
Table 267. flash_ssb_status_get function	242
Table 268. flash_interrupt_enable function	242
Table 269. flash_slib_enable function	243
Table 270. flash_slib_disable function	243
Table 271. flash_slib_state_get function.....	244
Table 272. flash_slib_start_sector_get function	244
Table 273. flash_slib_inststart_sector_get function.....	245
Table 274. flash_slib_end_sector_get function.....	245
Table 275. flash_crc_calibrate function.....	246
Table 276. flash_boot_memory_extension_mode_enable	246
Table 277. flash_extension_memory_slib_enable	247
Table 278. flash_extension_memory_slib_state_get.....	247

Table 279. flash_em_slib_inststart_sector_get.....	248
Table 280. Summary of GPIO registers	249
Table 281. GPIO and IOMUX library functions.....	249
Table 282. gpio_reset function	250
Table 283. gpio_init function	250
Table 284. gpio_default_para_init function.....	252
Table 285. gpio_init_struct default values	252
Table 286. gpio_input_data_bit_read function	253
Table 287. gpio_input_data_read function.....	253
Table 288. gpio_output_data_bit_read function.....	254
Table 289. gpio_output_data_read function	254
Table 290. gpio_bits_set function	255
Table 291. gpio_bits_reset function.....	255
Table 292. gpio_bits_write function.....	256
Table 293. gpio_bits_write function.....	256
Table 294. gpio_port_write function	257
Table 295. gpio_pin_wp_config function	257
Table 296. gpio_pins_huge_driven_config function.....	258
Table 297. gpio_pin_mux_config function.....	258
Table 298. Summary of I2C register.....	260
Table 299. Summary of I2C library functions.....	260
Table 300. I2C application-layer library functions.....	261
Table 301. i2c_reset function.....	262
Table 302. i2c_init function	263
Table 303. i2c_own_address1_set function.....	263
Table 304. i2c_own_address2_set function.....	264
Table 305. i2c_own_address2_enable function.....	265
Table 306. i2c_smbus_enable function	265
Table 307. i2c_enable function	266
Table 308. i2c_clock_stretch_enable function	266
Table 309. i2c_ack_enable function.....	267
Table 310. i2c_addr10_mode_enable function	267
Table 311. i2c_transfer_addr_set function.....	268
Table 312. i2c_transfer_addr_get function	268
Table 313. i2c_transfer_dir_set function.....	269

Table 314. i2c_transfer_dir_get function.....	269
Table 315. i2c_matched_addr_get function	270
Table 316. i2c_auto_stop_enable function	270
Table 317. i2c_reload_enable function	271
Table 318. i2c_cnt_set function.....	271
Table 319. i2c_addr10_header_enable function	272
Table 320. i2c_general_call_enable function.....	272
Table 321. i2c_smbus_alert_set function.....	273
Table 322. i2c_start_generate function	273
Table 323. i2c_pec_calculate_enable	274
Table 324. i2c_pec_transmit_enable function	274
Table 325. i2c_pec_value_get function	275
Table 326. i2c_timeout_set function	275
Table 327. i2c_timeout_dectet_set function	276
Table 328. i2c_timeout_enable function.....	276
Table 329. i2c_ext_timeout_set function.....	277
Table 330. i2c_ext_timeout_enable function	277
Table 331. i2c_interrupt_enable function.....	278
Table 332. i2c_interrupt_get function	279
Table 333. i2c_dma_enable function	280
Table 334. i2c_transmit_set function.....	280
Table 335. i2c_slave_transmit function.....	281
Table 336. i2c_stop_generate function.....	282
Table 337. i2c_data_send function	282
Table 338. i2c_data_receive function	283
Table 339. i2c_flag_get function.....	283
Table 340. i2c_interrupt_flag_get function	284
Table 341. i2c_flag_clear function	285
Table 342. i2c_wakeup_enable function.....	286
Table 343. i2c_analog_filter_enable function	286
Table 344. i2c_config function.....	287
Table 345. i2c_lowlevel_init function.....	289
Table 346. i2c_wait_end function.....	290
Table 347. i2c_wait_flag function.....	291
Table 348. i2c_master_transmit function	292

Table 349. i2c_master_receivefunction	293
Table 350. i2c_slave_transmit function	294
Table 351. i2c_slave_receive function	294
Table 352. i2c_master_transmit_int function	295
Table 353. i2c_master_receive_int function	296
Table 354. i2c_master_receive_int function	297
Table 355. i2c_master_receive_int function	297
Table 356. i2c_master_transmit_dma function	298
Table 357. i2c_master_receive_dma function	299
Table 358. i2c_slave_transmit_dma function	299
Table 359. i2c_slave_receive_dma function	300
Table 360. i2c_smbus_master_transmit function	301
Table 361. i2c_smbus_master_receive function	302
Table 362. i2c_smbus_slave_transmit function	303
Table 363. i2c_smbus_slave_receive function	304
Table 364. i2c_memory_write function	304
Table 365. i2c_memory_write_int function	305
Table 366. i2c_memory_write_dma function	306
Table 367. i2c_memory_write_dma function	307
Table 368. i2c_memory_write_dma function	308
Table 369. i2c_memory_write_dma function	309
Table 370. i2c_evt_irq_handler function	310
Table 371. i2c_err_irq_handler function	310
Table 372. i2c_dma_tx_irq_handler function	311
Table 373. i2c_dma_rx_irq_handler function	311
Table 374. Summary of NVIC registers	312
Table 375. Summary of NVIC library functions	312
Table 376. nvic_system_reset function	313
Table 377. nvic_irq_enable function	313
Table 378. nvic_irq_disable function	314
Table 379. nvic_priority_group_config function	314
Table 380. nvic_vector_table_set function	315
Table 381. nvic_lowpower_mode_config function	316
Table 382. Summary of PWC registers	317
Table 383. Summary of PWC library functions	317

Table 384. pwc_reset function.....	318
Table 385. pwc_battery_powered_domain_access function	318
Table 386. pwc_pvm_level_select function	319
Table 387. pwc_power_voltage_monitor_enable function.....	319
Table 388. pwc_wakeup_pin_enable function	320
Table 389. pwc_flag_clear function	320
Table 390. pwc_flag_get function	321
Table 391. pwc_sleep_mode_enter function	321
Table 392. pwc_deep_sleep_mode_enter function	322
Table 393. pwc_voltage_regulate_set function	322
Table 394. pwc_standby_mode_enter function.....	323
Table 395. pwc_ldo_output_voltage_set function.....	323
Table 396. Summary of SCFG registers	324
Table 397. Summary of SCFG library functions	324
Table 398. scfg_reset function	325
Table 399. scfg_infrared_config function.....	325
Table 400. scfg_mem_map_get function.....	326
Table 401. scfg_i2s_full_duplex_config function.....	326
Table 402. scfg_adc_dma_channel_remap function	327
Table 403. scfg_sram_operr_status_get function.....	327
Table 404. scfg_sram_operr_lock_enable function	328
Table 405. scfg_usart1_tx_dma_channel_remap function.....	328
Table 406. scfg_exint_line_config function	329
Table 407. scfg_pins_ultra_driven_enable function	330
Table 408. Summary of QSPI registers.....	331
Table 409. Summary of QSPI library functions.....	331
Table 410. qspi_encryption_enable function function	332
Table 411. qspi_sck_mode_set function	333
Table 412. qspi_clk_division_set function.....	333
Table 413. qspi_xip_cache_bypass_set function.....	334
Table 414. qspi_interrupt_enable function.....	334
Table 415. qspi_interrupt_flag_get function	335
Table 416. qspi_flag_get function	335
Table 417. qspi_flag_clear function	336
Table 418. qspi_dma_rx_threshold_set function	336

Table 419. qspi_dma_tx_threshold_set function	337
Table 420. qspi_dma_enable function	337
Table 421. qspi_busy_config function.....	338
Table 422. qspi_xip_enable function	338
Table 423. qspi_cmd_operation_kick function	339
Table 424. qspi_xip_init function	341
Table 425. qspi_byte_read function	343
Table 426. qspi_half_word_read function	343
Table 427. qspi_word_read function.....	344
Table 428. qspi_word_write function	344
Table 429. qspi_half_word_write function	345
Table 430. qspi_byte_write function	345
Table 431. Summary of SPI registers.....	346
Table 432. Summary of SPI library functions	346
Table 433. spi_i2s_reset function	347
Table 434. spi_default_para_init function	347
Table 435. spi_init function.....	348
Table 436. spi_ti_mode_enable function.....	350
Table 437. spi_crc_next_transmit function	350
Table 438. spi_crc_polynomial_set function	351
Table 439. spi_crc_polynomial_get function	351
Table 440. spi_crc_enable function	352
Table 441. spi_crc_value_get function	352
Table 442. spi_hardware_cs_output_enable function	353
Table 443. spi_software_cs_internal_level_set function.....	353
Table 444. spi_frame_bit_num_set function	354
Table 445. spi_half_duplex_direction_set function	354
Table 446. spi_enable function.....	355
Table 447. i2s_default_para_init function	355
Table 448. i2s_init function	356
Table 449. i2s_enable function	358
Table 450. spi_i2s_interrupt_enable function.....	358
Table 451. spi_i2s_dma_transmitter_enable function	359
Table 452. spi_i2s_dma_receiver_enable function	359
Table 453. spi_i2s_data_transmit function	360

Table 454. spi_i2s_data_receive function	360
Table 455. spi_i2s_flag_get function	361
Table 456. spi_i2s_interrupt_flag_get function	362
Table 457. spi_i2s_flag_clear function	363
Table 458. Summary of I2SF registers	364
Table 459. Summary of I2SF library functions	364
Table 460. i2sf_full_duplex_mode_enable function	365
Table 461. i2sf_pcm_sample_clock_set function	365
Table 462. Summary of SysTick registers	366
Table 463. Summary of SysTick library functions	366
Table 464. systick_clock_source_config function	366
Table 465. SysTick_Config function	367
Table 466. Summary of TMR registers	368
Table 467. Summary of TMR library functions	369
Table 468. tmr_reset function	370
Table 469. tmr_counter_enable function	371
Table 470. tmr_output_default_para_init function	371
Table 471. tmr_output_struct default values	371
Table 472. tmr_input_default_para_init function	372
Table 473. tmr_input_struct default values	372
Table 474. tmr_brkdt_default_para_init function	372
Table 475. tmr_brkdt_struct default values	373
Table 476. tmr_base_init function	373
Table 477. tmr_clock_source_div_set function	374
Table 478. tmr_cnt_dir_set function	374
Table 479. tmr_repetition_counter_set function	375
Table 480. tmr_counter_value_set function	375
Table 481. tmr_counter_value_get function	376
Table 482. tmr_div_value_set function	376
Table 483. tmr_div_value_get function	377
Table 484. tmr_output_channel_config function	377
Table 485. tmr_output_channel_mode_select function	379
Table 486. tmr_period_value_set function	380
Table 487. tmr_period_value_get function	380
Table 488. tmr_channel_value_set function	381

Table 489. tmr_channel_value_get function	382
Table 490. tmr_period_buffer_enable function.....	382
Table 491. tmr_output_channel_buffer_enable function.....	383
Table 492. tmr_output_channel_immediately_set function	384
Table 493. tmr_output_channel_switch_set function	385
Table 494. tmr_one_cycle_mode_enable function.....	385
Table 495. tmr_32_bit_function_enable function	386
Table 496. tmr_overflow_request_source_set function	386
Table 497. tmr_overflow_event_disable function.....	387
Table 498. tmr_input_channel_init function	387
Table 499. tmr_channel_enable function	389
Table 500. tmr_input_channel_filter_set function.....	390
Table 501. tmr_pwm_input_config function	390
Table 502. tmr_channel1_input_select function.....	391
Table 503. tmr_input_channel_divider_set function.....	392
Table 504. tmr_primary_mode_select function	393
Table 505. tmr_sub_mode_select function	394
Table 506. tmr_channel_dma_select function	395
Table 507. tmr_hall_select function	395
Table 508. tmr_channel_buffer_enable function	396
Table 509. tmr_trgout2_enable function.....	396
Table 510. tmr_trigger_input_select function.....	397
Table 511. tmr_sub_sync_mode_set function	397
Table 512. tmr_dma_request_enable function	398
Table 513. tmr_interrupt_enable function	399
Table 514. tmr_interrupt_flag_get function.....	400
Table 515. tmr_flag_get function.....	401
Table 516. tmr_flag_clear function.....	402
Table 517. tmr_event_sw_trigger function.....	402
Table 518. tmr_output_enable function.....	403
Table 519. tmr_internal_clock_set function.....	403
Table 520. tmr_output_channel_polarity_set function	404
Table 521. tmr_external_clock_config function	405
Table 522. tmr_external_clock_mode1_config function.....	406
Table 523. tmr_external_clock_mode2_config function.....	407

Table 524. tmr_encoder_mode_config function.....	408
Table 525. tmr_force_output_set function	409
Table 526. tmr_dma_control_config function.....	410
Table 527. tmr_brkdt_config function.....	411
Table 528. tmr_iremap_config function.....	413
Table 529. Summary of USART registers	414
Table 530. Summary of USART library functions	414
Table 531. usart_reset function	415
Table 532. usart_init function	416
Table 533. usart_parity_selection_config function	417
Table 534. usart_enable function	417
Table 535. usart_transmitter_enable function	418
Table 536. usart_receiver_enable function	418
Table 537. usart_clock_config function	419
Table 538. usart_clock_enable function.....	420
Table 539. usart_interrupt_enable function	420
Table 540. usart_dma_transmitter_enable function.....	421
Table 541. usart_dma_receiver_enable function.....	421
Table 542. usart_wakeup_id_set function.....	422
Table 543. usart_wakeup_mode_set function	422
Table 544. usart_receiver_mute_enable function	423
Table 545. usart_break_bit_num_set function	423
Table 546. usart_lin_mode_enable function	424
Table 547. usart_data_transmit function	424
Table 548. usart_data_receive function.....	425
Table 549. usart_break_send function	425
Table 550. usart_smartcard_guard_time_set function	426
Table 551. usart_irda_smartcard_division_set function	426
Table 552. usart_smartcard_mode_enable function.....	427
Table 553. usart_smartcard_nack_set function	427
Table 554. usart_single_line_halfduplex_select function	428
Table 555. usart_irda_mode_enable function.....	428
Table 556. usart_irda_low_power_enable function.....	429
Table 557. usart_hardware_flow_control_set function	429
Table 558. usart_flag_get function.....	430

Table 559. usart_interrupt_flag_get function.....	431
Table 560. usart_flag_clear function.....	432
Table 561. usart_flag_clear function.....	433
Table 562. usart_transmit_receive_pin_swap function	433
Table 563. usart_transmit_receive_pin_swap function	434
Table 564. usart_de_polarity_reverse function.....	434
Table 565. usart_rs485_mode_enable function.....	435
Table 566. usart_msb_transmit_first_enable function	435
Table 567. usart_dt_polarity_reverse function	436
Table 568. usart_dt_polarity_reverse function	436
Table 569. usart_receive_pin_polarity_reverse function	437
Table 570. usart_receiver_timeout_detection_enable function.....	437
Table 571. usart_receiver_timeout_value_set function.....	438
Table 572. Summary of WDT registers	439
Table 573. Summary of WDT library functions	439
Table 574. wdt_enable function.....	440
Table 575. wdt_counter_reload function	440
Table 576. wdt_reload_value_set function.....	441
Table 577. wdt_divider_set function	441
Table 578. wdt_register_write_enable function.....	442
Table 579. wdt_flag_get function	442
Table 580. wdt_window_counter_set function	443
Table 581. Summary of WWDT registers.....	444
Table 582. Summary of WWDT library functions.....	444
Table 583. wwdt_reset function.....	445
Table 584. wwdt_divider_set function	445
Table 585. wwdt_enable function	446
Table 586. wwdt_interrupt_enable function.....	446
Table 587. wwdt_counter_set function.....	447
Table 588. wwdt_window_counter_set function	447
Table 589. wwdt_flag_get function	448
Table 590. wwdt_interrupt_flag_get function	448
Table 591. wwdt_flag_clear function	449
Table 592. Clock configuration guideline.....	457
Table 593. Document revision history	458

List of figures

Figure 1. Pack kit	40
Figure 2. IAR Pack installation window	40
Figure 3. IAR Pack installation window	41
Figure 4. View IAR Pack installation status	42
Figure 5. View Keil_v5 Pack installation status	43
Figure 6. Keil_v4 Pack installation	44
Figure 7. Keil_v4 Pack installation process	45
Figure 8. Keil_v4 Pack installation complete	45
Figure 9. View Keil_v4 Pack installation status	46
Figure 10. Segger pack installation window	47
Figure 11. Segger pack installation process	47
Figure 12. Open J-Flash	48
Figure 13. Create a new project using J-Flash	48
Figure 14. View Device information	49
Figure 15. Keil algorithm file settings	50
Figure 16. Keil algorithm file configuration	51
Figure 17. Select algorithm files using Keil	51
Figure 18. Add algorithm files using Keil	52
Figure 19. IAR project name	53
Figure 20. IAR algorithm file configuration	53
Figure 21. IAR Flash Loader overview	54
Figure 22. IAR Flash Loader configuration	54
Figure 23. IAR Flash Loader configuration success	55
Figure 24. Template content	56
Figure 25. Keil_v5 template project example	56
Figure 26. Peripheral enable macro definitions	58
Figure 27. BSP folder structure	63
Figure 28. BSP function library structure	64
Figure 29. Change device part number in Keil	450
Figure 30. Change macro definition in Keil	451
Figure 31. Change device part number in IAR	452
Figure 32. Change macro definition in IAR	453
Figure 33. Error warning 1	453

Figure 34. Error warning 2	454
Figure 35. Error warning 3	454
Figure 36. JLinkLog and JLinkSettings.....	454
Figure 37. Unspecified Cortex-M4.....	455
Figure 38. AT32_New_Clock_Configuration window	456

1 Overview

In order to help users make efficient use of Artery MCU, we provide a complete set of BSP & Pack tools to speed up development. They include peripheral driver library, core-related documents and application cases as well as Pack documents supporting a variety of development environments such as Keil_v5, Keil_v4, IAR_v6, IAR_v7 and IAR_v8. The BSP and Pack are available on Artery official website.

This application note is written to present how to use BSP and Pack.

2 How to install Pack

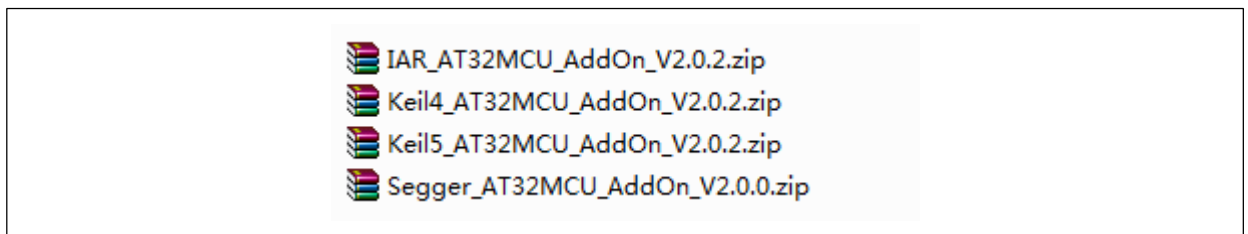
Artery Pack supports various development environment such as Keil_v5, Keil_v4, IAR_v6, IAR_v7 and IAR_v8.

Double click on the corresponding Pack to finish installation.

Note: This section takes AT32F403A as an example, and other AT32 MCUs have similar Pack installation methods.

The installation package is shown in Figure 1 (the specific version information is subject to the actual conditions).

Figure 1. Pack kit

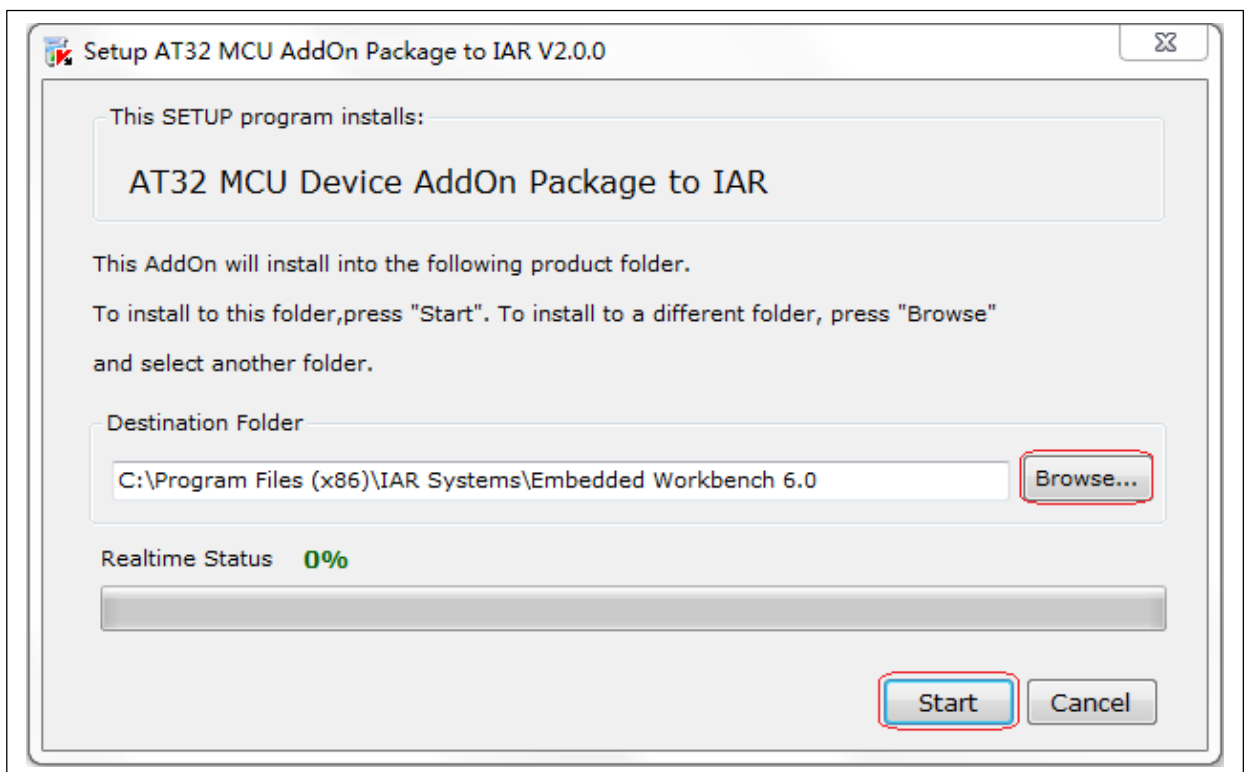


2.1 IAR Pack installation

IAR_AT32MCU_AddOn.zip: This is a zip file supporting IAR_V6, IAR_V7 and IAR_V8. Follow the steps below to install:

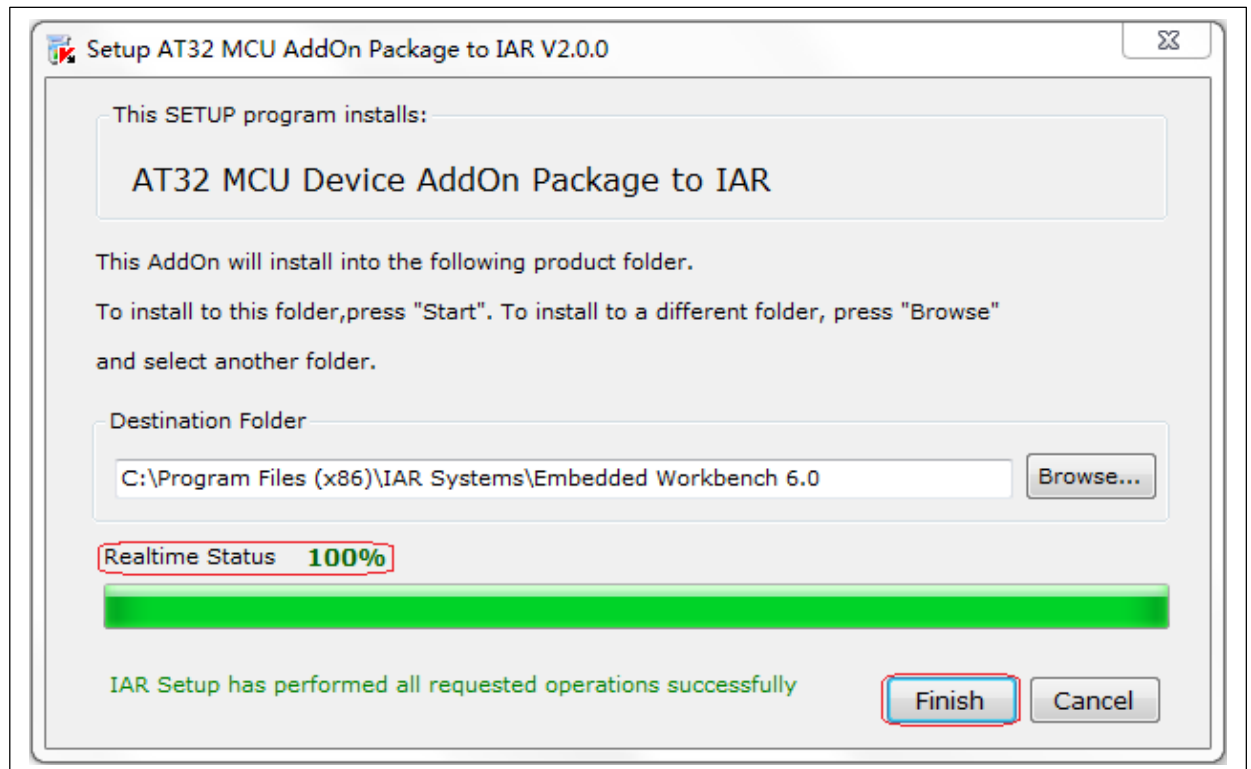
- ① Unzip *IAR_AT32MCU_AddOn.zip*;
- ② Double click on *IAR_AT32MCU_AddOn.exe*, and a dialog box pops up below (the specific version information is subject to the actual conditions).

Figure 2. IAR Pack installation window



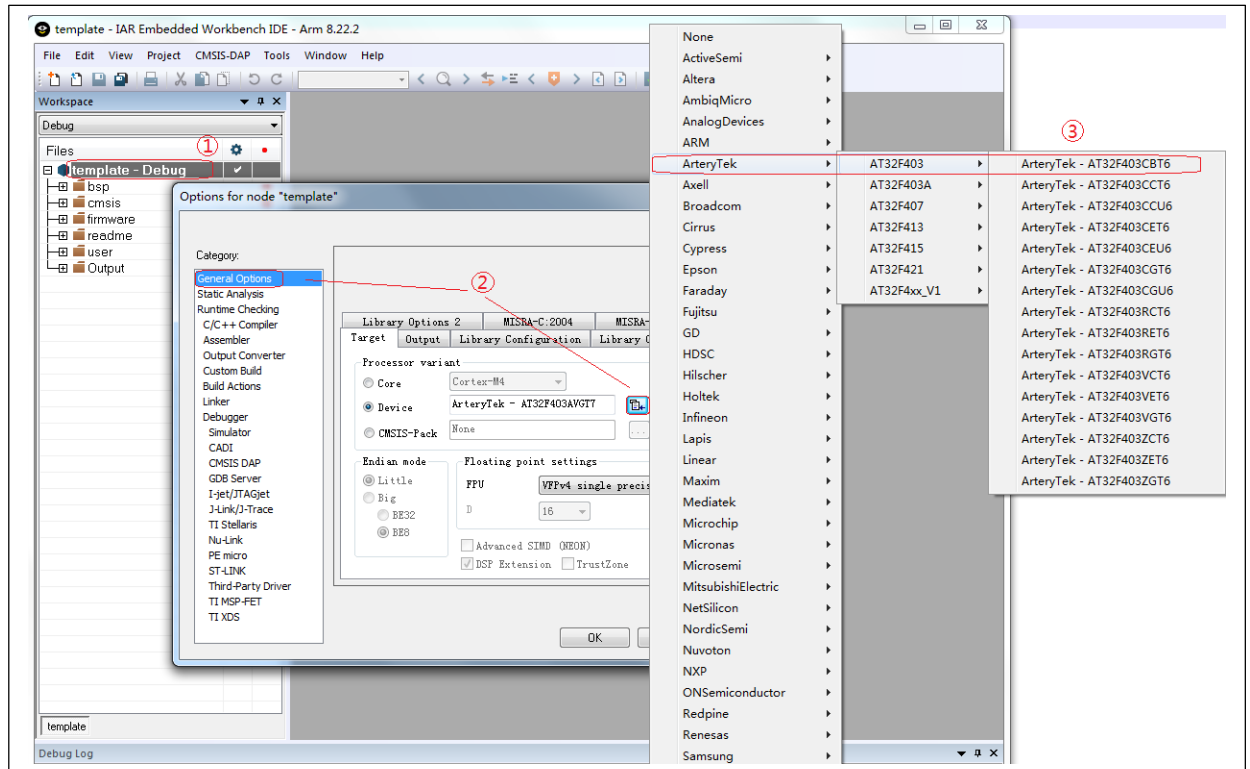
Note: If the installation path of IAR does not match the Destination Folder, click on "Browse" to select a correct path, then click on "Start", as shown below.

Figure 3. IAR Pack installation window



- ③ Click on "Finish";
- ④ To check whether the IAR Pack is installed successfully or not, open an IAR project and follow the steps below:
 - Right click on a project name, and select "Options...";
 - Select "General Options", and click on the check box;
 - Click on "ArteryTek" and view AT32 MCU-related information.

Figure 4. View IAR Pack installation status

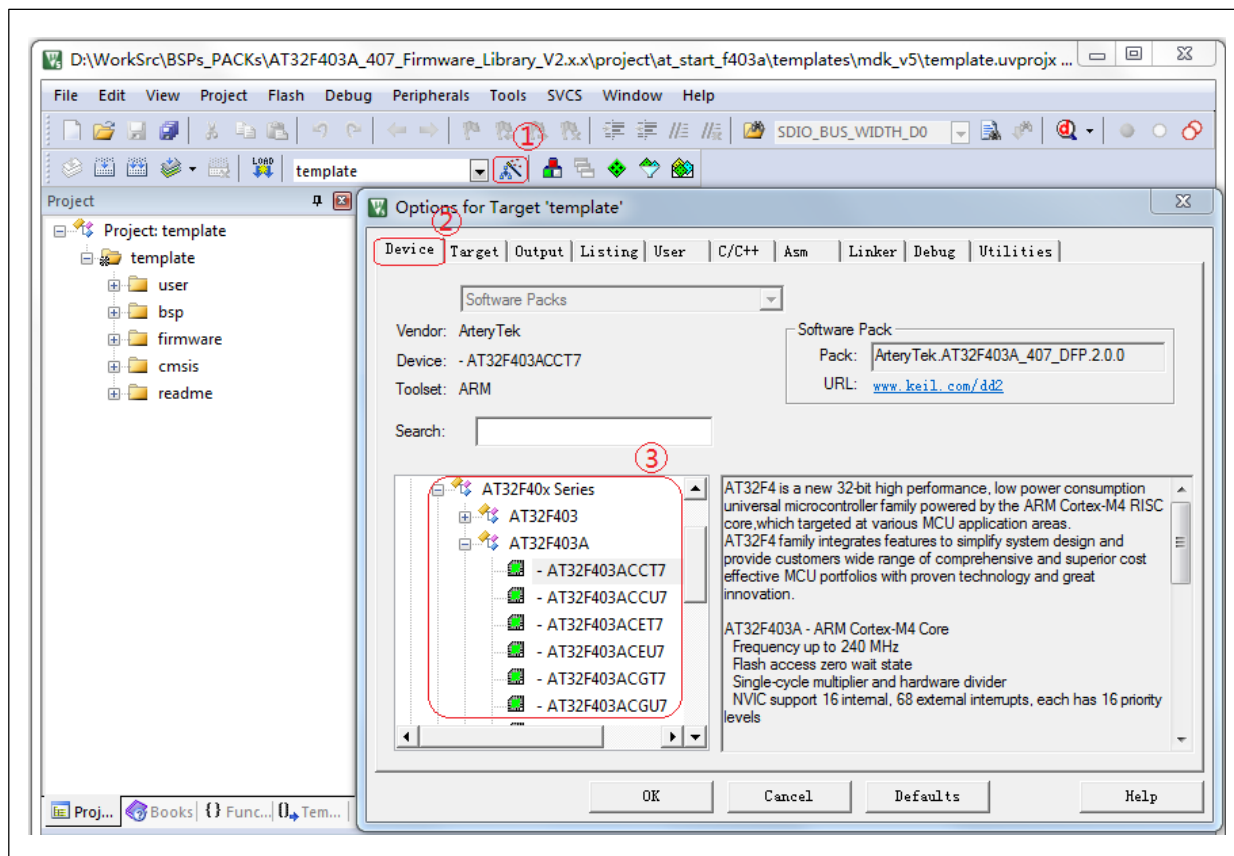


2.2 Keil_v5 Pack installation

Keil5_AT32MCU_AddOn.zip: This is a zip file supporting Keil_v5. Follow the steps below to install:

- ① Unzip *Keil5_AT32MCU_AddOn.zip*. This zip file includes all Keil5 packs supported, all of which are standard Keil_v5 DFP installation files.
- ② Select the desired Pack, and double click on *ArteryTek.AT32xxxx_DFP.2.x.x.pack* to get one-stop installation.
- ⑤ To check whether the Keil_v5 Pack is installed successfully or not, follow the steps below:
 - Click on wand;
 - Select “Device”;
 - View AT32 MCU-related information.

Figure 5. View Keil_v5 Pack installation status

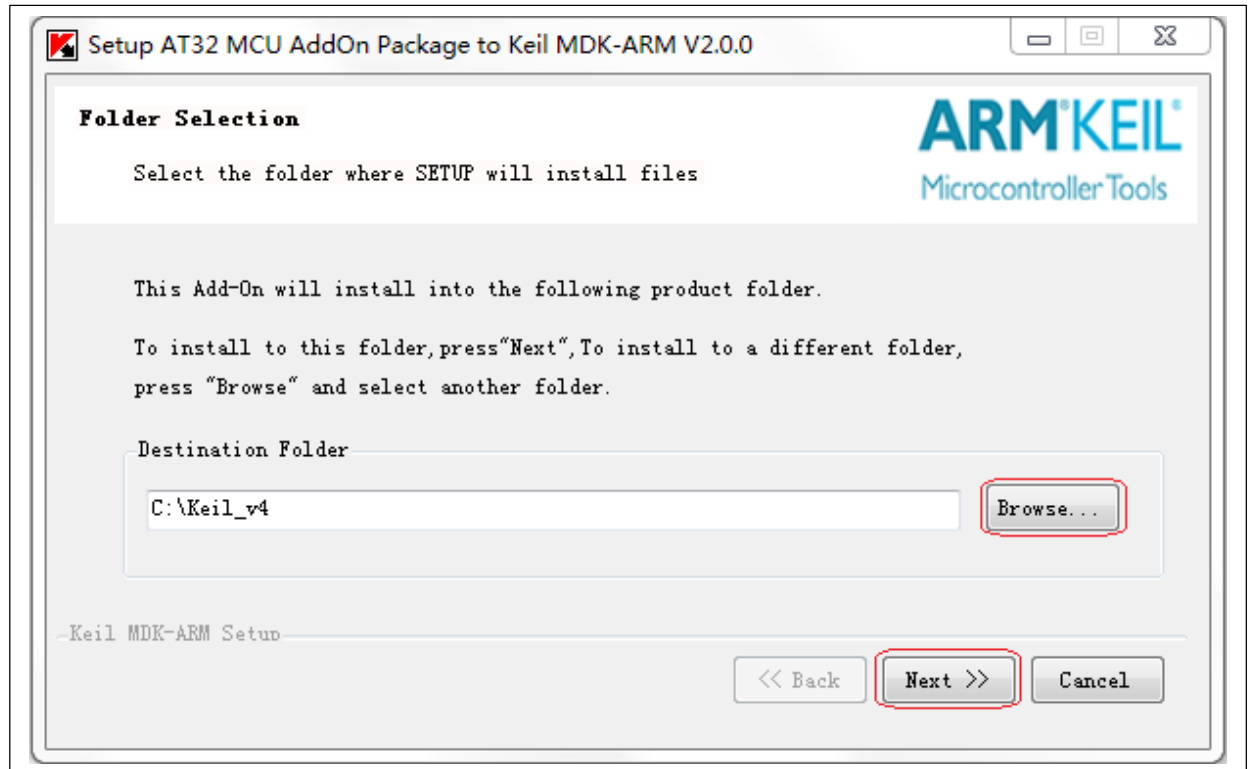


2.3 Keil_v4 Pack installation

Keil4_AT32MCU_AddOn.zip: This is a zip file supporting Keil_v4. Follow the steps below to install:

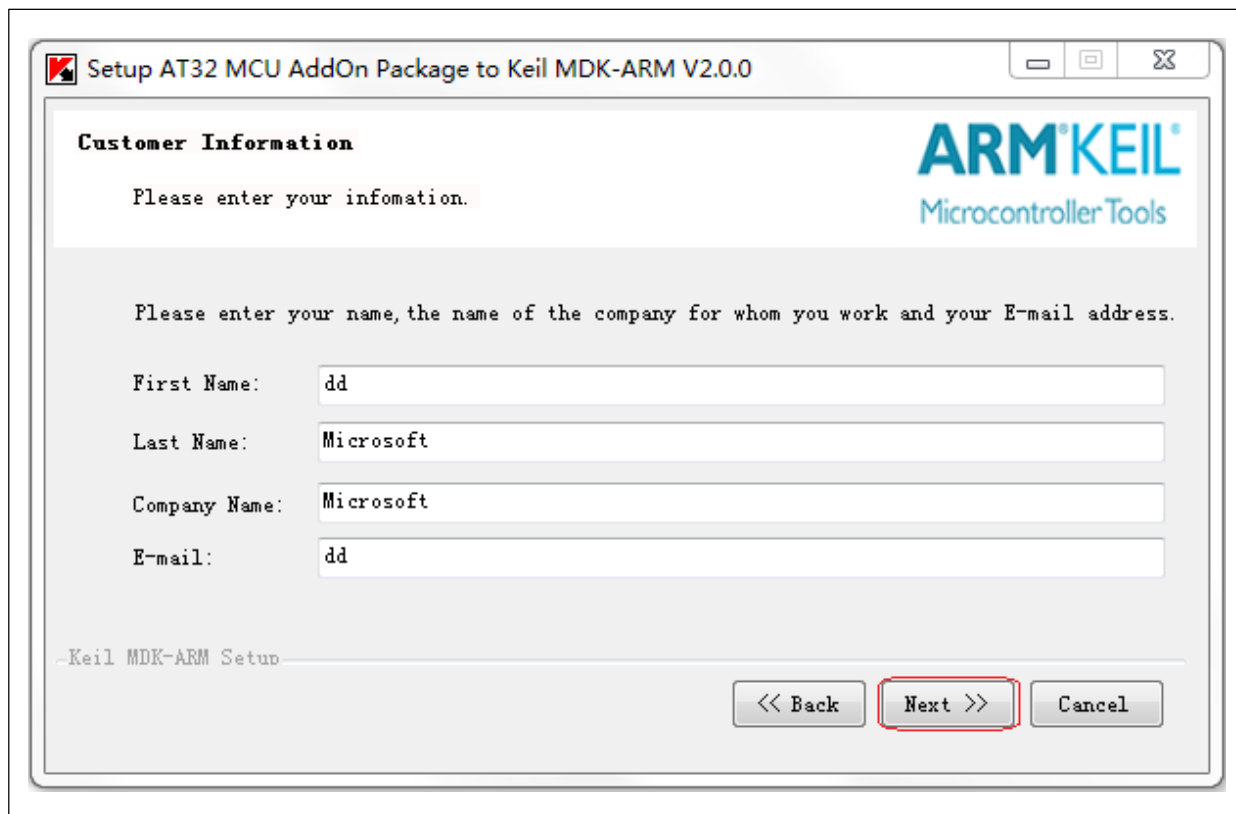
- ① Unzip *Keil4_AT32MCU_AddOn.zip*;
- ② Double click on *Keil4_AT32MCU_AddOn.exe*, and a dialog box pops up below (the specific version information is subject to the actual conditions).

Figure 6. Keil_v4 Pack installation



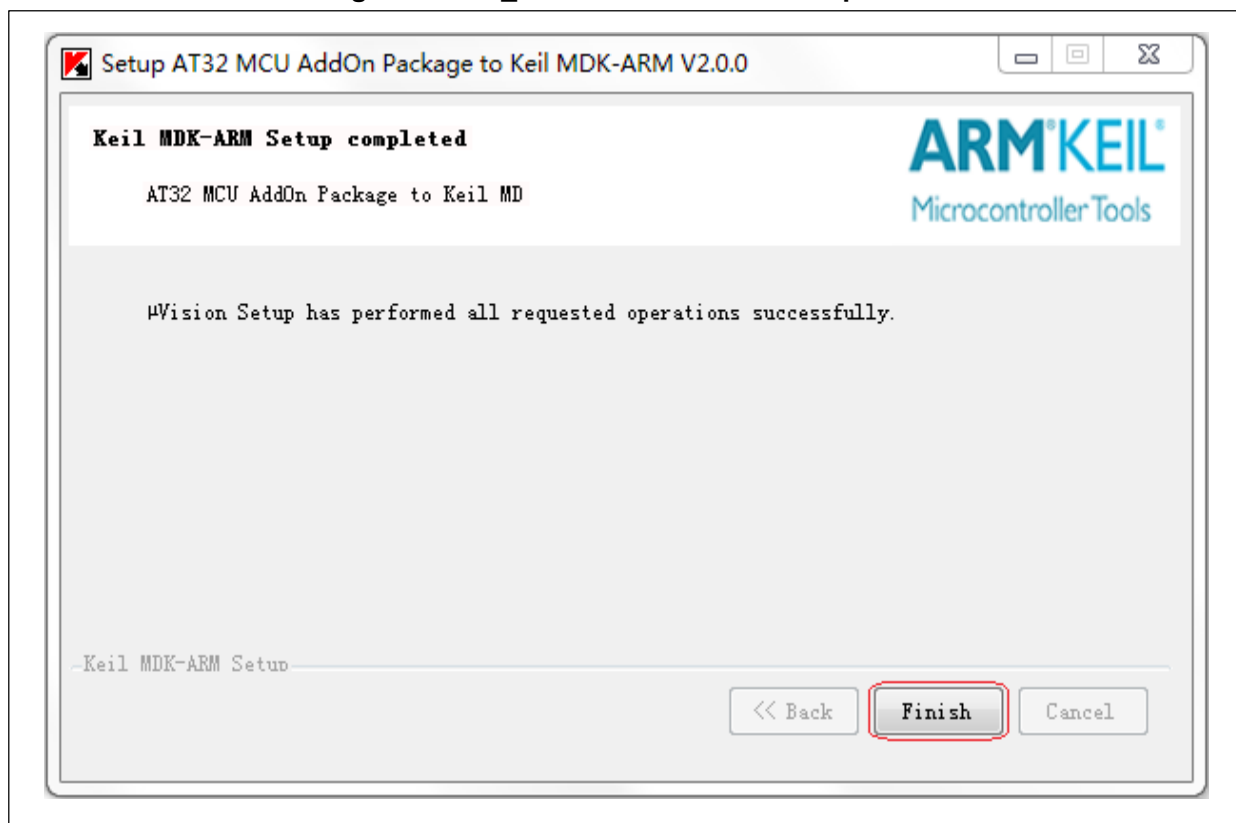
- ③ If the installation path of Keil_v4 does not match the "Destination Folder", click on "Browse" to select the actual correct path, then click on "Next", as shown below.

Figure 7. Keil_v4 Pack installation process



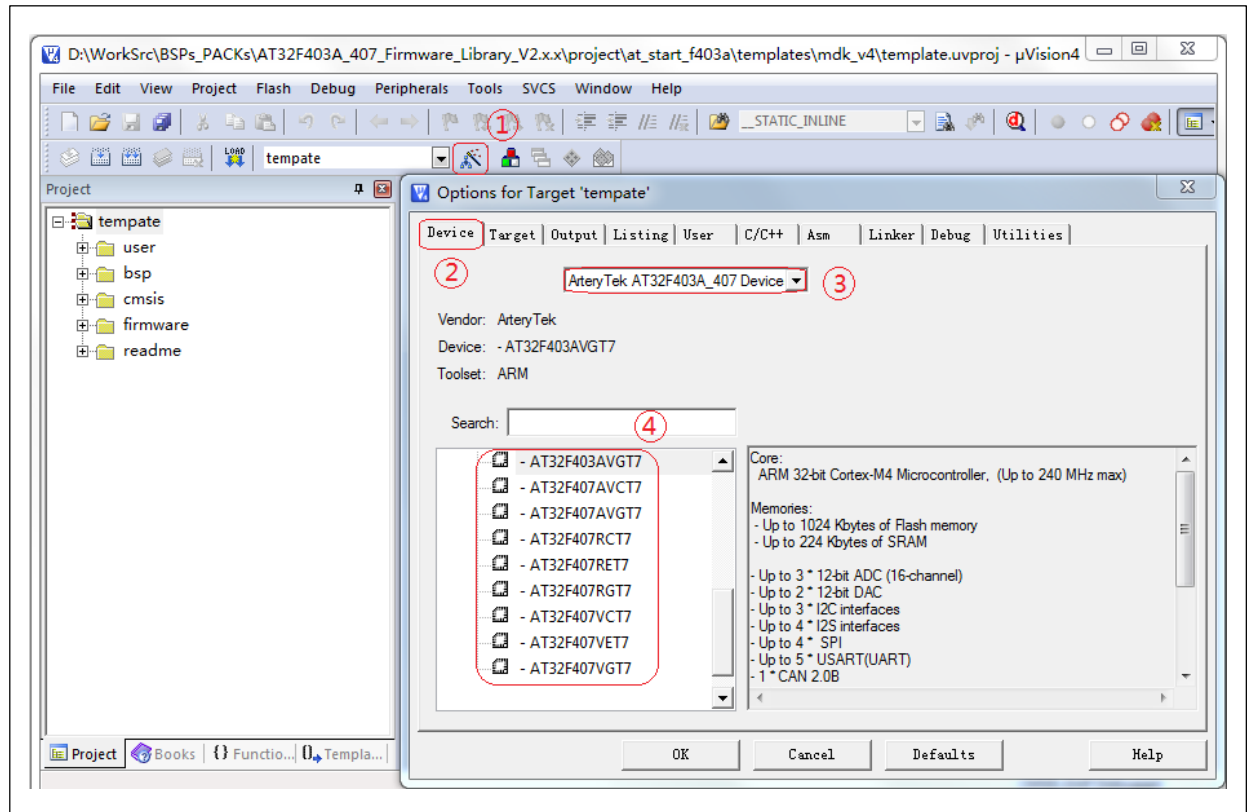
- ④ In the above “Customer Information” window, you can make some changes, but usually it is unnecessary. Then click on “Next” to start installation. The installation result is as follows.

Figure 8. Keil_v4 Pack installation complete



- ⑤ Click on “Finish”. To check whether Keil_v4 Pack is installed successfully or not, follow the below steps:
- Click on wand;
 - Select “Device”;
 - Select the desired pack file;
 - View ArteryTek-related information.

Figure 9. View Keil_v4 Pack installation status

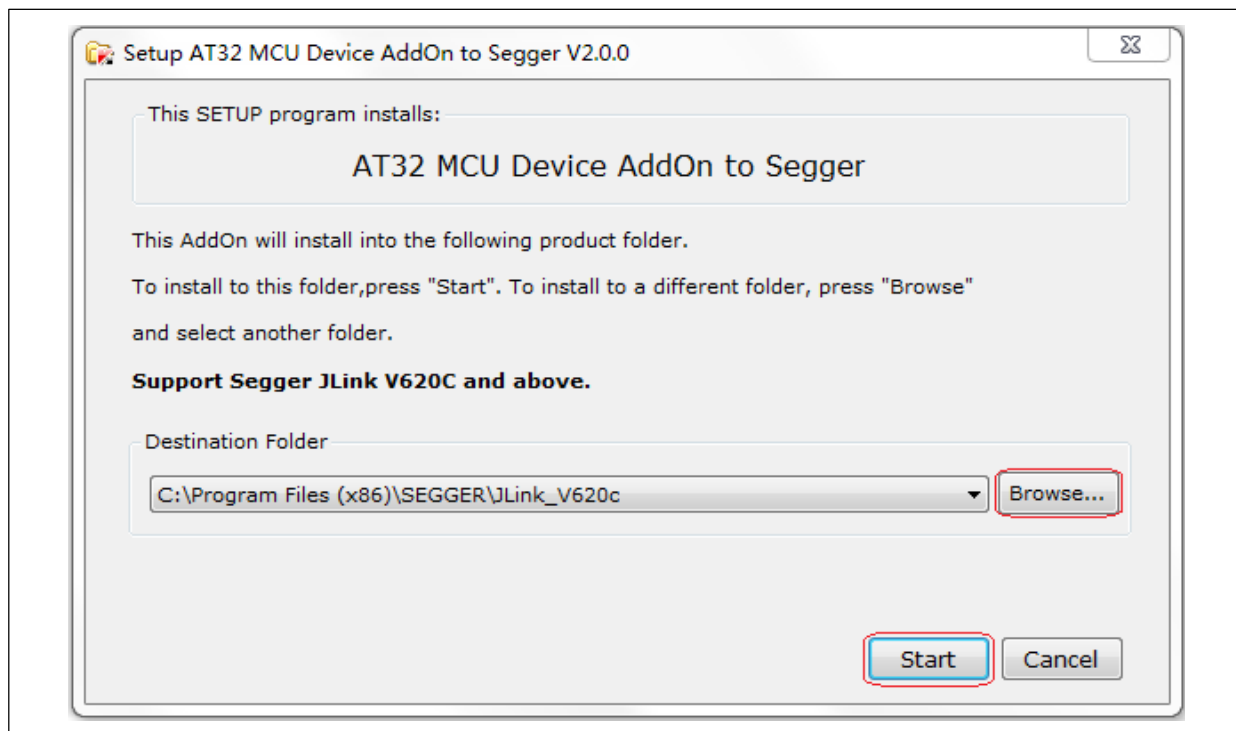


2.4 Segger Pack installation

Segger_AT32MCU_AddOn.zip: This is used to download J-Flash. Follow the steps below to install:

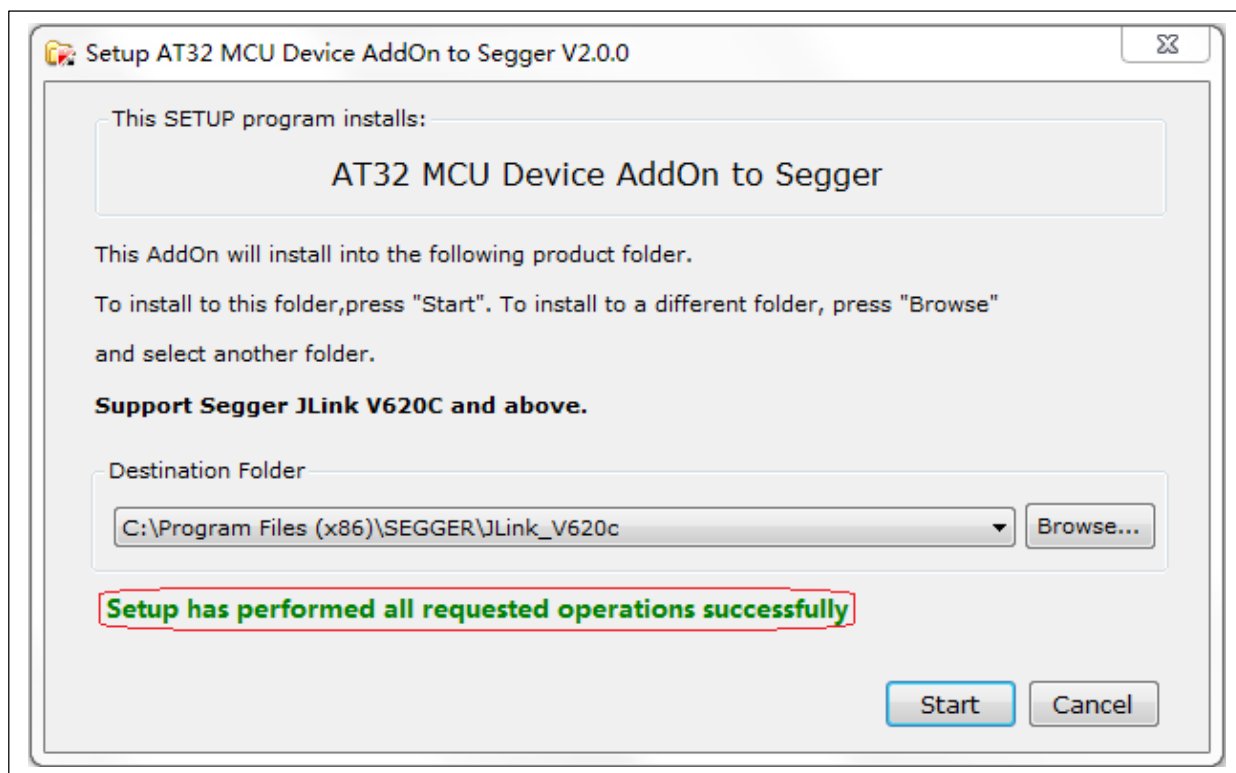
- ① Unzip *Segger_AT32MCU_AddOn.zip*;
- ② Double click on *Segger_AT32MCU_AddOn.exe*, and a dialog box pops up below (the specific version information is subject to the actual conditions).

Figure 10. Segger pack installation window



Note: If the installation path of Segger does not match the “Destination Folder”, click on “Browse” to select a correct path, then click on “Start”, as shown below.

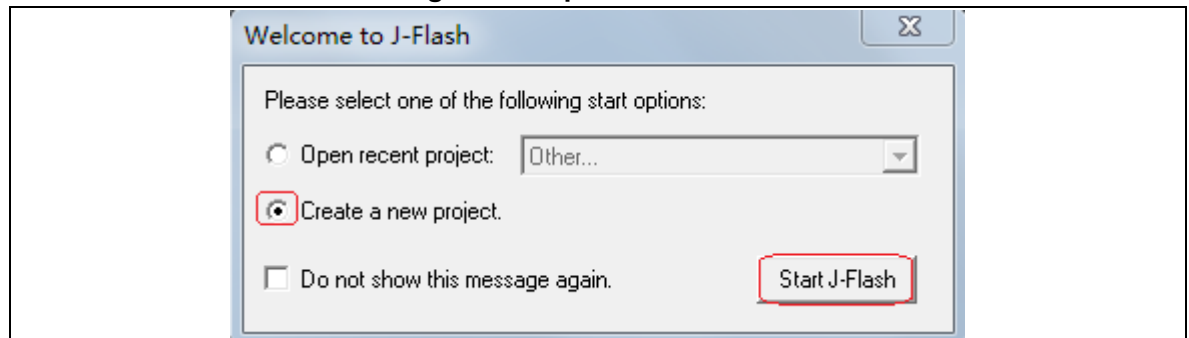
Figure 11. Segger pack installation process



- ③ If the “Setup has performed all requested operations successfully” appears, it indicates successful installation. To check whether the installation is successful or not, follow the steps below:
- Open *J-Flash.exe*, a dialog box appears; tick “Create a new project” and click on “Start J-

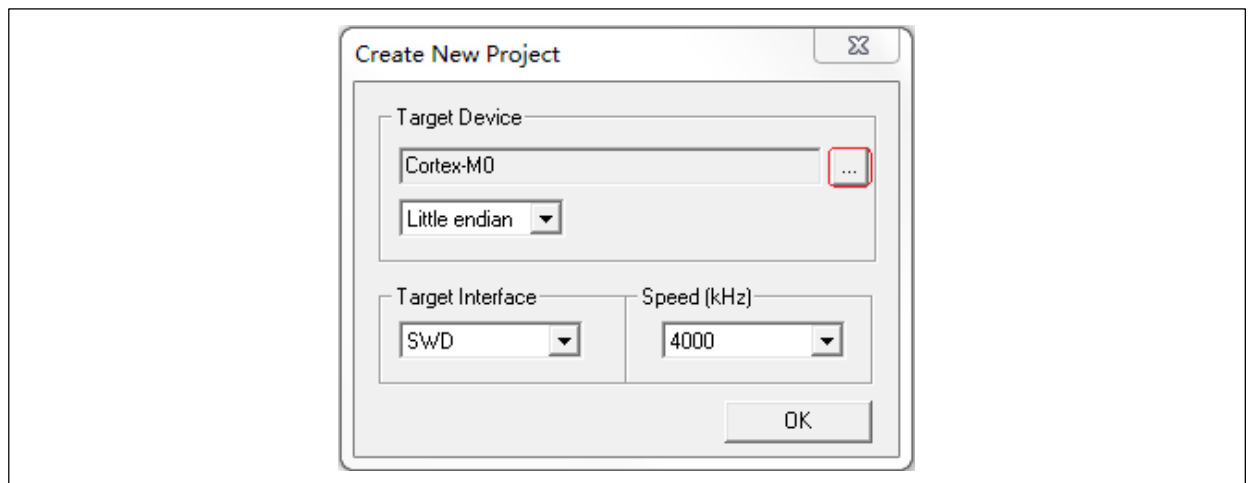
Flash”:

Figure 12. Open J-Flash



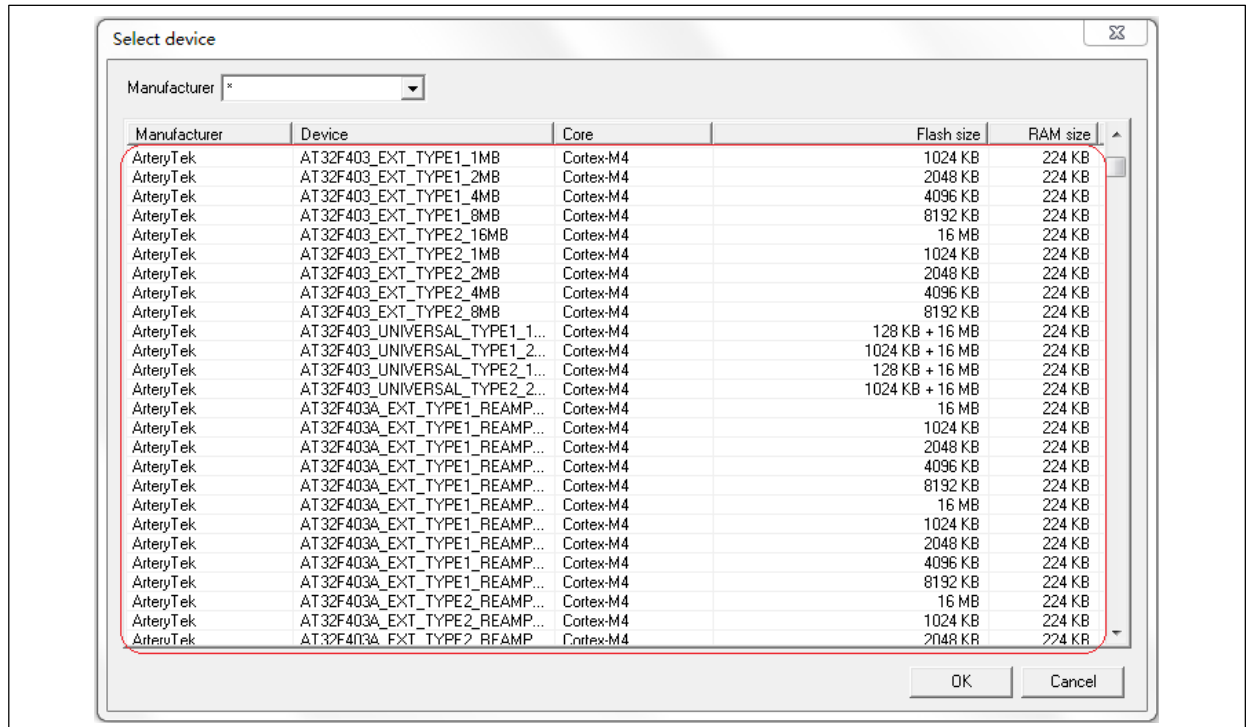
- After “Start J-Flash”, click on the check box under “Target Device”.

Figure 13. Create a new project using J-Flash



- Drag the scroll bar up and down in the check box. If the ArteryTek-related information and algorithm documents can be found, the installation is successful, as shown below:

Figure 14. View Device information



3 Flash algorithm file

Flash algorithm files are included in the Pack for online download through IDE tools such as KEIL/IAR. This section describes how to use Flash algorithm files.

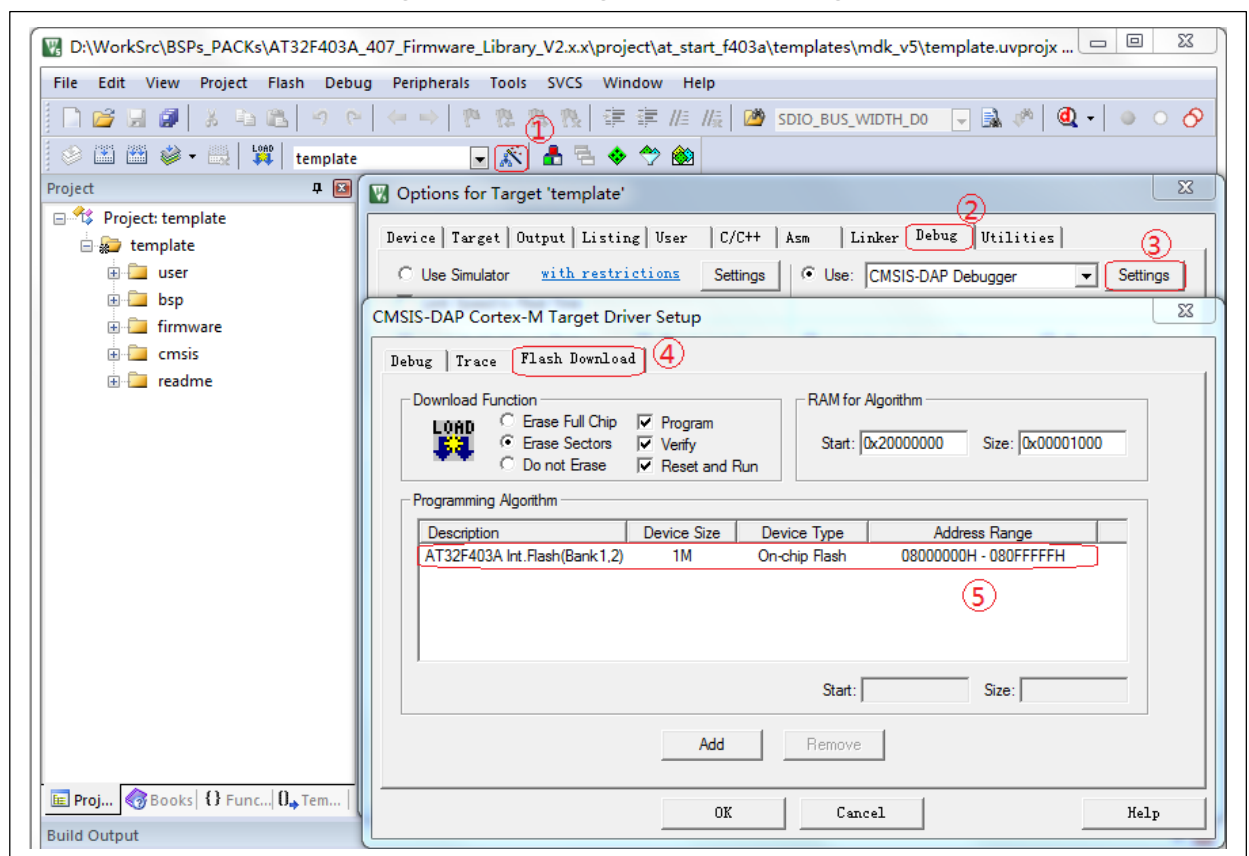
Note: AT32 MUCs have similar Flash algorithms, and this section uses AT32F403A as an example.

3.1 How to use Keil algorithm file

Common IDE tools such as Keil_v4 and Keil_v5 adopt a similar method to select and use the algorithm files. Here we take Keil_v5 as an example.

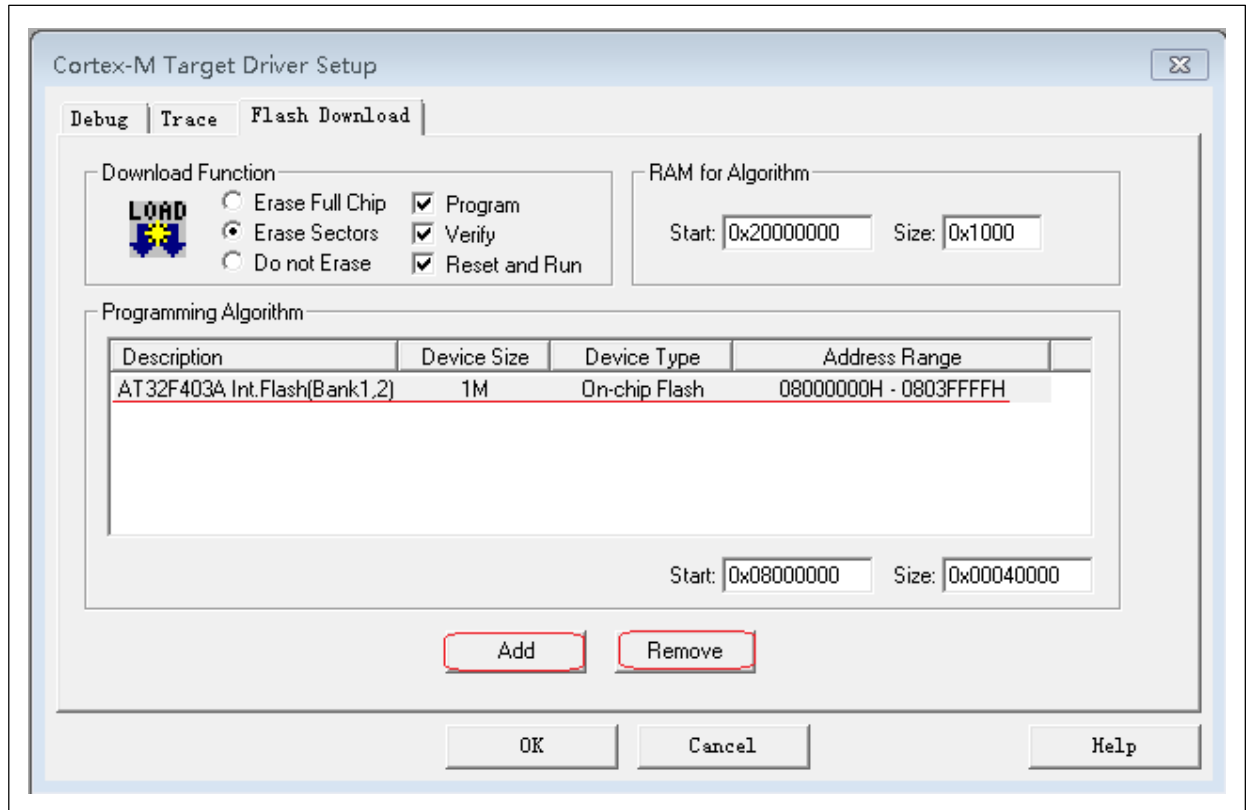
After creating a Keil IDE development tool project, the user can start Debug configuration and select the Flash algorithms. Go to *wand—>Debug—>Settings—>Flash Download*, as shown below:

Figure 15. Keil algorithm file settings



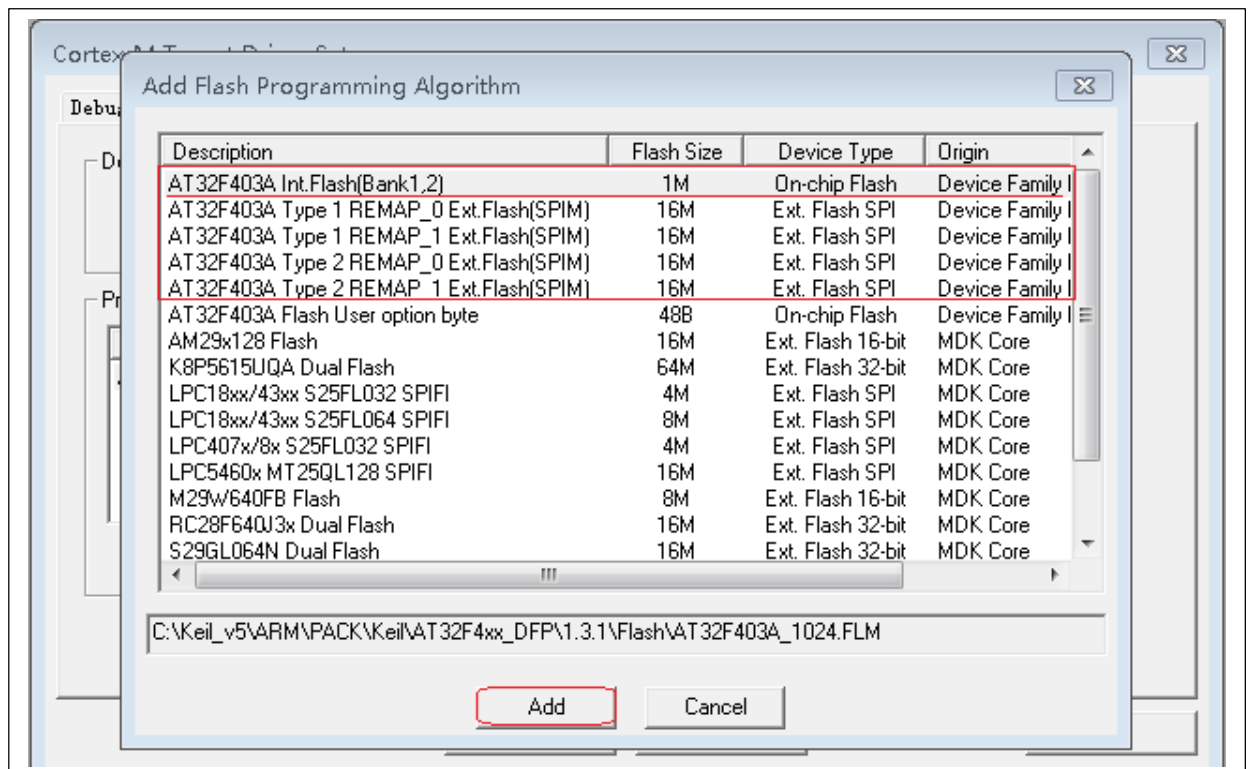
In this example, the selected Flash algorithm file is the default one. To change or remove it, click on this algorithm file, then click on *Add* or *Remove*. If the selected algorithm does not match the MCU, please follow the method below to modify.

Figure 16. Keil algorithm file configuration



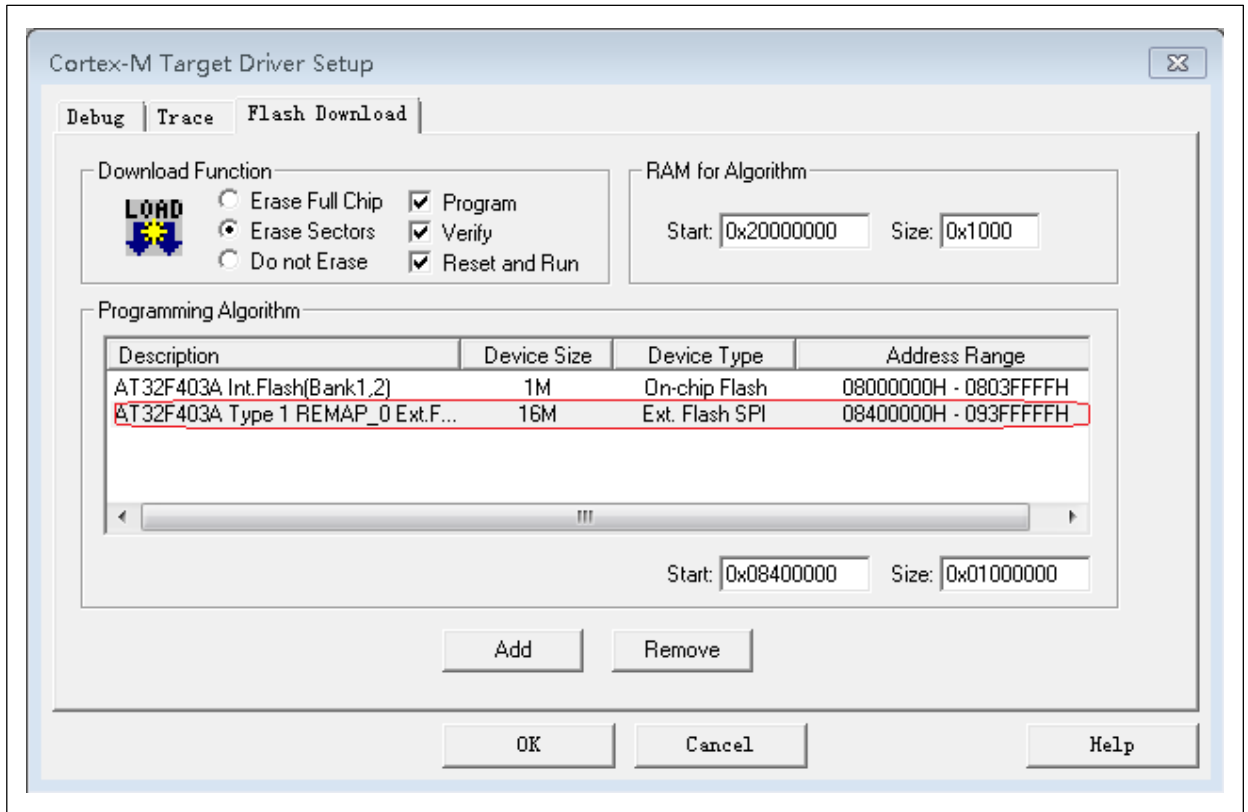
Click on *Remove* to remove the existing algorithm from the configuration, then click on *Add* to view the algorithm files associated with a MCU model and select them, as shown below:

Figure 17. Select algorithm files using Keil



After selection, click on *Add* to add the selected algorithm files into the current configuration. For example, a new SPI algorithm is added into the project.

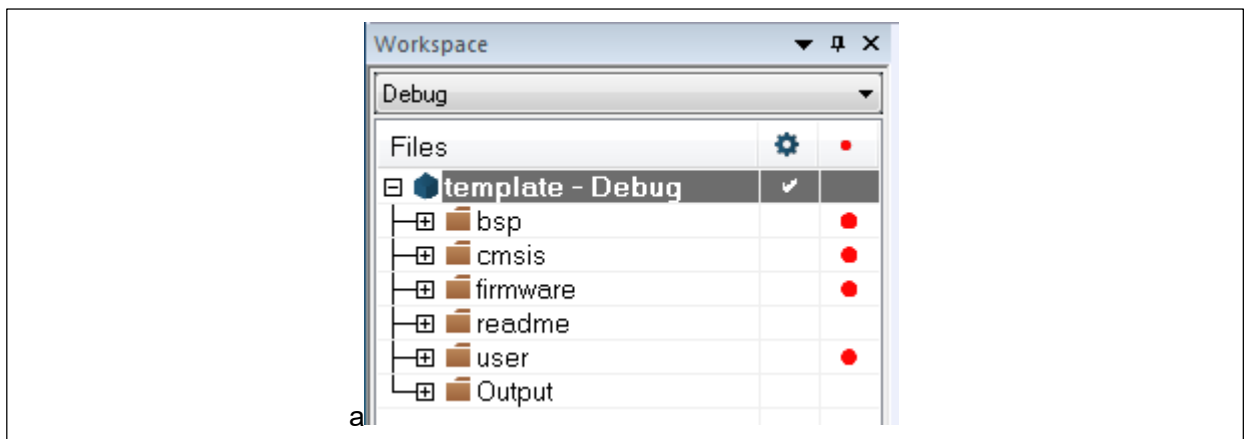
Figure 18. Add algorithm files using Keil



3.2 How to use IAR algorithm files

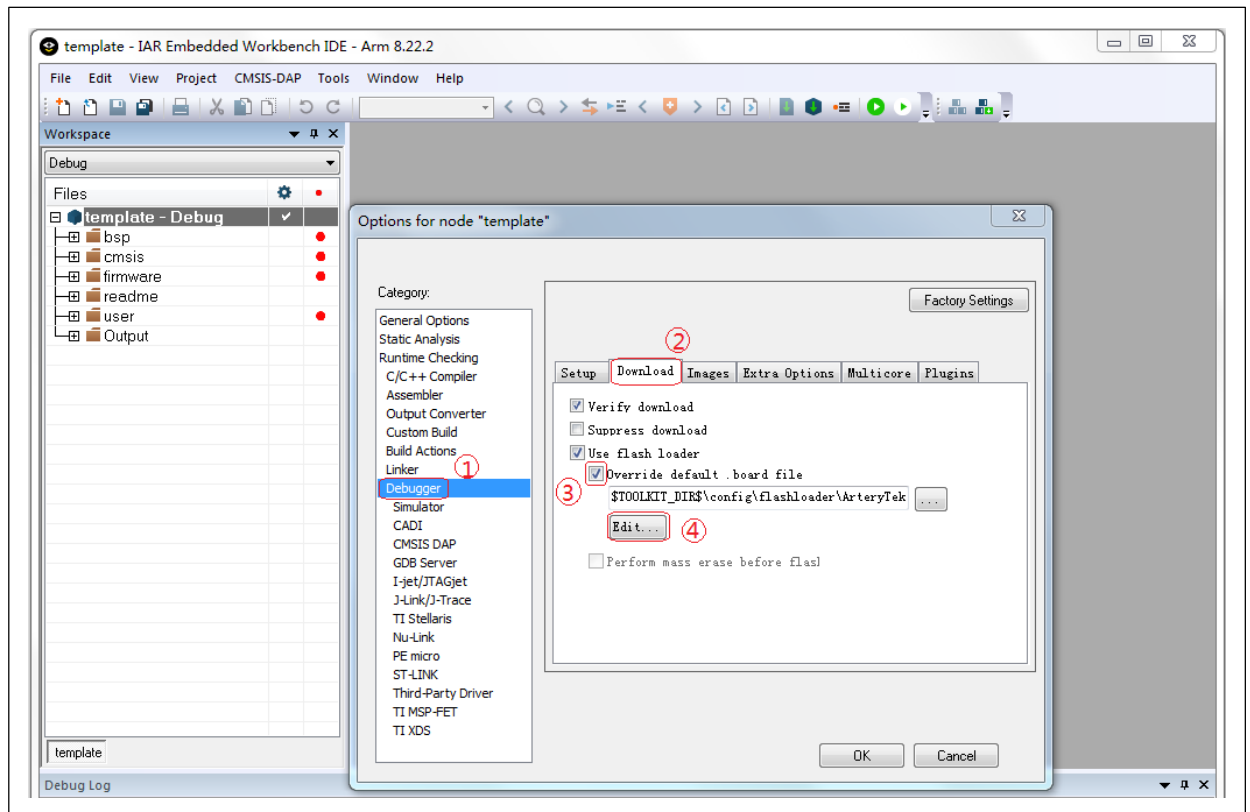
In IAR environment, the Flash algorithm files are automatically selected according to the selected MCU model during a new project configuration. To configure/modify an algorithm file manually, right-click on the file name (after an IAR project is created) in the following gray box:

Figure 19. IAR project name



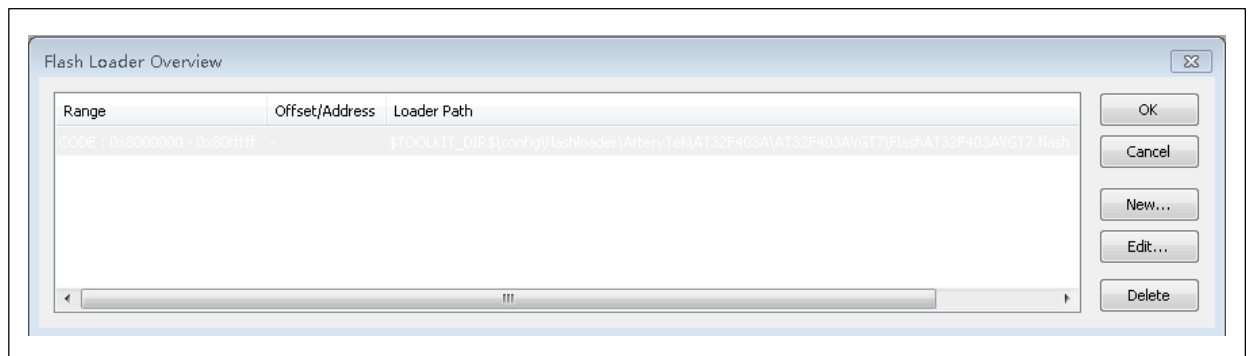
Go to *Options*—>*Debugger*—>*Download*—>Tick *Override default .board file*—>Click on *Edit*, as shown below:

Figure 20. IAR algorithm file configuration



Then the following window will be displayed.

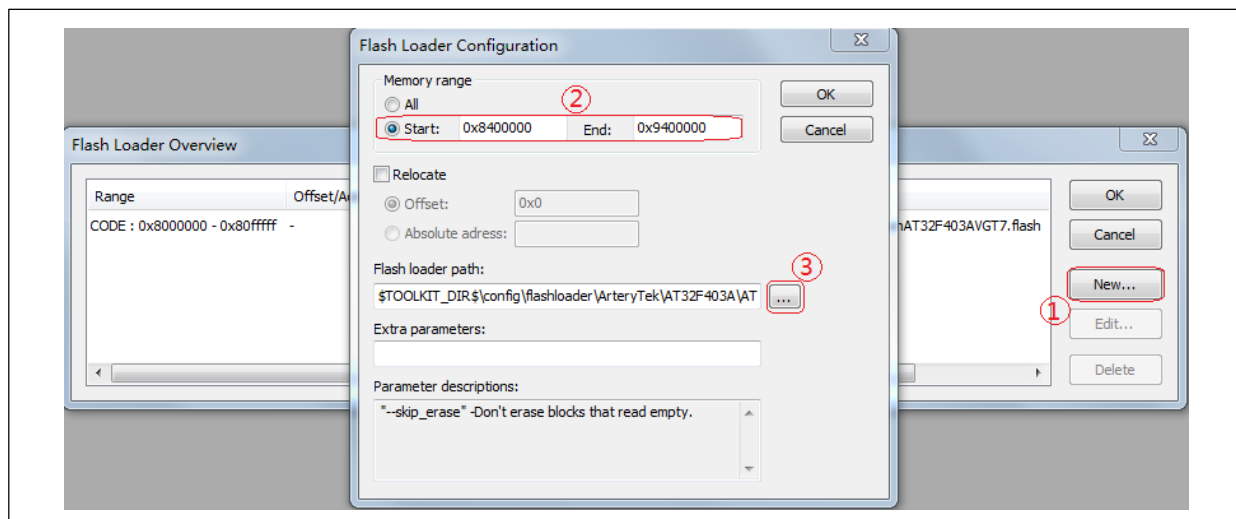
Figure 21. IAR Flash Loader overview



Flash algorithm configuration is designated by default after selecting a MCU part number. To modify it, click on *New/Edit/Delete*.

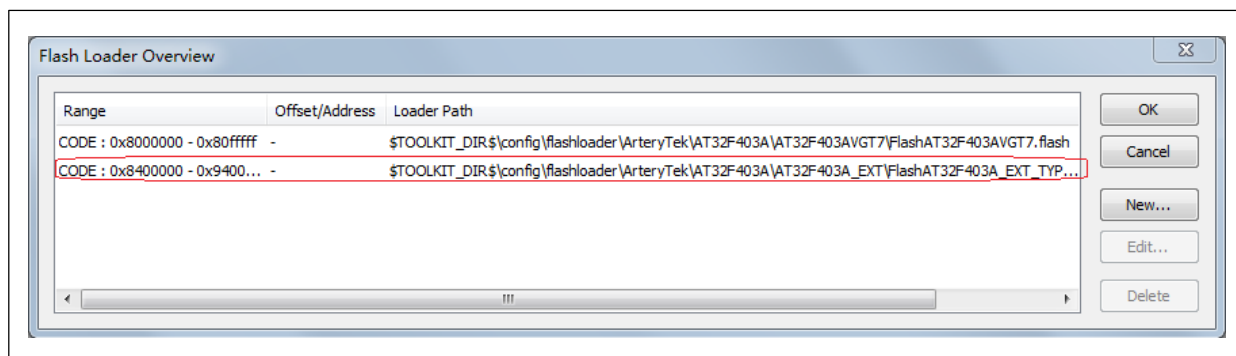
For example, click on *New*—>*Memory range*—> Select a Flash algorithm file, as shown below:

Figure 22. IAR Flash Loader configuration



This example shows how to add a SPIM Flash algorithm file. The user needs to select the corresponding MCU part number and a correct Flash algorithm file. The selected Flash algorithm configuration file is installed into IAR development environment using IAR_AT32MCU_AddOn tool. After a successful configuration, a new SPIM Flash algorithm is shown below:

Figure 23. IAR Flash Loader configuration success



1. Description of SPIM algorithms

Some Artery MCUs support Bank3 (refer to the Reference Manual or Datasheet on Artery official website for details), which can be used as an expansion of Flash memory in case of insufficient internal Flash or special application requirements. When the compiling addresses of some code or data are stored in the SPIM, these algorithm files are used for external Flash programming during online IDE tool download.

Naming rules of Artery SPIM algorithm file: AT32F4xxTypeNREMAP_P Ext.Flash.

N=1,2

P=0,1

TYPEN: External SPI Flash. Select it according to the external Flash type and part number. Refer to the FLASH_SELECT register section of the corresponding MCU Reference Manual.

REMAP_P: Select multiplex-function MCU SPIM PIN. Select it according to the connection method of pins connected to external Flash. Refer to the external SPIF remapping section in the corresponding MCU reference manual.

REMAP0: EXT_SPIF_GRP=000

REMAP1: EXT_SPIF_GRP=001

4 BSP introduction

4.1 Quick start

4.1.1 Template project

Artery firmware library BSP comes with a series of template projects built around Keil and IAR. For example, the template project of AT32F403A/407 is located in

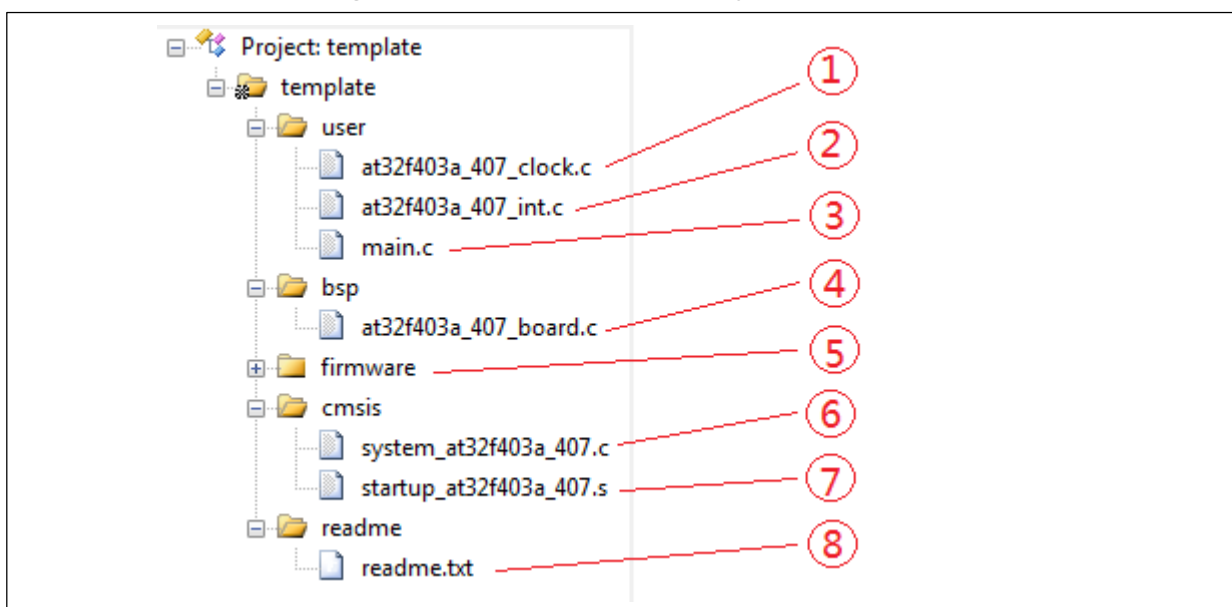
AT32F403A_407_Firmware_Library_V2.x.x/project/at_start_xxx/templates.

Figure 24. Template content

iar_v6.10	21/05/24 16:03	文件夹
iar_v7.4	21/05/24 16:03	文件夹
iar_v8.2	21/05/24 16:03	文件夹
inc	21/05/24 16:03	文件夹
mdk_v4	21/05/24 16:03	文件夹
mdk_v5	21/05/24 16:03	文件夹
src	21/05/24 16:03	文件夹
readme.txt	21/05/21 11:15	TXT 文件

The above template project includes various versions such as Keil_v5, Keil_v4, IAR_6.10, IAR_7.4 and IAR_8.2. Of those, “inc” and “src” folders contain header files and source code files, respectively. Open a corresponding folder and click on the corresponding file to open an IDE project. Figure 25 presents an example of Keil_v5 template project (its details and version are subject to the actual firmware library).

Figure 25. Keil_v5 template project example



The contents in a project include: (using AT32F403A/407 as an example, other products are similar)

- ① at32f403a_407_clock.c (clock configuration file) defines the default clock frequency and clock paths.
- ② at32f403a_407_int.c (interrupt file) contains some interrupt handling codes.

- ③ main.c contains the main code files.
- ④ at32f403a_407_board.c (board configuration file) contains common hardware configurations such as buttons and LEDs on the AT-START-Evaluation Board.
- ⑤ at32f403a_407_xx.c under firmware folder contains driver files of on-chip peripherals.
- ⑥ system_at32f403a_407.c is the system initialization file.
- ⑦ startup_at32f403a_407.s is a startup file.
- ⑧ readme.txt is a readme file, containing functional description and configuration information.

Note: AT32 MUCs share similar BSP usage method, and this section uses AT32F403A as an example.

4.1.2 BSP macro definitions

- ① To create a project, it is necessary to enable a startup code (startup_at32f403a_407.s) and open the appropriate macro definitions according to MCU part number before compiling code. Table 1 presents the correspondence between the MCU and their macro definitions.

Table 1. Summary of macro definitions

MCU part numbers	Macro definitions	PINs	Flash size (KB)
AT32F403ACCT7	AT32F403ACCT7	48	256
AT32F403ACET7	AT32F403ACET7	48	512
AT32F403ACGT7	AT32F403ACGT7	48	1024
AT32F403ACCU7	AT32F403ACCU7	48	256
AT32F403ACEU7	AT32F403ACEU7	48	512
AT32F403ACGU7	AT32F403ACGU7	48	1024
AT32F403ARCT7	AT32F403ARCT7	64	256
AT32F403ARET7	AT32F403ARET7	64	512
AT32F403ARGT7	AT32F403ARGT7	64	1024
AT32F403AVCT7	AT32F403AVCT7	100	256
AT32F403AVET7	AT32F403AVET7	100	512
AT32F403AVGT7	AT32F403AVGT7	100	1024
AT32F407RCT7	AT32F407RCT7	64	256
AT32F407RET7	AT32F407RET7	64	512
AT32F407RGT7	AT32F407RGT7	64	1024
AT32F407VCT7	AT32F407VCT7	100	256
AT32F407VET7	AT32F407VET7	100	512
AT32F407VGT7	AT32F407VGT7	100	1024
AT32F407AVCT7	AT32F407AVCT7	100	256
AT32F407AVGT7	AT32F407AVGT7	100	1024

- ② In the header file (at32f403a_407.h), USE_STDPERIPH_DRIVER (macro definition) is used to determine whether the Keil RTE feature is used or not. Enabling this definition while Keil RTE is unused can prevent some versions of Keil-MDK from opening _RTE_ accidentally.
- ③ The configuration header file (at32f403a_407_conf.h) defines macro definitions that enable peripherals. The file can be used to control the use of peripherals. The peripherals can be disabled simply by masking _MODULE_ENABLED pertaining to peripherals, as shown below:

Figure 26. Peripheral enable macro definitions

```
#define CRM_MODULE_ENABLED
#define TMR_MODULE_ENABLED
#define RTC_MODULE_ENABLED
#define BPR_MODULE_ENABLED
#define GPIO_MODULE_ENABLED
#define I2C_MODULE_ENABLED
#define USART_MODULE_ENABLED
#define PWC_MODULE_ENABLED
#define CAN_MODULE_ENABLED
#define ADC_MODULE_ENABLED
#define DAC_MODULE_ENABLED
#define SPI_MODULE_ENABLED
#define DMA_MODULE_ENABLED
#define DEBUG_MODULE_ENABLED
#define FLASH_MODULE_ENABLED
#define CRC_MODULE_ENABLED
#define WWDT_MODULE_ENABLED
#define WDT_MODULE_ENABLED
#define EXINT_MODULE_ENABLED
#define SDIO_MODULE_ENABLED
#define XMC_MODULE_ENABLED
#define USB_MODULE_ENABLED
#define ACC_MODULE_ENABLED
#define MISC_MODULE_ENABLED
#define EMAC_MODULE_ENABLED
```

at32f403a_407_conf.h also defines the HEXT_VALUE (high-speed external clock value), which should be modified accordingly when changing an external high-speed crystal oscillator.

- ④ The system clock configuration file (*at32f403a_407_clock.c/.h*) defines the default system clock frequency and clock paths. The user, if needed, can customize the frequency multiplication process and factors, or generate corresponding clock configuration files using the clock configuration host of ArteryTek.

4.2 BSP specifications

The subsequent sections give a description of BSP specifications.

4.2.1 List of abbreviations for peripherals

Table 2. List of abbreviations for peripherals

Abbreviations	Description
ADC	Analog-to-digital converter
BPR	Battery powered register
CAN	Controller area network
CRC	CRC calculation unit
CRM	Clock and reset manage
DAC	Digital-to-analog converter
DMA	Direct memory access
DEBUG	Debug
EXINT	External interrupt/event controller
GPIO	General-purpose I/Os
IOMUX	Multiplexed I/Os
I2C	Inter-integrated circuit interface
NVIC	Nested vectored interrupt controller
PWC	Power controller
RTC	Real-time clock
SPI	Serial peripheral interface
I2S	Inter-IC Sound
SysTick	System tick timer
TMR	Timer
USART	Universal synchronous/asynchronous receiver transmitter
WDT	Watchdog timer
WWDT	Window watchdog timer
XMC	External memory controller

4.2.2 Naming rules

The naming rules for BSP are described as follows:

“ip” indicates an abbreviation of a peripheral, for example, ADC, TMR, GPIO, etc., regardless of upper and lower case letters, such as, adc, tmr, gpio...

- **Source code file**

The file name starts with “at32fxxx_ip.c”, for example, at32f403a_407_adc.c

- **Header file**

The file name starts with “at32fxxx_ip.h”, such as, at32f403a_407_adc.h

- **Constant**

If it is used in a single one file, the constant is then defined in this file; if it is used in multiple files, the constant is defined in corresponding header file.

All constants are in written in English capital letters.

- **Variable**

If it is used in a single one file, the variable is then defined in this file; if it is used in multiple

files, the variable is declared with extern in the corresponding header file.

– Naming rules for functions

The peripheral functions are named based on the rule of “**peripheral abbreviatio_attribute_action**” or “**peripheral abbreviation_action**”.

The commonly used functions are as follows:

Function type	Naming rule	Example
Peripheral reset	ip_reset	adc_reset
Peripheral enable	ip_enable	adc_enable
Peripheral structure parameter initialize	ip_default_para_init	spi_default_para_init
Peripheral initialize	ip_init	spi_init
Peripheral interrupt enable	ip_interrupt_enable	adc_interrupt_enable
Peripheral flag get	ip_flag_get	adc_flag_get
Peripheral flag clear	ip_flag_clear	adc_flag_clear

4.2.3 Encoding rules

This section describes the encoding rules related to firmware function library.

Type of variables:

```
typedef int32_t INT32;
```

```
typedef int16_t INT16;
```

```
typedef int8_t INT8;
```

```
typedef uint32_t UINT32;
```

```
typedef uint16_t UINT16;
```

```
typedef uint8_t UINT8;
```

```
typedef int32_t s32;
```

```
typedef int16_t s16;
```

```
typedef int8_t s8;
```

```
typedef const int32_t sc32; /*!< read only */
```

```
typedef const int16_t sc16; /*!< read only */
```

```
typedef const int8_t sc8; /*!< read only */
```

```
typedef __IO int32_t vs32;
```

```
typedef __IO int16_t vs16;
```

```
typedef __IO int8_t vs8;
```

```
typedef __I int32_t vsc32; /*!< read only */
```

```
typedef __I int16_t vsc16; /*!< read only */
```

```
typedef __I int8_t vsc8; /*!< read only */
```

```
typedef uint32_t u32;
```

```
typedef uint16_t u16;
```

```
typedef uint8_t u8;
```

```
typedef const uint32_t uc32; /*!< read only */
```

```
typedef const uint16_t uc16; /*!< read only */
```

```
typedef const uint8_t uc8; /*!< read only */
```

```
typedef __IO uint32_t vu32;
```

```
typedef __IO uint16_t vu16;
```

```
typedef __IO uint8_t vu8;
```

```
typedef __I uint32_t vuc32; /*!< read only */
```

```
typedef __I uint16_t vuc16; /*!< read only */
```

```
typedef __I uint8_t vuc8; /*!< read only */
```

4.2.3.1 Flag type

```
typedef enum {RESET = 0, SET = !RESET} flag_status;
```

4.2.3.2 Function status type

```
typedef enum {FALSE = 0, TRUE = !FALSE} confirm_state;
```

4.2.3.3 Error status type

```
typedef enum {ERROR = 0, SUCCESS = !ERROR} error_status;
```

4.2.3.4 Peripheral type

① Peripherals

Define the base address of peripheral in the at32fxxx_ip.h, for example, in the at32f403a_407.h:

```
#define ADC1_BASE (APB2PERIPH_BASE + 0x2400)
```

```
#define ADC2_BASE (APB2PERIPH_BASE + 0x2800)
```

Define the type of a peripheral in the at32fxxx_ip.h, for example, in the at32f403a_407_adc.h:

```
#define ADC1 ((adc_type *) ADC1_BASE)
```

```
#define ADC2 ((adc_type *) ADC2_BASE)
```

② Peripheral registers and bits

Define the type of a peripheral in the at32fxxx_ip.h, for example, in the at32f403a_407_adc.h

```
/**
 * @brief type define adc register all
 */
typedef struct
{
    /**
     * @brief adc sts register, offset:0x00
     */
    union
    {
        __IO uint32_t sts;
    }
} struct
```



```

{
    __IO uint32_t vmor                : 1; /* [0] */
    __IO uint32_t cce                 : 1; /* [1] */
    __IO uint32_t pcce                : 1; /* [2] */
    __IO uint32_t pccs                : 1; /* [3] */
    __IO uint32_t occs                : 1; /* [4] */
    __IO uint32_t reserved1           : 27; /* [31:5] */
} sts_bit;
};
...
...
...
/**
 * @brief adc odt register, offset:0x4C
 */
union
{
    __IO uint32_t odt;
    struct
    {
        __IO uint32_t odt                : 16; /* [15:0] */
        __IO uint32_t adc2odt           : 16; /* [31:16] */
    } odt_bit;
};
} adc_type;

```

③ Examples of peripheral register access

Read peripheral	i = ADC1-> ctrl1;
Write peripheral	ADC1-> ctrl1 = i;
Read bit 5 in bit-field mode	i = ADC1-> ctrl1. cceien;
Write 1 to bit 5 in bit-field mode	ADC1-> ctrl1. cceien= TRUE;
Write 1 to bit 5	ADC1-> ctrl1 = 1<<5;
Write 0 to bit 5	ADC1-> ctrl1&= ~(1<<5);

4.3 BSP structure

4.3.1 BSP folder structure

BSP(Board Support Package) structure is shown in Figure 27.

Figure 27. BSP folder structure

 document	21/05/18 10:32	文件夹
 libraries	21/05/18 10:32	文件夹
 middlewares	21/05/18 10:32	文件夹
 project	21/05/18 10:32	文件夹
 utilities	21/05/14 11:35	文件夹

Document:

- AT32Fxxx firmware library BSP&Pack user guide.pdf: refer to BSP/Pack user manual
- ReleaseNotes_AT32F403A_407_Firmware_Library.pdf: document revision history

Libraries:

- **Drivers:** driver library for peripherals
Src folder: low-level driver source file for peripherals, such as, at32fxxx_ip.c
inc folder: low-level driver header file for peripherals, such as, at32fxxx_ip.h
- **Cmsis:** Core-related files
cm4 folder: core-related files, including cortex-m4 library, system initialization file, startup file, etc.
dsp folder: dsp-related files

Middlewares:

Third-party software or public protocols, including USB protocol layer driver, network protocol driver, operating system source code, etc.

Project:

Examples: demo

Templates: template projects, including Keil4, keil5, IAR6, IAR7, IAR8 and eclipse_gcc

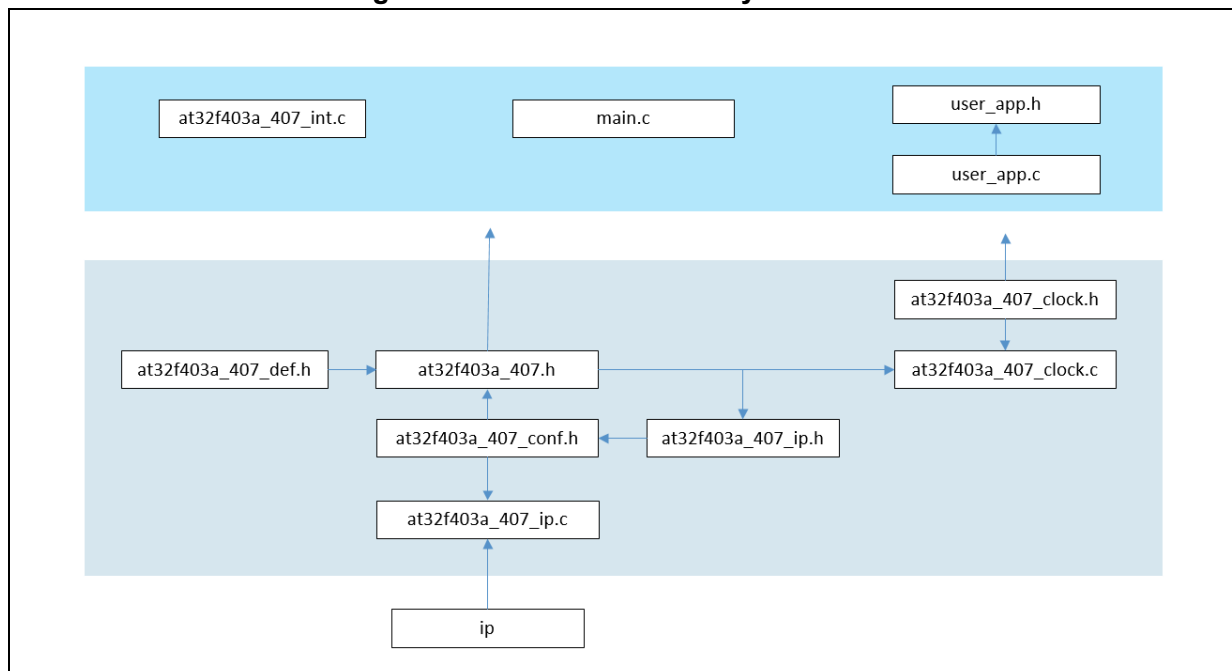
Utilities:

Store application cases

4.3.2 BSP function library structure

Figure 28 shows the architecture of BSP function library.

Figure 28. BSP function library structure



BSP function library files are described in Table 3.

Table 3. Summary of BSP function library files

File name	Description
at32f403a_407_conf.h	Macro definition for peripheral enable, and external high-speed clock HEXT_VALUE
main.c	Main function
at32f403a_407_ip.c	Driver source file for a peripheral, for example, at32f403a_407_adc.c
at32f403a_407_ip.h	Driver header file for a peripheral, for example, at32f403a_407_adc.h
at32f403a_407.h	In the header file (at32f403a_407.h), the definition USE_STDPERIPH_DRIVER is used to determine whether the Keil RTE is used or not. Enabling the definition while Keil RTE is unused can prevent Keil-MDK from enabling _RTE_ accidentally.
at32f403a_407_clock.c	This is a clock configuration file used to configure default clock frequency and clock path.
at32f403a_407_clock.h	This is a clock configure header file.
at32f403a_407_int.c	This is a source file for interrupt functions that programs interrupt handling code.
at32f403a_407_int.h	This is a header file for interrupt functions.
at32f403a_407_misc.c	This is a source file for other configurations, such as, nvic configuration function, systick clock source selection.
at32f403a_407_misc.h	This is a header file for other configurations.
startup_at32f403a_407.s	This is a startup file.

4.3.3 Initialization and configuration for peripherals

This section describes how to initialize and configure peripherals using GPIO as an example.

GPIO initialization

- Step 1: Define the `gpio_init_type`, for example, `gpio_init_type gpio_init_struct;`
- Step 2: Enable GPIO clock using the function `crm_periph_clock_enable;`
- Step 3: De-initialize the structure `gpio_init_struct` to allow the values of other members (mostly default values) to be correctly written, for example, `gpio_default_para_init(&gpio_init_struct);`
- Step 4: Configure member of the structure, and write structure parameters into GPIO registers through the `gpio_init`, for example,

```
gpio_init_struct.gpio_pins = GPIO_PINS_2 | GPIO_PINS_3;
gpio_init_struct.gpio_mode = GPIO_MODE_OUTPUT;
gpio_init_struct.gpio_out_type = GPIO_OUTPUT_PUSH_PULL;
gpio_init_struct.gpio_pull = GPIO_PULL_NONE;
gpio_init_struct.gpio_drive_strength = GPIO_DRIVE_STRENGTH_STRONGER;
gpio_init(GPIOA, &gpio_init_struct);
```

For more information on peripheral initialization procedure, refer to the section of peripherals of the reference manual, and the section of peripherals of the `AT32Fxxx_Firmware_Library_V2.x.x.zip\project\at_start_fx\examples`.

4.3.4 Peripheral functions format description

Table 4. Function format description for peripherals

Name	Description
Function name	The name of a peripheral function
Function prototype	Prototype declaration
Function description	Brief description of how the function is executed
Input parameter (n)	Description of the input parameters
Output parameter (n)	Description of the output parameters
Return value	Value returned by the function
Required preconditions	Requirements before calling the function
Called functions	Other library functions called

5 AT32F402/405 peripheral library functions

5.1 HICK automatic clock calibration (ACC)

The ACC register structure `acc_type` is defined in the “at32f402/405_acc.h”:

```
/**
 * @brief type define acc register all
 */
typedef struct
{
    .....
} acc_type;
```

The table below gives a list of the ACC registers.

Table 5. Summary of ACC registers

Register	Description
<code>acc_sts</code>	ACC status register
<code>acc_ctrl1</code>	ACC control register 1
<code>acc_ctrl2</code>	ACC control register 2
<code>acc_c1</code>	ACC compare value 1
<code>acc_c2</code>	ACC compare value 2
<code>acc_c3</code>	ACC compare value 3

The table below gives a list of the ACC library functions.

Table 6. Summary of ACC library functions

Function name	Description
<code>acc_calibration_mode_enable</code>	ACC calibration mode enable
<code>acc_step_set</code>	Configure ACC calibration step length
<code>acc_interrupt_enable</code>	ACC interrupt enable
<code>acc_hicktrim_get</code>	Get ACC trimming calibration value
<code>acc_hickcal_get</code>	Get ACC coarse calibration value
<code>acc_write_c1</code>	Write ACC C1 register value
<code>acc_write_c2</code>	Write ACC C2 register value
<code>acc_write_c3</code>	Write ACC C3 register value
<code>acc_read_c1</code>	Read ACC C1 register value
<code>acc_read_c2</code>	Read ACC C2 register value
<code>acc_read_c3</code>	Read ACC C3 register value
<code>acc_flag_get</code>	Get ACC interrupt flag
<code>acc_flag_clear</code>	Clear ACC interrupt flag

5.1.1 acc_calibration_mode_enable function

The table below describes the function acc_calibration_mode_enable.

Table 7. acc_calibration_mode_enable function

Name	Description
Function name	acc_calibration_mode_enable
Function prototype	void acc_calibration_mode_enable(uint16_t acc_trim, confirm_state new_state);
Function description	ACC calibration mode enable
Input parameter 1	acc_trim: calibration mode selection This parameter can be ACC_CAL_HICKCAL or ACC_CAL_HICKTRIM.
Input parameter 2	new_state: Enable or disable ACC
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

acc_trim

Calibration mode selection

ACC_CAL_HICKCAL: Coarse calibration mode

ACC_CAL_HICKTRIM: Fine calibration mode

new_state

Enable or disable ACC

FALSE: Disabled

TRUE: Enabled

Example:

```
/* open acc calibration */  
acc_calibration_mode_enable(ACC_CAL_HICKTRIM, TRUE);
```

5.1.2 acc_step_set function

The table below describes the function acc_step_set.

Table 8. acc_step_set function

Name	Description
Function name	acc_step_set
Function prototype	void acc_step_set(uint8_t step_value);
Function description	Configure ACC calibration step length
Input parameter 1	step_value: step value for calibration
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

step_value

This 4-bit field defines the value to be changed for each calibration.

Note: To obtain better calibration accuracy, it is recommended to set the step value to 1.

When ENTRIM=0 only the HICKCAL is calibrated. If the step value is incremented or decremented by one, the corresponding HICKCAL follows the change rule (increased or decreased by one), and the HICK frequency will increase or decrease by 40 KHz (design value), meaning a positive correlation between them.

When ENTRIM=1, only the HICKTRIM is calibrated. If the step value is incremented or decremented by one, the corresponding HICKTRIM follows the change rule (increased or decreased by one), and the HICK frequency will increase or decrease by 20KHz (design value), meaning a positive correlation between them.

Example:

```
/* set acc step value */
acc_step_set(0x1);
```

5.1.3 acc_interrupt_enable function

The table below describes the function acc_interrupt_enable.

Table 9. acc_interrupt_enable function

Name	Description
Function name	dma_interrupt_enable
Function prototype	void acc_interrupt_enable(uint16_t acc_int, confirm_state new_state);
Function description	Enable acc interrupts
Input parameter 1	acc_int: interrupt source selection
Input parameter 2	new_state: enable or disable interrupts
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

acc_int

interrupt source selection

ACC_CALRDYIEN_INT: Calibration complete interrupt

ACC_EIEN_INT: Reference signal lost interrupt

new_state

Enable or disable interrupts.

FALSE: Interrupt disabled

TRUE: Interrupt enabled

Example:

```
/* enable the acc reference signal lost interrupt */
acc_interrupt_enable(ACC_EIEN_INT, TRUE);
```

5.1.4 acc_hicktrim_get function

The table below describes the function acc_hicktrim_get.

Table 10. acc_hicktrim_get function

Name	Description
Function name	acc_hicktrim_get
Function prototype	uint8_t acc_hicktrim_get(void);
Function description	Get ACC trimming value
Input parameter	NA
Output parameter	NA
Return value	Return ACC trimming value
Required preconditions	NA
Called functions	NA

Example:

```
/* get trim value*/
uint8_t trim_value;
trim_value = acc_hicktrim_get();
```

5.1.5 acc_hickcal_get function

The table below describes the function acc_hickcal_get.

Table 11. acc_hickcal_get function

Name	Description
Function name	acc_hickcal_get
Function prototype	uint8_t acc_hickcal_get(void);
Function description	Get ACC coarse calibration value
Input parameter	NA
Output parameter	NA
Return value	Return ACC coarse calibration value
Required preconditions	NA
Called functions	NA

Example:

```
/* get cal value*/
uint8_t cal_value;
cal_value = acc_hickcal_get ();
```


5.1.6 acc_write_c1 function

The table below describes the function acc_write_c1.

Table 12. acc_write_c1 function

Name	Description
Function name	acc_write_c1
Function prototype	void acc_write_c1(uint16_t acc_c1_value);
Function description	Write ACC C1 register value
Input parameter	acc_c1_value: the value to be written in ACC C1 register
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* update the c1 value */
acc_c2_value = 8000;
acc_write_c1(acc_c2_value - 10);
```

5.1.7 acc_write_c2 function

The table below describes the function acc_write_c2.

Table 13. acc_write_c2 function

Name	Description
Function name	acc_write_c2
Function prototype	void acc_write_c2(uint16_t acc_c2_value);
Function overview	Write ACC C2 register value
Input parameter	acc_c2_value: the value to be written in ACC C2 register
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* update the c2 value */
acc_c2_value = 8000;
acc_write_c2(acc_c2_value - 10);
```

5.1.8 acc_write_c3 function

The table below describes the function acc_write_c3.

Table 14. acc_write_c3 function

Name	Description
Function name	acc_write_c3
Function prototype	void acc_write_c3(uint16_t acc_c3_value);
Function description	Write ACC C3 register value
Input parameter	acc_c3_value: the value to be written in ACC C3 register
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* update the c3 value */  
acc_c2_value = 8000;  
acc_write_c3(acc_c2_value - 10);
```

5.1.9 acc_read_c1 function

The table below describes the function acc_read_c1.

Table 15. acc_read_c1 function

Name	Description
Function name	acc_read_c1
Function prototype	uint16_t acc_read_c1(void);
Function description	Read ACC C1 register value
Input parameter	NA
Output parameter	NA
Return value	ACC C1 register value
Required preconditions	NA
Called functions	NA

Example:

```
/* get the c1 value */  
uint16_t acc_c1_value;  
acc_c1_value = acc_read_c1();
```

5.1.10 acc_read_c2 function

The table below describes the function acc_read_c2.

Table 16. acc_read_c2 function

Name	Description
Function name	acc_read_c2
Function prototype	uint16_t acc_read_c2(void);
Function description	Read ACC C2 register value
Input parameter	NA
Output parameter	NA
Return value	ACC C2 register value
Required preconditions	NA
Called functions	NA

Example:

```
/* get the c2 value */
uint16_t acc_c2_value;
acc_c2_value = acc_read_c2();
```

5.1.11 acc_read_c3 function

The table below describes the function acc_read_c3.

Table 17. acc_read_c3 function

Name	Description
Function name	acc_read_c3
Function prototype	uint16_t acc_read_c3(void);
Function description	Read ACC C3 register value
Input parameter	NA
Output parameter	NA
Return value	ACC C3 register value
Required preconditions	NA
Called functions	NA

Example:

```
/* get the c3 value */
uint16_t acc_c3_value;
acc_c3_value = acc_read_c3();
```

5.1.12 acc_flag_get function

The table below describes the function acc_flag_get.

Table 18. acc_flag_get function

Name	Description
Function name	acc_flag_get
Function prototype	flag_status acc_flag_get(uint16_t acc_flag);
Function description	Get ACC flag status
Input parameter 1	acc_flag: ACC flag selection
Output parameter	NA
Return value	flag_status: indicates whether or not the flag has been set
Required preconditions	NA
Called functions	NA

acc_flag

The acc_flag is used for flag selection, including:

ACC_RSLOST_FLAG: Reference signal lost interrupt

ACC_CALRDY_FLAG: Calibration complete interrupt

flag_status

RESET: Corresponding flag bit is not set

SET: Corresponding flag bit is set

Example:

```
if(acc_flag_get(ACC_CALRDY_FLAG) != RESET)
{
    at32_led_toggle(LED2);
    /* clear acc calibration ready flag */
    acc_flag_clear(ACC_CALRDY_FLAG);
}
```

5.1.13 acc_interrupt_flag_get function

The table below describes the function acc_interrupt_flag_get.

Table 19. acc_interrupt_flag_get function

Name	Description
Function name	acc_interrupt_flag_get
Function prototype	flag_status acc_interrupt_flag_get(uint16_t acc_flag);
Function description	Get acc interrupt flag
Input parameter 1	acc_flag: ACC interrupt flag selection
Output parameter	NA
Return value	flag_status: indicates whether or not the interrupt flag has been set
Required preconditions	NA
Called function	NA

acc_flag

The acc_flag is used for flag selection, including:

ACC_RSLOST_FLAG: Reference signal lost interrupt

ACC_CALRDY_FLAG: Calibration complete interrupt

flag_status

RESET: Corresponding flag bit is not set

SET: Corresponding flag bit is set

Example

```
if(acc_interrupt_flag_get(ACC_CALRDY_FLAG) != RESET)
{
    at32_led_toggle(LED2);
    /* clear acc calibration ready flag */
    acc_flag_clear(ACC_CALRDY_FLAG);
}
```

5.1.14 acc_flag_clear function

The table below describes the function acc_flag_clear.

Table 20. acc_flag_clear function

Name	Description
Function name	acc_flag_clear
Function prototype	void acc_flag_clear(uint16_t acc_flag);
Function description	Clear ACC flag
Input parameter 1	acc_flag: ACC flag selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

acc_flag

The acc_flag is used for flag selection, including:

ACC_RSLOST_FLAG: Reference signal lost interrupt

ACC_CALRDY_FLAG: Calibration complete interrupt

Example:

```
if(acc_flag_get(ACC_CALRDY_FLAG) != RESET)
{
    at32_led_toggle(LED2);
    /* clear acc calibration ready flag */
    acc_flag_clear(ACC_CALRDY_FLAG);
}
```

5.2 Analog-to-digital converter (ADC)

ADC register structure `adc_type` is defined in the “at32f402/405_adc.h”.

```
/**
 * @brief type define adc register all
 */
typedef struct
{
    .....
} adc_type;
```

The table below gives a list of the ADC registers.

Table 21. Summary of ADC registers

Register	Description
sts	ADC status register
ctrl1	ADC control register 1
ctrl2	ADC control register 2
spt1	ADC sample time register 1
spt2	ADC sample time register 2
pcdto1	ADC preempted channel data offset register 1
pcdto2	ADC preempted channel data offset register 2
pcdto3	ADC preempted channel data offset register 3
pcdto4	ADC preempted channel data offset register 4
vmhb	ADC voltage monitor high boundary register
vmhb	ADC voltage monitor low boundary register
osq1	ADC ordinary sequence register 1
osq2	ADC ordinary sequence register 2
osq3	ADC ordinary sequence register 3
psq	ADC preempted sequence register
pdt1	ADC preempted data register 1
pdt2	ADC preempted data register 2
pdt3	ADC preempted data register 3
pdt4	ADC preempted data register 4
odt	ADC ordinary data register
ovsp	ADC oversampling register

The table below gives a list of ADC library functions.

Table 22. Summary of ADC library functions

Function name	Description
adc_reset	Reset all ADC registers to their reset values
adc_enable	Enable A/D converter
adc_base_default_para_init	Define an initial value for adc_base_struct
adc_base_config	Configure ADC registers with the initialized parameters of the adc_base_struct
adc_clock_div_set	ADC clock division setting
adc_dma_mode_enable	Enable DMA transfer for ordinary group
adc_interrupt_enable	Enable the selected ADC event interrupt
adc_calibration_init	Initialization calibration
adc_calibration_init_status_get	Get initialization calibration status
adc_calibration_start	Start calibration
adc_calibration_status_get	Get calibration status
adc_voltage_monitor_enable	Enable voltage monitoring for ordinary/preempted channels and a single channel
adc_voltage_monitor_threshold_value_set	Set the threshold of voltage monitoring
adc_voltage_monitor_single_channel_select	Select a single channel for voltage monitoring
adc_ordinary_channel_set	Configure ordinary channels, including channel selection, conversion sequence number and sampling time
adc_preempt_channel_length_set	Configure the length of preempted group conversion sequence
adc_preempt_channel_set	Configure preempted channels, including channel selection, conversion sequence number and sampling time
adc_ordinary_conversion_trigger_set	Enable trigger mode and trigger event selection for ordinary conversion
adc_preempt_conversion_trigger_set	Enable trigger mode and trigger event selection for preempted conversion
adc_preempt_offset_value_set	Set data offset for preempted conversion
adc_ordinary_part_count_set	Set the number of ordinary channels for each triggered conversion in partition mode
adc_ordinary_part_mode_enable	Enable partition mode for ordinary channels
adc_preempt_part_mode_enable	Enable partition mode for preempted channels
adc_preempt_auto_mode_enable	Enable auto conversion of preempted group at the end of ordinary conversion
adc_temperSENSOR_vintrv_enable	Enable internal temperature sensor V _{INTRV}
adc_ordinary_software_trigger_enable	Software trigger ordinary group conversion
adc_ordinary_software_trigger_status_get	Get the status of ordinary group conversion triggered by software
adc_preempt_software_trigger_enable	Software trigger preempted group conversion
adc_preempt_software_trigger_status_get	Get the status of preempted group conversion triggered by software
adc_ordinary_conversion_data_get	Get data of ordinary group conversion in non-master-slave mode
adc_preempt_conversion_data_get	Get the converted data of preempted group
adc_flag_get	Get the status of flag bits
adc_flag_clear	Clear flag bits

Function name	Description
adc_ordinary_oversample_enable	Enable oversampling of ordinary group
adc_preempt_oversample_enable	Enable oversampling of preempted group
adc_oversample_ratio_shift_set	Set oversampling ratio and its shift length
adc_ordinary_oversample_trig_enable	Enable oversampling trigger mode of ordinary group
adc_ordinary_oversample_restart_set	Enable oversampling restart mode of ordinary group

5.2.1 adc_reset function

The table below describes the function adc_reset.

Table 23. adc_reset function

Name	Description
Function name	adc_reset
Function prototype	void adc_reset(adc_type *adc_x)
Function description	Reset all ADC registers to their reset values
Input parameter	adc_x: ADC peripheral selected This parameter can be ADC1
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset()

Example:

```
/* deinitialize adc1 */
adc_reset(ADC1);
```

5.2.2 adc_enable function

The table below describes the function adc_enable.

Table 24. adc_enable function

Name	Description
Function name	adc_enable
Function prototype	void adc_enable(adc_type *adc_x, confirm_state new_state)
Function description	Enable/disable A/D converter
Input parameter 1	adc_x: indicates the selected ADC This parameter can be ADC1.
Input parameter 2	new_state: indicates the pre-configured status of A/D converter This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable adc1 */
adc_enable(ADC1, TRUE);
```


Note: Call the `adc_enable` while ADC is enabled will trigger regular group conversions.

5.2.3 `adc_base_default_para_init` function

The table below describes the function `adc_base_default_para_init`.

Table 25. `adc_base_default_para_init` function

Name	Description
Function name	<code>adc_base_default_para_init</code>
Function prototype	<code>void adc_base_default_para_init(adc_base_config_type *adc_base_struct)</code>
Function description	Set the initial value for the <code>adc_base_struct</code> .
Input parameter	<code>adc_base_struct</code> : <code>adc_base_config_type</code> pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The default values of members in the `adc_base_struct`:

```
sequence_mode:      FALSE
repeat_mode:        FALSE
data_align:          ADC_RIGHT_ALIGNMENT
ordinary_channel_length: 1
```

Example:

```
/* initialize a adc_base_config_type structure */
adc_base_config_type adc_base_struct;
adc_base_default_para_init(&adc_base_struct);
```

5.2.4 `adc_base_config` function

The table below describes the function `adc_base_config`.

Table 26. `adc_base_config` function

Name	Description
Function name	<code>adc_base_config</code>
Function prototype	<code>void adc_base_config(adc_type *adc_x, adc_base_config_type *adc_base_struct);</code>
Function description	Initialize ADC registers with the specified parameters in the <code>adc_base_struct</code> .
Input parameter 1	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Input parameter 2	<code>adc_base_struct</code> : <code>adc_base_config_type</code> structure pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

`adc_base_config_type` structure

The `adc_base_config_type` is defined in the `at32f402_405_adc.h`:

```
typedef struct
{
```

```
confirm_state      sequence_mode;
confirm_state      repeat_mode;
adc_data_align_type data_align;
uint8_t            ordinary_channel_length;
} adc_base_config_type; the member parameters are described as follows
```

sequence_mode

Set ADC sequence mode.

FALSE: Select a single channel for conversion

TRUE: Select multiple channels for conversion

repeat_mode

Set ADC repeat mode.

FALSE: when SQEN=0, trigger a single channel conversion each time; when SQEN=1, trigger the conversion of a group of channels each time

TRUE: when SQEN =0, repeatedly convert a single channel at each trigger; when SQEN=1, repeatedly convert a group of channels at each trigger until the ADCEN bit is cleared.

data_align

Set data alignment of ADC

ADC_RIGHT_ALIGNMENT: right-aligned

ADC_LEFT_ALIGNMENT: left-aligned

ordinary_channel_length

Set the length of ordinary group ADC conversion

Example:

```
adc_base_config_type adc_base_struct;
adc_base_struct.sequence_mode = TRUE;
adc_base_struct.repeat_mode = FALSE;
adc_base_struct.data_align = ADC_RIGHT_ALIGNMENT;
adc_base_struct.ordinary_channel_length = 3;
adc_base_config(ADC1, &adc_base_struct);
```

5.2.5 adc_clock_div_set

The table below describes the function `adc_clock_div_set`.

Table 27. `adc_clock_div_set` function

Name	Description
Function name	<code>adc_clock_div_set</code>
Function prototype	<code>void adc_clock_div_set(adc_div_type div_value)</code>
Function description	Set ADC clock division
Input parameter	<code>div_value</code> : ADC clock division value Refer to the “ <code>div_value</code> ” for details
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

`div_value`

`div_value` is used to select ADC clock division value from one of the following:

`ADC_DIV_2`: HCLK/2
`ADC_DIV_3`: HCLK/3
`ADC_DIV_4`: HCLK/4
`ADC_DIV_5`: HCLK/5
`ADC_DIV_6`: HCLK/6
`ADC_DIV_7`: HCLK/7
`ADC_DIV_8`: HCLK/8
`ADC_DIV_9`: HCLK 9
`ADC_DIV_10`: HCLK/10
`ADC_DIV_11`: HCLK/11
`ADC_DIV_12`: HCLK/12
`ADC_DIV_13`: HCLK/13
`ADC_DIV_14`: HCLK/14
`ADC_DIV_15`: HCLK/15
`ADC_DIV_16`: HCLK/16
`ADC_DIV_17`: HCLK/17

Example:

```

/* config ADC clock */
adc_clock_div_set(ADC_DIV_8);
    
```

5.2.6 adc_dma_mode_enable function

The table below describes the function `adc_dma_mode_enable`.

Table 28. `adc_dma_mode_enable` function

Name	Description
Function name	<code>adc_dma_mode_enable</code>
Function prototype	<code>void adc_dma_mode_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Enable DMA transfer for ordinary group conversion
Input parameter 1	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : pre-configured status of ordinary group in DMA transfer mode This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable dma transfer adc ordinary conversion data */
adc_dma_mode_enable(ADC1, TRUE);
```

5.2.7 adc_interrupt_enable function

The table below describes the function `adc_interrupt_enable`.

Table 29. `adc_interrupt_enable` function

Name	Description
Function name	<code>adc_interrupt_enable</code>
Function prototype	<code>void adc_interrupt_enable(adc_type *adc_x, uint32_t adc_int, confirm_state new_state)</code>
Function description	Enable the selected ADC event interrupt
Input parameter 1	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Input parameter 2	<code>adc_int</code> : ADC event interrupt selection This parameter is used to select any event interrupt supported by ADC.
Input parameter3	<code>new_state</code> : indicates the pre-configured status of ADC event interrupts This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_int

The `adc_int` is used to select and set event interrupts, with the following parameters:

ADC_CCE_INT: Interrupt enabled at the end of a channel conversion

ADC_VMOR_INT: Interrupt enabled when voltage monitor is outside a threshold

ADC_PCCE_INT: Interrupt enabled at the end of preempted group conversion

Example:

```
/* enable voltage monitoring out of range interrupt */  
adc_interrupt_enable(ADC1, ADC_VMOR_INT, TRUE);
```

5.2.8 adc_calibration_init function

The table below describes the function `adc_calibration_init`.

Table 30. adc_calibration_init function

Name	Description
Function name	<code>adc_calibration_init</code>
Function prototype	<code>void adc_calibration_init(adc_type *adc_x)</code>
Function description	Initialization calibration
Input parameter	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* initialize A/D calibration */  
adc_calibration_init(ADC1);
```

5.2.9 adc_calibration_init_status_get function

The table below describes the function `adc_calibration_init_status_get`.

Table 31. adc_calibration_init_status_get function

Name	Description
Function name	<code>adc_calibration_init_status_get</code>
Function prototype	<code>flag_status adc_calibration_init_status_get(adc_type *adc_x)</code>
Function description	Get the status of initialization calibration
Input parameter	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Output parameter	NA
Return value	<code>flag_status</code> : indicates the status of calibration initialization Return SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* wait initialize A/D calibration success */  
while(adc_calibration_init_status_get(ADC1));
```

5.2.10 adc_calibration_start function

The table below describes the function adc_calibration_start.

Table 32. adc_calibration_start function

Name	Description
Function name	adc_calibration_start
Function prototype	void adc_calibration_start(adc_type *adc_x)
Function description	Start calibration
Input parameter	adc_x: indicates the selected ADC peripheral This parameter can be ADC1.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* start calibration process */  
adc_calibration_start(ADC1);
```

5.2.11 adc_calibration_status_get function

The table below describes the function adc_calibration_status_get.

Table 33. adc_calibration_status_get function

Name	Description
Function name	adc_calibration_status_get
Function prototype	flag_status adc_calibration_status_get(adc_type *adc_x)
Function description	Get the status of calibration
Input parameter	adc_x: indicates the selected ADC peripheral This parameter can be ADC1.
Output parameter	NA
Return value	flag_status: indicates the status of calibration Return SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* wait calibration success */  
while(adc_calibration_status_get(ADC1));
```

5.2.12 adc_voltage_monitor_enable function

The table below describes the function `adc_voltage_monitor_enable`.

Table 34. `adc_voltage_monitor_enable` function

Name	Description
Function name	<code>adc_voltage_monitor_enable</code>
Function prototype	<code>void adc_voltage_monitor_enable(adc_type *adc_x, adc_voltage_monitoring_type adc_voltage_monitoring)</code>
Function description	Enable voltage monitor for ordinary/preempted group and a single channel
Input parameter 1	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Input parameter 2	<code>adc_voltage_monitoring</code> : select ordinary group, preempted group or a single channel for voltage monitoring This parameter can be any enumerated value in the <code>adc_voltage_monitoring_type</code> .
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

`adc_voltage_monitoring`

The `adc_voltage_monitoring` is used to select one or more channels of ordinary group/preempted group for voltage monitoring, including:

`ADC_VMONITOR_SINGLE_ORDINARY`:

Select a single ordinary channel for voltage monitoring

`ADC_VMONITOR_SINGLE_PREEMPT`:

Select a single preempted channel for voltage monitoring

`ADC_VMONITOR_SINGLE_ORDINARY_PREEMPT`:

Select a single channel from ordinary or preempted group for voltage monitoring

`ADC_VMONITOR_ALL_ORDINARY`:

Select all ordinary channels for voltage monitoring

`ADC_VMONITOR_ALL_PREEMPT`:

Select all preempted channels for voltage monitoring

`ADC_VMONITOR_ALL_ORDINARY_PREEMPT`:

Select all ordinary and preempted channels for voltage monitoring

`ADC_VMONITOR_NONE`:

No channels need voltage monitoring

Example:

```
/* enable the voltage monitoring on all ordinary and preempt channels */
adc_voltage_monitor_enable(ADC1, ADC_VMONITOR_ALL_ORDINARY_PREEMPT);
```

5.2.13 adc_voltage_monitor_threshold_value_set function

The table below describes the function `adc_voltage_monitor_threshold_value_set`.

Table 35. adc_voltage_monitor_threshold_value_set function

Name	Description
Function name	<code>adc_voltage_monitor_threshold_value_set</code>
Function prototype	<code>void adc_voltage_monitor_threshold_value_set(adc_type *adc_x, uint16_t adc_high_threshold, uint16_t adc_low_threshold)</code>
Function description	Configure the threshold of voltage monitoring
Input parameter 1	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Input parameter 2	<code>adc_high_threshold</code> : indicates the upper limit for voltage monitoring This parameter can be any value between 0x000 and 0xFFFF.
Input parameter3	<code>adc_low_threshold</code> : indicates the lower limit for voltage monitoring This parameter can be any value lower than that of <code>adc_high_threshold</code> in the range of 0x000~0xFFFF.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set voltage monitoring's high and low thresholds value */
adc_voltage_monitor_threshold_value_set(ADC1, 0xBBB, 0xAAA);
```

5.2.14 adc_voltage_monitor_single_channel_select function

The table below describes the function `adc_voltage_monitor_single_channel_select`.

Table 36. adc_voltage_monitor_single_channel_select function

Name	Description
Function name	<code>adc_voltage_monitor_single_channel_select</code>
Function prototype	<code>void adc_voltage_monitor_single_channel_select(adc_type *adc_x, adc_channel_select_type adc_channel)</code>
Function description	Select a single channel for voltage monitoring
Input parameter 1	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Input parameter 2	<code>adc_channel</code> : select a single channel for voltage monitoring Refer to adc_channel for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_channel

The `adc_channel` is used to select a single channel for voltage monitoring, including:

ADC_CHANNEL_0: ADC channel 0

ADC_CHANNEL_1: ADC channel 1

.....

ADC_CHANNEL_16: ADC channel 16

ADC_CHANNEL_17: ADC channel 17

Example:

```
/* select the voltage monitoring's channel */
adc_voltage_monitor_single_channel_select(ADC1, ADC_CHANNEL_5);
```

5.2.15 adc_ordinary_channel_set function

The table below describes the function `adc_ordinary_channel_set`.

Table 37. adc_ordinary_channel_set function

Name	Description
Function name	<code>adc_ordinary_channel_set</code>
Function prototype	<code>void adc_ordinary_channel_set(adc_type *adc_x, adc_channel_select_type adc_channel, uint8_t adc_sequence, adc_sampletime_select_type adc_sampletime)</code>
Function description	Configure ordinary channels, including parameters such as channel selection, conversion sequence number and sampling time
Input parameter 1	<code>adc_x</code> : indicates the selected ADC peripheral This parameter can be ADC1.
Input parameter 2	<code>adc_channel</code> : indicates the selected channel Refer to adc_channel for details.
Input parameter3	<code>adc_sequence</code> : defines the sequence of channel conversion This parameter can be any value from 1 to 16.
Input parameter4	<code>adc_sampletime</code> : defines the sampling time for channel Refer to adc_sampletime for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_sampletime

The `adc_sampletime` is used to configure the sampling time of channels, including:

ADC_SAMPLETIME_1_5: sampling time is 1.5 ADCCLK cycles

ADC_SAMPLETIME_7_5: sampling time is 7.5 ADCCLK cycles

ADC_SAMPLETIME_13_5: sampling time is 13.5 ADCCLK cycles

ADC_SAMPLETIME_28_5: sampling time is 28.5 ADCCLK cycles

ADC_SAMPLETIME_41_5: sampling time is 41.5 ADCCLK cycles

ADC_SAMPLETIME_55_5: sampling time is 55.5 ADCCLK cycles

ADC_SAMPLETIME_71_5: sampling time is 71.5 ADCCLK cycles

ADC_SAMPLETIME_239_5: sampling time is 239.5 ADCCLK cycles

Example:

```
/* set ordinary channel's corresponding rank in the sequencer and sample time */
adc_ordinary_channel_set(ADC1, ADC_CHANNEL_4, 1, ADC_SAMPLETIME_239_5);
adc_ordinary_channel_set(ADC1, ADC_CHANNEL_5, 2, ADC_SAMPLETIME_239_5);
```

5.2.16 adc_preempt_channel_length_set function

The table below describes the function `adc_preempt_channel_length_set`.

Table 38. adc_preempt_channel_length_set function

Name	Description
Function name	<code>adc_preempt_channel_length_set</code>
Function prototype	<code>void adc_preempt_channel_length_set(adc_type *adc_x, uint8_t adc_channel_lenght)</code>
Function description	Set the length of preempted channel conversion
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_channel_lenght</code> : set the length of preempted channel conversion This parameter can be any value from 0x1 to 0x4.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set preempt channel length */
adc_preempt_channel_length_set(ADC1, 3);
```

5.2.17 adc_preempt_channel_set function

The table below describes the function `adc_preempt_channel_set`.

Table 39. adc_preempt_channel_set function

Name	Description
Function name	<code>adc_preempt_channel_set</code>
Function prototype	<code>void adc_preempt_channel_set(adc_type *adc_x, adc_channel_select_type adc_channel, uint8_t adc_sequence, adc_sampletime_select_type adc_sampletime)</code>
Function description	Configure preempted group, including parameters such as channel selection, conversion sequence number and sampling time
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_channel</code> : indicates the selected channel Refer to adc_channel for details.
Input parameter3	<code>adc_sequence</code> : set the sequence number for channel conversion This parameter can be any value from 1 to 4.
Input parameter4	<code>adc_sampletime</code> : set the sampling time for channels Refer to adc_sampletime for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set ordinary channel's corresponding rank in the sequencer and sample time */
adc_preempt_channel_set(ADC1, ADC_CHANNEL_7, 1, ADC_SAMPLETIME_239_5);
adc_preempt_channel_set(ADC1, ADC_CHANNEL_8, 2, ADC_SAMPLETIME_239_5);
```

5.2.18 adc_ordinary_conversion_trigger_set function

The table below describes the function `adc_ordinary_conversion_trigger_set`.

Table 40. adc_ordinary_conversion_trigger_set function

Name	Description
Function name	<code>adc_ordinary_conversion_trigger_set</code>
Function prototype	<code>void adc_ordinary_conversion_trigger_set(adc_type *adc_x, adc_ordinary_trig_select_type adc_ordinary_trig, confirm_state new_state)</code>
Function description	Enable trigger mode and select trigger events for ordinary group conversion
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_ordinary_trig</code> : indicates the selected trigger event for ordinary group This parameter can be any enumerated value in the <code>adc_ordinary_trig_select_type</code> .
Input parameter3	<code>new_state</code> : the pre-defined state of trigger mode This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_ordinary_trig

The `adc_ordinary_trig` is used to select a trigger event for ordinary group conversion, including:

<code>ADC12_ORDINARY_TRIG_TMR1TRGOUT</code> :	TMR1 TRGOUT event
<code>ADC12_ORDINARY_TRIG_TMR1CH4</code> :	TMR1 CH4 event
<code>ADC12_ORDINARY_TRIG_TMR2TRGOUT</code> :	TMR2 TRGOUT event
<code>ADC12_ORDINARY_TRIG_TMR3TRGOUT</code> :	TMR3 TRGOUT event
<code>ADC12_ORDINARY_TRIG_TMR9TRGOUT</code> :	TMR9 TRGOUT event
<code>ADC12_ORDINARY_TRIG_TMR1CH1</code> :	TMR1 CH1 event
<code>ADC12_ORDINARY_TRIG_EXINT11</code> :	EXINT 11 event
<code>ADC12_ORDINARY_TRIG_SOFTWARE</code> :	Software trigger event

Example:

```
/* set ordinary external trigger event */
adc_ordinary_conversion_trigger_set(ADC1, ADC12_ORDINARY_TRIG_TMR1CH4, TRUE);
```

5.2.19 adc_preempt_conversion_trigger_set function

The table below describes the function `adc_preempt_conversion_trigger_set`.

Table 41. `adc_preempt_conversion_trigger_set` function

Name	Description
Function name	<code>adc_preempt_conversion_trigger_set</code>
Function prototype	<code>void adc_preempt_conversion_trigger_set(adc_type *adc_x, adc_preempt_trig_select_type adc_preempt_trig, confirm_state new_state)</code>
Function description	Enable trigger mode and trigger events for preempted group conversion
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_preempt_trig</code> : indicates the selected trigger event for preempted group This parameter can be any enumerated value in the <code>adc_preempt_trig_select_type</code> .
Input parameter3	<code>new_state</code> : the pre-defined state of trigger mode This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

`adc_preempt_trig`

The `adc_preempt_trig` is used to select a trigger event for preempted group conversion, including:

<code>ADC12_PREEMPT_TRIG_TMR1CH2:</code>	TMR1 CH2 event
<code>ADC12_PREEMPT_TRIG_TMR1CH3:</code>	TMR1 CH3 event
<code>ADC12_PREEMPT_TRIG_TMR2CH4:</code>	TMR2 CH4 event
<code>ADC12_PREEMPT_TRIG_TMR3CH4:</code>	TMR3 CH4 event
<code>ADC12_PREEMPT_TRIG_TMR9CH1:</code>	TMR9 CH1 event
<code>ADC12_PREEMPT_TRIG_TMR6TRGOUT:</code>	TMR6 TRGOUT event
<code>ADC12_PREEMPT_TRIG_EXINT15:</code>	EXINT 15 event
<code>ADC12_PREEMPT_TRIG_SOFTWARE:</code>	Software trigger event

Example:

```
/* set preempt external trigger event */
adc_preempt_conversion_trigger_set(ADC1, ADC12_PREEMPT_TRIG_SOFTWARE, TRUE);
```

5.2.20 adc_preempt_offset_value_set function

The table below describes the function `adc_preempt_offset_value_set`.

Table 42. adc_preempt_offset_value_set function

Name	Description
Function name	<code>adc_preempt_offset_value_set</code>
Function prototype	<code>void adc_preempt_offset_value_set(adc_type *adc_x, adc_preempt_channel_type adc_preempt_channel, uint16_t adc_offset_value)</code>
Function description	Set the offset value of preempted group conversion
Input parameter 1	<code>adc_x</code> : selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_preempt_channel</code> : indicates the selected channel Refer to adc_preempt_channel for details.
Input parameter3	<code>adc_offset_value</code> : set the offset value for the selected channel This parameter can be any value from 0x000 to 0xFFFF.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_preempt_channel

The `adc_preempt_channel` is used to set an offset value for the selected channel, including:

<code>ADC_PREEMPT_CHANNEL_1</code> :	Preempted channel 1
<code>ADC_PREEMPT_CHANNEL_2</code> :	Preempted channel 2
<code>ADC_PREEMPT_CHANNEL_3</code> :	Preempted channel 3
<code>ADC_PREEMPT_CHANNEL_4</code> :	Preempted channel 4

Example:

```
/* set preempt channel's conversion value offset */
adc_preempt_offset_value_set(ADC1, ADC_PREEMPT_CHANNEL_1, 0x111);
adc_preempt_offset_value_set(ADC1, ADC_PREEMPT_CHANNEL_2, 0x222);
adc_preempt_offset_value_set(ADC1, ADC_PREEMPT_CHANNEL_3, 0x333);
```

5.2.21 adc_ordinary_part_count_set function

The table below describes the function `adc_ordinary_part_count_set`.

Table 43. adc_ordinary_part_count_set function

Name	Description
Function name	<code>adc_ordinary_part_count_set</code>
Function prototype	<code>void adc_ordinary_part_count_set(adc_type *adc_x, uint8_t adc_channel_count)</code>
Function description	Set the number of ordinary channels at each triggered conversion in partition mode
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_channel_count</code> : indicates the number of ordinary group in partition mode This parameter can be any value from 0x1 to 0x8.

Name	Description
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set partitioned mode channel count */
adc_ordinary_part_count_set(ADC1, 2);
```

Note: In partition mode, only the number of ordinary group is settable, and that of preempted group is fixed at 1.

5.2.22 adc_ordinary_part_mode_enable function

The table below describes the function `adc_ordinary_part_mode_enable`.

Table 44. `adc_ordinary_part_mode_enable` function

Name	Description
Function name	<code>adc_ordinary_part_mode_enable</code>
Function prototype	<code>void adc_ordinary_part_mode_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Enable partition mode for ordinary channels
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status for partition mode of ordinary channels This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the partitioned mode on ordinary channel */
adc_ordinary_part_mode_enable(ADC1, TRUE);
```

5.2.23 adc_preempt_part_mode_enable function

The table below describes the function `adc_preempt_part_mode_enable`.

Table 45. `adc_preempt_part_mode_enable` function

Name	Description
Function name	<code>adc_preempt_part_mode_enable</code>
Function prototype	<code>void adc_preempt_part_mode_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Enable partition mode for preempted channels
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status for partition mode of preempted channels

Name	Description
	This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the partitioned mode on preempt channel */
adc_preempt_part_mode_enable(ADC1, TRUE);
```

5.2.24 adc_preempt_auto_mode_enable function

The table below describes the function `adc_preempt_auto_mode_enable`.

Table 46. `adc_preempt_auto_mode_enable` function

Name	Description
Function name	<code>adc_preempt_auto_mode_enable</code>
Function prototype	<code>void adc_preempt_auto_mode_enable(adc_type *adc_x, confirm_state, new_state)</code>
Function description	Enable auto preempted group conversion at the end of ordinary group conversion
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status for auto preempted group conversion This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable automatic preempt group conversion */
adc_preempt_auto_mode_enable(ADC1, TRUE);
```

5.2.25 adc_tempersensor_vintrv_enable

The table below describes the function `adc_tempersensor_vintrv_enable`.

Table 47. `adc_tempersensor_vintrv_enable` function

Name	Description
Function name	<code>adc_tempersensor_vintrv_enable</code>
Function prototype	<code>void adc_tempersensor_vintrv_enable(confirm_state new_state)</code>
Function description	Enable internal temperature sensor and V_{INTRV}
Input parameter	<code>new_state</code> : the pre-defined state of internal temperature sensor and V_{INTRV} This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the temperature sensor and vintrv channel */
adc_tempersensor_vintrv_enable(TRUE);
```

5.2.26 adc_ordinary_software_trigger_enable function

The table below describes the function `adc_ordinary_software_trigger_enable`.

Table 48. `adc_ordinary_software_trigger_enable` function

Name	Description
Function name	<code>adc_ordinary_software_trigger_enable</code>
Function prototype	<code>void adc_ordinary_software_trigger_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Trigger ordinary group conversion by software
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status for software-triggered ordinary group conversion This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable ordinary software start conversion */
adc_ordinary_software_trigger_enable(ADC1, TRUE);
```


5.2.27 adc_ordinary_software_trigger_status_get function

The table below describes the function `adc_ordinary_software_trigger_status_get`

Table 49. adc_ordinary_software_trigger_status_get function

Name	Description
Function name	<code>adc_ordinary_software_trigger_status_get</code>
Function prototype	<code>flag_status adc_ordinary_software_trigger_status_get(adc_type *adc_x)</code>
Function description	Get the status of software-triggered ordinary group conversion
Input parameter	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Output parameter	NA
Return value	<code>flag_status</code> : indicates the status of software-triggered ordinary group conversion This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* wait ordinary software start conversion */  
while(adc_ordinary_software_trigger_status_get(ADC1));
```

5.2.28 adc_preempt_software_trigger_enable function

The table below describes the function `adc_preempt_software_trigger_enable`

Table 50. adc_preempt_software_trigger_enable function

Name	Description
Function name	<code>adc_preempt_software_trigger_enable</code>
Function prototype	<code>void adc_preempt_software_trigger_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Preempted group conversion triggered by software
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status of software-triggered preempted group conversion This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable preempt software start conversion */  
adc_preempt_software_trigger_enable(ADC1, TRUE);
```

5.2.29 adc_preempt_software_trigger_status_get function

The table below describes the function `adc_preempt_software_trigger_status_get`.

Table 51. adc_preempt_software_trigger_status_get function

Name	Description
Function name	<code>adc_preempt_software_trigger_status_get</code>
Function prototype	<code>flag_status adc_preempt_software_trigger_status_get(adc_type *adc_x)</code>
Function description	Get the status of software-triggered preempted group conversion
Input parameter	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Output parameter	NA
Return value	<code>flag_status</code> : indicates the status of software-triggered preempted group conversion This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* wait preempt software start conversion */
while(adc_preempt_software_trigger_status_get(ADC1));
```

5.2.30 adc_ordinary_conversion_data_get function

The table below describes the function `adc_ordinary_conversion_data_get`.

Table 52. adc_ordinary_conversion_data_get function

Name	Description
Function name	<code>adc_ordinary_conversion_data_get</code>
Function prototype	<code>uint16_t adc_ordinary_conversion_data_get(adc_type *adc_x)</code>
Function description	Get the converted data of ordinary group in non-master/slave mode
Input parameter	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Output parameter	NA
Return value	16-bit converted data by ordinary group
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t adc1_ordinary_index = 0;
adc1_ordinary_index = adc_ordinary_conversion_data_get(ADC1);
```

Note: This function is available only for single-channel ADC configuration.

5.2.31 adc_preempt_conversion_data_get function

The table below describes the function `adc_preempt_conversion_data_get`.

Table 53. adc_preempt_conversion_data_get function

Name	Description
Function name	<code>adc_preempt_conversion_data_get</code>
Function prototype	<code>uint16_t adc_preempt_conversion_data_get(adc_type *adc_x, adc_preempt_channel_type adc_preempt_channel)</code>
Function description	Get the converted data of preempted group
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_preempt_channel</code> : the selected preempted channel Refer to adc_preempt_channel for details.
Output parameter	NA
Return value	16-bit converted data by preempted group
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t adc1_preempt_valuetab[3] = {0};
adc1_preempt_valuetab[0] = adc_preempt_conversion_data_get(ADC1, ADC_PREEMPT_CHANNEL_1);
adc1_preempt_valuetab[1] = adc_preempt_conversion_data_get(ADC1, ADC_PREEMPT_CHANNEL_2);
adc1_preempt_valuetab[2] = adc_preempt_conversion_data_get(ADC1, ADC_PREEMPT_CHANNEL_3);
```

5.2.32 adc_flag_get function

The table below describes the function `adc_flag_get`.

Table 54. adc_flag_get function

Name	Description
Function name	<code>adc_flag_get</code>
Function prototype	<code>flag_status adc_flag_get(adc_type *adc_x, uint8_t adc_flag)</code>
Function description	Get the status of the flag bit
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_flag</code> : indicates the selected flag Refer to adc_flag for details.
Output parameter	NA
Return value	<code>flag_status</code> : the status for the selected flag bit. This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

adc_flag

The `adc_flag` is used to select a flag to get its status, including:

ADC_VMOR_FLAG: Voltage monitor outside threshold
ADC_CCE_FLAG: End of a channel conversion

ADC_PCCE_FLAG: End of preempted channel conversion
 ADC_PCCS_FLAG: Start of preempted channel conversion
 ADC_OCCS_FLAG: Start of ordinary channel conversion

Example:

```
/* check if wakeup preempted channelsconversion end flag is set */
if(adc_flag_get(ADC1, ADC_PCCE_FLAG) != RESET)
```

5.2.33 adc_interrupt_flag_get function

The table below describes the function `adc_interrupt_flag_get`.

Table 55. adc_interrupt_flag_get function

Name	Description
Function name	<code>adc_interrupt_flag_get</code>
Function prototype	<code>flag_status adc_interrupt_flag_get(adc_type *adc_x, uint8_t adc_flag)</code>
Function description	Get the status of the flag bit
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_flag</code> : indicates the selected flag Refer to adc_flag
Output parameter	NA
Return value	<code>flag_status</code> : the status for the selected flag bit This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

adc_flag

The `adc_flag` is used to select a flag to get its status, including:

ADC_VMOR_FLAG: Voltage monitor outside threshold
 ADC_CCE_FLAG: End of a channel conversion
 ADC_PCCE_FLAG: End of preempted channel conversion

Example

```
/* check if wakeup preempted channelsconversion end flag is set */
if(adc_interrupt_flag_get(ADC1, ADC_PCCE_FLAG) != RESET)
```

5.2.34 adc_flag_clear function

The table below describes the function `adc_flag_clear`.

Table 56. adc_flag_clear function

Name	Description
Function name	<code>adc_flag_clear</code>
Function prototype	<code>void adc_flag_clear(adc_type *adc_x, uint32_t adc_flag)</code>
Function description	Clear the flag bits that have been set.
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_flag</code> : select a flag to be clear Refer to adc_flag
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* preempted channelsconversion end flag clear */  
adc_flag_clear(ADC1, ADC_PCCE_FLAG);
```

5.2.35 adc_ordinary_oversample_enable function

The table below describes the function `adc_ordinary_oversample_enable`.

Table 57. adc_ordinary_oversample_enable function

Name	Description
Function name	<code>adc_ordinary_oversample_enable</code>
Function prototype	<code>void adc_ordinary_oversample_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Enable oversampling for ordinary group
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status of oversampling of ordinary group This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable ordinary oversampling */  
adc_ordinary_oversample_enable(ADC1, TRUE);
```

5.2.36 adc_preempt_oversample_enable function

The table below describes the function `adc_preempt_oversample_enable`.

Table 58. adc_preempt_oversample_enable function

Name	Description
Function name	<code>adc_preempt_oversample_enable</code>
Function prototype	<code>void adc_preempt_oversample_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Enable oversampling for preempted group
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status of oversampling of preempted group This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable preempt oversampling */
adc_preempt_oversample_enable(ADC1, TRUE);
```

5.2.37 adc_oversample_ratio_shift_set function

The table below describes the function `adc_oversample_ratio_shift_set`.

Table 59. adc_oversample_ratio_shift_set function

Name	Description
Function name	<code>adc_oversample_ratio_shift_set</code>
Function prototype	<code>void adc_oversample_ratio_shift_set(adc_type *adc_x, adc_oversample_ratio_type adc_oversample_ratio, adc_oversample_shift_type adc_oversample_shift)</code>
Function description	Set oversampling ratio and shift mode
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_oversample_ratio</code> : indicates the oversampling rate This parameter can be any enumerated value in the <code>adc_oversample_ratio_type</code> .
Input parameter3	<code>adc_oversample_shift</code> : indicates the oversampling shift mode This parameter can be any enumerated value in the <code>adc_oversample_shift_type</code> .
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_oversample_ratio

The `adc_oversample_ratio` is used for ADC oversampling ratio selection, including:

ADC_OVERSAMPLE_RATIO_2: 2 x oversampling

ADC_OVERSAMPLE_RATIO_4:	4 x oversampling
ADC_OVERSAMPLE_RATIO_8:	5 x oversampling
ADC_OVERSAMPLE_RATIO_16:	16 x oversampling
ADC_OVERSAMPLE_RATIO_32:	32 x oversampling
ADC_OVERSAMPLE_RATIO_64:	64 x oversampling
ADC_OVERSAMPLE_RATIO_128:	128 x oversampling
ADC_OVERSAMPLE_RATIO_256:	256 x oversampling

adc_oversample_shift

The `adc_oversample_shift` is used for ADC oversampling bit width selection, including:

ADC_OVERSAMPLE_SHIFT_0:	No shift
ADC_OVERSAMPLE_SHIFT_1:	1-bit shift
ADC_OVERSAMPLE_SHIFT_2:	2-bit shift
ADC_OVERSAMPLE_SHIFT_3:	3-bit shift
ADC_OVERSAMPLE_SHIFT_4:	4-bit shift
ADC_OVERSAMPLE_SHIFT_5:	5-bit shift
ADC_OVERSAMPLE_SHIFT_6:	6-bit shift
ADC_OVERSAMPLE_SHIFT_7:	7-bit shift
ADC_OVERSAMPLE_SHIFT_8:	8-bit shift

Example:

```
/* set oversampling ratio and shift */  
adc_oversample_ratio_shift_set(ADC1, ADC_OVERSAMPLE_RATIO_8, ADC_OVERSAMPLE_SHIFT_3);
```

5.2.38 adc_ordinary_oversample_trig_enable function

The table below describes the function `adc_ordinary_oversample_trig_enable`.

Table 60. adc_ordinary_oversample_trig_enable function

Name	Description
Function name	<code>adc_ordinary_oversample_trig_enable</code>
Function prototype	<code>void adc_ordinary_oversample_trig_enable(adc_type *adc_x, confirm_state new_state)</code>
Function description	Enable oversampling trigger mode for ordinary group
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>new_state</code> : indicates the pre-configured status of oversampling trigger mode of ordinary group This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* disable ordinary oversampling trigger mode */  
adc_ordinary_oversample_trig_enable(ADC1, FALSE);
```

5.2.39 adc_ordinary_oversample_restart_set function

The table below describes the function `adc_ordinary_oversample_restart_set`.

Table 61. adc_ordinary_oversample_restart_set function

Name	Description
Function name	<code>adc_ordinary_oversample_restart_set</code>
Function prototype	<code>void adc_ordinary_oversample_restart_set(adc_type *adc_x, adc_ordinary_oversample_restart_type adc_ordinary_oversample_restart)</code>
Function description	Select oversampling restart mode for ordinary group
Input parameter 1	<code>adc_x</code> : indicates the selected ADC This parameter can be ADC1.
Input parameter 2	<code>adc_ordinary_oversample_restart</code> : indicates the pre-configured status of oversampling restart mode for ordinary group This parameter can be any enumerated value in the <code>adc_ordinary_oversample_restart_type</code> .
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

adc_ordinary_oversample_restart

The `adc_ordinary_oversample_restart` is used to select oversampling restart mode for ordinary group ADC conversion, including:

`ADC_OVERSAMPLE_CONTINUE`:

Continuous mode (the oversampling buffer area of ordinary group is preserved)

`ADC_OVERSAMPLE_RESTART`:

Restart mode (the oversampling buffer area of ordinary group is cleared, that is, the sampling times of the current channel is cleared)

Example:

```
/* set ordinary oversample restart mode */  
adc_ordinary_oversample_restart_set(ADC1, ADC_OVERSAMPLE_CONTINUE);
```


5.3 Controller area network (CAN)

CAN register structure `can_type` is defined in the “at32f402_405_can.h”:

```
/**
 * @brief type define can register all
 */
typedef struct
{
    ...
} can_type;
```

The table below gives a list of the CAN registers.

Table 62. Summary of CAN registers

Register	Description
mctrl	CAN master control register
msts	CAN master status register
tsts	CAN transmit status register
rf0	CAN receive FIFO 0 register
fr1	CAN receive FIFO 1 register
inten	CAN interrupt enable register
ests	CAN error status register
btmg	CAN bit timing register
tmi0	Transmit mailbox 0 identifier register
tmc0	Transmit mailbox 0 data length and time stamp register
tmdtl0	Transmit mailbox 0 data byte low register
tmdth0	Transmit mailbox 0 data byte high register
tmi1	Transmit mailbox 1 identifier register
tmc1	Transmit mailbox 1 data length and time stamp register
tmdtl1	Transmit mailbox 1 data byte low register
tmdth1	Transmit mailbox 1 data byte high register
tmi2	Transmit mailbox 2 identifier register
tmc2	Transmit mailbox 2 data length and time stamp register
tmdtl2	Transmit mailbox 2 data byte low register
tmdth2	Transmit mailbox 2 data byte high register
rfi0	Receive FIFO0 mailbox identifier register
rfc0	Receive FIFO0 mailbox data length and time stamp register
rfdtl0	Receive FIFO0 mailbox data byte low register
rfdth0	Receive FIFO0 mailbox data byte high register
rfi1	Receive FIFO1 mailbox identifier register
rfc1	Receive FIFO1 mailbox data length and time stamp register
rfdtl1	Receive FIFO1 mailbox data byte low register
rfdth1	Receive FIFO1 mailbox data byte high register
fctrl	CAN filter control register
fmcfgr	CAN filter mode configuration register

Register	Description
fscfg	CAN filter size configuration register
frf	CAN filter FIFO association register
facfg	CAN filter activate control register
fb0f1	CAN filter bank 0 filter register 1
fb0f2	CAN filter bank 0 filter register 2
fb1f1	CAN filter bank 1 filter register 1
fb1f2	CAN filter bank 1 filter register 2
...	...
Fb13f1	CAN filter bank 13 filter register 1
Fb13f2	CAN filter bank 13 filter register 2

The table below gives a list of CAN library functions.

Table 63. Summary of CAN library functions

Function name	Description
can_reset	Reset all CAN registers to their reset values
can_baudrate_default_para_init	Configure the CAN baud rate initial structure with the initial value
can_baudrate_set	Configure CAN baud rate
can_default_para_init	Configure the CAN initial structure with the initial value
can_base_init	Initialize CAN registers with the specified parameters in the can_base_struct
can_filter_default_para_init	Configure the CAN filter initial structure with the initial value
can_filter_init	Initialize CAN registers with the specified parameters in the can_filter_init_struct
can_debug_transmission_prohibit	Select to disalbe/enable message reception and transmission when debug
can_ttc_mode_enable	Enable time-triggered mode
can_message_transmit	Transmit a frame of message
can_transmit_status_get	Get the status of transmission
can_transmit_cancel	Cancel transmission
can_message_receive	Receive a frame of message
can_receive_fifo_release	Release receive FIFO
can_receive_message_pending_get	Get the count of pending messages in FIFO
can_operating_mode_set	Configure CAN operating mode
can_doze_mode_enter	Enter sleep mode
can_doze_mode_exit	Exit sleep mode
can_error_type_record_get	Read CAN error type
can_receive_error_counter_get	Read CAN receive error counter
can_transmit_error_counter_get	Read CAN transmit error counter
can_interrupt_enable	Enable the selected CAN interrupt
can_flag_get	Read the selected CAN flag
can_interrupt_flag_get	Read the selected CAN interrupt flag
can_flag_clear	Clear the selected CAN flag

5.3.1 can_reset function

The table below describes the function can_reset.

Table 64. can_reset function

Name	Description
Function name	can_reset
Function prototype	void can_reset(can_type* can_x);
Function description	Reset CAN registers to their default values.
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
can_reset(CAN1);
```

5.3.2 can_baudrate_default_para_init function

The table below describes the function can_baudrate_default_para_init.

Table 65. can_baudrate_default_para_init function

Name	Description
Function name	can_baudrate_default_para_init
Function prototype	void can_baudrate_default_para_init(can_baudrate_type* can_baudrate_struct);
Function description	Configure the CAN baud rate initial structure with the initial value
Input parameter 1	can_baudrate_struct: can_baudrate_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of the can_baudrate_type before starting.
Called functions	NA

Example:

```
can_baudrate_type can_baudrate_struct;  
can_baudrate_default_para_init(&can_baudrate_struct);
```

5.3.3 can_baudrate_set function

The table below describes the function can_baudrate_set.

Table 66. can_baudrate_set function

Name	Description
Function name	can_baudrate_set
Function prototype	error_status can_baudrate_set(can_type* can_x, can_baudrate_type* can_baudrate_struct);
Function description	Set baud rate for CAN
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1 or CAN2.
Input parameter 2	can_baudrate_struct: can_baudrate_type pointer
Output parameter	NA
Return value	status_index: check if baud rate is configured successfully
Required preconditions	It is necessary to define a variable of the can_baudrate_type before starting.
Called functions	NA

The can_baudrate_type is defined in the at32f402_405_can.h:

typedef struct

```
{
    uint16_t      baudrate_div;
    can_rsaw_type rsaw_size;
    can_bts1_type bts1_size;
    can_bts2_type bts2_size;
} can_baudrate_type;
```

baudrate_div

CAN clock division factor

Value range: 0x001~0x400

rsaw_size

Defines the maximum of time unit that the CAN is allowed to lengthen or shorten in a bit

CAN_RSAW_1TQ: Resynchronization width is 1 time unit

CAN_RSAW_2TQ: Resynchronization width is 2 time units

CAN_RSAW_3TQ: Resynchronization width is 3 time units

CAN_RSAW_4TQ: Resynchronization width is 4 time units

bts1_size

segment1 time duration

bts1_size description

CAN_BTS1_1TQ: the bit time segment 1 has 1 time unit

.....

CAN_BTS1_16TQ: the bit time segment 1 has 16 time units

bts2_size

segment2 time duration

CAN_BTS2_1TQ: the bit time segment 2 has 1 time unit

.....

CAN_BTS2_8TQ: the bit time segment 2 has 8 time units

Example:

```
/* can baudrate, set baudrate = pclk/(baudrate_div *(1 + bts1_size + bts2_size)) */
can_baudrate_struct.baudrate_div = 10;
can_baudrate_struct.rsaw_size = CAN_RSAW_3TQ;
can_baudrate_struct.bts1_size = CAN_BTS1_8TQ;
can_baudrate_struct.bts2_size = CAN_BTS2_3TQ;
can_baudrate_set(CAN1, &can_baudrate_struct);
```

5.3.4 can_default_para_init function

The table below describes the function can_default_para_init.

Table 67. can_default_para_init function

Name	Description
Function name	can_default_para_init
Function prototype	void can_default_para_init(can_base_type* can_base_struct);
Function description	Set an initial value for CAN initial structure
Input parameter 1	can_base_struct: can_base_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of the can_base_type before starting.
Called functions	NA

Example:

```
can_base_type can_base_struct;
can_default_para_init (&can_base_struct);
```

5.3.5 can_base_init function

The table below describes the function can_base_init.

Table 68. can_base_init function

Name	Description
Function name	can_base_init
Function prototype	error_status can_base_init(can_type* can_x, can_base_type* can_base_struct);
Function description	Initialize CAN registers with the specified parameters in the can_base_struct
Input parameter 1	can_x: the selected CAN peripheral This parameter can be CAN1.
Input parameter 2	can_base_struct: can_base_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of the can_base_type before starting.
Called functions	NA

The can_base_type is defined in the at32f402_405_can.h:

```
typedef struct
{
```

```

    can_mode_type          mode_selection;
    confirm_state          ttc_enable;
    confirm_state          aebo_enable;
    confirm_state          aed_enable;
    confirm_state          prsf_enable;
    can_msg_discarding_rule_type mdrsel_selection;
    can_msg_sending_rule_type  mmssr_selection;
} can_base_type;

```

mode_selection

Test mode selection

CAN_MODE_COMMUNICATE:	Communication mode
CAN_MODE_LOOPBACK:	Loopback mod
CAN_MODE_LISTENONLY:	Listen only mode
CAN_MODE_LISTENONLY_LOOPBACK:	Loopback + listen only mod

ttc_enable

Enable/disable time-triggered communication mode.

FALSE: Disable time-triggered communication mode

TRUE: Enable time-triggered communication mode (while receiving/sending messages, capture time stamp and store it into the CAN_RFCx and CAN_TMCx registers)

aebo_enable

Enable auto exit of bus-off mode.

FALSE: Automatic exit of bus-off mode is disabled

TRUE: Automatic exit of bus-off mode is enabled

aed_enable

Enable auto exit of sleep mode.

FALSE: Auto exit of sleep mode is disabled

TRUE: Auto exit of sleep mode is enabled

prsf_enable

Disable retransmission when transmit failed.

FALSE: Retransmission is enabled

TRUE: Retransmission is disabled

mdrsel_selection

Define message discard rule when reception overflows.

CAN_DISCARDING_FIRST_RECEIVED: The previous message is discarded.

CAN_DISCARDING_LAST_RECEIVED: The new incoming message is discarded.

mmssr_selection

Define multiple message transmit sequence rule.

CAN_SENDING_BY_ID: The message with the smallest identifier number is first transmitted.

CAN_SENDING_BY_REQUEST: The message with the first request order is first transmitted.

Example:

```

/* can base init */
can_base_struct.mode_selection = CAN_MODE_COMMUNICATE;
can_base_struct.ttc_enable = FALSE;
can_base_struct.aebo_enable = TRUE;
can_base_struct.aed_enable = TRUE;

```

```

can_base_struct.prsf_enable = FALSE;
can_base_struct.mdrsel_selection = CAN_DISCARDING_FIRST_RECEIVED;
can_base_struct.mmssr_selection = CAN_SENDING_BY_ID;
can_base_init(CAN1, &can_base_struct);

```

5.3.6 can_filter_default_para_init function

The table below describes the function can_filter_default_para_init.

Table 69. can_filter_default_para_init function

Name	Description
Function name	can_filter_default_para_init
Function prototype	void can_filter_default_para_init(can_filter_init_type* can_filter_init_struct);
Function description	Configure CAN filter initialization structure with the initial value
Input parameter 1	can_filter_init_struct: can_filter_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of can_filter_init_type before starting.
Called functions	NA

Example:

```

can_filter_init_type can_filter_init_struct;
can_filter_default_para_init(&can_filter_init_struct);

```

5.3.7 can_filter_init function

The table below describes the function can_filter_init.

Table 70. can_filter_init function

Name	Description
Function name	can_filter_init
Function prototype	void can_filter_init(can_type* can_x, can_filter_init_type* can_filter_init_struct);
Function description	Initialize all CAN registers with the specified parameters in the can_base_struct
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	can_filter_init_struct: can_filter_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of can_filter_init_type before starting.
Called functions	NA

The can_filter_init_type is defined in the at32f402_405_can.h:

typedef struct

```

{
    confirm_state          filter_activate_enable;
    can_filter_mode_type   filter_mode;
    can_filter_fifo_type   filter_fifo;
    uint8_t                filter_number;
}

```

```

    can_filter_bit_width_type  filter_bit;
    uint16_t                  filter_id_high;
    uint16_t                  filter_id_low;
    uint16_t                  filter_mask_high;
    uint16_t                  filter_mask_low;

```

} can_filter_init_type;

filter_activate_enable

Enable/disable filter bank

FALSE: Disable filter bank

TRUE: Enable filter bank

filter_mode

Select filter mode.

CAN_FILTER_MODE_ID_MASK: Identifier mask mode

CAN_FILTER_MODE_ID_LIST: Identifier list mode

filter_fifo

Select filter associated FIFO.

CAN_FILTER_FIFO0: Associated with FIFO0

CAN_FILTER_FIFO1: Associated with FIFO1

filter_number

Select filter bank.

Value range: 0~13

filter_bit

Select filter bit width

CAN_FILTER_16BIT: 16-bit

CAN_FILTER_32BIT: 32-bit

filter_id_high

The filter_id_high is used to configure the upper 16 bits (32-bit width, Mask/List mode) of the filter identifier 1, the filter identifier 2 (16-bit width, List mode) or the filter mask identifier 1 (16-bit width, Mask mode).

Value range: 0x0000~0xFFFF

filter_id_low

The filter_id_low is used to configure the lower 16 bits of the filter identifier 1 (32-bit width, Mask/List mode), or the filter identifier 1 (16-bit width, List mode).

Value range: 0x0000~0xFFFF

filter_mask_high

The filter_mask_high is used to configure the upper 16 bits of the filter mask identifier 1 (32-bit width, Mask mode), the filter mask identifier 2 (16-bit width, Mask mode), the upper 16 bits of the filter identifier 2 (32-bit width, List mode) or the filter identifier 4 (16-bit width, List mode).

Value range: 0x0000~0xFFFF

filter_mask_low

The filter_mask_low is used to configure the lower 16 bits of the filter mask identifier 1 (32-bit width, Mask mode), the filter identifier 2 (16-bit width, Mask mode), the lower 16 bits of the filter identifier 2 (32-bit width, List mode) or the filter identifier 3 (16-bit width, List mode).

Value range: 0x0000~0xFFFF

Example:


```

/* can filter init */
can_filter_init_struct.filter_activate_enable = TRUE;
can_filter_init_struct.filter_mode = CAN_FILTER_MODE_ID_MASK;
can_filter_init_struct.filter_fifo = CAN_FILTER_FIFO0;
can_filter_init_struct.filter_number = 0;
can_filter_init_struct.filter_bit = CAN_FILTER_32BIT;
can_filter_init_struct.filter_id_high = 0;
can_filter_init_struct.filter_id_low = 0;
can_filter_init_struct.filter_mask_high = 0;
can_filter_init_struct.filter_mask_low = 0;
can_filter_init(CAN1, &can_filter_init_struct);

```

5.3.8 can_debug_transmission_prohibit function

The table below describes the function can_debug_transmission_prohibit.

Table 71. can_debug_transmission_prohibit function

Name	Description
Function name	can_debug_transmission_prohibit
Function prototype	void can_debug_transmission_prohibit(can_type* can_x, confirm_state new_state);
Function description	Disable/enable message transceiver when debugging
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```

/* prohibit can trans when debug*/
can_debug_transmission_prohibit(CAN1, TRUE);

```

5.3.9 can_ttc_mode_enable function

The table below describes the function can_ttc_mode_enable.

Table 72. can_ttc_mode_enable function

Name	Description
Function name	can_ttc_mode_enable
Function prototype	void can_ttc_mode_enable(can_type* can_x, confirm_state new_state);
Function description	Enable time-triggered mode
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.

Name	Description
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* can time trigger operation communication mode enable*/
can_ttc_mode_enable (CAN1, TRUE);
```

Note: When the ttc_enable is enabled in the can_base_init, it indicates that only the time stamp is enabled (During message receive and transmit, the time stamp is captured and stored in the CAN_RFCx and CAN_TMCx registers). But when the can_ttc_mode_enable is enabled, not only the time stamp is enabled, and but the time stamp transmission feature is enabled (During message transmission, the time stamp is sent on the 7th and 8th data byte).

5.3.10 can_message_transmit function

The table below describes the function can_message_transmit

Table 73. can_message_transmit function

Name	Description
Function name	can_message_transmit
Function prototype	uint8_t can_message_transmit(can_type* can_x, can_tx_message_type* tx_message_struct);
Function description	Transmit a frame of message
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	tx_message_struct: message pending for transmission, refer to can_tx_message_type
Output parameter	NA
Return value	transmit_mailbox: indicates the mailbox number required to send message
Required preconditions	Write the to-be-sent message in the tx_message_struct
Called functions	NA

The can_tx_message_type is defined in the at32f402_405_can.h:

typedef struct

```
{
    uint32_t          standard_id;
    uint32_t          extended_id;
    can_identifier_type id_type;
    can_trans_frame_type frame_type;
    uint8_t           dlc;
    uint8_t           data[8];
}
```

} can_tx_message_type;

standard_id

Standard identifier (11 bits active)

Value range: 0x000~0x7FF

extended_id

Extended identifier (29 bits active)

Value range: 0x000~0x1FFFFFFF

id_type

Identifier type

CAN_ID_STANDARD: Standard identifier

CAN_ID_EXTENDED: Extended identifier

frame_type

Frame type

CAN_TFT_DATA: Data frame

CAN_TFT_REMOTE: Remote frame

dlc

Data length (in byte)

Value range:0~8

data[8]

Data pending for transmission

Value range:0x00~0xFF

Example:

```
/* can transmit data */
static void can_transmit_data(void)
{
    uint8_t transmit_mailbox;
    can_tx_message_type tx_message_struct;
    tx_message_struct.standard_id = 0x400;
    tx_message_struct.extended_id = 0;
    tx_message_struct.id_type = CAN_ID_STANDARD;
    tx_message_struct.frame_type = CAN_TFT_DATA;
    tx_message_struct.dlc = 8;
    tx_message_struct.data[0] = 0x11;
    tx_message_struct.data[1] = 0x22;
    tx_message_struct.data[2] = 0x33;
    tx_message_struct.data[3] = 0x44;
    tx_message_struct.data[4] = 0x55;
    tx_message_struct.data[5] = 0x66;
    tx_message_struct.data[6] = 0x77;
    tx_message_struct.data[7] = 0x88;
    transmit_mailbox = can_message_transmit(CAN1, &tx_message_struct);
    while(can_transmit_status_get(CAN1, (can_tx_mailbox_num_type)transmit_mailbox) !=
CAN_TX_STATUS_SUCCESSFUL);
}
```

5.3.11 can_transmit_status_get function

The table below describes the function can_transmit_status_get.

Table 74. can_transmit_status_get function

Name	Description
Function name	can_transmit_status_get
Function prototype	can_transmit_status_type can_transmit_status_get(can_type* can_x, can_tx_mailbox_num_type transmit_mailbox);
Function description	Get the status of transmission
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	transmit_mailbox: indicates the mailbox number required to send message
Output parameter	NA
Return value	state_index: transmission status
Required preconditions	First send a frame of message and get a transmit mailbox number
Called functions	NA

Example:

```

/* can transmit data */
static void can_transmit_data(void)
{
    uint8_t transmit_mailbox;
    can_tx_message_type tx_message_struct;
    tx_message_struct.standard_id = 0x400;
    tx_message_struct.extended_id = 0;
    tx_message_struct.id_type = CAN_ID_STANDARD;
    tx_message_struct.frame_type = CAN_TFT_DATA;
    tx_message_struct.dlc = 8;
    tx_message_struct.data[0] = 0x11;
    tx_message_struct.data[1] = 0x22;
    tx_message_struct.data[2] = 0x33;
    tx_message_struct.data[3] = 0x44;
    tx_message_struct.data[4] = 0x55;
    tx_message_struct.data[5] = 0x66;
    tx_message_struct.data[6] = 0x77;
    tx_message_struct.data[7] = 0x88;
    transmit_mailbox = can_message_transmit(CAN1, &tx_message_struct);
    while(can_transmit_status_get(CAN1, (can_tx_mailbox_num_type)transmit_mailbox) !=
CAN_TX_STATUS_SUCCESSFUL);
}

```

5.3.12 can_transmit_cancel function

The table below describes the function can_transmit_cancel.

Table 75. can_transmit_cancel function

Name	Description
Function name	can_transmit_cancel
Function prototype	void can_transmit_cancel(can_type* can_x, can_tx_mailbox_num_type transmit_mailbox);
Function description	Cancel transmission
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	transmit_mailbox: indicates the mailbox number required to send message
Output parameter	NA
Return value	NA
Required preconditions	First send a frame of message and get a transmit mailbox number
Called functions	NA

Example:

```
/* cancel a transmit request */
uint8_t transmit_mailbox;
transmit_mailbox = can_message_transmit(CAN1, &tx_message_struct);
can_transmit_cancel(CAN1, (can_tx_mailbox_num_type)transmit_mailbox);
```

5.3.13 can_message_receive function

The table below describes the function can_message_receive.

Table 76. can_message_receive function

Name	Description
Function name	can_message_receive
Function prototype	void can_message_receive(can_type* can_x, can_rx_fifo_num_type fifo_number, can_rx_message_type* rx_message_struct);
Function description	Receive message
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	fifo_number: receive FIFO This parameter can be CAN_RX_FIFO0 or CAN_RX_FIFO1.
Output parameter	rx_message_struct: indicates the received message, refer to can_rx_message_type
Return value	NA
Required preconditions	Receive FIFO not empty (FIFO message count is not zero)
Called functions	void can_receive_fifo_release(can_type* can_x, can_rx_fifo_num_type fifo_number);

The can_rx_message_type is defined in the at32f402_405_can.h:

typedef struct

```
{
    uint32_t          standard_id;
    uint32_t          extended_id;
```

```

        can_identifier_type    id_type;
        can_trans_frame_type   frame_type;
        uint8_t                dlc;
        uint8_t                data[8];
        uint8_t                filter_index;
    } can_rx_message_type;

```

standard_id

Standard identifier (11 bits active)

Value range:0x000~0x7FF

extended_id

Extended identifier (29 bits active)

Value range:0x000~0x1FFFFFFF

id_type

Identifier type

CAN_ID_STANDARD: Standard identifier

CAN_ID_EXTENDED: Extended identifier

frame_type

Frame type

CAN_TFT_DATA: Data frame

CAN_TFT_REMOTE: Remote frame

dlc

Data length (in byte)

Value range:0~8

data[8]

Data pending for transmission

Value range:0x00~0xFF

filter_index

Filter match index (indicating the filter number that a message has passed through)

Value range:0x00~0xFF

Example:

```

/* can receive message */
can_rx_message_type rx_message_struct;
can_message_receive(CAN1, CAN_RX_FIFO0, &rx_message_struct);

```

5.3.14 can_receive_fifo_release function

The table below describes the function can_receive_fifo_release.

Table 77. can_receive_fifo_release function

Name	Description
Function name	can_receive_fifo_release
Function prototype	void can_receive_fifo_release(can_type* can_x, can_rx_fifo_num_type fifo_number);
Function description	Release receive FIFO
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	fifo_number: receive FIFO number This parameter can be CAN_RX_FIFO0 or CAN_RX_FIFO1.
Output parameter	NA
Return value	NA
Required preconditions	Message in FIFO has already been read
Called functions	NA

Example:

```

/* can receive message */
void can_message_receive(can_type* can_x, can_rx_fifo_num_type fifo_number, can_rx_message_type*
rx_message_struct)
{
    /* get the id type */
    rx_message_struct->id_type = (can_identifier_type)can_x->fifo_mailbox[fifo_number].rfdi_bit.rfidi;
    ...

    /* get the data field */
    rx_message_struct->data[0] = can_x->fifo_mailbox[fifo_number].rfdtl_bit.rfdt0;
    ...
    rx_message_struct->data[7] = can_x->fifo_mailbox[fifo_number].rfdth_bit.rfdt7;

    /* FIFO must be read before releasing FIFO */
    /* release the fifo */
    can_receive_fifo_release(can_x, fifo_number);
}

```

5.3.15 can_receive_message_pending_get function

The table below describes the function can_receive_message_pending_get.

Table 78. can_receive_message_pending_get function

Name	Description
Function name	can_receive_message_pending_get
Function prototype	uint8_t can_receive_message_pending_get(can_type* can_x, can_rx_fifo_num_type fifo_number);
Function description	Get the number of message pending for read in FIFO
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	fifo_number: receive FIFO number This parameter can be CAN_RX_FIFO0 or CAN_RX_FIFO1.
Output parameter	NA
Return value	message_pending: the count of message pending for read in FIFO
Required preconditions	NA
Called functions	NA

Example:

```
/* return the number of pending messages of */
can_receive_message_pending_get (CAN1, CAN_RX_FIFO0);
```

5.3.16 can_operating_mode_set function

The table below describes the function can_operating_mode_set.

Table 79. can_operating_mode_set function

Name	Description
Function name	can_operating_mode_set
Function prototype	error_status can_operating_mode_set(can_type* can_x, can_operating_mode_type can_operating_mode);
Function description	Configure CAN operating modes
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	can_operating_mode : CAN operating mode selection
Output parameter	NA
Return value	status: indicates whether configuration is successful or not
Required preconditions	NA
Called functions	NA

can_operating_mode

CAN_OPERATINGMODE_FREEZE: Freeze mode—for CAN controller initialization

CAN_OPERATINGMODE_DOZE: Sleep mode—CAN clock stopped to save power consumption

CAN_OPERATINGMODE_COMMUNICATE: Communication mode—for communication

Example:

```
/* set the operation mode –enter freeze mode*/
can_operating_mode_set (CAN1, CAN_OPERATINGMODE_FREEZE);
```



```

/* Initialize CAN controller */
...

/* set the operation mode –enter communicate mode*/
can_operating_mode_set (CAN1, CAN_OPERATINGMODE_COMMUNICATE);

/* Starts communication: send and receive message */
...

```

5.3.17 can_doze_mode_enter function

The table below describes the function can_doze_mode_enter.

Table 80. can_doze_mode_enter function

Name	Description
Function name	can_doze_mode_enter
Function prototype	can_enter_doze_status_type can_doze_mode_enter(can_type* can_x);
Function description	Enter sleep mode
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Output parameter	NA
Return value	can_enter_doze_status : indicates wheter the Sleep mode is entered
Required preconditions	NA
Called functions	NA

can_enter_doze_status

Indicates whether the Sleep mode is entered or not

CAN_ENTER_DOZE_FAILED: Sleep mode entry failure

CAN_ENTER_DOZE_SUCCESSFUL: Sleep mode entry success

Example:

```

/* can enter the low power mode */
can_enter_doze_status_type can_enter_doze_status;
can_enter_doze_status = can_doze_mode_enter(CAN1);

```

5.3.18 can_doze_mode_exit function

The table below describes the function can_doze_mode_exit.

Table 81. can_doze_mode_exit function

Name	Description
Function name	can_doze_mode_exit
Function prototype	can_quit_doze_status_type can_doze_mode_exit(can_type* can_x);
Function description	Exit Sleep mode
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Output parameter	NA
Return value	can_quit_doze_status : indicates whethe the Sleep mode has been left
Required preconditions	NA
Called functions	NA

can_quit_doze_status

Indicates whethe the Sleep mode has been left successfully

CAN_QUIT_DOZE_FAILED: Sleep mode exit failure

CAN_QUIT_DOZE_SUCCESSFUL: Sleep mode exit success

Example:

```
/* can exit the low power mode */
can_quit_doze_status_type can_quit_doze_status;
can_quit_doze_status = can_doze_mode_exit (CAN1);
```

5.3.19 can_error_type_record_get function

The table below describes the function can_error_type_record_get.

Table 82. can_error_type_record_get function

Name	Description
Function name	can_error_type_record_get
Function prototype	can_error_record_type can_error_type_record_get(can_type* can_x);
Function description	Read CAN error type
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Output parameter	NA
Return value	can_error_record : Error type
Required preconditions	NA
Called functions	NA

can_error_record

CAN error record

CAN_ERRORRECORD_NOERR: No error

CAN_ERRORRECORD_STUFFERR: Bit stuffing error

CAN_ERRORRECORD_FORMERR: Format error

CAN_ERRORRECORD_ACKERR: Acknowledge error

CAN_ERRORRECORD_BITRECESSIVEERR: Recessive bit error

CAN_ERRORRECORD_BITDOMINANTERR: Dominant bit error
 CAN_ERRORRECORD_CRCERR: CRC error
 CAN_ERRORRECORD_SOFTWARESETERR: Set by software

Example:

```
/* get the error type record (etr) */
can_error_record_type can_error_record;
can_error_record = can_error_type_record_get (CAN1);
```

5.3.20 can_receive_error_counter_get function

The table below describes the function can_receive_error_counter_get.

Table 83. can_receive_error_counter_get function

Name	Description
Function name	can_receive_error_counter_get
Function prototype	uint8_t can_receive_error_counter_get(can_type* can_x);
Function description	Read CAN receive error counter
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Output parameter	NA
Return value	receive_error_counter: Receive error counter Value range: 0x00~0xFF
Required preconditions	NA
Called functions	NA

Example:

```
/* get the receive error counter (rec) */
uint8_t receive_error_counter;
receive_error_counter = can_receive_error_counter_get (CAN1);
```

5.3.21 can_transmit_error_counter_get function

The table below describes the function can_transmit_error_counter_get.

Table 84. can_transmit_error_counter_get function

Name	Description
Function name	can_transmit_error_counter_get
Function prototype	uint8_t can_transmit_error_counter_get(can_type* can_x);
Function description	Read CAN transmit error counter
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Output parameter	NA
Return value	transmit_error_counter: Transmit error counter Value range: 0x00~0xFF
Required preconditions	NA
Called functions	NA

Example:

```
/* get the transmit error counter (tec) */
uint8_t transmit_error_counter;
transmit_error_counter = can_transmit_error_counter_get (CAN1);
```

5.3.22 can_interrupt_enable function

The table below describes the function can_interrupt_enable.

Table 85. can_interrupt_enable function

Name	Description
Function name	can_interrupt_enable
Function prototype	void can_interrupt_enable(can_type* can_x, uint32_t can_int, confirm_state new_state);
Function description	Enable the selected CAN interrupt
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	can_int : Select CAN interrupts
Input parameter3	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

can_int

CAN interrupt select

CAN_TCIEN_INT:	Transmit mailbox empty interrupt enable
CAN_RF0MIEN_INT:	FIFO 0 receive message interrupt enable
CAN_RF0FIEN_INT:	Receive FIFO0 full interrupt enable
CAN_RF0OIEN_INT:	Receive FIFO0 overflow interrupt enable
CAN_RF1MIEN_INT:	FIFO 1 receive message interrupt enable
CAN_RF1FIEN_INT:	Receive FIFO1 full interrupt enable
CAN_RF1OIEN_INT:	Receive FIFO1 overflow interrupt enable
CAN_EAIEN_INT:	Error active interrupt enable
CAN_EPIEN_INT:	Error passive interrupt enable
CAN_BOIEN_INT:	Bus-off interrupt enable
CAN_ETRIEN_INT:	Error type record interrupt enable
CAN_EOIEN_INT:	Error occur interrupt enable
CAN_QDZIEN_INT:	Quit Sleep mode interrupt enable
CAN_EDZIEN_INT:	Enter Sleep mode interrupt enable

Example:

```
/* can interrupt config */
nvic_irq_enable(CAN1_SE_IRQn, 0x00, 0x00);/*CAN1 error/status change interrupt */
nvic_irq_enable(USBFS_L_CAN1_RX0_IRQn, 0x00, 0x00);/*CAN1 FIFO0 receive interrupt */

/* FIFO 0 receive message interrupt enable */
```

```

can_interrupt_enable(CAN1, CAN_RF0MIEN_INT, TRUE);
/* error type record interrupt enable */
can_interrupt_enable(CAN1, CAN_ETRIEN_INT, TRUE);

/* This parameter is an error interrupt controller and it is enabled before error-related interrupts */
can_interrupt_enable(CAN1, CAN_EOIEIN_INT, TRUE);

```

5.3.23 can_flag_get function

The table below describes the function can_flag_get.

Table 86. can_flag_get function

Name	Description
Function name	can_flag_get
Function prototype	flag_status can_flag_get(can_type* can_x, uint32_t can_flag);
Function description	Get the status of the selected CAN flag
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1.
Input parameter 2	can_flag: indicates the selected flag Refer to the “can_flag” description below for details.
Output parameter	NA
Return value	flag_status: the status of the selected flag Return value can be SET or RESET.
Required preconditions	NA
Called functions	NA

can_flag

This is used to select a flag and get its status, including:

CAN_EAF_FLAG: Error active flag
 CAN_EPF_FLAG: Error passive flag
 CAN_BOF_FLAG: Bus-off flag
 CAN_ETR_FLAG: Error type record (non-zero error type flag)
 CAN_EOIF_FLAG: Error occur interrupt flag
 CAN_TM0TCF_FLAG: Mailbox 0 transmission complete flag
 CAN_TM1TCF_FLAG: Mailbox 1 transmission complete flag
 CAN_TM2TCF_FLAG: Mailbox 2 transmission complete flag
 CAN_RF0MN_FLAG: Receive FIFO0 non-empty flag
 CAN_RF0FF_FLAG: FIFO0 full flag
 CAN_RF0OF_FLAG: FIFO0 overflow flag
 CAN_RF1MN_FLAG: FIFO1 non-empty flag
 CAN_RF1FF_FLAG: FIFO1 full flag
 CAN_RF1OF_FLAG: FIFO1 overflow flag
 CAN_QDZIF_FLAG: Quit Sleep mode flag
 CAN_EDZC_FLAG: Enter Sleep mode flag
 CAN_TMEF_FLAG: Transmit mailbox empty flag (any one of three transmit mailboxes is empty)

Example:

```

/* get receive fifo 0 message num flag */

```

```
flag_status bit_status = RESET;
bit_status = can_flag_get (CAN1, CAN_RF0MN_FLAG);
```

5.3.24 can_interrupt_flag_get function

The table below describes the function can_interrupt_flag_get.

Table 87. can_interrupt_flag_get function

Name	Description
Function name	can_interrupt_flag_get
Function prototype	flag_status can_interrupt_flag_get(can_type* can_x, uint32_t can_flag);
Function description	Get the status of the selected CAN interrupt flag
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1.
Input parameter 2	can_flag : indicates the selected flag Refer to the “can_flag” description below for details.
Output parameter	NA
Return value	flag_status: the status of the selected flag Return value can be SET or RESET.
Required preconditions	NA
Called functions	NA

can_flag

This is used to select a flag and get its status, including:

CAN_EAF_FLAG: Error active flag
 CAN_EPF_FLAG: Error passive flag
 CAN_BOF_FLAG: Bus-off flag
 CAN_ETR_FLAG: Error type record (non-zero error type flag)
 CAN_EOIF_FLAG: Error occur interrupt flag
 CAN_TM0TCF_FLAG: Mailbox 0 transmission complete flag
 CAN_TM1TCF_FLAG: Mailbox 1 transmission complete flag
 CAN_TM2TCF_FLAG: Mailbox 2 transmission complete flag
 CAN_RF0MN_FLAG: FIFO0 non-empty flag
 CAN_RF0FF_FLAG: FIFO0 full flag
 CAN_RF0OF_FLAG: FIFO0 overflow flag
 CAN_RF1MN_FLAG: FIFO1 non-empty flag
 CAN_RF1FF_FLAG: FIFO1 full flag
 CAN_RF1OF_FLAG: FIFO1 overflow flag
 CAN_QDZIF_FLAG: Quit Sleep mode flag
 CAN_EDZC_FLAG: Enter Sleep mode flag
 CAN_TMEF_FLAG: Transmit mailbox empty flag (any one of three transmit mailboxes is empty)

Example

```
/* check receive fifo 0 message num interrupt flag */
if(can_interrupt_flag_get(CAN1, CAN_RF0MN_FLAG) != RESET)
{
}
}
```

5.3.25 can_flag_clear function

The table below describes the function can_flag_clear.

Table 88. can_flag_clear function

Name	Description
Function name	can_flag_clear
Function prototype	void can_flag_clear(can_type* can_x, uint32_t can_flag);
Function description	Clear the selected CAN flag
Input parameter 1	can_x: indicates the selected CAN This parameter can be CAN1
Input parameter 2	<i>can_flag</i> : indicates the selected flag Refer to can_flag
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

can_flag:

This is used to clear the selected flag, including:

- CAN_EAF_FLAG: Error active flag
- CAN_EPF_FLAG: Error passive flag
- CAN_BOF_FLAG: Bus-off flag
- CAN_ETR_FLAG: Error type record (non-zero Error type flag)
- CAN_EOIF_FLAG: Error occur interrupt flag
- CAN_TM0TCF_FLAG: Mailbox 0 transmission complete flag
- CAN_TM1TCF_FLAG: Mailbox 1 transmission complete flag
- CAN_TM2TCF_FLAG: Mailbox 2 transmission complete flag
- CAN_RF0FF_FLAG: FIFO0 full flag
- CAN_RF0OF_FLAG: FIFO0 overflow flag
- CAN_RF1FF_FLAG: FIFO1 full flag
- CAN_RF1OF_FLAG: FIFO1 overflow flag
- CAN_QDZIF_FLAG: Quit Sleep mode flag
- CAN_EDZC_FLAG: Enter Sleep mode flag
- CAN_TMEF_FLAG: Transmit mailbox empty flag (any one of three transmit mailboxes is empty)

Note: The CAN_RF0MN_FLAG (FIFO0 non-empty flag) and CAN_RF1MN_FLAG (FIFO1 non-empty flag) have no clear operations since both are defined by software.

Example:

```
/* clear receive fifo 0 overflow flag */
can_flag_clear (CAN1, CAN_RF1OF_FLAG);
```

5.4 CRC calculation unit (CRC)

The CRC register structure `crc_type` is defined in the “`at32f402_405_crc.h`”:

```
/**
 * @brief type define crc register all
 */
typedef struct
{
    ...

} crc_type;
```

The table below gives a list of the CRC registers.

Table 89. Summary of CRC registers

Register	Description
dt	Data register
cdt	General-purpose data register
ctrl	Control register
idt	Control register
poly	Polynomial generator

The table below gives a list of CRC library functions.

Table 90. Summary of CRC library functions

Function name	Description
<code>crc_data_reset</code>	Data register reset
<code>crc_one_word_calculate</code>	Calculate the CRC value using combination of a new 32-bit data and the previous CRC value
<code>crc_block_calculate</code>	Write a data block in order into CRC check and return the calculated result
<code>crc_data_get</code>	Get the currently calculated CRC result
<code>crc_common_data_set</code>	Configure common registers
<code>crc_common_data_get</code>	Get the value of common registers
<code>crc_init_data_set</code>	Set the CRC initialization register
<code>crc_reverse_input_data_set</code>	Set CRC input data bit reverse type
<code>crc_reverse_output_data_set</code>	Set CRC output data reverse type
<code>crc_poly_value_set</code>	Set polynomial value
<code>crc_poly_value_get</code>	Get polynomial value
<code>crc_poly_size_set</code>	Set polynomial valid width
<code>crc_poly_size_get</code>	Get polynomial valid width

5.4.1 crc_data_reset function

The table below describes the function `crc_data_reset`.

Table 91. `crc_data_reset` function

Name	Description
Function name	<code>crc_data_reset</code>
Function prototype	<code>void crc_data_reset(void);</code>
Function description	When the data register is reset, the value of the initialization register is added into the data register as an initial value. The default reset value is 0xFFFFFFFF.
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reset crc data register */
crc_data_reset();
```

5.4.2 crc_one_word_calculate function

The table below describes the function `crc_one_word_calculate`.

Table 92. `crc_one_word_calculate` function

Name	Description
Function name	<code>crc_one_word_calculate</code>
Function prototype	<code>uint32_t crc_one_word_calculate(uint32_t data);</code>
Function description	Calculate the CRC value using a combination of a new 32-bit data and the previous CRC value.
Input parameter 1	data: input a 32-bit data
Input parameter 2	NA
Output parameter	NA
Return value	<code>uint32_t</code> : return CRC calculation result
Required preconditions	NA
Called functions	NA

Example:

```
/* calculate and return result */
uint32_t data = 0x12345678, result = 0;
result = crc_one_word_calculate (data);
```

5.4.3 crc_block_calculate function

The table below describes the function `crc_block_calculate`

Table 93. `crc_block_calculate` function

Name	Description
Function name	<code>crc_block_calculate</code>
Function prototype	<code>uint32_t crc_block_calculate(uint32_t *pbuffer, uint32_t length);</code>
Function description	Input a data block in sequence to go through CRC calculation and return a result
Input parameter 1	<code>pbuffer</code> : point to the data block pending for CRC check
Input parameter 2	<code>length</code> : data block length pending for CRC check, in terms of 32-bit
Output parameter	NA
Return value	<code>uint32_t</code> : return CRC calculation result
Required preconditions	NA
Called functions	NA

Example:

```
/* calculate and return result */
uint32_t pbuffer[2] = {0x12345678, 0x87654321};
uint32_t result = 0;
result = crc_block_calculate (pbuffer, 2);
```

5.4.4 crc_data_get function

The table below describes the function `crc_data_get`.

Table 94. `crc_data_get` function

Name	Description
Function name	<code>crc_data_get</code>
Function prototype	<code>uint32_t crc_data_get(void);</code>
Function description	Return the current CRC calculation result
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	<code>uint32_t</code> : return CRC calculation result
Required preconditions	NA
Called functions	NA

Example:

```
/* get result */
uint32_t result = 0;
result = crc_data_get ();
```

5.4.5 crc_common_data_set function

The table below describes the function `crc_common_data_set`.

Table 95. `crc_common_data_set` function

Name	Description
Function name	<code>crc_common_data_set</code>
Function prototype	<code>void crc_common_data_set(uint8_t cdt_value);</code>
Function description	Configure common data register
Input parameter 1	<code>cdt_value</code> : 8-bit common data that can be used as temporary storage data
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set common data */
crc_common_data_set (0x88);
```

5.4.6 crc_common_data_get function

The table below describes the function `crc_common_data_get`.

Table 96. `crc_common_data_get` function

Name	Description
Function name	<code>crc_common_data_get</code>
Function prototype	<code>uint8_t crc_common_data_get(void);</code>
Function description	Return the value of the command data register
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	<code>uint8_t</code> : return the value of the previously programmed common data register
Required preconditions	NA
Called functions	NA

Example:

```
/* get common data */
uint8_t cdt_value = 0;
cdt_value = crc_common_data_get ();
```

5.4.7 crc_init_data_set function

The table below describes the function `crc_init_data_set`.

Table 97. `crc_init_data_set` function

Name	Description
Function name	<code>crc_init_data_set</code>
Function prototype	<code>void crc_init_data_set(uint32_t value);</code>
Function description	Set the value of the CRC initialization register
Input parameter 1	value: the value of the CRC initialization register
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

After the value of the CRC initialization register is programmed, the CRC data register is updated with this value whenever the `crc_data_reset` function is called.

Example:

```
/* set initial data */  
uint32_t init_value = 0x11223344;  
crc_init_data_set (init_value);
```

5.4.8 crc_reverse_input_data_set function

The table below describes the function `crc_reverse_input_data_set`.

Table 98. `crc_reverse_input_data_set` function

Name	Description
Function name	<code>crc_reverse_input_data_set</code>
Function prototype	<code>void crc_reverse_input_data_set(crc_reverse_input_type value);</code>
Function description	Define the CRC input data bit reverse type
Input parameter 1	value: input data bit reverse type. Refer to “value” below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

Define the reverse type of input data bit.

CRC_REVERSE_INPUT_NO_AFFECTE: No effect
CRC_REVERSE_INPUT_BY_BYTE: Byte reverse
CRC_REVERSE_INPUT_BY_HALFWORD: Half-word reverse
CRC_REVERSE_INPUT_BY_WORD: Word reverse

Example:

```
/* set input data reversing type */  
crc_reverse_input_data_set(CRC_REVERSE_INPUT_BY_WORD);
```

5.4.9 crc_reverse_output_data_set function

The table below describes the function `crc_reverse_output_data_set`.

Table 99. `crc_reverse_output_data_set` function

Name	Description
Function name	<code>crc_reverse_output_data_set</code>
Function prototype	<code>void crc_reverse_output_data_set(crc_reverse_output_type value);</code>
Function description	Define the CRC output data reverse type
Input parameter 1	value: output data bit reverse type. Refer to “value” below for details.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

Define the reverse type of output data bit.

CRC_REVERSE_OUTPUT_NO_AFFECTE: No effect

CRC_REVERSE_OUTPUT_DATA: Word reverse

Example:

```
/* set output data reversing type */
crc_reverse_output_data_set (CRC_REVERSE_OUTPUT_DATA);
```

5.4.10 crc_poly_value_set function

The table below describes the function `crc_poly_value_set`.

Table 100. `crc_poly_value_set` function

Name	Description
Function name	<code>crc_poly_value_set</code>
Function prototype	<code>void crc_poly_value_set(uint32_t value);</code>
Function description	Set CRC polynomial value
Input parameter 1	value: polynomial value
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set poly value */
crc_poly_value_set(0x12345671);
```

5.4.11 crc_poly_value_get function

The table below describes the function `crc_poly_value_get`.

Table 101. `crc_poly_value_get` function

Name	Description
Function name	<code>crc_poly_value_get</code>
Function prototype	<code>uint32_t crc_poly_value_get(void);</code>
Function description	Get CRC polynomial value
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	<code>uint32_t</code> : return polynomial value
Required preconditions	NA
Called functions	NA

Example:

```
/* get poly value */
uint32_t poly = 0;
poly = crc_poly_value_get();
```

5.4.12 crc_poly_size_set function

The table below describes the function `crc_poly_size_set`.

Table 102. `crc_poly_size_set` function

Name	Description
Function name	<code>crc_poly_size_set</code>
Function prototype	<code>void crc_poly_size_set(crc_poly_size_type size);</code>
Function description	Set CRC polynomial valid width
Input parameter 1	size: polynomial valid width
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

size

Define the valid width of polynomial.

CRC_POLY_SIZE_32B: 32-bit
CRC_POLY_SIZE_16B: 16-bit
CRC_POLY_SIZE_8B: 8-bit
CRC_POLY_SIZE_7B: 7-bit

Example:

```
/* set poly size 32-bit */
crc_poly_size_set(CRC_POLY_SIZE_32B);
```

5.4.13 crc_poly_size_get function

The table below describes the function `crc_poly_size_get`.

Table 103. `crc_poly_size_get` function

Name	Description
Function name	<code>crc_poly_size_get</code>
Function prototype	<code>crc_poly_size_type crc_poly_size_get(void);</code>
Function description	Get CRC polynomial valid width
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	<code>crc_poly_size_type</code> : polynomial valid width
Required preconditions	NA
Called functions	NA

`crc_poly_size_type`

Define the valid width of polynomial.

`CRC_POLY_SIZE_32B`: 32-bit

`CRC_POLY_SIZE_16B`: 16-bit

`CRC_POLY_SIZE_8B`: 8-bit

`CRC_POLY_SIZE_7B`: 7-bit

Example:

```
/* get poly size */  
crc_poly_size_type size;  
size = crc_poly_size_get();
```

5.5 Clock and reset management (CRM)

The CRM register structure `crm_type` is defined in the “at32f402_405_crm.h”:

```
/**
 * @brief type define crm register all
 */
typedef struct
{
    ...

} crm_type;
```

The table below gives a list of the CRM registers.

Table 104. Summary of CRM registers

Register	Description
ctrl	Clock control register
pllcfg	PLL clock configuration register
cfg	Clock configuration register
clkint	Clock interrupt register
ahbrst1	AHB peripheral reset register 1
ahbrst2	AHB peripheral reset register 2
ahbrst3	AHB peripheral reset register 3
apb1rst	APB1 peripheral reset register
apb2rst	APB2 peripheral reset register
ahben1	AHB peripheral clock enable register 1
ahben2	AHB peripheral clock enable register 2
ahben3	AHB peripheral clock enable register 3
apb1en	APB1 peripheral clock enable register
apb2en	APB2 peripheral clock enable register
ahblpen1	AHB peripheral clock enable in low power mode register 1
ahblpen2	AHB peripheral clock enable in low power mode register 2
ahblpen3	AHB peripheral clock enable in low power mode register 3
apb1lpen	APB1 peripheral clock enable in low power mode register
apb2lpen	APB2 peripheral clock enable in low power mode register
bpdcc	Battery powered domain control register
ctrlsts	Control/status register
otghs	High-speed USB PHY clock control register
misc1	Extra register 1
misc2	Extra register 2

The table below gives a list of CRM library functions.

Table 105. Summary of CRM library functions

Function name	Description
crm_reset	Reset clock reset management register and control status
crm_lxt_bypass	Configure low-speed external clock bypass
crm_hext_bypass	Configure highed external clock bypass
crm_flag_get	Check if the selected flag is set or not
crm_hext_stable_wait	Wait HEXT to get stable
crm_hick_clock_trimming_set	High speed internal clock trimming
crm_hick_clock_calibration_set	High speed internal clock calibration
crm_periph_clock_enable	Peripheral clock enable
crm_periph_reset	Peripheral set
crm_periph_lowpower_mode_enable	Enable peripheral clock in low-power mode
crm_clock_source_enable	Clock source enable
crm_flag_clear	Clear flag
crm_ertc_clock_select	ERTC clock source selection
crm_ertc_clock_enable	ERTC clock enable
crm_ahb_div_set	AHB clock division
crm_apb1_div_set	APB1 clock division
crm_apb2_div_set	APB2 clock division
crm_hext_sclk_div_set	HEXT clock division
crm_hick_sclk_div_set	HICK clock division
crm_clock_failure_detection_enable	Clock failure detection enable
crm_battery_powered_domain_reset	Battery powered domain reset
crm_auto_step_mode_enable	Enable auto step mode
crm_i2sf5_clock_select	I2SF5 clock selection
crm_hick_sclk_frequency_select	When HICK is used as a system clock, the system clock frequency can be 8M or 48M
crm_usb_clock_source_select	Select PLL or internal high-speed clock (48M) as USB clock source
crm_usb_phy12_clock_select	High-speed USB PHY 12MHz clock selection
crm_pllu_output_set	Enable PLLU clock output
crm_pll_config	PLL clock source and frequency multiplication factor
crm_pllu_div_set	Set PLLU post-frequency division
crm_sysclk_switch	System clock source switch
crm_sysclk_switch_status_get	Get the status of system clock source
crm_clocks_freq_get	Get clock frequency
crm_clock_out_set	Clock output clock source
crm_clkout_div_set	Clock frequency division on clock out pins
crm_interrupt_enable	Interrupt enable
crm_pll_parameter_calculate	Calculate PLL parameters automatically

5.5.1 crm_reset function

The table below describes the function crm_reset.

Table 106. crm_reset function

Name	Description
Function name	crm_reset
Function prototype	void crm_reset(void);
Function description	Reset the clock reset management register and control status
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

1. This function does not change the HICKTRIM[5:0] in the CRM_CTRL register;
2. Modifying the function does not reset the CRM_BPDC and CRM_CTRLSTS registers.

Example:

```
/* reset crm */
crm_reset();
```

5.5.2 crm_lext_bypass function

The table below describes the function crm_lext_bypass.

Table 107. crm_lext_bypass function

Name	Description
Function name	crm_lext_bypass
Function prototype	void crm_lext_bypass(confirm_state new_state);
Function description	Configure low-speed external clock bypass
Input parameter 1	new_state: Enable bypass (TRUE), disable bypass (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	The LEXT configuration must be done before being enabled.
Called functions	NA

Example:

```
/* enable lext bypass mode */
crm_lext_bypass(TRUE);
```

5.5.3 crm_hext_bypass function

The table below describes the function crm_hext_bypass.

Table 108. crm_hext_bypass function

Name	Description
Function name	crm_hext_bypass
Function prototype	void crm_hext_bypass(confirm_state new_state);
Function description	Configure high-speed external clock bypass
Input parameter 1	new_state: Enable bypass (TRUE), disable bypass (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	The HEXT configuration must be done before being enabled.
Called functions	NA

Example:

```
/* enable hext bypass mode */
crm_hext_bypass(TRUE);
```

5.5.4 crm_flag_get function

The table below describes the function crm_flag_get.

Table 109. crm_flag_get function

Name	Description
Function name	crm_flag_get
Function prototype	flag_status crm_flag_get(uint32_t flag);
Function description	Check if the selected flag has been set.
Input parameter 1	flag: flag selection
Input parameter 2	NA
Output parameter	NA
Return value	flag_status: check the status of the selected flag. (SET or RESET)
Required preconditions	NA
Called functions	NA

flag

Select a flag to read, including:

CRM_HICK_STABLE_FLAG:	HICK clock stable flag
CRM_HEXT_STABLE_FLAG:	HEXT clock stable flag
CRM_PLL_STABLE_FLAG:	PLL clock stable flag
CRM_PLLU_STABLE_FLAG:	PLLU clock stable flag
CRM_LEXT_STABLE_FLAG:	LEXT clock stable flag
CRM_LICK_STABLE_FLAG:	LICK clock stable flag
CRM_NRST_RESET_FLAG:	NRST pin reset flag
CRM_POR_RESET_FLAG:	Power-on/low voltage reset flag
CRM_SW_RESET_FLAG:	Software reset flag
CRM_WDT_RESET_FLAG:	Watchdog reset flag

CRM_WWDT_RESET_FLAG:	Window watchdog reset flag
CRM_LOWPOWER_RESET_FLAG:	Low-power consumption reset flag
CRM_LICK_READY_INT_FLAG:	LICK clock ready interrupt flag
CRM_LEXT_READY_INT_FLAG:	LEXT clock ready interrupt flag
CRM_HICK_READY_INT_FLAG:	HICK clock ready interrupt flag
CRM_HEXT_READY_INT_FLAG:	HEXT clock ready interrupt flag
CRM_PLL_READY_INT_FLAG:	PLL clock ready interrupt flag
CRM_CLOCK_FAILURE_INT_FLAG:	Clock failure interrupt flag

Example:

```
/* wait till pll is ready */
while(crm_flag_get(CRM_PLL_STABLE_FLAG) != SET)
{
}
```

5.5.5 crm_interrupt_flag_get function

The table below describes the function `crm_interrupt_flag_get`.

Table 110. crm_interrupt_flag_get function

Name	Description
Function name	<code>crm_interrupt_flag_get</code>
Function prototype	<code>flag_status crm_interrupt_flag_get(uint32_t flag);</code>
Function description	Check if the selected interrupt flag has been set
Input parameter 1	flag: flag selection Refer to the “flag” description below for details.
Input parameter 2	NA
Output parameter	NA
Return value	flag_status: check the status of the selected flag. (SET or RESET)
Required preconditions	NA
Called functions	NA

flag

Select a flag to read, including:

CRM_LICK_READY_INT_FLAG:	LICK clock ready interrupt flag
CRM_LEXT_READY_INT_FLAG:	LEXT clock ready interrupt flag
CRM_HICK_READY_INT_FLAG:	HICK clock ready interrupt flag
CRM_HEXT_READY_INT_FLAG:	HEXT clock ready interrupt flag
CRM_PLL_READY_INT_FLAG:	PLL clock ready interrupt flag
CRM_CLOCK_FAILURE_INT_FLAG:	Clock failure interrupt flag

Example

```
/* check pll ready interrupt flag */
if(crm_interrupt_flag_get(CRM_PLL_READY_INT_FLAG) != RESET)
{
}
```

5.5.6 crm_hext_stable_wait function

The table below describes the function `crm_hext_stable_wait`

Table 111. crm_hext_stable_wait function

Name	Description
Function name	<code>crm_hext_stable_wait</code>
Function prototype	<code>error_status crm_hext_stable_wait(void);</code>
Function description	Wait for HEXT to activate and become stable
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	<code>error_status</code> : Return the status of HEXT (SUCCESS or ERROR).
Required preconditions	NA
Called functions	NA

Example:

```
/* wait till hext is ready */
while(crm_hext_stable_wait() == ERROR)
{
}
```

5.5.7 crm_hick_clock_trimming_set function

The table below describes the function `crm_hick_clock_trimming_set`.

Table 112. crm_hick_clock_trimming_set function

Name	Description
Function name	<code>crm_hick_clock_trimming_set</code>
Function prototype	<code>void crm_hick_clock_trimming_set(uint8_t trim_value);</code>
Function description	Trim HICK clock
Input parameter 1	<code>trim_value</code> : trimming value. Default value is 0x20, configurable range is from 0 to 0x3F.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set trimming value */
crm_hick_clock_trimming_set(0x1F);
```

5.5.8 crm_hick_clock_calibration_set function

The table below describes the function `crm_hick_clock_calibration_set`.

Table 113. crm_hick_clock_calibration_set function

Name	Description
Function name	<code>crm_hick_clock_calibration_set</code>
Function prototype	<code>void crm_hick_clock_calibration_set(uint8_t cali_value);</code>
Function description	Set HICK clock calibration value
Input parameter 1	<code>cali_value</code> : calibration compensation value. The factory gate value is the default value, Its configurable range is from 0 to 0xFF.
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set trimming value */
crm_hick_clock_calibration_set(0x80);
```

5.5.9 crm_periph_clock_enable

The table below describes the function `crm_periph_clock_enable`.

Table 114. crm_periph_clock_enable function

Name	Description
Function name	<code>crm_periph_clock_enable</code>
Function prototype	<code>void crm_periph_clock_enable(crm_periph_clock_type value, confirm_state new_state);</code>
Function description	Enable peripheral clock
Input parameter 1	<code>value</code> : defines peripheral clock type
Input parameter 2	<code>new_state</code> : TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

The `crm_periph_clock_type` is defined in the `at32f402_405_crm.h`.

The naming rule of this parameter is: `CRM_peripheral_PERIPH_CLOCK`.

`CRM_DMA1_PERIPH_CLOCK`: DMA1 peripheral clock enable

`CRM_DMA2_PERIPH_CLOCK`: DMA2 peripheral clock enable

...

`CRM_PWC_PERIPH_CLOCK`: PWC peripheral clock enable

Example:

```
/* enable gpioa periph clock */
crm_periph_clock_enable(CRM_GPIOA_PERIPH_CLOCK, TRUE);
```

5.5.10 crm_periph_reset function

The table below describes the function `crm_periph_reset`.

Table 115. crm_periph_reset function

Name	Description
Function name	<code>crm_periph_reset</code>
Function prototype	<code>void crm_periph_reset(crm_periph_reset_type value, confirm_state new_state);</code>
Function description	Reset peripherals
Input parameter 1	value: Peripheral reset type
Input parameter 2	new_state: TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

This indicates the selected peripheral. The `crm_periph_reset_type` is defined in the `at32f402_405_crm.h`.

The naming rule of this parameter is: CRM_peripheral_PERIPH_RESET.

CRM_DMA1_PERIPH_RESET: DMA1 peripheral reset

CRM_DMA2_PERIPH_RESET: DMA2 peripheral reset

...

CRM_PWC_PERIPH_RESET: PWC peripheral reset

Example:

```
/* reset gpioa periph */
crm_periph_reset(CRM_GPIOA_PERIPH_RESET, TRUE);
```

5.5.11 crm_periph_lowpower_mode_enable function

The table below describes the function `crm_periph_lowpower_mode_enable`.

Table 116. crm_periph_lowpower_mode_enable function

Name	Description
Function name	<code>crm_periph_lowpower_mode_enable</code>
Function prototype	<code>void crm_periph_lowpower_mode_enable(crm_periph_clock_lowpower_type value, confirm_state new_state);</code>
Function description	Enable peripheral clock in low-power mode
Input parameter 1	value: indicates peripheral clock type in low-power mode
Input parameter 2	new_state: TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

It indicates the selected peripheral. The `crm_periph_lowpower_type` is defined in `at32f402_405_crm.h`.

The naming rule of this parameter is: CRM_peripheral_PERIPH_LOWPOWER.

CRM_DMA1_PERIPH_LOWPOWER: DMA1 peripheral low-power clock definition

CRM_DMA2_PERIPH_LOWPOWER: DMA2 peripheral low-power clock definition

...

CRM_PWC_PERIPH_LOWPOWER: PWC peripheral low-power clock definition

Example:

```
/* disable gpioa periph clock at sleep mode */  
crm_periph_reset(CRM_GPIOA_PERIPH_LOWPOWER, FALSE);
```

5.5.12 crm_clock_source_enable function

The table below describes the function crm_clock_source_enable function.

Table 117. crm_clock_source_enable function

Name	Description
Function name	crm_clock_source_enable
Function prototype	void crm_clock_source_enable(crm_clock_source_type source, confirm_state new_state);
Function description	Enable clock source
Input parameter 1	source: Clock type
Input parameter 2	new_state: TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

Clock source selection.

CRM_CLOCK_SOURCE_HICK: HICK

CRM_CLOCK_SOURCE_HEXT: HEXT

CRM_CLOCK_SOURCE_PLL: PLL

CRM_CLOCK_SOURCE_LEXT: LEXT

CRM_CLOCK_SOURCE_LICK: LICK

Example:

```
/* enable hext */  
crm_clock_source_enable (CRM_CLOCK_SOURCE_HEXT, FALSE);
```


5.5.13 crm_flag_clear function

The table below describes the function crm_flag_clear function.

Table 118. crm_flag_clear function

Name	Description
Function name	crm_flag_clear
Function prototype	void crm_flag_clear(uint32_t flag);
Function description	Clear the selected flags
Input parameter 1	Flag: indicates the flag to clear
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

Select a flag to clear.

CRM_NRST_RESET_FLAG:	NRST pin reset flag
CRM_POR_RESET_FLAG:	Power-on/low voltage reset flag
CRM_SW_RESET_FLAG:	Software reset flag
CRM_WDT_RESET_FLAG:	Watchdog reset flag
CRM_WWDT_RESET_FLAG:	Window watchdog reset flag
CRM_LOWPOWER_RESET_FLAG:	Low-power reset flag
CRM_ALL_RESET_FLAG:	All reset flags
CRM_LICK_READY_INT_FLAG:	LICK clock ready interrupt flag
CRM_LEXT_READY_INT_FLAG:	LEXT clock ready interrupt flag
CRM_HICK_READY_INT_FLAG:	HICK clock ready interrupt flag
CRM_HEXT_READY_INT_FLAG:	HEXT clock ready interrupt flag
CRM_PLL_READY_INT_FLAG:	PLL clock ready interrupt flag
CRM_CLOCK_FAILURE_INT_FLAG:	Clock failure interrupt flag

Example:

```
/* clear clock failure detection flag */
crm_flag_clear(CRM_CLOCK_FAILURE_INT_FLAG);
```

5.5.14 crm_ertc_clock_select function

The table below describes the function `crm_ertc_clock_select` function.

Table 119. crm_ertc_clock_select function

Name	Description
Function name	<code>crm_ertc_clock_select</code>
Function prototype	<code>void crm_ertc_clock_select(crm_ertc_clock_type value);</code>
Function description	Select ERTC clock source
Input parameter 1	value: indicates ertc clock source type
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

ERTC clock source selection.

CRM_ERTC_CLOCK_NOCLK: No clock source for ERTC
 CRM_ERTC_CLOCK_LEXT: LEXT selected as ERTC clock
 CRM_ERTC_CLOCK_LICK: LICK selected as ERTC clock
 CRM_ERTC_CLOCK_HEXT_DIV2: HEXT/2 selected as ERTC clock
 ...
 CRM_ERTC_CLOCK_HEXT_DIV31: HEXT/31 selected as ERTC clock

Example:

```
/* config text as ertc clock */
crm_ertc_clock_select (CRM_ERTC_CLOCK_LEXT);
```

5.5.15 crm_ertc_clock_enable function

The table below describes the function `crm_ertc_clock_enable`.

Table 120. crm_ertc_clock_enable function

Name	Description
Function name	<code>crm_ertc_clock_enable</code>
Function prototype	<code>void crm_ertc_clock_enable(confirm_state new_state);</code>
Function description	Enable ERTC clock
Input parameter 1	<code>new_state</code> : TRUE or FALSE
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable ertc clock */
crm_ertc_clock_enable (TRUE);
```

5.5.16 crm_ahb_div_set function

The table below describes the function crm_ahb_div_set.

Table 121. crm_ahb_div_set function

Name	Description
Function name	crm_ahb_div_set
Function prototype	void crm_ahb_div_set(crm_ahb_div_type value);
Function description	Configure AHB clock division
Input parameter 1	value: indicates the division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_AHB_DIV_1:	SCLK/1 used as AHB clock
CRM_AHB_DIV_2:	SCLK/2 used as AHB clock
CRM_AHB_DIV_4:	SCLK/4 used as AHB clock
CRM_AHB_DIV_8:	SCLK/8 used as AHB clock
CRM_AHB_DIV_16:	SCLK/16 used as AHB clock
CRM_AHB_DIV_64:	SCLK/64 used as AHB clock
CRM_AHB_DIV_128:	SCLK/128 used as AHB clock
CRM_AHB_DIV_256:	SCLK/256 used as AHB clock
CRM_AHB_DIV_512:	SCLK/512 used as AHB clock

Example:

```
/* config ahbclk */
crm_ahb_div_set(CRM_AHB_DIV_1);
```

5.5.17 crm_apb1_div_set function

The table below describes the function crm_apb1_div_set.

Table 122. crm_apb1_div_set function

Name	Description
Function name	crm_apb1_div_set
Function prototype	void crm_apb1_div_set(crm_apb1_div_type value);
Function description	Configure APB1 clock division
Input parameter 1	value: indicates the division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_APB1_DIV_1: AHB/1 used as APB1 clock
 CRM_APB1_DIV_2: AHB/2 used as APB1 clock
 CRM_APB1_DIV_4: AHB/4 used as APB1 clock
 CRM_APB1_DIV_8: AHB/8 used as APB1 clock
 CRM_APB1_DIV_16: AHB/16 used as APB1 clock

Example:

```
/* config apb1clk */
crm_apb1_div_set(CRM_APB1_DIV_2);
```

5.5.18 crm_apb2_div_set function

The table below describes the function crm_apb2_div_set.

Table 123. crm_apb2_div_set function

Name	Description
Function name	crm_apb2_div_set
Function prototype	void crm_apb2_div_set(crm_apb2_div_type value);
Function description	Configure APB2 clock division
Input parameter 1	value: indicates the division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_APB2_DIV_1: AHB/1 used as APB2 clock
 CRM_APB2_DIV_2: AHB/2 used as APB2 clock
 CRM_APB2_DIV_4: AHB/4 used as APB2 clock
 CRM_APB2_DIV_8: AHB/8 used as APB2 clock
 CRM_APB2_DIV_16: AHB/16 used as APB2 clock

Example:

```
/* config apb2clk */
crm_apb2_div_set(CRM_APB2_DIV_2);
```

5.5.19 crm_hext_sclk_div_set function

The table below describes the function `crm_hext_sclk_div_set`.

Table 124. crm_hext_sclk_div_set function

Name	Description
Function name	<code>crm_hext_sclk_div_set</code>
Function prototype	<code>void crm_hext_sclk_div_set(crm_hext_sclk_div_type value);</code>
Function description	Configure HEXT clock division
Input parameter 1	value: division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_HEXT_SCLK_DIV_1: HEXT/1 used as system clock

CRM_HEXT_SCLK_DIV_2: HEXT/2 used as system clock

CRM_HEXT_SCLK_DIV_4: HEXT/4 used as system clock

CRM_HEXT_SCLK_DIV_8: HEXT/8 used as system clock

CRM_HEXT_SCLK_DIV_16: HEXT/16 used as system clock

CRM_HEXT_SCLK_DIV_32: HEXT/32 used as system clock

Example:

```
/* config hext to sysclk div */
crm_hext_sclk_div_set(CRM_HEXT_SCLK_DIV_1);
```

5.5.20 crm_hick_sclk_div_set function

The table below describes the function `crm_hick_sclk_div_set`.

Table 125. crm_hick_sclk_div_set function

Name	Description
Function name	<code>crm_hick_sclk_div_set</code>
Function prototype	<code>void crm_hick_sclk_div_set(crm_hick_sclk_div_type value);</code>
Function description	Configure HICK clock division
Input parameter 1	value: division factor
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_HICK_SCLK_DIV_1: HICK/1 used as system clock

CRM_HICK_SCLK_DIV_2: HICK/2 used as system clock

CRM_HICK_SCLK_DIV_4: HICK/4 used as system clock

CRM_HICK_SCLK_DIV_8: HICK/8 used as system clock

CRM_HICK_SCLK_DIV_16: HICK/16 used as system clock

Example:

```
/* config hick to sysclk div */
crm_hick_sclk_div_set(CRM_HICK_SCLK_DIV_1);
```

5.5.21 crm_clock_failure_detection_enable function

The table below describes the function crm_clock_failure_detection_enable.

Table 126. crm_clock_failure_detection_enable function

Name	Description
Function name	crm_clock_failure_detection_enable
Function prototype	void crm_clock_failure_detection_enable(confirm_state new_state);
Function description	Enable clock failure detection
Input parameter 1	new_state: TRUE or FALSE
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable clock failure detection */
crm_clock_failure_detection_enable(TRUE);
```

5.5.22 crm_battery_powered_domain_reset function

The table below describes the function crm_battery_powered_domain_reset.

Table 127. crm_battery_powered_domain_reset

Name	Description
Function name	crm_battery_powered_domain_reset
Function prototype	void crm_battery_powered_domain_reset(confirm_state new_state);
Function description	Reset battery powered domain
Input parameter 1	new_state: Reset (TRUE), Not reset (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

When it comes to resetting battery powered domain, it is usually necessary to reset battery powered domain through TRUE operation and then disable battery powered domain reset through FALSE operation after the completion of reset.

Example:

```
/* reset battery powered domain */
crm_battery_powered_domain_reset (TRUE);
```

5.5.23 crm_auto_step_mode_enable function

The table below describes the crm_auto_step_mode_enable.

Table 128. crm_battery_powered_domain_reset

Name	Description
Function name	crm_auto_step_mode_enable
Function prototype	void crm_auto_step_mode_enable(confirm_state new_state);
Function description	Enable auto step mode
Input parameter 1	new_state: Enable (TRUE), Disable (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable auto step mode */
crm_auto_step_mode_enable(TRUE);
```

5.5.24 crm_i2sf5_clock_select function

The table below describes the crm_i2sf5_clock_select.

Table 129. crm_i2sf5_clock_select

Name	Description
Function name	crm_i2sf5_clock_select
Function prototype	void crm_i2sf5_clock_select(crm_i2sf5_clock_source_type value);
Function description	i2sf5 peripheral clock selection
Input parameter 1	Value: clock source
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_I2SF5_CLOCK_SOURCE_SCLK: HICK/1 is used as system clock
 CRM_I2SF5_CLOCK_SOURCE_PLLP: HICK/2 is used as system clock
 CRM_I2SF5_CLOCK_SOURCE_HICK: HICK/4 is used as system clock
 CRM_I2SF5_CLOCK_SOURCE_EXTERNAL: HICK/8 is used as system clock

Example:

```
/* select hick as i2sf5 periph clock */
crm_i2sf5_clock_select (CRM_I2SF5_CLOCK_SOURCE_HICK);
```

5.5.25 crm_hick_sclk_frequency_select function

The table below describes the crm_hick_sclk_frequency_select.

Table 130. crm_hick_sclk_frequency_select

Name	Description
Function name	crm_hick_sclk_frequency_select
Function prototype	void crm_hick_sclk_frequency_select(crm_hick_sclk_frequency_type value);
Function description	When HICK is used as system clock, the system clock frequency can be set as 8M or 48M
Input parameter 1	Value: 8M or 48M HICK
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_I2SF5_CLOCK_SOURCE_SCLK:	HICK/1 is used as system clock
CRM_I2SF5_CLOCK_SOURCE_PLLP:	HICK/2 is used as system clock
CRM_I2SF5_CLOCK_SOURCE_HICK:	HICK/4 is used as system clock
CRM_I2SF5_CLOCK_SOURCE_EXTERNAL:	HICK/8 is used as system clock
CRM_HICK_SCLK_8MHZ:	8MHz HICK is used as system clock
CRM_HICK_SCLK_48MHZ:	48MHz HICK is used as system clock

Example:

```
/* config sysclk with hick 48mhz */
crm_hick_sclk_frequency_select (CRM_HICK_SCLK_48MHZ);
```


5.5.26 crm_usb_clock_source_select function

The table below describes the crm_usb_clock_source_select.

Table 131. crm_usb_clock_source_select

Name	Description
Function name	crm_usb_clock_source_select
Function prototype	void crm_usb_clock_source_select(crm_usb_clock_source_type value);
Function description	PLL or 48M HICK is selected as USB clock source
Input parameter 1	Value: PLL or HICK (48M)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_USB_CLOCK_SOURCE_PLLU: PLUL clock is used as USB clock source

CRM_USB_CLOCK_SOURCE_HICK: HICK clock is used as USB clock source

Example:

```
/* select hick48 as usb clock */
crm_usb_clock_source_select (CRM_USB_CLOCK_SOURCE_HICK);
```

5.5.27 crm_usb_phy12_clock_select function

The table below describes the crm_usb_phy12_clock_select.

Table 132. crm_usb_phy12_clock_select

Name	Description
Function name	crm_usb_phy12_clock_select
Function prototype	void crm_usb_phy12_clock_select(crm_usb_phy12_clock_type value);
Function description	High-speed USB PHY clock (12M) selection
Input parameter 1	value: PLLU/4 or HEXT/1
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_USB_PHY12_CLOCK_HEXT_DIV_1: HEXT/1 is used as USB PHY clock source

CRM_USB_PHY12_CLOCK_PLLU_DIV_4: PLLU/4 is used as USB PHY clock source

Example:

```
/* select hext div1 as usb phy clock */
crm_usb_phy12_clock_select (CRM_USB_PHY12_CLOCK_HEXT_DIV_1);
```

5.5.28 crm_pllu_output_set function

The table below describes the crm_pllu_output_set.

Table 133. crm_pllu_output_set

Name	Description
Function name	crm_pllu_output_set
Function prototype	void crm_pllu_output_set(confirm_state new_state);
Function description	Enable PLLU clock output
Input parameter 1	new_state: Enable (TRUE), disable (FALSE)
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable output pllu clock */
crm_pllu_output_set(TRUE);
```

5.5.29 crm_pll_config function

The table below describes the function crm_pll_config.

Table 134. crm_pll_config function

Name	Description
Function name	crm_pll_config
Function prototype	void crm_pll_config(crm_pll_clock_source_type clock_source, uint16_t pll_ns, uint16_t pll_ms, crm_pll_fr_type pll_fr);
Function description	Configure PLL clock source and frequency multiplication and division factor
Input parameter 1	clock_source: clock source for PLL frequency multiplication
Input parameter 2	pll_ns: frequency multiplication factor from 31 to 500
Input parameter3	pll_ms: pre-division frequency factor from 1 to 15
Input parameter4	pll_fp: post-division frequency factor
Output parameter	NA
Return value	NA
Required preconditions	PLL clock source must be enabled and stabilized before configuring and enabling PLL
Called functions	NA

Frequency multiplication formula: $PLLCLK = PLL \text{ input clock} / PLL_MS * PLL_NS / PLL_FP$

Requirements:

$2MHz \leq PLL \text{ input clock} / PLL_MS \leq 16MHz$

$500MHz \leq PLL \text{ input clock} / PLL_MS * PLL_NS \leq 1000MHz$

clock_source

CRM_PLL_SOURCE_HICK: HICK is selected as PLL clock source

CRM_PLL_SOURCE_HEXT: HEXT is selected as PLL clock source

pll_fp

CRM_PLL_FP_1: PLL/1
 CRM_PLL_FP_2: PLL/2
 CRM_PLL_FP_4: PLL/4
 CRM_PLL_FP_8: PLL/8
 CRM_PLL_FP_16: PLL/16
 CRM_PLL_FP_32: PLL/32

Example:

```
/* config pll clock resource */
crm_pll_config(CRM_PLL_SOURCE_HEXT, 96, 1, CRM_PLL_FP_8);
```

5.5.30 crm_pllu_div_set function

The table below describes the function crm_pllu_div_set.

Table 135. crm_pllu_div_set function

Name	Description
Function name	crm_pllu_div_set
Function prototype	void crm_pllu_div_set(crm_pll_fu_type pll_fu);
Function description	Select PLLU clock post-frequency division parameter
Input parameter 1	pll_fu: PLLU post-frequency division value
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_PLL_FU_11: VCO/11
 CRM_PLL_FU_13: VCO/13
 CRM_PLL_FU_12: VCO/12
 CRM_PLL_FU_14: VCO/14
 CRM_PLL_FU_16: VCO/16
 CRM_PLL_FU_18: VCO/18
 CRM_PLL_FU_20: VCO/20

Example:

```
/* config pllu div14 */
crm_pllu_div_set (CRM_PLL_FU_14);
```

5.5.31 crm_sysclk_switch function

The table below describes the function crm_sysclk_switch.

Table 136. crm_sysclk_switch function

Name	Description
Function name	crm_sysclk_switch
Function prototype	void crm_sysclk_switch(crm_sclk_type value);
Function description	Switch system clock source
Input parameter 1	value: indicates the clock source for system clock
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

value

CRM_SCLK_HICK: HICK as system clock

CRM_SCLK_HEXT: HEXT as system clock

CRM_SCLK_PLL: PLL as system clock

Example:

```
/* select pll as system clock source */
crm_sysclk_switch(CRM_SCLK_PLL);
```

5.5.32 crm_sysclk_switch_status_get function

The table below describes the function crm_sysclk_switch_status_get.

Table 137. crm_sysclk_switch_status_get function

Name	Description
Function name	crm_sysclk_switch_status_get
Function prototype	crm_sclk_type crm_sysclk_switch_status_get(void);
Function description	Get the clock source of system clock
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	crm_sclk_type: return value is the clock source of system clock
Required preconditions	NA
Called functions	NA

Example:

```
/* wait till pll is used as system clock source */
while(crm_sysclk_switch_status_get() != CRM_SCLK_PLL)
{
}
```

5.5.33 crm_clocks_freq_get function

The table below describes the function `crm_clocks_freq_get`.

Table 138. crm_clocks_freq_get function

Name	Description
Function name	<code>crm_clocks_freq_get</code>
Function prototype	<code>void crm_clocks_freq_get(crm_clocks_freq_type *clocks_struct);</code>
Function description	Get clock frequency
Input parameter 1	<code>clocks_struct</code> : <code>crm_clocks_freq_type</code> pointer, including clock frequency
Input parameter 2	NA
Output parameter	NA
Return value	<code>crm_sclk_type</code> : return the clock source for system clock
Required preconditions	NA
Called functions	NA

crm_clocks_freq_type

The `crm_clocks_freq_type` is defined in the `at32f402_405_crm.h`:

typedef struct

{

uint32_t sclk_freq;

uint32_t ahb_freq;

uint32_t apb2_freq;

uint32_t apb1_freq;

} `crm_clocks_freq_type`;

sclk_freq

Get the system clock frequency, in Hz

ahb_freq

Get the clock frequency of AHB, in Hz

apb2_freq

Get the clock frequency of APB2, in Hz

apb1_freq

Get the clock frequency of APB1, in Hz

Example:

```
/* get frequency */
crm_clocks_freq_type clocks_struct;
crm_clocks_freq_get(&clocks_struct);
```

5.5.34 crm_clock_out_set function

The table below describes the function crm_clock_out_set.

Table 139. crm_clock_out_set function

Name	Description
Function name	crm_clock_out_set
Function prototype	void crm_clock_out_set(crm_clkout_select_type clkout);
Function description	Select clock source output on clkout pin
Input parameter 1	clkout: clock source output on clkout pin
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

clkout

Select clock source output on the clkout1 pin.

CRM_CLKOUT_SCLK: SCLK output on clkout pin

CRM_CLKOUT_HEXT: HEXT output on clkout pin

CRM_CLKOUT_PLL: PLL output on clkout pin

CRM_CLKOUT_USB: USB output on clkout pin

CRM_CLKOUT_ADC: ADC output on clkout pin

CRM_CLKOUT_HICK: HICK output on clkout pin

CRM_CLKOUT_LICK: LICK output on clkout pin

CRM_CLKOUT_LEXT: LEXT output on clkout pin

Example:

```
/* config clkout output hick */
crm_clock_out_set(CRM_CLKOUT_HICK);
```

5.5.35 crm_clkout_div_set function

The table below describes the function crm_clkout_div_set.

Table 140. crm_clkout_div_set function

Name	Description
Function name	crm_clkout_div_set
Function prototype	void crm_clkout_div_set(crm_clkout_div1_type div1, crm_clkout_div2_type div2)
Function description	Clock frequency division on clockout pin
Input parameter 1	div1: divider 1 clock frequency division
Input parameter 2	div2: divider 2 clock frequency division
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

div1

Divider 1 clock frequency division

CRM_CLKOUT_DIV1_1: divided by 1
 CRM_CLKOUT_DIV1_2: Divided by 2
 CRM_CLKOUT_DIV1_3: Divided by 3
 CRM_CLKOUT_DIV1_4: Divided by 4
 CRM_CLKOUT_DIV1_5: Divided by 5

div2

Divider 2 clock frequency division

CRM_CLKOUT_DIV2_1: divided by 1
 CRM_CLKOUT_DIV2_2: divided by 2
 CRM_CLKOUT_DIV2_4: divided by 4
 CRM_CLKOUT_DIV2_8: divided by 8
 CRM_CLKOUT_DIV2_16: divided by 16
 CRM_CLKOUT_DIV2_64: divided by 64
 CRM_CLKOUT_DIV2_128: divided by 128
 CRM_CLKOUT_DIV2_256: divided by 256
 CRM_CLKOUT_DIV2_512: divided by 512

Example:

```
/* config clkout div */
crm_clkout_div_set (CRM_CLKOUT_DIV1_1, CRM_CLKOUT_DIV2_8);
```

5.5.36 crm_interrupt_enable function

The table below describes the function crm_interrupt_enable.

Table 141. crm_interrupt_enable function

Name	Description
Function name	crm_interrupt_enable
Function prototype	void crm_interrupt_enable(uint32_t crm_int, confirm_state new_state);
Function description	Enable interrupts
Input parameter 1	crm_int: indicates the selected interrupt
Input parameter 2	new_state: Enable (TRUE), disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

crm_int

CRM_LICK_STABLE_INT: LICK stable interrupt
 CRM_LEXT_STABLE_INT: LEXT stable interrupt
 CRM_HICK_STABLE_INT: HICK stable interrupt
 CRM_HEXT_STABLE_INT: HEXT stable interrupt
 CRM_PLL_STABLE_INT: PLL stable interrupt
 CRM_CLOCK_FAILURE_INT: Clock failure interrupt

Example:

```
/* enable pll stable interrupt */
crm_interrupt_enable (CRM_PLL_STABLE_INT);
```

5.5.37 crm_pll_parameter_calculate function

The table below describes the function `crm_pll_parameter_calculate`.

Table 142. crm_pll_parameter_calculate function

Name	Description
Function name	<code>crm_pll_parameter_calculate</code>
Function prototype	<code>error_status crm_pll_parameter_calculate(crm_pll_clock_source_type pll_rcs, uint32_t target_sclk_freq, uint16_t *ret_ms, uint16_t *ret_ns, uint16_t *ret_fr);</code>
Function description	PLL parameter auto calculation
Input parameter 1	<code>pll_rcs</code> : pll input clock source
Input parameter 2	<code>target_sclk_freq</code> : target clock frequency multiplication, for example, for 200 MHz, this parameter can be <code>target_sclk_freq=200000000</code> .
Output parameter1	<code>ret_ms</code> : return <code>pll_ms</code> parameter
Output parameter2	<code>ret_ns</code> : return <code>pll_ns</code> parameter
Output parameter3	<code>ret_fr</code> : return <code>pll_fr</code> parameter
Return value	<code>error_status</code> : Calculation status. SUCCESS: the calculated result equals the target clock PLL parameter ERROR: the calculated result is close to the target clock PLL parameter
Required preconditions	NA
Called functions	NA

Example:

```
/* pll parameter calculate automatic */
uint16_t pll_ms = 0, pll_ns = 0, pll_fr = 0;
crm_pll_parameter_calculate (CRM_PLL_SOURCE_HEXT, 200000000, &pll_ms, &pll_ns, &pll_fr);
```


5.6 Debug

The DEBUG register structure `debug_type` is defined in the “at32f402_405_debug.h”:

```
/**
 * @brief type define debug register all
 */
typedef struct
{
    ...

} debug_type;
```

The table below gives a list of the DEBUG registers.

Table 143. Summary of DEBUG registers

Register	Description
idcode	Device ID
ctrl	Control register
apb1_pause	APB1 pause control register
apb2_pause	APB2 pause control register

The table below gives a list of DEBUG library functions.

Table 144. Summary of DEBUG library functions

Function name	Description
debug_device_id_get	Read device idcode
debug_low_power_mode_set	Low-power debug mode configuration
debug_apb1_periph_mode_set	apb1 debug mode configuration
debug_apb2_periph_mode_set	apb2 debug mode configuration

5.6.1 debug_device_id_get function

The table below describes the function `debug_device_id_get`.

Table 145. debug_device_id_get function

Name	Description
Function name	<code>debug_device_id_get</code>
Function prototype	<code>uint32_t debug_device_id_get(void);</code>
Function description	Read device idcode
Input parameter 1	NA
Input parameter 2	NA
Output parameter	NA
Return value	Return 32-bit idcode
Required preconditions	NA
Called functions	NA

Example:

```
/* get idcode */
```

```
uint32_t idcode = 0;
idcode = debug_device_id_get();
```

5.6.2 debug_low_power_mode_set function

The table below describes the function debug_low_power_mode_set.

Table 146. debug_low_power_mode_set function

Name	Description
Function name	debug_low_power_mode_set
Function prototype	void debug_low_power_mode_set(uint32_t low_power_mode, confirm_state new_state)
Function description	Debug in low-power mode
Input parameter 1	periph_debug_mode: Selected peripheral or mode
Input parameter 2	new_state: enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

low_power_mode

Select a low-power mode to debug

DEBUG_SLEEP: DEBUG in SLEEP mode

DEBUG_DEEPSLEEP: DEBUG in DEEPSLEEP mode

DEBUG_STANDBY: DEBUG in STANDBY mode

Example:

```
/* enable sleep debug mode */
debug_low_power_mode_set (DEBUG_SLEEP, TRUE);
```

5.6.3 debug_apb1_periph_mode_set function

The table below describes the function debug_apb1_periph_mode_set.

Table 147. debug_apb1_periph_mode_set function

Name	Description
Function name	debug_apb1_periph_mode_set
Function prototype	void debug_apb1_periph_mode_set(uint32_t apb1_periph, confirm_state new_state)
Function description	Select a peripheral to debug
Input parameter 1	apb1_periph: Select a peripheral
Input parameter 2	new_state: enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

apb1_periph

Select a peripheral or mode to debug

DEBUG_WDT_PAUSE:	Watchdog pause control bit
DEBUG_WWDT_PAUSE:	Window watchdog pause control bit
DEBUG_TMR2_PAUSE:	TMR2 pause control bit
DEBUG_TMR3_PAUSE:	TMR3 pause control bit
DEBUG_TMR4_PAUSE:	TMR4 pause control bit
DEBUG_TMR6_PAUSE:	TMR6 pause control bit
DEBUG_TMR7_PAUSE:	TMR7 pause control bit
DEBUG_TMR13_PAUSE:	TMR13 pause control bit
DEBUG_TMR14_PAUSE:	TMR14 pause control bit
DEBUG_I2C1_SMBUS_TIMEOUT:	I2C1 SMBUS TIMEOUT pause control bit
DEBUG_I2C2_SMBUS_TIMEOUT:	I2C2 SMBUS TIMEOUT pause control bit
DEBUG_I2C3_SMBUS_TIMEOUT:	I2C3 SMBUS TIMEOUT pause control bit
DEBUG_CAN1_PAUSE:	CAN1 receive register pause control bit
DEBUG_ERTC_PAUSE:	ERTC pause control bit
DEBUG_ERTC_512_PAUSE:	ERTC 512Hz pulse output pause control bit

Example:

```
/* enable tmr2 debug mode */
debug_apb1_periph_mode_set (DEBUG_TMR2_PAUSE, TRUE);
```

5.6.4 debug_apb2_periph_mode_set function

The table below describes the function debug_apb2_periph_mode_set.

Table 148. debug_apb2_periph_mode_set function

Name	Description
Function name	debug_apb2_periph_mode_set
Function prototype	void debug_apb2_periph_mode_set(uint32_t apb2_periph, confirm_state new_state)
Function description	Select a peripheral to debug
Input parameter 1	apb2_periph: Selected peripheral or mode
Input parameter 2	new_state: enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

apb2_periph

Select a peripheral or mode to debug

DEBUG_TMR9_PAUSE:	TMR9 pause control bit
DEBUG_TMR10_PAUSE:	TMR10 pause control bit
DEBUG_TMR11_PAUSE:	TMR11 pause control bit
DEBUG_TMR1_PAUSE:	TMR1 pause control bit

Example:

```
/* enable tmr1 debug mode */
debug_apb2_periph_mode_set (DEBUG_TMR1_PAUSE, TRUE);
```

5.7 DMA controller

The DMA register structure `dma_type` is defined in the “at32f402_405_dma.h”:

```
/**
 * @brief type define dma register
 */
typedef struct
{
    ...

} dma_type;
```

DMA channel register structure `dma_channel_type` is defined in the “at32f402_405_dma.h”:

```
/**
 * @brief type define dma channel register all
 */
typedef struct
{
    ...

} dma_channel_type;
```

The table below gives a list of the DMA registers.

Table 149. Summary of DMA registers

Register	Description
<code>dma_sts</code>	DMA status register
<code>dma_clr</code>	DMA status clear register
<code>dma_c1ctrl</code>	DMA channel 1 configuration register
<code>dma_c1dtcnt</code>	DMA channel 1 number of data register
<code>dma_c1paddr</code>	DMA channel 1 peripheral address register
<code>dma_c1maddr</code>	DMA channel 1 memory address register
<code>dma_c2ctrl</code>	DMA channel 2 configuration register
<code>dma_c2dtcnt</code>	DMA channel 2 number of data register
<code>dma_c2paddr</code>	DMA channel 2 peripheral address register
<code>dma_c2maddr</code>	DMA channel 2 memory address register
<code>dma_c3ctrl</code>	DMA channel 3 configuration register
<code>dma_c3dtcnt</code>	DMA channel 3 number of data register
<code>dma_c3paddr</code>	DMA channel 3 peripheral address register
<code>dma_c3maddr</code>	DMA channel 3 memory address register
<code>dma_c4ctrl</code>	DMA channel 4 configuration register
<code>dma_c4dtcnt</code>	DMA channel 4 number of data register
<code>dma_c4paddr</code>	DMA channel 4 peripheral address register
<code>dma_c4maddr</code>	DMA channel 4 memory address register
<code>dma_c5ctrl</code>	DMA channel 5 configuration register

Register	Description
dma_c5dtcnt	DMA channel 5 number of data register
dma_c5paddr	DMA channel 5 peripheral address register
dma_c5maddr	DMA channel 5 memory address register
dma_c6ctrl	DMA channel 6 configuration register
dma_c6dtcnt	DMA channel 6 number of data register
dma_c6paddr	DMA channel 6 peripheral address register
dma_c6maddr	DMA channel 6 memory address register
dma_c7ctrl	DMA channel 7 configuration register
dma_c7dtcnt	DMA channel 7 number of data register
dma_c7paddr	DMA channel 7 peripheral address register
dma_c7maddr	DMA channel 7 memory address register
dma_muxsel	DMAMUX enable register
dma_muxc1ctrl	DMAMUX channel 1 control register
dma_muxc2ctrl	DMAMUX channel 2 control register
dma_muxc3ctrl	DMAMUX channel 3 control register
dma_muxc4ctrl	DMAMUX channel 4 control register
dma_muxc5ctrl	DMAMUX channel 5 control register
dma_muxc6ctrl	DMAMUX channel 6 control register
dma_muxc7ctrl	DMAMUX channel 7 control register
dma_muxg1ctrl	DMAMUX request generator 1 control register
dma_muxg2ctrl	DMAMUX request generator 2 control register
dma_muxg3ctrl	DMAMUX request generator 3 control register
dma_muxg4ctrl	DMAMUX request generator 4 control register
dma_muxsyncsts	DMAMUX synchronous status register
dma_muxsyncclr	DMAMUX synchronous status clear register
dma_muxgsts	DMAMUX request generator status register
dma_muxgclr	DMAMUX request generator status clear register

Table 169 gives a list of DMA library functions.

Table 150. Summary of DMA library functions

Function name	Description
dma_default_para_init	Initialize the parameters of the dma_init_struct
dma_init	Initialize the selected DMA channel
dma_reset	Reset the selected DMA channel
dma_data_number_set	Set the number of data transfer of a given channel
dma_data_number_get	Get the number of data transfer of a given channel
dma_interrupt_enable	Enable DMA channel interrupt
dma_channel_enable	Enable DMA channel
dma_flexible_config	Configure flexible DMA request mapping
dma_flag_get	Get the flag of DMA channels
dma_flag_clear	Clear the flag of DMA channels
dmamux_enable	Enable DMAMUX

dmamux_init	Initialize DMAMUX
dmamux_sync_default_para_init	Initialize DMAMUX synchronous module
dmamux_sync_config	Configure DMAMUX synchronous module
dmamux_generator_default_para_init	Initialize DMAMUX request generator
dmamux_generator_config	Configure DMAMUX request generator
dmamux_sync_interrupt_enable	Enable DMAMUX synchronous module interrupt
dmamux_generator_interrupt_enable	Enable DMAMUX request generator interrupt
dmamux_sync_flag_get	Get the flag of DMAMUX synchronous module
dmamux_sync_flag_clear	Clear the flag of DMAMUX synchronous module
dmamux_generator_flag_get	Get the flag of DMAMUX request generator
dmamux_generator_flag_clear	Clear the flag of DMAMUX request generator

5.7.1 dma_default_para_init function

The table below describes the function dma_default_para_init.

Table 151. dma_default_para_init function

Name	Description
Function name	dma_default_para_init
Function prototype	void dma_default_para_init(dma_init_type* dma_init_struct);
Function description	Initialize the parameters of the dma_init_struct
Input parameter 1	dma_init_struct: dma_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of the dma_init_struct members.

Table 152. dma_init_struct default values

Member	Default values
peripheral_base_addr	0x0
memory_base_addr	0x0
direction	DMA_DIR_PERIPHERAL_TO_MEMORY
buffer_size	0x0
peripheral_inc_enable	FALSE
memory_inc_enable	FALSE
peripheral_data_width	DMA_PERIPHERAL_DATA_WIDTH_BYTE
memory_data_width	DMA_MEMORY_DATA_WIDTH_BYTE
loop_mode_enable	FALSE
priority	DMA_PRIORITY_LOW

Example:

```
/* dma init config with its default value */
dma_init_type dma_init_struct = {0};
dma_default_para_init(&dma_init_struct);
```

5.7.2 dma_init function

The table below describes the function dma_init.

Table 153. dma_init function

Name	Description
Function name	dma_init
Function prototype	void dma_init(dma_channel_type* dma_channel, dma_init_type* dma_init_struct)
Function description	Initialize the selected DMA channel
Input parameter 1	dma_channel: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	dma_init_struct: dma_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_init_type structure

The dma_init_type is defined in the at32f402_405_dma.h:

typedef struct

```
{
    uint32_t          peripheral_base_addr;
    uint32_t          memory_base_addr;
    dma_dir_type      direction;
    uint16_t          buffer_size;
    confirm_state     peripheral_inc_enable;
    confirm_state     memory_inc_enable;
    dma_peripheral_data_size_type peripheral_data_width;
    dma_memory_data_size_type memory_data_width;
    confirm_state     loop_mode_enable;
    dma_priority_level_type priority;
} dma_init_type;
```

peripheral_base_addr

Set the peripheral address of a DMA channel

memory_base_addr

Set the memory address of a DMA channel

direction

Set the transfer direction of a DMA channel

DMA_DIR_PERIPHERAL_TO_MEMORY: Peripheral to memory
 DMA_DIR_MEMORY_TO_PERIPHERAL: Memory to peripheral
 DMA_DIR_MEMORY_TO_MEMORY: Memory to memory

buffer_size

Set the number of data transfer of a DMA channel

peripheral_inc_enable

Enable/disable DMA channel peripheral address auto increment

FALSE: Peripheral address is not incremented

TRUE: Peripheral address is incremented

memory_inc_enable

Enable/disable DMA channel memory address auto increment

FALSE: Memory address is not incremented

TRUE: Memory address is incremented

peripheral_data_width

Set DMA peripheral data width

DMA_PERIPHERAL_DATA_WIDTH_BYTE: Byte

DMA_PERIPHERAL_DATA_WIDTH_HALFWORD: Half-word

DMA_PERIPHERAL_DATA_WIDTH_WORD: Word

memory_data_width

Set DMA memory data width

DMA_MEMORY_DATA_WIDTH_BYTE: Byte

DMA_MEMORY_DATA_WIDTH_HALFWORD: Half-word

DMA_MEMORY_DATA_WIDTH_WORD: Word

loop_mode_enable

Set DMA loop mode

FALSE: DMA single mode

TRUE: DMA loop mode

priority

Set DMA channel priority

DMA_PRIORITY_LOW: Low

DMA_PRIORITY_MEDIUM: Medium

DMA_PRIORITY_HIGH: High

DMA_PRIORITY_VERY_HIGH: Very high

Example:

```
dma_init_type dma_init_struct = {0};
/* dma2 channel1 configuration */
dma_init_struct.buffer_size = BUFFER_SIZE;
dma_init_struct.direction = DMA_DIR_MEMORY_TO_PERIPHERAL;
dma_init_struct.memory_base_addr = (uint32_t)src_buffer;
dma_init_struct.memory_data_width = DMA_MEMORY_DATA_WIDTH_HALFWORD;
dma_init_struct.memory_inc_enable = TRUE;
dma_init_struct.peripheral_base_addr = (uint32_t)0x4001100C;
dma_init_struct.peripheral_data_width = DMA_PERIPHERAL_DATA_WIDTH_HALFWORD;
dma_init_struct.peripheral_inc_enable = FALSE;
dma_init_struct.priority = DMA_PRIORITY_MEDIUM;
dma_init_struct.loop_mode_enable = FALSE;
dma_init(DMA2_CHANNEL1, &dma_init_struct);
```


5.7.3 dma_reset function

The table below describes the function dma_reset.

Table 154. dma_reset function

Name	Description
Function name	dma_reset
Function prototype	void dma_reset(dma_channel_type* dma_channel);
Function description	Reset the selected DMA channel
Input parameter 1	dma_channel: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reset dma2 channel1 */
dma_reset(DMA2_CHANNEL1);
```

5.7.4 dma_data_number_set function

The table below describes the function dma_data_number_set.

Table 155. dma_data_number_set function

Name	Description
Function name	dma_data_number_set
Function prototype	void dma_data_number_set(dma_channel_type* dma_channel, uint16_t data_number);
Function description	Set the number of data transfer of the selected DMA channel
Input parameter 1	dma_channel: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	data_number: indicates the number of data transfer, up to 65535
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* set dma2 channel1 data count is 0x100*/
dma_data_number_set(DMA2_CHANNEL1, 0x100);
```

5.7.5 dma_data_number_get function

The table below describes the function dma_data_number_get.

Table 156. dma_data_number_get function

Name	Description
Function name	dma_data_number_get
Function prototype	uint16_t dma_data_number_get(dma_channel_type* dmax_channely);
Function description	Get the number of data transfer of the selected DMA channel
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Output parameter	NA
Return value	Get the number of data transfer of a DMA channel
Required preconditions	NA
Called functions	NA

Example:

```
/* get dma2 channel1 data count*/
uint16_t data_counter;
data_counter = dma_data_number_get(DMA2_CHANNEL1);
```

5.7.6 dma_interrupt_enable function

The table below describes the function dma_interrupt_enable.

Table 157. dma_interrupt_enable function

Name	Description
Function name	dma_interrupt_enable
Function prototype	void dma_interrupt_enable(dma_channel_type* dmax_channely, uint32_t dma_int, confirm_state new_state);
Function description	Enable DMA channels interrupt
Input parameter 1	dmax_channely: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	dma_int: interrupt source selection
Input parameter3	new_state: interrupt enable/disable
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_int

Select DMA interrupt source

DMA_FDT_INT: Transfer complete interrupt

DMA_HDT_INT: Half transfer complete interrupt

DMA_DTERR_INT: Transfer error interrupt

new_state

Enable or disable DMA channel interrupt

FALSE: Disabled

TRUE: Enabled

Example:

```
/* enable dma2 channel1 transfer full data interrupt */  
dma_interrupt_enable(DMA2_CHANNEL1, DMA_FDT_INT, TRUE);
```

5.7.7 dma_channel_enable function

The table below describes the function dma_channel_enable.

Table 158. dma_channel_enable function

Name	Description
Function name	dma_channel_enable
Function prototype	void dma_channel_enable(dma_channel_type* dma_channel, confirm_state new_state);
Function description	Enable the selected DMA channel
Input parameter 1	dma_channel: DMAx_CHANNELy defines a DMA channel number, x=1 or 2, y=1...7
Input parameter 2	new_state: Enable or disable the selected DMA channel
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Enable or disable DMA channels

FALSE: Disabled

TRUE: Enabled

Example:

```
/* enable dma channel */  
dma_channel_enable(DMA2_CHANNEL1, TRUE);
```

5.7.8 dma_flag_get function

The table below describes the function dma_flag_get.

Table 159. dma_flag_get function

Name	Description
Function name	dma_flag_get
Function prototype	flag_status dma_flag_get(uint32_t dma_flag);
Function description	Get the flag of the selected DMA channel
Input parameter 1	dma_flag: select the desired flag
Output parameter	NA
Return value	flag_status: indicates whether the desired flag is set or not
Required preconditions	NA
Called functions	NA

dma_flag

The dma_flag is used for flag section, including:

DMA1_GL1_FLAG:	DMA1 channel 1 global flag
DMA1_FDT1_FLAG:	DMA1 channel 1 transfer complete flag
DMA1_HDT1_FLAG:	DMA1 channel 1 half transfer complete flag
DMA1_DTERR1_FLAG:	DMA1 channel 1 transfer error flag
DMA1_GL2_FLAG:	DMA1 channel 2 global flag
DMA1_FDT2_FLAG:	DMA1 channel 2 transfer complete flag
DMA1_HDT2_FLAG:	DMA1 channel 2 half transfer complete flag
DMA1_DTERR2_FLAG:	DMA1 channel 2 transfer error flag
DMA1_GL3_FLAG:	DMA1 channel 3 global flag
DMA1_FDT3_FLAG:	DMA1 channel 3 transfer complete flag
DMA1_HDT3_FLAG:	DMA1 channel 3 half transfer complete flag
DMA1_DTERR3_FLAG:	DMA1 channel 3 transfer error flag
DMA1_GL4_FLAG:	DMA1 channel 4 global flag
DMA1_FDT4_FLAG:	DMA1 channel 4 transfer complete flag
DMA1_HDT4_FLAG:	DMA1 channel 4 half transfer complete flag
DMA1_DTERR4_FLAG:	DMA1 channel 4 transfer error flag
DMA1_GL5_FLAG:	DMA1 channel 5 global flag
DMA1_FDT5_FLAG:	DMA1 channel 5 transfer complete flag
DMA1_HDT5_FLAG:	DMA1 channel 5 half transfer complete flag
DMA1_DTERR5_FLAG:	DMA1 channel 5 transfer error flag
DMA1_GL6_FLAG:	DMA1 channel 6 global flag
DMA1_FDT6_FLAG:	DMA1 channel 6 transfer complete flag
DMA1_HDT6_FLAG:	DMA1 channel 6 half transfer complete flag
DMA1_DTERR6_FLAG:	DMA1 channel 6 transfer error flag
DMA1_GL7_FLAG:	DMA1 channel 7 global flag
DMA1_FDT7_FLAG:	DMA1 channel 7 transfer complete flag
DMA1_HDT7_FLAG:	DMA1 channel 7 half transfer complete flag
DMA1_DTERR7_FLAG:	DMA1 channel 7 transfer error flag
DMA2_GL1_FLAG:	DMA2 channel 1 global flag
DMA2_FDT1_FLAG:	DMA2 channel 1 transfer complete flag
DMA2_HDT1_FLAG:	DMA2 channel 1 half transfer complete flag
DMA2_DTERR1_FLAG:	DMA2 channel 1 transfer error flag
DMA2_GL2_FLAG:	DMA2 channel 2 global flag
DMA2_FDT2_FLAG:	DMA2 channel 2 transfer complete flag
DMA2_HDT2_FLAG:	DMA2 channel 2 half transfer complete flag
DMA2_DTERR2_FLAG:	DMA2 channel 2 transfer error flag
DMA2_GL3_FLAG:	DMA2 channel 3 global flag
DMA2_FDT3_FLAG:	DMA2 channel 3 transfer complete flag
DMA2_HDT3_FLAG:	DMA2 channel 3 half transfer complete flag
DMA2_DTERR3_FLAG:	DMA2 channel 3 transfer error flag
DMA2_GL4_FLAG:	DMA2 channel 4 global flag
DMA2_FDT4_FLAG:	DMA2 channel 4 transfer complete flag
DMA2_HDT4_FLAG:	DMA2 channel 4 half transfer complete flag
DMA2_DTERR4_FLAG:	DMA2 channel 4 transfer error flag
DMA2_GL5_FLAG:	DMA2 channel 5 global flag

DMA2_FDT5_FLAG:	DMA2 channel 5 transfer complete flag
DMA2_HDT5_FLAG:	DMA2 channel 5 half transfer complete flag
DMA2_DTERR5_FLAG:	DMA2 channel 5 transfer error flag
DMA2_GL6_FLAG:	DMA2 channel 6 global flag
DMA2_FDT6_FLAG:	DMA2 channel 6 transfer complete flag
DMA2_HDT6_FLAG:	DMA2 channel 6 half transfer complete flag
DMA2_DTERR6_FLAG:	DMA2 channel 6 transfer error flag
DMA2_GL7_FLAG:	DMA2 channel 7 global flag
DMA2_FDT7_FLAG:	DMA2 channel 7 transfer complete flag
DMA2_HDT7_FLAG:	DMA2 channel 7 half transfer complete flag
DMA2_DTERR7_FLAG:	DMA2 channel 7 transfer error flag

flag_status

RESET: Flag is reset

SET: Flag is set

Example:

```
if(dma_flag_get(DMA2_FDT1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
}
```

5.7.9 dma_flag_clear function

The table below describes the function dma_flag_clear.

Table 160. dma_flag_clear function

Name	Description
Function name	dma_flag_clear
Function prototype	void dma_flag_clear(uint32_t dmax_flag);
Function description	Clear the selected flag
Input parameter 1	dmax_flag: a flag that needs to be cleared
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dmax_flag

dmax_flag is used to select the desired flag, including:

DMA1_GL1_FLAG:	DMA1 channel 1 global flag
DMA1_FDT1_FLAG:	DMA1 channel 1 transfer complete flag
DMA1_HDT1_FLAG:	DMA1 channel 1 half transfer complete flag
DMA1_DTERR1_FLAG:	DMA1 channel 1 transfer error flag
DMA1_GL2_FLAG:	DMA1 channel 2 global flag
DMA1_FDT2_FLAG:	DMA1 channel 2 transfer complete flag
DMA1_HDT2_FLAG:	DMA1 channel 2 half transfer complete flag

DMA1_DTERR2_FLAG:	DMA1 channel 2 transfer error flag
DMA1_GL3_FLAG:	DMA1 channel 3 global flag
DMA1_FDT3_FLAG:	DMA1 channel 3 transfer complete flag
DMA1_HDT3_FLAG:	DMA1 channel 3 half transfer complete flag
DMA1_DTERR3_FLAG:	DMA1 channel 3 transfer error flag
DMA1_GL4_FLAG:	DMA1 channel 4 global flag
DMA1_FDT4_FLAG:	DMA1 channel 4 transfer complete flag
DMA1_HDT4_FLAG:	DMA1 channel 4 half transfer complete flag
DMA1_DTERR4_FLAG:	DMA1 channel 4 transfer error flag
DMA1_GL5_FLAG:	DMA1 channel 5 global flag
DMA1_FDT5_FLAG:	DMA1 channel 5 transfer complete flag
DMA1_HDT5_FLAG:	DMA1 channel 5 half transfer complete flag
DMA1_DTERR5_FLAG:	DMA1 channel 5 transfer error flag
DMA1_GL6_FLAG:	DMA1 channel 6 global flag
DMA1_FDT6_FLAG:	DMA1 channel 6 transfer complete flag
DMA1_HDT6_FLAG:	DMA1 channel 6 half transfer complete flag
DMA1_DTERR6_FLAG:	DMA1 channel 6 transfer error flag
DMA1_GL7_FLAG:	DMA1 channel 7 global flag
DMA1_FDT7_FLAG:	DMA1 channel 7 transfer complete flag
DMA1_HDT7_FLAG:	DMA1 channel 7 half transfer complete flag
DMA1_DTERR7_FLAG:	DMA1 channel 7 transfer error flag
DMA2_GL1_FLAG:	DMA2 channel 1 global flag
DMA2_FDT1_FLAG:	DMA2 channel 1 transfer complete flag
DMA2_HDT1_FLAG:	DMA2 channel 1 half transfer complete flag
DMA2_DTERR1_FLAG:	DMA2 channel 1 transfer error flag
DMA2_GL2_FLAG:	DMA2 channel 2 global flag
DMA2_FDT2_FLAG:	DMA2 channel 2 transfer complete flag
DMA2_HDT2_FLAG:	DMA2 channel 2 half transfer complete flag
DMA2_DTERR2_FLAG:	DMA2 channel 2 transfer error flag
DMA2_GL3_FLAG:	DMA2 channel 3 global flag
DMA2_FDT3_FLAG:	DMA2 channel 3 transfer complete flag
DMA2_HDT3_FLAG:	DMA2 channel 3 half transfer complete flag
DMA2_DTERR3_FLAG:	DMA2 channel 3 transfer error flag
DMA2_GL4_FLAG:	DMA2 channel 4 global flag
DMA2_FDT4_FLAG:	DMA2 channel 4 transfer complete flag
DMA2_HDT4_FLAG:	DMA2 channel 4 half transfer complete flag
DMA2_DTERR4_FLAG:	DMA2 channel 4 transfer error flag
DMA2_GL5_FLAG:	DMA2 channel 5 global flag
DMA2_FDT5_FLAG:	DMA2 channel 5 transfer complete flag
DMA2_HDT5_FLAG:	DMA2 channel 5 half transfer complete flag
DMA2_DTERR5_FLAG:	DMA2 channel 5 transfer error flag
DMA2_GL6_FLAG:	DMA2 channel 6 global flag
DMA2_FDT6_FLAG:	DMA2 channel 6 transfer complete flag
DMA2_HDT6_FLAG:	DMA2 channel 6 half transfer complete flag
DMA2_DTERR6_FLAG:	DMA2 channel 6 transfer error flag

DMA2_GL7_FLAG: DMA2 channel 7 global flag
 DMA2_FDT7_FLAG: DMA2 channel 7 transfer complete flag
 DMA2_HDT7_FLAG: DMA2 channel 7 half transfer complete flag
 DMA2_DTERR7_FLAG: DMA2 channel 7 transfer error flag

Example:

```

if(dma_flag_get(DMA2_FDT1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
    dma_flag_clear(DMA2_FDT1_FLAG);
}
  
```

5.7.10 dma_flexible_config function

The table below describes the function dma_flexible_enable.

Table 161. dma_flexible_config function

Name	Description
Function name	dma_flexible_config
Function prototype	void dma_flexible_config(dma_type* dma_x, dmamux_channel_type *dmamux_channelx, dmamux_request_id_sel_type dmamux_req_sel);
Function description	Configure DMAMUX
Input parameter 1	dma_x: DMAx, x=1 or 2
Input parameter 2	dmamux_channelx :DMAMUX channel selection, x=1...7
Input parameter3	dmamux_req_sel :DMAMUX channel request ID
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dmamux_channelx

DMAMUX channel selection, including:

DMA1MUX_CHANNEL1
 DMA1MUX_CHANNEL2
 DMA1MUX_CHANNEL3
 DMA1MUX_CHANNEL4
 DMA1MUX_CHANNEL5
 DMA1MUX_CHANNEL6
 DMA1MUX_CHANNEL7
 DMA2MUX_CHANNEL1
 DMA2MUX_CHANNEL2
 DMA2MUX_CHANNEL3
 DMA2MUX_CHANNEL4

DMA2MUX_CHANNEL5

DMA2MUX_CHANNEL6

DMA2MUX_CHANNEL7

dmamux_req_sel

Table 181 shows the DMAMUX channel request ID:

Table 162. DMAMUX channel request source ID

Request source ID	Description	Request source ID	Description
0x01	DMAMUX_DMAREQ_ID_REQ_G1	0x02	DMAMUX_DMAREQ_ID_REQ_G2
0x03	DMAMUX_DMAREQ_ID_REQ_G3	0x04	DMAMUX_DMAREQ_ID_REQ_G4
0x05	DMAMUX_DMAREQ_ID_ADC1	0x24	DMAMUX_DMAREQ_ID_ADC2
0x25	DMAMUX_DMAREQ_ID_ADC3	0x06	DMAMUX_DMAREQ_ID_DAC1
0x29	DMAMUX_DMAREQ_ID_DAC2	0x08	DMAMUX_DMAREQ_ID_TMR6_OVERFLOW
0x09	DMAMUX_DMAREQ_ID_TMR7_OVERFLOW	0x0A	DMAMUX_DMAREQ_ID_SPI1_RX
0x0B	DMAMUX_DMAREQ_ID_SPI1_TX	0x0C	DMAMUX_DMAREQ_ID_SPI2_RX
0x0D	DMAMUX_DMAREQ_ID_SPI2_TX	0x0E	DMAMUX_DMAREQ_ID_SPI3_RX
0x0F	DMAMUX_DMAREQ_ID_SPI3_TX	0x6A	DMAMUX_DMAREQ_ID_SPI4_RX
0x6B	DMAMUX_DMAREQ_ID_SPI4_TX	0x6E	DMAMUX_DMAREQ_ID_I2S2_EXT_RX
0x6F	DMAMUX_DMAREQ_ID_I2S2_EXT_TX	0x70	DMAMUX_DMAREQ_ID_I2S3_EXT_RX
0x71	DMAMUX_DMAREQ_ID_I2S3_EXT_TX	0x10	DMAMUX_DMAREQ_ID_I2C1_RX
0x11	DMAMUX_DMAREQ_ID_I2C1_TX	0x12	DMAMUX_DMAREQ_ID_I2C2_RX
0x13	DMAMUX_DMAREQ_ID_I2C2_TX	0x14	DMAMUX_DMAREQ_ID_I2C3_RX
0x15	DMAMUX_DMAREQ_ID_I2C3_TX	0x18	DMAMUX_DMAREQ_ID_USART1_RX
0x19	DMAMUX_DMAREQ_ID_USART1_TX	0x1A	DMAMUX_DMAREQ_ID_USART2_RX
0x1B	DMAMUX_DMAREQ_ID_USART2_TX	0x1C	DMAMUX_DMAREQ_ID_USART3_RX
0x1D	DMAMUX_DMAREQ_ID_USART3_TX	0x1E	DMAMUX_DMAREQ_ID_UART4_RX
0x1F	DMAMUX_DMAREQ_ID_UART4_TX	0x20	DMAMUX_DMAREQ_ID_UART5_RX
0x21	DMAMUX_DMAREQ_ID_UART5_TX	0x72	DMAMUX_DMAREQ_ID_USART6_RX
0x73	DMAMUX_DMAREQ_ID_USART6_TX	0x74	DMAMUX_DMAREQ_ID_UART7_RX
0x75	DMAMUX_DMAREQ_ID_UART7_TX	0x76	DMAMUX_DMAREQ_ID_UART8_RX
0x77	DMAMUX_DMAREQ_ID_UART8_TX	0x27	DMAMUX_DMAREQ_ID_SDIO1
0x67	DMAMUX_DMAREQ_ID_SDIO2	0x28	DMAMUX_DMAREQ_ID_QSPI1
0x2B	DMAMUX_DMAREQ_ID_TMR1_CH2	0x2A	DMAMUX_DMAREQ_ID_TMR1_CH1
0x2D	DMAMUX_DMAREQ_ID_TMR1_CH4	0x2C	DMAMUX_DMAREQ_ID_TMR1_CH3
0x2F	DMAMUX_DMAREQ_ID_TMR1_TRIG	0x2E	DMAMUX_DMAREQ_ID_TMR1_OVERFLOW
0x31	DMAMUX_DMAREQ_ID_TMR8_CH1	0x30	DMAMUX_DMAREQ_ID_TMR1_HALL
0x33	DMAMUX_DMAREQ_ID_TMR8_CH3	0x32	DMAMUX_DMAREQ_ID_TMR8_CH2
0x35	DMAMUX_DMAREQ_ID_TMR8_OVERFLOW	0x34	DMAMUX_DMAREQ_ID_TMR8_CH4
0x37	DMAMUX_DMAREQ_ID_TMR8_HALL	0x36	DMAMUX_DMAREQ_ID_TMR8_TRIG
0x39	DMAMUX_DMAREQ_ID_TMR2_CH2	0x38	DMAMUX_DMAREQ_ID_TMR2_CH1
0x3B	DMAMUX_DMAREQ_ID_TMR2_CH4	0x3A	DMAMUX_DMAREQ_ID_TMR2_CH3
0x7E	DMAMUX_DMAREQ_ID_TMR2_TRIG	0x3C	DMAMUX_DMAREQ_ID_TMR2_OVERFLOW
0x3E	DMAMUX_DMAREQ_ID_TMR3_CH2	0x3D	DMAMUX_DMAREQ_ID_TMR3_CH1

Request source ID	Description	Request source ID	Description
0x40	DMAMUX_DMAREQ_ID_TMR3_CH4	0x3F	DMAMUX_DMAREQ_ID_TMR3_CH3
0x42	DMAMUX_DMAREQ_ID_TMR3_TRIG	0x41	DMAMUX_DMAREQ_ID_TMR3_OVERFLOW
0x44	DMAMUX_DMAREQ_ID_TMR4_CH2	0x43	DMAMUX_DMAREQ_ID_TMR4_CH1
0x46	DMAMUX_DMAREQ_ID_TMR4_CH4	0x45	DMAMUX_DMAREQ_ID_TMR4_CH3
0x7F	DMAMUX_DMAREQ_ID_TMR4_TRIG	0x47	DMAMUX_DMAREQ_ID_TMR4_OVERFLOW
0x49	DMAMUX_DMAREQ_ID_TMR5_CH2	0x48	DMAMUX_DMAREQ_ID_TMR5_CH1
0x4B	DMAMUX_DMAREQ_ID_TMR5_CH4	0x4A	DMAMUX_DMAREQ_ID_TMR5_CH3
0x4D	DMAMUX_DMAREQ_ID_TMR5_TRIG	0x4C	DMAMUX_DMAREQ_ID_TMR5_OVERFLOW
0x57	DMAMUX_DMAREQ_ID_TMR20_CH2	0x56	DMAMUX_DMAREQ_ID_TMR20_CH1
0x59	DMAMUX_DMAREQ_ID_TMR20_CH4	0x58	DMAMUX_DMAREQ_ID_TMR20_CH3
0x5D	DMAMUX_DMAREQ_ID_TMR20_TRIG	0x5A	DMAMUX_DMAREQ_ID_TMR20_OVERFLOW
0x69	DMAMUX_DMAREQ_ID_DVP	0x5E	DMAMUX_DMAREQ_ID_TMR20_HALL

Example:

```
/* tmr20 hall dmamux function enable */
dma_flexible_config(DMA2, DMA1MUX_CHANNEL1, DMAMUX_DMAREQ_ID_TMR20_HALL);
```

5.7.11 dmamux_enable function

The table below describes the function dmamux_enable.

Table 163. dmamux_enable function

Name	Description
Function name	dmamux_enable
Function prototype	void dmamux_enable(dma_type *dma_x, confirm_state new_state);
Function description	Enable DMAMUX feature
Input parameter 1	dma_x: DMAx, x=1 or 2
Input parameter 2	new_state: enable or disable a channel
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Enable or disable a DMA channel

FALSE: Channel disabled

TRUE: Channel enabled

Example:

```
/* dmamux function enable */
dmamux_enable(DMA2, TRUE);
```

5.7.12 dmamux_init

The table below describes the function dmamux_init.

Table 164. dmamux_init function

Name	Description
Function name	dmamux_init
Function prototype	void dmamux_init(dmamux_channel_type *dmamux_channelx, dmamux_reqst_id_sel_type dmamux_req_sel);
Function description	Configure DMAMUX
Input parameter 1	dmamux_channelx :DMAMUX channel selection, x=1...7
Input parameter 2	dmamux_req_sel :DMAMUX channel request ID
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* generator1 for dmamux channel4 as dma request */
dmamux_init(DMA2MUX_CHANNEL4, DMAMUX_DMAREQ_ID_REQ_G1);
```

5.7.13 dmamux_sync_default_para_init function

The table below describes the function dmamux_sync_default_para_init.

Table 165. dmamux_sync_default_para_init function

Name	Description
Function name	dmamux_sync_default_para_init
Function prototype	void dmamux_sync_default_para_init(dmamux_sync_init_type *dmamux_sync_init_struct);
Function description	Initialize the parameters in the dmamux_sync_init_struct
Input parameter 1	dmamux_sync_init_struct: dmamux_sync_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below shows the default values of members in the dmamux_sync_init_struct.

Table 166. dmamux_sync_init_struct default values

Member	Default value
sync_enable	FALSE
sync_event_enable	FALSE
sync_polarity	DMAMUX_SYNC_POLARITY_DISABLE
sync_request_number	0x0
sync_signal_sel	(dmamux_sync_id_sel_type)0

Example

```
/* dmamux sync init config with its default value */
dmamux_sync_init_type dmamux_sync_init_struct = {0};
dmamux_sync_default_para_init (&dmamux_sync_init_struct);
```

5.7.14 dmamux_sync_config function

The table below describes the function dmamux_sync_config.

Table 167. dmamux_sync_config function

Name	Description
Function name	dmamux_sync_config
Function prototype	void dmamux_sync_config(dmamux_channel_type *dmamux_channelx, dmamux_sync_init_type *dmamux_sync_init_struct);
Function description	Configure DMAMUX synchronous feature
Input parameter 1	dmamux_channelx DMAMUX channel selection, x=1...7
Input parameter 2	dmamux_sync_init_struct: dmamux_sync_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dmamux_sync_init_struct

The dmamux_sync_init_type is defined in the “at32f402_405_dma.h”:

typedef struct

```
{
    dmamux_sync_id_sel_type    sync_signal_sel;
    uint32_t                   sync_polarity;
    uint32_t                   sync_request_number;
    confirm_state               sync_event_enable;
    confirm_state               sync_enable;
} dmamux_sync_init_type;
```

sync_signal_sel

Select signal source for synchronous module.

DMAMUX_SYNC_ID_EXINT0: External exint0 signal
DMAMUX_SYNC_ID_EXINT1: External exint1 signal
DMAMUX_SYNC_ID_EXINT2: External exint2 signal
DMAMUX_SYNC_ID_EXINT3: External exint3 signal

DMAMUX_SYNC_ID_EXINT4: External exint4 signal
 DMAMUX_SYNC_ID_EXINT5: External exint5 signal
 DMAMUX_SYNC_ID_EXINT6: External exint6 signal
 DMAMUX_SYNC_ID_EXINT7: External exint7 signal
 DMAMUX_SYNC_ID_EXINT8: External exint8 signal
 DMAMUX_SYNC_ID_EXINT9: External exint9 signal
 DMAMUX_SYNC_ID_EXINT10: External exint10 signal
 DMAMUX_SYNC_ID_EXINT11: External exint11 signal
 DMAMUX_SYNC_ID_EXINT12: External exint12 signal
 DMAMUX_SYNC_ID_EXINT13: External exint13 signal
 DMAMUX_SYNC_ID_EXINT14: External exint14 signal
 DMAMUX_SYNC_ID_EXINT15: External exint15 signal
 DMAMUX_SYNC_ID_DMAMUX_CH1_EVT: dmamux channel 1 event
 DMAMUX_SYNC_ID_DMAMUX_CH2_EVT: dmamux channel 2 event
 DMAMUX_SYNC_ID_DMAMUX_CH3_EVT: dmamux channel 3 event
 DMAMUX_SYNC_ID_DMAMUX_CH4_EVT: dmamux channel 4 event
 DMAMUX_SYNC_ID_DMAMUX_CH5_EVT: dmamux channel 5 event
 DMAMUX_SYNC_ID_DMAMUX_CH6_EVT: dmamux channel 6 event
 DMAMUX_SYNC_ID_DMAMUX_CH7_EVT: dmamux channel 7 event

sync_polarity

Polarity selection for synchronous signal

DMAMUX_SYNC_POLARITY_RISING: Rising edge
 DMAMUX_SYNC_POLARITY_FALLING: Falling edge
 DMAMUX_SYNC_POLARITY_RISING_FALLING: Rising edge and falling edge

sync_request_number

The number of DMA requests that can be synchronized

Range: 1~32

sync_event_enable

Enable or disable synchronous event generation

TRUE: Synchronous event generated

FALSE: No synchronous event generated

sync_enable

Enable or disable synchronous module

FALSE: Disabled

TRUE: Enabled

Example:

```

dmamux_sync_default_para_init(&dmamux_sync_init_struct);
dmamux_sync_init_struct.sync_request_number = 4;
dmamux_sync_init_struct.sync_signal_sel = DMAMUX_SYNC_ID_EXINT1;
dmamux_sync_init_struct.sync_polarity = DMAMUX_SYNC_POLARITY_RISING;
dmamux_sync_init_struct.sync_event_enable = TRUE;
dmamux_sync_init_struct.sync_enable = TRUE;
dmamux_sync_config(DMA2MUX_CHANNEL4, &dmamux_sync_init_struct);
  
```

5.7.15 dmamux_generator_default_para_init function

The table below describes the function dmamux_generator_default_para_init.

Table 168. dmamux_generator_default_para_init function

Name	Description
Function name	dmamux_generator_default_para_init
Function prototype	void dmamux_generator_default_para_init(dmamux_gen_init_type *dmamux_gen_init_struct);
Function description	Initialize the parameters of the dmamux_gen_init_struct
Input parameter 1	dmamux_gen_init_struct: dmamux_gen_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Table 188 shows the default values of members in the dmamux_gen_init_struct:

Table 169. dmamux_gen_init_struct default values

Member	Default value
gen_signal_sel	(dmamux_gen_id_sel_type)0x0
gen_polarity	DMAMUX_GEN_POLARITY_DISABLE
gen_request_number	0x0
gen_enable	FALSE

Example:

```
/* dmamux gen init config with its default value */
dmamux_gen_init_type dmamux_gen_init_struct = {0};
dmamux_gen_default_para_init (&dmamux_gen_init_struct);
```

5.7.16 dmamux_generator_config function

The table below describes the function dmamux_generator_config.

Table 170. dmamux_generator_config function

Name	Description
Function name	dmamux_generator_config
Function prototype	void dmamux_generator_config(dmamux_generator_type * dmamux_gen_x, dmamux_gen_init_type *dmamux_gen_init_struct);
Function description	Configure DMAMUX request generator feature
Input parameter 1	dmamux_gen_x request generator channel
Input parameter 2	dmamux_sync_init_struct: dmamux_sync_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dmamux_gen_x

dma request generator channel selection

DMA1MUX_GENERATOR1

DMA1MUX_GENERATOR2

DMA1MUX_GENERATOR3

DMA1MUX_GENERATOR4

DMA2MUX_GENERATOR1

DMA2MUX_GENERATOR2

DMA2MUX_GENERATOR3

DMA2MUX_GENERATOR4

dmamux_sync_init_struct

The dmamux_gen_init_type is defined in the "at32f402_405_dma.h":

typedef struct

```
{
    dmamux_gen_id_sel_type      gen_signal_sel;
    uint32_t                    gen_polarity;
    uint32_t                    gen_request_number;
    confirm_state                gen_enable;
} dmamux_gen_init_type;
```

gen_signal_sel

Select signal source for synchronous module

DMAMUX_GEN_ID_EXINT0:	External exint0 singal
DMAMUX_GEN_ID_EXINT1:	External exint1 singal
DMAMUX_GEN_ID_EXINT2:	External exint2 singal
DMAMUX_GEN_ID_EXINT3:	External exint3 singal
DMAMUX_GEN_ID_EXINT4:	External exint4 singal
DMAMUX_GEN_ID_EXINT5:	External exint5 singal
DMAMUX_GEN_ID_EXINT6:	External exint6 singal
DMAMUX_GEN_ID_EXINT7:	External exint7 singal
DMAMUX_GEN_ID_EXINT8:	External exint8 singal
DMAMUX_GEN_ID_EXINT9:	External exint9 singal
DMAMUX_GEN_ID_EXINT10:	External exint10 singal
DMAMUX_GEN_ID_EXINT11:	External exint11 singal
DMAMUX_GEN_ID_EXINT12:	External exint12 singal
DMAMUX_GEN_ID_EXINT13:	External exint13 singal
DMAMUX_GEN_ID_EXINT14:	External exint14 singal
DMAMUX_GEN_ID_EXINT15:	External exint15 singal
DMAMUX_GEN_ID_DMAMUX_CH1_EVT:	dmamux channel 1 event
DMAMUX_GEN_ID_DMAMUX_CH2_EVT:	dmamux channel 2 event
DMAMUX_GEN_ID_DMAMUX_CH3_EVT:	dmamux channel 3 event
DMAMUX_GEN_ID_DMAMUX_CH4_EVT:	dmamux channel 4 event
DMAMUX_GEN_ID_DMAMUX_CH5_EVT:	dmamux channel 5 event
DMAMUX_GEN_ID_DMAMUX_CH6_EVT:	dmamux channel 6 event
DMAMUX_GEN_ID_DMAMUX_CH7_EVT:	dmamux channel 7 event

gen_polarity

Polarity selection for request generator signal

DMAMUX_GEN_POLARITY_RISING: Rising edge

DMAMUX_GEN_POLARITY_FALLING: Falling edge

DMAMUX_GEN_POLARITY_RISING_FALLING: Rising edge and falling edge

gen_request_number

The number of DMA requests that are generated by request generator

Range: 1~32

gen_enable

Enable or disable request generator

FALSE: Disabled

TRUE: Enabled

Example:

```
/* genertor1 configuration */
dmamux_generator_default_para_init(&dmamux_gen_init_struct);
dmamux_gen_init_struct.gen_polarity = DMAMUX_GEN_POLARITY_RISING;
dmamux_gen_init_struct.gen_request_number = 4;
dmamux_gen_init_struct.gen_signal_sel = DMAMUX_GEN_ID_EXINT1;
dmamux_gen_init_struct.gen_enable = TRUE;
dmamux_generator_config(DMA2MUX_GENERATOR1, &dmamux_gen_init_struct);
```

5.7.17 dmamux_sync_interrupt_enable function

The table below describes the function dmamux_sync_interrupt_enable.

Table 171. dmamux_sync_interrupt_enable function

Name	Description
Function name	dmamux_sync_interrupt_enable
Function prototype	void dmamux_sync_interrupt_enable(dmamux_channel_type *dmamux_channelx, confirm_state new_state);
Function description	Enable synchronous module overflow interrupt
Input parameter 1	dmamux_channelx DMAMUX channel selection, x=1...7
Input parameter 2	new_state: enable or disable interrupts
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Enable or disable DMA channel interrupts

FALSE: Interrupt disabled

TRUE: Interrupt enabled

Example:

```
/* enable sync overrun interrupt */
dmamux_sync_interrupt_enable (DMA2MUX_CHANNEL1, TRUE);
```

5.7.18 dmamux_generator_interrupt_enable function

The table below describes the function dmamux_generator_interrupt_enable.

Table 172. dmamux_generator_interrupt_enable function

Name	Description
Function name	dmamux_generator_interrupt_enable
Function prototype	void dmamux_generator_interrupt_enable(dmamux_generator_type *dmamux_gen_x, confirm_state new_state);
Function description	Enable request generator overflow interrupt
Input parameter 1	dmamux_gen_x DMAMUX request generator channel selection, x=1...4
Input parameter 2	new_state: enable or disable interrupts
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Enable or disable request generator channel interrupts

FALSE: Interrupt disabled

TRUE: Interrupt enabled

Example:

```
/* enable gen overrun interrupt */
dmamux_generator_interrupt_enable(DMA2MUX_GENERATOR3, TRUE);
```

5.7.19 dmamux_sync_flag_get function

The table below describes the function dmamux_sync_flag_get.

Table 173. dmamux_sync_flag_get function

Name	Description
Function name	dmamux_sync_flag_get
Function prototype	flag_status dmamux_sync_flag_get(dma_type *dma_x, uint32_t flag);
Function description	Get dmamux synchronous flag
Input parameter 1	dma_x: DMAx, x=1 or 2
Input parameter 2	Flag: flag selection
Output parameter	NA
Return value	flag_status: indicates whether or not the selected flag is set
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

DMAMUX_SYNC_OV1_FLAG

DMAMUX_SYNC_OV2_FLAG

DMAMUX_SYNC_OV3_FLAG

DMAMUX_SYNC_OV4_FLAG

DMAMUX_SYNC_OV5_FLAG

DMAMUX_SYNC_OV6_FLAG

DMAMUX_SYNC_OV7_FLAG

flag_status

RESET: Corresponding flag is not set

SET: Corresponding flag is set

Example:

```
if(dmamux_sync_flag_get (DMA2, DMAMUX_SYNC_OV1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
}
```

5.7.20 dmamux_sync_flag_clear function

The table below describes the function dmamux_sync_flag_clear.

Table 174. dmamux_sync_flag_clear function

Name	Description
Function name	dmamux_sync_flag_clear
Function prototype	void dmamux_sync_flag_clear(dma_type *dma_x, uint32_t flag);
Function description	Clear synchronous module flag
Input parameter 1	dma_x: DMAx, x=1 or 2
Input parameter 2	Flag: flag selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

DMAMUX_SYNC_OV1_FLAG

DMAMUX_SYNC_OV2_FLAG

DMAMUX_SYNC_OV3_FLAG

DMAMUX_SYNC_OV4_FLAG

DMAMUX_SYNC_OV5_FLAG

DMAMUX_SYNC_OV6_FLAG

DMAMUX_SYNC_OV7_FLAG

Example:

```
if(dmamux_sync_flag_get (DMA2, DMAMUX_SYNC_OV1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
}
```

```

at32_led_on(LED4);
dmamux_sync_flag_clear(DMA2, DMAMUX_SYNC_OV1_FLAG);
}

```

5.7.21 dmamux_generator_flag_get function

The table below describes the function dmamux_generator_flag_get.

Table 175. dmamux_generator_flag_get function

Name	Description
Function name	dmamux_generator_flag_get
Function prototype	flag_status dmamux_generator_flag_get(dma_type *dma_x, uint32_t flag);
Function description	Get dmamux request generator flag
Input parameter 1	dma_x: DMAx, x=1 or 2
Input parameter 2	Flag: flag selection
Output parameter	NA
Return value	flag_status: indicates whether or not the selected flag is set
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

DMAMUX_GEN_TRIG_OV1_FLAG

DMAMUX_GEN_TRIG_OV2_FLAG

DMAMUX_GEN_TRIG_OV3_FLAG

DMAMUX_GEN_TRIG_OV4_FLAG

flag_status

RESET: Corresponding flag is not set

SET: Corresponding flag is set

Example:

```

if(dmamux_generator_flag_get (DMA2, DMAMUX_GEN_TRIG_OV1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
}

```

5.7.22 dmamux_generator_flag_clear function

The table below describes the function dmamux_generator_flag_clear.

Table 176. dmamux_generator_flag_clear function

Name	Description
Function name	dmamux_generator_flag_clear
Function prototype	void dmamux_generator_flag_clear(dma_type *dma_x, uint32_t flag);
Function description	Clear request generator flag
Input parameter 1	dma_x: DMAx, x=1 or 2
Input parameter 2	Flag: flag selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

DMAMUX_GEN_TRIG_OV1_FLAG

DMAMUX_GEN_TRIG_OV2_FLAG

DMAMUX_GEN_TRIG_OV3_FLAG

DMAMUX_GEN_TRIG_OV4_FLAG

Example:

```
if(dmamux_generator_flag_get (DMA2, DMAMUX_GEN_TRIG_OV1_FLAG) != RESET)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
    dmamux_generator_flag_clear(DMA2, DMAMUX_GEN_TRIG_OV1_FLAG);
}
```

5.8 Real-time clock (ERTC)

The ERTC register structure ertc_type is defined in the “at32f402_405_ertc.h”:

```
/**
 * @brief type define ertc register all
 */
typedef struct
{

} ertc_type;
```

The table below gives a list of the ERTC registers:

Table 177. Summary of ERTC registers

Register	Description
time	ERTC time register
date	ERTC date register
ctrl	ERTC control register
sts	ERTC initialization and status register
div	ERTC divider register
wat	ERTC wakeup timer register
ccal	ERTC coarse calibration register
ala	ERTC alarm clock A register
alb	ERTC alarm clock B register
wp	ERTC write protection register
sbs	ERTC subsecond register
tadj	ERTC time adjustment register
tstm	ERTC time stamp time register
tsdt	ERTC time stamp date register
tssbs	ERTC time stamp subsecond register
scal	ERTC smooth calibration register
tamp	ERTC tamper configuration register
alasbs	ERTC alarm clock A subsecond register
albsbs	ERTC alarm clock B subsecond register
bprx	ERTC battery powered domain data register

The table below gives a list of ERTC library functions.

Table 178. Summary of ERTC library functions

Function name	Description
ertc_num_to_bcd	Convert number to BCD code
ertc_bcd_to_num	Convert BCD code to number
ertc_write_protect_enable	Enable write protection
ertc_write_protect_disable	Disable write protection
ertc_wait_update	Wait for register update complete
ertc_wait_flag	Wait flag
ertc_init_mode_enter	Enter initialization mode
ertc_init_mode_exit	Exit initialization mode
ertc_reset	Reset ERTC registers
ertc_divider_set	Divider setting
ertc_hour_mode_set	Hour mode setting
ertc_date_set	Date setting
ertc_time_set	Time setting
ertc_calendar_get	Get calendar
ertc_sub_second_get	Get the current subsecond
ertc_alarm_mask_set	Set alarm mask
ertc_alarm_week_date_select	Alarm time format selection (week/date)
ertc_alarm_set	Set alarm
ertc_alarm_sub_second_set	Set alarm subsecond
ertc_alarm_enable	Enable alarm
ertc_alarm_get	Get alarm value
ertc_alarm_sub_second_get	Get alarm subsecond
ertc_wakeup_clock_set	Select wakeup clock source
ertc_wakeup_counter_set	Set wakeup counter value
ertc_wakeup_counter_get	Get wakeup counter value
ertc_wakeup_enable	Enable wakeup timer
ertc_smooth_calibration_config	Configure smooth calibration
ertc_coarse_calibration_set	Configure coarse calibration
ertc_coarse_calibration_enable	Enable coarse calibration
ertc_cal_output_select	Calibration output source selection
ertc_cal_output_enable	Enable calibration output
ertc_time_adjust	Time adjustment
ertc_daylight_set	Set daylight saving time
ertc_daylight_bpr_get	Get daylight saving time battery powered domain data register value (BPR)
ertc_refer_clock_detect_enable	Enable reference clock detection
ertc_direct_read_enable	Enable direct read mode
ertc_output_set	Set event output
ertc_timestamp_pin_select	Time stamp detection pin selection
ertc_timestamp_valid_edge_set	Set time stamp detection valid edge
ertc_timestamp_enable	Enable time stamp
ertc_timestamp_get	Get time stamp
ertc_timestamp_sub_second_get	Get time stamp subsecond

ertc_tamper_1_pin_select	Tamper detection 1 pin selection
ertc_tamper_pull_up_enable	Enable tamper pin pull-up resistor
ertc_tamper_precharge_set	Set tamper pin precharge time
ertc_tamper_filter_set	Set tamper filter time
ertc_tamper_detect_freq_set	Set tamper detection frequency
ertc_tamper_valid_edge_set	Set tamper detection valid edge
ertc_tamper_timestamp_enable	Enable time stamp upon a tamper event
ertc_tamper_enable	Enable tamper detection
ertc_interrupt_enable	Enable interrupts
ertc_interrupt_get	Get the status of interrupt enable
ertc_flag_get	Get flag status
ertc_flag_clear	Clear flag
ertc_bpr_data_write	Write data to battery powered data register (BPR)
ertc_bpr_data_read	Read from battery powered data register (BPR)

5.8.1 ertc_num_to_bcd function

The table below describes the function ertc_num_to_bcd.

Table 179. ertc_num_to_bcd function

Name	Description
Function name	ertc_num_to_bcd
Function prototype	uint8_t ertc_num_to_bcd(uint8_t num);
Function description	Convert number into BCD format
Input parameter 1	num: number to be converted
Output parameter	NA
Return value	BCD code
Required preconditions	NA
Called functions	NA

Example:

```
ertc_num_to_bcd(12);
```

5.8.2 ertc_bcd_to_num function

The table below describes the function ertc_bcd_to_num.

Table 180. ertc_bcd_to_num function

Name	Description
Function name	ertc_bcd_to_num
Function prototype	uint8_t ertc_bcd_to_num(uint8_t bcd);
Function description	Convert BCD code into number
Input parameter 1	bcd: BCD code to be converted
Output parameter	NA
Return value	Return the number corresponding to BCD code
Required preconditions	NA
Called functions	NA

Example:

```
ertc_bcd_to_num(0x12);
```

5.8.3 ertc_write_protect_enable function

The table below describes the function ertc_write_protect_enable.

Table 181. ertc_write_protect_enable function

Name	Description
Function name	ertc_write_protect_enable
Function prototype	void ertc_write_protect_enable(void);
Function description	Write protection enable
Input parameter 1	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_write_protect_enable();
```

5.8.4 ertc_write_protect_disable function

The table below describes the function ertc_write_protect_disable.

Table 182. ertc_write_protect_disable function

Name	Description
Function name	ertc_write_protect_disable
Function prototype	void ertc_write_protect_disable(void);
Function description	Write protection disable
Input parameter 1	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_write_protect_disable();
```

5.8.5 ertc_wait_update function

The table below describes the function ertc_wait_update.

Table 183. ertc_wait_update function

Name	Description
Function name	ertc_wait_update
Function prototype	error_status ertc_wait_update(void);
Function description	Wait for register to finish update
Input parameter 1	NA
Output parameter	NA
Return value	SUCCESS: register update complete ERROR: flag wait timeout
Required preconditions	NA
Called functions	NA

Example:

```
ertc_wait_update();
```


5.8.6 ertc_wait_flag function

The table below describes the function ertc_wait_flag.

Table 184. ertc_wait_flag function

Name	Description
Function name	ertc_wait_flag
Function prototype	error_status ertc_wait_flag(uint32_t flag, flag_status status);
Function description	Wait flag
Input parameter 1	flag: flag selection Refer to the “flag” description below for details.
Input parameter 1	status: flag status. After the flag status is set, the function remains stuck here until flag status changes. This parameter can be SET or RESET.
Output parameter	NA
Return value	SUCCESS: flag state changed ERROR: flag wait timeout
Required preconditions	NA
Called functions	NA

flag

Flag selection

ERTC_ALAWF_FLAG:	Alarm A write enable flag
ERTC_ALBWF_FLAG:	Alarm B write enable flag
ERTC_WATWF_FLAG:	Wakeup timer register write enable flag
ERTC_TADJF_FLAG:	Time adjustment flag
ERTC_CALUPDF_FLAG:	Calibration value update complete flag

Example:

```
ertc_wait_flag(ERTC_ALAWF_FLAG, RESET);
```

5.8.7 ertc_init_mode_enter function

The table below describes the function ertc_init_mode_enter.

Table 185. ertc_init_mode_enter function

Name	Description
Function name	ertc_init_mode_enter
Function prototype	error_status ertc_init_mode_enter(void);
Function description	Enter initialization mode
Input parameter 1	NA
Output parameter	NA
Return value	SUCCESS: Initialization mode is entered successfully ERROR: Timeout
Required preconditions	NA
Called functions	NA

Example:

```
ertc_init_mode_enter();
```

5.8.8 ertc_init_mode_exit function

The table below describes the function ertc_init_mode_exit.

Table 186. ertc_init_mode_exit function

Name	Description
Function name	ertc_init_mode_exit
Function prototype	void ertc_init_mode_exit(void);
Function description	Exit initialization mode
Input parameter 1	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_init_mode_exit();
```

5.8.9 ertc_reset function

The table below describes the function ertc_reset.

Table 187. ertc_reset function

Name	Description
Function name	ertc_reset
Function prototype	error_status ertc_reset(void);
Function description	Reset all ERTC registers
Input parameter 1	NA
Output parameter	NA
Return value	SUCCESS: Reset successful ERROR: Reset failed
Required preconditions	NA
Called functions	NA

Example:

```
ertc_reset();
```

5.8.10 ertc_divider_set function

The table below describes the function ertc_divider_set.

Table 188. ertc_divider_set function

Name	Description
Function name	ertc_divider_set
Function prototype	error_status ertc_divider_set(uint16_t div_a, uint16_t div_b);
Function description	Divider settings, frequency division value $(div_a + 1) * (div_b + 1) = ERTC_CLK$ frequency For example, if 32768Hz is used, the frequency division should be $div_a = 127$, $div_b = 255$
Input parameter 1	div_a: divider A, range: 0~0x7F
Input parameter 2	div_b: divider B, range: 0~0x7FFF
Output parameter	NA
Return value	SUCCESS: Reset successful ERROR: Reset failed
Required preconditions	NA
Called functions	NA

Example:

```
ertc_divider_set(127, 255);
```

5.8.11 ertc_hour_mode_set function

The table below describes the function ertc_hour_mode_set.

Table 189. ertc_hour_mode_set function

Name	Description
Function name	ertc_hour_mode_set
Function prototype	error_status ertc_hour_mode_set(ertc_hour_mode_set_type mode);
Function description	Hour mode settings
Input parameter 1	mode: hour mode Refer to the following description "Mode" for details.
Output parameter	NA
Return value	SUCCESS: Setting success ERROR: Setting error
Required preconditions	NA
Called functions	NA

mode

ERTC_HOUR_MODE_24: 24-hour format

ERTC_HOUR_MODE_12: 12-hour format

Example:

```
ertc_hour_mode_set(ERTC_HOUR_MODE_24);
```

5.8.12 ertc_date_set function

The table below describes the function ertc_date_set.

Table 190. ertc_date_set function

Name	Description
Function name	ertc_date_set
Function prototype	error_status ertc_date_set(uint8_t year, uint8_t month, uint8_t date, uint8_t week);
Function description	Set date: year, month, date, weekday
Input parameter 1	year: range 0~99
Input parameter 2	month: range 1~12
Input parameter3	date: range 1~31
Input parameter4	week: range 1~7
Output parameter	NA
Return value	SUCCESS: Setting success ERROR: Setting error
Required preconditions	NA
Called functions	NA

Example:

```
ertc_date_set(22, 5, 26, 4);
```

5.8.13 ertc_time_set function

The table below describes the function ertc_time_set.

Table 191. ertc_time_set function

Name	Description
Function name	ertc_time_set
Function prototype	error_status ertc_time_set(uint8_t hour, uint8_t min, uint8_t sec, ertc_am_pm_type ampm);
Function description	Set time: hour, minute, second, AM/PM (for 12-hour format only)
Input parameter 1	hour: range 0~23
Input parameter 2	min: range 0~59
Input parameter3	sec: range 0~59
Input parameter4	ampm: AM/PM in 12-hour format (for 12-hour format only, don't care in 24-hour format) Refer to the following description "ampm" for details.
Output parameter	NA
Return value	SUCCESS: Setting success ERROR: Setting error
Required preconditions	NA
Called functions	NA

ampm

AM/PM in 12-hour format (for 12-hour format only, don't care in 24-hour format)

ERTC_24H: 24-hour format (for 24-hour format)

ERTC_AM: AM in 12-hour format

ERTC_PM: PM in 12-hour format

Example:

```
ertc_time_set(12, 1, 20, ERTC_24H);
```

5.8.14 ertc_calendar_get function

The table below describes the function ertc_calendar_get.

Table 192. ertc_calendar_get function

Name	Description
Function name	ertc_calendar_get
Function prototype	void ertc_calendar_get(ertc_time_type* time);
Function description	Get calendar, including year, month, date, weekday, hour, minute, second, AM/PM
Input parameter 1	time: ertc_time_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

ertc_time_type* time

The ertc_time_type is defined in the "at32f402_405_ertc.h":

typedef struct

```
{  
    uint8_t      year;  
    uint8_t      month;  
    uint8_t      day;  
    uint8_t      hour;  
    uint8_t      min;  
    uint8_t      sec;  
    uint8_t      week;  
    ertc_am_pm_type ampm;  
} ertc_time_type;
```

year

Range 0~99

month

Range 1~12

day

Range 1~31

week

Range 1~7

hour

Range 0~23

min

Range 0~59

sec

Range 0~59

ampm

AM/PM in 12-hour format (for 12-hour format only, doesn't care in 24 hour), including:

ERTC_AM: AM in 12 hour format

ERTC_PM: PM in 12 hour format

Example:

```
ertc_calendar_get(&time);
```

5.8.15 ertc_sub_second_get function

The table below describes the function ertc_sub_second_get.

Table 193. ertc_sub_second_get function

Name	Description
Function name	ertc_sub_second_get
Function prototype	uint32_t ertc_sub_second_get(void);
Function description	Get current subsecond (the current value of divider B)
Input parameter 1	NA
Output parameter	NA
Return value	Current subsecond
Required preconditions	NA
Called functions	NA

Example:

```
ertc_sub_second_get();
```

5.8.16 ertc_alarm_mask_set function

The table below describes the function ertc_alarm_mask_set.

Table 194. ertc_alarm_mask_set function

Name	Description
Function name	ertc_alarm_mask_set
Function prototype	void ertc_alarm_mask_set(ertc_alarm_type alarm_x, uint32_t mask);
Function description	Set alarm mask
	alarm_x: alarm selection Refer to the following description “alarm_x” for details.
Input parameter 1	mask: Set alarm mask Refer to the following description “mask” for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

alarm_x

Alarm selection

ERTC_ALA: Alarm A

ERTC_ALB: Alarm B

mask

Set alarm mask

ERTC_ALARM_MASK_NONE:	No mask, alarm is relevant to all fields
ERTC_ALARM_MASK_SEC:	Mask second, alarm is not relevant to second
ERTC_ALARM_MASK_MIN:	Mask minute, alarm is not relevant to minute
ERTC_ALARM_MASK_HOUR:	Mask hour, alarm is not relevant to hour
ERTC_ALARM_MASK_DATE_WEEK:	Mask date, alarm is not relevant to date
ERTC_ALARM_MASK_ALL:	Mask all. Generate an alarm per one second

Example:

```
ertc_alarm_mask_set(ERTC_ALA, ERTC_ALARM_MASK_NONE);
```

5.8.17 ertc_alarm_week_date_select function

The table below describes the function ertc_alarm_week_date_select.

Table 195. ertc_alarm_week_date_select function

Name	Description
Function name	ertc_alarm_week_date_select
Function prototype	void ertc_alarm_week_date_select(ertc_alarm_type alarm_x, ertc_week_date_select_type wk);
Function description	Alarm time format selection: week/date
Input parameter 1	alarm_x: alarm selection Refer to the following description “alarm_x” for details.
Input parameter 2	wk: alarm week/date format selection Refer to the following description “wk” for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

alarm_x

Alarm selection

ERTC_ALA: Alarm A

ERTC_ALB: Alarm B

wk

Alarm week/date format selection

ERTC_SELECT_DATE: Date mode

ERTC_SELECT_WEEK: Week mode

Example:

```
ertc_alarm_week_date_select(ERTC_ALA, ERTC_SELECT_DATE);
```


5.8.18 ertc_alarm_set function

The table below describes the function ertc_alarm_set.

Table 196. ertc_alarm_set function

Name	Description
Function name	ertc_alarm_set
Function prototype	void ertc_alarm_set(ertc_alarm_type alarm_x, uint8_t week_date, uint8_t hour, uint8_t min, uint8_t sec, ertc_am_pm_type ampm);
Function description	Set alarm
Input parameter 1	alarm_x: alarm selection Refer to the following description “alarm_x” for details.
Input parameter 2	week_date: date or week, depending on the ertc_alarm_week_date_select() Date: range 1~31 Week: range 1~7
Input parameter3	hour: range 0~23
Input parameter4	min: range 0~59
Input parameter5	sec: range 0~59
Input parameter6	ampm: AM/PM in 12-hour format (12 hour format only, doesn't care in 24-hour format) Refer to the following description “ampm” for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

alarm_x

Alarm selection

ERTC_ALA: Alarm A

ERTC_ALB: Alarm B

ampm

AM/PM in 12-hour format (for 12 hour format only, doesn't care in 24 hour)

ERTC_24H: 24-hour format (for 24 hour format)

ERTC_AM: AM in 12-hour format

ERTC_PM: PM in 12-hour format

Example:

```
ertc_alarm_set(ERTC_ALA, 15, 8, 0, 0, ERTC_24H);
```

5.8.19 ertc_alarm_sub_second_set function

The table below describes the function ertc_alarm_sub_second_set.

Table 197. ertc_alarm_sub_second_set function

Name	Description
Function name	ertc_alarm_sub_second_set
Function prototype	void ertc_alarm_sub_second_set(ertc_alarm_type alarm_x, uint32_t value, ertc_alarm_sbs_mask_type mask);
Function description	Set alarm subsecond
Input parameter 1	alarm_x: alarm selection Refer to the following description “alarm_x” for details.
Input parameter 2	value: subsecond value, range 0~0x7FFF
Input parameter3	mask: alarm mask settings Refer to the following description “mask” for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

alarm_x

Alarm selection

ERTC_ALA: Alarm A

ERTC_ALB: Alarm B

mask

Subsecond mask

ERTC_ALARM_SBS_MASK_ALL:	Mask all
ERTC_ALARM_SBS_MASK_14_1:	Only match SBS bit [0]
ERTC_ALARM_SBS_MASK_14_2:	Only match SBS bit [1:0]
ERTC_ALARM_SBS_MASK_14_3:	Only match SBS bit [2:0]
ERTC_ALARM_SBS_MASK_14_4:	Only match SBS bit [3:0]
ERTC_ALARM_SBS_MASK_14_5:	Only match SBS bit [4:0]
ERTC_ALARM_SBS_MASK_14_6:	Only match SBS bit [5:0]
ERTC_ALARM_SBS_MASK_14_7:	Only match SBS bit [6:0]
ERTC_ALARM_SBS_MASK_14_8:	Only match SBS bit [7:0]
ERTC_ALARM_SBS_MASK_14_9:	Only match SBS bit [8:0]
ERTC_ALARM_SBS_MASK_14_10:	Only match SBS bit [9:0]
ERTC_ALARM_SBS_MASK_14_11:	Only match SBS bit [10:0]
ERTC_ALARM_SBS_MASK_14_12:	Only match SBS bit [11:0]
ERTC_ALARM_SBS_MASK_14_13:	Only match SBS bit [12:0]
ERTC_ALARM_SBS_MASK_14:	Only match SBS bit [13:0]
ERTC_ALARM_SBS_MASK_NONE:	Only match SBS bit [14:0]

Example:

```
ertc_alarm_sub_second_set(ERTC_ALA, 200, ERTC_ALARM_SBS_MASK_NONE);
```

5.8.20 ertc_alarm_enable function

The table below describes the function ertc_alarm_enable.

Table 198. ertc_alarm_enable function

Name	Description
Function name	ertc_alarm_enable
Function prototype	error_status ertc_alarm_enable(ertc_alarm_type alarm_x, confirm_state new_state);
Function description	Alarm enable
Input parameter 1	alarm_x: alarm selection Refer to the following description “alarm_x” for details.
Input parameter 2	new_state: alarm enable status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	SUCCESS: Setting success ERROR: Setting error
Required preconditions	NA
Called functions	NA

alarm_x

Alarm selection

ERTC_ALA: Alarm A

ERTC_ALB: Alarm B

Example:

```
ertc_alarm_enable(ERTC_ALA, TRUE);
```

5.8.21 ertc_alarm_get function

The table below describes the function ertc_alarm_get.

Table 199. ertc_alarm_get function

Name	Description
Function name	ertc_alarm_get
Function prototype	void ertc_alarm_get(ertc_alarm_type alarm_x, ertc_alarm_value_type* alarm);
Function description	Get alarm value
Input parameter 1	alarm_x: alarm selection Refer to the following description “alarm_x” for details.
Input parameter 2	alarm: ertc_alarm_value_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

alarm_x

Alarm selection

ERTC_ALA: Alarm A

ERTC_ALB: Alarm B

ertc_alarm_value_type* alarm

ertc_alarm_value_type is defined in the “at32f402_405_ertc.h”:

typedef struct

```
{
    uint8_t      day;
    uint8_t      hour;
    uint8_t      min;
    uint8_t      sec;
    ertc_am_pm_type ampm;
    uint32_t      mask;
    uint8_t      week_date_sel;
    uint8_t      week;
}
```

} ertc_alarm_value_type;

day

Range 1~31

hour

Range 0~23

min

Range 0~59

sec

Range 0~59

ampm

AM/PM in 12-hour format (for 12-hour format only, doesn't care in 24 hour), including:

ERTC_AM: AM in 12 hour format

ERTC_PM: PM in 12 hour format

mask

Alarm mask value, including:

ERTC_ALARM_MASK_NONE: No mask

ERTC_ALARM_MASK_SEC: Mask second

ERTC_ALARM_MASK_MIN: Mask minute

ERTC_ALARM_MASK_HOUR: Mask hour

ERTC_ALARM_MASK_DATE_WEEK: Mask date

ERTC_ALARM_MASK_ALL: Mask all

week_date_sel

Alarm week/date format, including:

ERTC_SELECT_DATE: date mode

ERTC_SELECT_WEEK: week mode

week

Range 1~7

Example:

```
ertc_alarm_get(ERTC_ALA, &alarm);
```

5.8.22 ertc_alarm_sub_second_get function

The table below describes the function ertc_alarm_sub_second_get.

Table 200. ertc_alarm_sub_second_get function

Name	Description
Function name	ertc_alarm_sub_second_get
Function prototype	uint32_t ertc_alarm_sub_second_get(ertc_alarm_type alarm_x);
Function description	Get alarm subsecond value
Input parameter 1	alarm_x: alarm selection Refer to the following description “alarm_x” for details.
Output parameter	NA
Return value	Alarm subsecond value
Required preconditions	NA
Called functions	NA

alarm_x

Alarm selection

ERTC_ALA: Alarm A

ERTC_ALB: Alarm B

Example:

```
ertc_alarm_sub_second_get(ERTC_ALA);
```

5.8.23 ertc_wakeup_clock_set function

The table below describes the function ertc_wakeup_clock_set.

Table 201. ertc_wakeup_clock_set function

Name	Description
Function name	ertc_wakeup_clock_set
Function prototype	void ertc_wakeup_clock_set(ertc_wakeup_clock_type clock);
Function description	Select wakeup timer clock source
Input parameter 1	clock: clock source for wakeup timer Refer to the following description “clock” for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

clock

Wakeup timer clock source

ERTC_WAT_CLK_ERTCCLK_DIV16: ERTC_CLK / 16

ERTC_WAT_CLK_ERTCCLK_DIV8: ERTC_CLK / 8

ERTC_WAT_CLK_ERTCCLK_DIV4: ERTC_CLK / 4

ERTC_WAT_CLK_ERTCCLK_DIV2: ERTC_CLK / 2

ERTC_WAT_CLK_CK_B_16BITS: CK_B (1Hz calendar clock), wakeup counter value = ERTC_WAT

ERTC_WAT_CLK_CK_B_17BITS: CK_B (1Hz calendar clock), wakeup counter value =

ERTC_WAT + 65535

Example:

```
ertc_wakeup_clock_set(ERTC_WAT_CLK_CK_B_16BITS);
```

5.8.24 ertc_wakeup_counter_set function

The table below describes the function ertc_wakeup_counter_set.

Table 202. ertc_wakeup_counter_set function

Name	Description
Function name	ertc_wakeup_counter_set
Function prototype	void ertc_wakeup_counter_set(uint32_t counter);
Function description	Set wakeup counter value
Input parameter 1	counter: ounter value, range 0~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_wakeup_counter_set(0x7FFF);
```

5.8.25 ertc_wakeup_counter_get function

The table below describes the function ertc_wakeup_counter_get.

Table 203. ertc_wakeup_counter_get function

Name	Description
Function name	ertc_wakeup_counter_get
Function prototype	uint16_t ertc_wakeup_counter_get(void);
Function description	Get the current wakeup counter value
Input parameter 1	NA
Output parameter	NA
Return value	Return the current wakeup counter value
Required preconditions	NA
Called functions	NA

Example:

```
ertc_wakeup_counter_get();
```

5.8.26 ertc_wakeup_enable function

The table below describes the function ertc_wakeup_enable.

Table 204. ertc_wakeup_enable function

Name	Description
Function name	ertc_wakeup_enable
Function prototype	error_status ertc_wakeup_enable(confirm_state new_state);
Function description	Enable wakeup timer
Input parameter 1	new_state: wakeup timer enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	SUCCESS: Set success ERROR: Set error
Required preconditions	NA
Called functions	NA

Example:

```
ertc_wakeup_enable(TRUE);
```

5.8.27 ertc_smooth_calibration_config function

The table below describes the function ertc_smooth_calibration_config.

Table 205. ertc_smooth_calibration_config function

Name	Description
Function name	ertc_smooth_calibration_config
Function prototype	error_status ertc_smooth_calibration_config(ertc_smooth_cal_period_type period, ertc_smooth_cal_clk_add_type clk_add, uint32_t clk_dec);
Function description	et smooth digital calibration
Input parameter 1	period: calibration period Refer to the following “period” descriptions for details.
Input parameter 2	clk_add: add ERTC CLK cycles Refer to the following “clk_add” descriptions for details.
Input parameter3	clk_dec: reduce ERTC CLK cycles, range 0~511
Output parameter	NA
Return value	SUCCESS: Set success ERROR: Set error
Required preconditions	NA
Called functions	NA

period

Calibration periods

ERTC_SMOOTH_CAL_PERIOD_32: 32 seconds

ERTC_SMOOTH_CAL_PERIOD_16: 16 seconds

ERTC_SMOOTH_CAL_PERIOD_8: 8 seconds

clk_add

Add ERTC CLK

ERTC_SMOOTH_CAL_CLK_ADD_0: No effect
ERTC_SMOOTH_CAL_CLK_ADD_512: Add 512 ERTC_CLK cycles

Example:

```
ertc_smooth_calibration_config(ERTC_SMOOTH_CAL_PERIOD_32, ERTC_SMOOTH_CAL_CLK_ADD_0, 511);
```

5.8.28 ertc_cal_output_select function

The table below describes the function ertc_cal_output_select.

Table 206. ertc_cal_output_select function

Name	Description
Function name	ertc_cal_output_select
Function prototype	void ertc_cal_output_select(ertc_cal_output_select_type output);
Function description	Calibration output source selection
Input parameter 1	output: Calibration output source Refer to the following “output” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

output

Calibration output source

ERTC_CAL_OUTPUT_512HZ: 512 Hz output

ERTC_CAL_OUTPUT_1HZ: 1 Hz output

Example:

```
ertc_cal_output_select(ERTC_CAL_OUTPUT_1HZ);
```

5.8.29 ertc_cal_output_enable function

The table below describes the function ertc_cal_output_enable.

Table 207. ertc_cal_output_enable function

Name	Description
Function name	ertc_cal_output_enable
Function prototype	void ertc_cal_output_enable(confirm_state new_state);
Function description	Calibration output enable
Input parameter 1	new_state: calibration output enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_cal_output_enable(TRUE);
```


5.8.30 ertc_time_adjust function

The table below describes the function ertc_time_adjust.

Table 208. ertc_time_adjust function

Name	Description
Function name	ertc_time_adjust
Function prototype	error_status ertc_time_adjust(ertc_time_adjust_type add1s, uint32_t decsbs);
Function description	Adjust time
Input parameter 1	add1s: add seconds Refer to the following “add1s” descriptions for details.
Input parameter 2	decsbs: reduce subseconds, range 0~0x7FFF
Output parameter	NA
Return value	SUCCESS: Set success ERROR: Set error
Required preconditions	NA
Called functions	NA

add1s

This bit is used to add seconds.

ERTC_TIME_ADD_NONE: No effect

ERTC_TIME_ADD_1S: Add 1 second

Example:

```
ertc_time_adjust(ERTC_TIME_ADD_1S, 254);
```

5.8.31 ertc_daylight_set function

The table below describes the function ertc_daylight_set.

Table 209. ertc_daylight_set function

Name	Description
Function name	ertc_daylight_set
Function prototype	void ertc_daylight_set(ertc_dst_operation_type operation, ertc_dst_save_type save);
Function description	Set daylight-saving time
Input parameter 1	operation: daylight-saving time settings Refer to the following “operation” descriptions for details.
Input parameter 2	save: save daylight time Refer to the following “save” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

operation

Daylight-saving time settings

ERTC_DST_ADD_1H: Add 1 hour

ERTC_DST_DEC_1H: Reduce 1 hour

save

Save daylight time

ERTC_DST_SAVE_0: set BPR bit to 0 in the CTRL register

ERTC_DST_SAVE_1: set BPR bit to 1 in the CTRL register

Example:

```
ertc_daylight_set(ERTC_DST_ADD_1H, ERTC_DST_SAVE_1);
```

5.8.32 ertc_daylight_bpr_get function

The table below describes the function ertc_daylight_bpr_get.

Table 210. ertc_daylight_bpr_get function

Name	Description
Function name	ertc_daylight_bpr_get
Function prototype	uint8_t ertc_daylight_bpr_get(void);
Function description	Get the value of daylight-saving time battery powered register (BPR bit in the CTRL register)
Input parameter 1	NA
Output parameter	NA
Return value	Return the value of daylight-saving time battery powered register (BPR bit in the CTRL register)
Required preconditions	NA
Called functions	NA

Example:

```
ertc_daylight_bpr_get();
```

5.8.33 ertc_refer_clock_detect_enable function

The table below describes the function ertc_refer_clock_detect_enable.

Table 211. ertc_refer_clock_detect_enable function

Name	Description
Function name	ertc_refer_clock_detect_enable
Function prototype	error_status ertc_refer_clock_detect_enable(confirm_state new_state);
Function description	Enable reference clock detection
Input parameter 1	new_state: reference clock detection enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	SUCCESS: Set success ERROR: Set error
Required preconditions	NA
Called functions	NA

Example:

```
ertc_refer_clock_detect_enable(TRUE);
```

5.8.34 ertc_direct_read_enable function

The table below describes the function ertc_direct_read_enable.

Table 212. ertc_direct_read_enable function

Name	Description
Function name	ertc_direct_read_enable
Function prototype	void ertc_direct_read_enable(confirm_state new_state);
Function description	Enable direct read mode
Input parameter 1	new_state: direct read mode enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_direct_read_enable(TRUE);
```

5.8.35 ertc_output_set function

The table below describes the function ertc_output_set.

Table 213. ertc_output_set function

Name	Description
Function name	ertc_output_set
Function prototype	void ertc_output_set(ertc_output_source_type source, ertc_output_polarity_type polarity, ertc_output_type type);
Function description	Set event output, event output on PC13
Input parameter 1	source: output source selection Refer to the following “source” descriptions for details.
Input parameter 2	polarity: output polarity Refer to the following “polarity” descriptions for details.
Input parameter3	type: output type Refer to the following “type” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

Output source selection

ERTC_OUTPUT_DISABLE:	Output disabled
ERTC_OUTPUT_ALARM_A:	Output alarm A event
ERTC_OUTPUT_ALARM_B:	Output alarm B event
ERTC_OUTPUT_WAKEUP:	Output wakeup event

polarity

Output polarity

ERTC_OUTPUT_POLARITY_HIGH: Output high when an event occurred

ERTC_OUTPUT_POLARITY_LOW: Output low when an event occurred

type

Output type

ERTC_OUTPUT_TYPE_OPEN_DRAIN: Open-drain output

ERTC_OUTPUT_TYPE_PUSH_PULL: Push-pull output

Example:

```
ertc_output_set(ERTC_OUTPUT_ALARM_A, ERTC_OUTPUT_POLARITY_HIGH,  
ERTC_OUTPUT_TYPE_PUSH_PULL);
```

5.8.36 ertc_timestamp_pin_select function

The table below describes the function ertc_timestamp_pin_select.

Table 214. ertc_timestamp_pin_select function

Name	Description
Function name	ertc_timestamp_pin_select
Function prototype	void ertc_timestamp_pin_select(ertc_pin_select_type pin);
Function description	Timestamp detection pin selection
Input parameter 1	pin: Timestamp detection pin Refer to the following “pin” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pin

Timestampe detection pin

ERTC_PIN_PC13: PC13 is selected as timestampe detection pin

ERTC_PIN_PA0: PA0 is selected as timestampe detection pin

Example:

```
ertc_timestamp_pin_select(ERTC_PIN_PC13);
```

5.8.37 ertc_timestamp_valid_edge_set function

The table below describes the function ertc_timestamp_valid_edge_set.

Table 215. ertc_timestamp_valid_edge_set function

Name	Description
Function name	ertc_timestamp_valid_edge_set
Function prototype	void ertc_timestamp_valid_edge_set(ertc_timestamp_valid_edge_type edge);
Function description	Set timestamp detection valid edge
Input parameter 1	edge: timestamp detection valid edge Refer to the following “edge” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

edge

Timestamp detection valid edge

ERTC_TIMESTAMP_EDGE_RISING: Rising edge

ERTC_TIMESTAMP_EDGE_FALLING: Falling edge

Example:

```
ertc_timestamp_valid_edge_set(ERTC_TIMESTAMP_EDGE_RISING);
```

5.8.38 ertc_timestamp_enable function

The table below describes the function ertc_timestamp_enable.

Table 216. ertc_timestamp_enable function

Name	Description
Function name	ertc_timestamp_enable
Function prototype	void ertc_timestamp_enable(confirm_state new_state);
Function description	Enable timestamp
Input parameter 1	new_state: timestamp enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_timestamp_enable(TRUE);
```

5.8.39 ertc_timestamp_get function

The table below describes the function ertc_timestamp_get.

Table 217. ertc_timestamp_get function

Name	Description
Function name	ertc_timestamp_get
Function prototype	void ertc_timestamp_get(ertc_time_type* time);
Function description	Get timestamp
Input parameter 1	time: ertc_time_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

ertc_time_type* time

The ertc_time_type is defined in the “at32f402_405_ertc.h”:

typedef struct

```
{
    uint8_t      year;
    uint8_t      month;
    uint8_t      day;
    uint8_t      hour;
    uint8_t      min;
    uint8_t      sec;
    uint8_t      week;
    ertc_am_pm_type ampm;
}
```

} ertc_time_type;

year

Range 0~99

month

Range 1~12

day

Range 1~31

week

Range 1~7

hour

Range 0~23

min

Range 0~59

sec

Range 0~59

ampm

AM/PM in 12-hour format (only for 12-hour format, doesn't care in 24-hour format), including:

ERTC_AM: AM in 12-hour format

ERTC_PM: PM in 12-hour format

Example:

```
ertc_timestamp_get(&time);
```

5.8.40 ertc_timestamp_sub_second_get function

The table below describes the function ertc_timestamp_sub_second_get.

Table 218. ertc_timestamp_sub_second_get function

Name	Description
Function name	ertc_timestamp_sub_second_get
Function prototype	uint32_t ertc_timestamp_sub_second_get(void);
Function description	Get timestamp subsecond
Input parameter 1	NA
Output parameter	NA
Return value	Return timestamp subsecond
Required preconditions	NA
Called functions	NA

Example:

```
ertc_timestamp_sub_second_get();
```

5.8.41 ertc_tamper_1_pin_select function

The table below describes the function ertc_tamper_1_pin_select.

Table 219. ertc_tamper_1_pin_select function

Name	Description
Function name	ertc_tamper_1_pin_select
Function prototype	void ertc_tamper_1_pin_select(ertc_pin_select_type pin);
Function description	Tamper detection 1 pin selection
Input parameter 1	pin: Tamper detection pin Refer to the following “pin” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pin

Tamper detection pin

ERTC_PIN_PC13: PC13 is selected as a tamper detection pin

ERTC_PIN_PA0: PA0 is selected as a tamper detection pin

Example:

```
ertc_tamper_1_pin_select(ERTC_PIN_PC13);
```

5.8.42 ertc_tamper_pull_up_enable function

The table below describes the function ertc_tamper_pull_up_enable.

Table 220. ertc_tamper_pull_up_enable function

Name	Description
Function name	ertc_tamper_pull_up_enable
Function prototype	void ertc_tamper_pull_up_enable(confirm_state new_state);
Function description	Enable tamper pin pull-up resistor
Input parameter 1	new_state: tamper pin pull-up resistor enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_tamper_pull_up_enable(TRUE);
```

5.8.43 ertc_tamper_precharge_set function

The table below describes the function ertc_tamper_precharge_set.

Table 221. ertc_tamper_precharge_set function

Name	Description
Function name	ertc_tamper_precharge_set
Function prototype	void ertc_tamper_precharge_set(ertc_tamper_precharge_type precharge);
Function description	Set tamper pin precharge time. This setting is needed only when the tamper pull-up resistor is enabled through the function ertc_tamper_pull_up_enable.
Input parameter 1	precharge: tamper pin precharge time Refer to the following “precharge” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

precharge

Tamper pin precharge time

ERTC_TAMPER_PR_1_ERTCCLK: One ERTC_CLK cycle

ERTC_TAMPER_PR_2_ERTCCLK: Two ERTC_CLK cycles

ERTC_TAMPER_PR_4_ERTCCLK: Four ERTC_CLK cycles

ERTC_TAMPER_PR_8_ERTCCLK: Eight ERTC_CLK cycles

Example:

```
ertc_tamper_precharge_set(ERTC_TAMPER_PR_2_ERTCCLK);
```


5.8.44 ertc_tamper_filter_set function

The table below describes the function ertc_tamper_filter_set.

Table 222. ertc_tamper_filter_set function

Name	Description
Function name	ertc_tamper_filter_set
Function prototype	void ertc_tamper_filter_set(ertc_tamper_filter_type filter);
Function description	Set tamper filtering time
Input parameter 1	filter: tamper filtering time Refer to the following “filter” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

filter

Set tamper filtering time

ERTC_TAMPER_FILTER_DISABLE:

No filtering

ERTC_TAMPER_FILTER_2:

Tamper event is considered to have occur after two valid consecutive sampling

ERTC_TAMPER_FILTER_4:

Tamper event is considered to have occur after four valid consecutive sampling

ERTC_TAMPER_FILTER_8:

Tamper event is considered to have occur after eight valid consecutive sampling

Example:

```
ertc_tamper_filter_set(ERTC_TAMPER_FILTER_2);
```

5.8.45 ertc_tamper_detect_freq_set function

The table below describes the function ertc_tamper_detect_freq_set.

Table 223. ertc_tamper_detect_freq_set function

Name	Description
Function name	ertc_tamper_detect_freq_set
Function prototype	void ertc_tamper_detect_freq_set(ertc_tamper_detect_freq_type freq);
Function description	Set tamper detection frequency
Input parameter 1	freq: tamper detection frequency Refer to the following “freq” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

freq

Select tamper detection frequency

ERTC_TAMPER_FREQ_DIV_32768: ERTC_CLK / 32768

ERTC_TAMPER_FREQ_DIV_16384: ERTC_CLK / 16384
 ERTC_TAMPER_FREQ_DIV_8192: ERTC_CLK / 8192
 ERTC_TAMPER_FREQ_DIV_4096: ERTC_CLK / 4096
 ERTC_TAMPER_FREQ_DIV_2048: ERTC_CLK / 2048
 ERTC_TAMPER_FREQ_DIV_1024: ERTC_CLK / 1024
 ERTC_TAMPER_FREQ_DIV_512: ERTC_CLK / 512
 ERTC_TAMPER_FREQ_DIV_256: ERTC_CLK / 256

Example:

```
ertc_tamper_detect_freq_set(ERTC_TAMPER_FREQ_DIV_512);
```

5.8.46 ertc_tamper_valid_edge_set function

The table below describes the function ertc_tamper_valid_edge_set.

Table 224. ertc_tamper_valid_edge_set function

Name	Description
Function name	ertc_tamper_valid_edge_set
Function prototype	void ertc_tamper_valid_edge_set(ertc_tamper_select_type tamper_x, ertc_tamper_valid_edge_type trigger);
Function description	Set tamper detection valid edge
Input parameter 1	tamper_x: tamper selection Refer to the following “tamper_x” descriptions for details.
Input parameter 2	trigger: tamper detection valid edge Refer to the following “trigger” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tamper_x

Tamper selection

ERTC_TAMPER_1: Tamper detection 1

ERTC_TAMPER_2: Tamper detection 2

trigger

Tamper detection valid edge selection

ERTC_TAMPER_EDGE_RISING: Rising edge

ERTC_TAMPER_EDGE_FALLING: Falling edge

ERTC_TAMPER_EDGE_LOW: Low level

ERTC_TAMPER_EDGE_HIGH: High level

Example:

```
ertc_tamper_valid_edge_set(ERTC_TAMPER_1, ERTC_TAMPER_EDGE_RISING);
```

5.8.47 ertc_tamper_timestamp_enable function

The table below describes the function ertc_tamper_timestamp_enable.

Table 225. ertc_tamper_timestamp_enable function

Name	Description
Function name	ertc_tamper_timestamp_enable
Function prototype	void ertc_tamper_timestamp_enable(confirm_state new_state);
Function description	Enable timestamp when a tamper event occurred
Input parameter 1	new_state: timestamp feature enable state when a tamper event occurred This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
ertc_tamper_timestamp_enable(TRUE);
```

5.8.48 ertc_tamper_enable function

The table below describes the function ertc_tamper_enable.

Table 226. ertc_tamper_enable function

Name	Description
Function name	ertc_tamper_enable
Function prototype	void ertc_tamper_enable(ertc_tamper_select_type tamper_x, confirm_state new_state);
Function description	Enable tamper detection
Input parameter 1	tamper_x: tamper selection Refer to the following “tamper_x” descriptions for details.
Input parameter 2	new_state: tamper detection enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tamper_x

Tamper selection

ERTC_TAMPER_1: Tamper detection 1

ERTC_TAMPER_2: Tamper detection 2

Example:

```
ertc_tamper_enable(ERTC_TAMPER_1, TRUE);
```

5.8.49 ertc_interrupt_enable function

The table below describes the function ertc_interrupt_enable.

Table 227. ertc_interrupt_enable function

Name	Description
Function name	ertc_interrupt_enable
Function prototype	void ertc_interrupt_enable(uint32_t source, confirm_state new_state);
Function description	Interrupt enable
	source: interrupt source to be enabled Refer to the following “source” descriptions for details.
Input parameter 1	new_state: interrupt enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

Interrupt source to be enabled

ERTC_TP_INT: Tamper detection interrupt

ERTC_ALA_INT: Alarm A interrupt

ERTC_ALB_INT: Alarm B interrupt

ERTC_WAT_INT: Wakeup timer interrupt

ERTC_TS_INT: Time stamp interrupt

Example:

```
ertc_interrupt_enable(ERTC_TP_INT, TRUE);
```

5.8.50 ertc_interrupt_get function

The table below describes the function ertc_interrupt_get.

Table 228. ertc_interrupt_get function

Name	Description
Function name	ertc_interrupt_get
Function prototype	flag_status ertc_interrupt_get(uint32_t source);
Function description	Get interrupt enable state
Input parameter 1	source: interrupt source Refer to the following “source” descriptions for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

source

Interrupt source

ERTC_TP_INT: Tamper detection interrupt

ERTC_ALA_INT: Alarm A interrupt
 ERTC_ALB_INT: Alarm B interrupt
 ERTC_WAT_INT: Wakeup timer interrupt
 ERTC_TS_INT: Time stamp interrupt

Example:

```
ertc_interrupt_get(ERTC_TP_INT);
```

5.8.51 ertc_flag_get function

The table below describes the function ertc_flag_get.

Table 229. ertc_flag_get function

Name	Description
Function name	ertc_flag_get
Function prototype	flag_status ertc_flag_get(uint32_t flag);
Function description	Get flag status
Input parameter 1	flag: flag selection Refer to the following “flag” descriptions for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This bit is used to select a flag. Optional parameters are as follows:

ERTC_ALAWF_FLAG: Alarm A write enable flag
 ERTC_ALBWF_FLAG: Alarm B write enable flag
 ERTC_WATWF_FLAG: Wakeup timer register write enable flag
 ERTC_TADJF_FLAG: Time adjust flag
 ERTC_INITF_FLAG: Calendar initialization flag
 ERTC_UPDF_FLAG: Calendar update flag
 ERTC_IMF_FLAG: Initialization mode entry flag
 ERTC_ALAF_FLAG: Alarm A flag
 ERTC_ALBF_FLAG: Alarm B flag
 ERTC_WATF_FLAG: Wakeup timer flag
 ERTC_TSF_FLAG: Time stamp flag
 ERTC_TSOFF_FLAG: Time stamp overflow flag
 ERTC_TP1F_FLAG: Tamper detection 1 flag
 ERTC_TP2F_FLAG: Tamper detection 2 flag
 ERTC_CALUPDF_FLAG: Calibration value update complete flag

Example:

```
ertc_flag_get(ERTC_TP1F_FLAG);
```

5.8.52 ertc_interrupt_flag_get function

The table below describes the function ertc_interrupt_flag_get.

Table 230. ertc_interrupt_flag_get function

Name	Description
Function name	ertc_interrupt_flag_get
Function prototype	flag_status ertc_interrupt_flag_get(uint32_t flag);
Function description	Get flag status and judge the corresponding interrupt enable bit
Input parameter 1	Flag: flag selection Refer to the following “flag” descriptions for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This bit is used to select a flag. Optional parameters are as follows:

ERTC_ALAF_FLAG:	Alarm A flag
ERTC_ALBF_FLAG:	Alarm B flag
ERTC_WATF_FLAG:	Wakeup timer flag
ERTC_TSF_FLAG:	Time stamp flag
ERTC_TP1F_FLAG:	Tamper detection 1 flag
ERTC_TP2F_FLAG:	Tamper detection 2 flag

Example

```
ertc_interrupt_flag_get(ERTC_TP1F_FLAG);
```

5.8.53 ertc_flag_clear function

The table below describes the function ertc_flag_clear.

Table 231. ertc_flag_clear function

Name	Description
Function name	ertc_flag_clear
Function prototype	void ertc_flag_clear(uint32_t flag);
Function description	Clear flag
Input parameter 1	flag: flag selection Refer to the following “flag” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

This bit is used to select a flag. Optional parameters are as follows:

ERTC_ALAWF_FLAG:	Alarm A write enable flag
ERTC_ALBWF_FLAG:	Alarm B write enable flag

ERTC_WATWF_FLAG:	Wakeup timer register write enable flag
ERTC_TADJF_FLAG:	Time adjust flag
ERTC_INITF_FLAG:	Calendar initialization flag
ERTC_UPDF_FLAG:	Calendar update flag
ERTC_IMF_FLAG:	Initialization mode entry flag
ERTC_ALAF_FLAG:	Alarm A flag
ERTC_ALBF_FLAG:	Alarm B flag
ERTC_WATF_FLAG:	Wakeup timer flag
ERTC_TSF_FLAG:	Time stamp flag
ERTC_TSOF_FLAG:	Time stamp overflow flag
ERTC_TP1F_FLAG:	Tamper detection 1 flag
ERTC_TP2F_FLAG:	Tamper detection 2 flag
ERTC_CALUPDF_FLAG:	Calibration value update complete flag

Example:

```
ertc_flag_clear(ERTC_TP1F_FLAG);
```

5.8.54 ertc_bpr_data_write function

The table below describes the function ertc_bpr_data_write.

Table 232. ertc_bpr_data_write function

Name	Description
Function name	ertc_bpr_data_write
Function prototype	void ertc_bpr_data_write(ertc_dt_type dt, uint32_t data);
Function description	Write data to BPR register (battery powered data register)
Input parameter 1	dt: data register Refer to the following "dt" descriptions for details.
Input parameter 1	data: 32-bit data
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dt

Data register

ERTC_DT1: Data register 1

ERTC_DT2: Data register 2

ERTC_DT19: Data register 19

ERTC_DT20: Data register 20

Example:

```
ertc_bpr_data_write(ERTC_DT1, 0x12345678);
```

5.8.55 ertc_bpr_data_read function

The table below describes the function ertc_bpr_data_read.

Table 233. ertc_bpr_data_read function

Name	Description
Function name	ertc_bpr_data_read
Function prototype	uint32_t ertc_bpr_data_read(ertc_dt_type dt);
Function description	Read data from BPR register (battery pwered data register)
Input parameter 1	dt: data register Refer to the following “dt” descriptions for details.
Output parameter	NA
Return value	Data from BPR register
Required preconditions	NA
Called functions	NA

dt

Data register

ERTC_DT1: Data register 1

ERTC_DT2: Data register 2

...

ERTC_DT19: Data register19

ERTC_DT20: Data register 20

Example:

```
ertc_bpr_data_read(ERTC_DT1);
```


5.9 External interrupt/event controller (EXINT)

The EXINT register structure `exint_type` is defined in the “at32f402_405_exint.h”:

```
/**
 * @brief type define exint register all
 */
typedef struct
{
    ...
} exint_type;
```

The table below gives a list of the EXINT registers:

Table 234. Summary of EXINT registers

Register	Description
inten	Interrupt enable register
evten	Event enable register
polcfg1	Polarity configuration register 1
polcfg2	Polarity configuration register 2
swtrg	Software trigger register
intsts	Interrupt status register

The table below gives a list of EXINT library functions.

Table 235. Summary of EXINT library functions

Function name	Description
<code>exint_reset</code>	Reset all EXINT registers to their reset values
<code>exint_default_para_init</code>	Configure the EXINT initial structure with the initial value
<code>exint_init</code>	Initialize EXINT
<code>exint_flag_clear</code>	Clear the selected EXINT interrupt flag
<code>exint_flag_get</code>	Read the selected EXINT flag
<code>exint_interrupt_flag_get</code>	Read the selected EXINT interrupt flag
<code>exint_software_interrupt_event_generate</code>	Software interrupt event generation
<code>exint_interrupt_enable</code>	Enable the selected EXINT interrupt
<code>exint_event_enable</code>	Enable the selected EXINT event

5.9.1 exint_reset function

The table below describes the function `exint_reset`.

Table 236. exint_reset function

Name	Description
Function name	<code>exint_reset</code>
Function prototype	<code>void exint_reset(void);</code>
Function description	Reset all EXINT registers to their reset values.
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	<code>crm_periph_reset();</code>

Example:

```
exint_reset ();
```

5.9.2 exint_default_para_init function

The table below describes the function `exint_default_para_init`.

Table 237. exint_default_para_init function

Name	Description
Function name	<code>exint_default_para_init</code>
Function prototype	<code>void exint_default_para_init(exint_init_type *exint_struct);</code>
Function description	Configure the EXINT initial structure with the initial value
Input parameter 1	<code>exint_struct</code> : exint_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of <code>exint_init_type</code> before starting.
Called functions	NA

Example:

```
exint_init_type exint_init_struct;
exint_default_para_init(&exint_init_struct);
```

5.9.3 exint_init function

The table below describes the function `exint_init`.

Table 238. exint_init function

Name	Description
Function name	<code>exint_init</code>
Function prototype	<code>void exint_init(exint_init_type *exint_struct);</code>
Function description	Initialize EXINT
Input parameter 1	<i>exint_init_type</i> : <code>exint_init_struct</code> pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of <code>exint_init_type</code> before starting.
Called functions	NA

The `exint_init_type` is defined in the “`at32f402_405_exint.h`”:

typedef struct

```
{
    exint_line_mode_type          line_mode;
    uint32_t                      line_select;
    exint_polarity_config_type    line_polarity;
    confirm_state                 line_enable;
} exint_init_type;
```

line_mode

Select event mode or interrupt mode

EXINT_LINE_INTERRUPT: Interrupt mode

EXINT_LINE_EVENT: Event mode

line_select

Line selection

EXINT_LINE_NONE: No e

EXINT_LINE_0: line0

EXINT_LINE_1: line1

...

EXINT_LINE_18: line18

EXINT_LINE_20: line20

EXINT_LINE_21: line21

EXINT_LINE_22: line22

line_polarity

Trigger edge selection

EXINT_TRIGGER_RISING_EDGE: Rising edge

EXINT_TRIGGER_FALLING_EDGE: Falling edge

EXINT_TRIGGER_BOTH_EDGE: Rising/Falling edge

line_enable

Enable/disable line

FALSE: Disable line

TRUE: Enable line

Example:

```
exint_init_type exint_init_struct;  
exint_default_para_init(&exint_init_struct);  
exint_init_struct.line_enable = TRUE;  
exint_init_struct.line_mode = EXINT_LINE_INTERRUPT;  
exint_init_struct.line_select = EXINT_LINE_0;  
exint_init_struct.line_polarity = EXINT_TRIGGER_RISING_EDGE;  
exint_init(&exint_init_struct);
```

5.9.4 exint_flag_clear function

The table below describes the function `exint_flag_clear`.

Table 239. exint_flag_clear function

Name	Description
Function name	<code>exint_flag_clear</code>
Function prototype	<code>void exint_flag_clear(uint32_t exint_line);</code>
Function description	Clear the selected EXINT interrupt flag
Input parameter	<code>exint_line</code> : line selection Refer to the line_select for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_flag_clear(EXINT_LINE_0);
```

5.9.5 exint_flag_get function

The table below describes the function `exint_flag_get`.

Table 240. exint_flag_get function

Name	Description
Function name	<code>exint_flag_get</code>
Function prototype	<code>flag_status exint_flag_get(uint32_t exint_line);</code>
Function description	Get the selected EXINT interrupt flag
Input parameter	<code>exint_line</code> : line selection Refer to line_select for details.
Output parameter	NA
Return value	<code>flag_status</code> : indicates the status of the selected flag This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status = RESET;
```

```
status = exint_flag_get(EXINT_LINE_0);
```

5.9.6 exint_interrupt_flag_get function

The table below describes the function `exint_interrupt_flag_get`.

Table 241. exint_interrupt_flag_get function

Name	Description
Function name	<code>exint_interrupt_flag_get</code>
Function prototype	<code>flag_status exint_interrupt_flag_get(uint32_t exint_line)</code>
Function description	Get the selected EXINT interrupt flag
Input parameter	<code>exint_line</code> : line selection Refer to line_select for details.
Output parameter	NA
Return value	<code>flag_status</code> : indicates the status of the selected flag This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example

```
flag_status status = RESET;  
status = exint_interrupt_flag_get (EXINT_LINE_0);
```

5.9.7 exint_software_interrupt_event_generate function

The table below describes the function `exint_software_interrupt_event_generate`.

Table 242. exint_software_interrupt_event_generate function

Name	Description
Function name	<code>exint_software_interrupt_event_generate</code>
Function prototype	<code>void exint_software_interrupt_event_generate(uint32_t exint_line);</code>
Function description	Generate software interrupt event
Input parameter	<code>exint_line</code> : line selection Refer to line_select for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_software_interrupt_event_generate (EXINT_LINE_0);
```

5.9.8 exint_interrupt_enable function

The table below describes the function `exint_interrupt_enable`.

Table 243. exint_interrupt_enable function

Name	Description
Function name	<code>exint_interrupt_enable</code>
Function prototype	<code>void exint_interrupt_enable(uint32_t exint_line, confirm_state new_state);</code>
Function description	Enable the selected EXINT interrupt
Input parameter 1	<code>exint_line</code> : line selection Refer to line_select for details.
Input parameter 2	<code>new_state</code> : Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_interrupt_enable (EXINT_LINE_0);
```

5.9.9 exint_event_enable function

The table below describes the function `exint_event_enable`.

Table 244. exint_event_enable function

Name	Description
Function name	<code>exint_event_enable</code>
Function prototype	<code>void exint_event_enable(uint32_t exint_line, confirm_state new_state);</code>
Function description	Enable the selected EXINT event
Input parameter 1	<code>exint_line</code> : line selection Refer to line_select for details.
Input parameter 2	<code>new_state</code> : Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
exint_event_enable (EXINT_LINE_0);
```

5.10 Flash memory controller (FLASH)

The FLASH register structure flash_type is defined in the “at32f402_405_flash.h”:

```
/**
 * @brief type define flash register all
 */
typedef struct
{
    ...
} flash_type;
```

The table below gives a list of the FLASH registers

Table 245. Summary of FLASH registers

Register	Description
flash_psr	Flash performance select register
flash_unlock	Flash unlock register
flash_usd_unlock	Flash user system data unlock register
flash_sts	Flash status register
flash_ctrl	Flash control register
flash_addr	Flash address register
flash_usd	User system data register
flash_epps	Erase/program protection status register
slib_sts0	Flash security library status register 0
slib_sts1	Flash security library status register 1
slib_pwd_clr	Flash security library password clear register
slib_misc_sts	Flash security library extra status register
Flash_crc_addr	Flash CRC address register
flash_crc_ctrl	Flash CRC check control register
flash_crc_chkr	Flash CRC check result register
slib_set_pwd	Flash security library password setting register
slib_set_range	Flash security library address setting register
em_slib_set	Extended memory security library setting register
btm_mode_set	Boot memory mode setting register
slib_unlock	Flash security library unlock register

The table below gives a list of FLASH library functions.

Table 246. Summary of FLASH library functions

Function name	Description
flash_flag_get	Get flag status
flash_flag_clear	Clear flag
flash_operation_wait_for	Wait for operation complete (Flash memory bank 1)
flash_operation_status_get	Get Flash operation status
flash_operation_wait_for	Wait for Flash operation complete
flash_unlock	Unlock Flash (Flash memory bank 1 and 2)
flash_lock	Lock Flash (Flash memory bank 1 and 2)
flash_sector_erase	Erase Flash sector
flash_internal_all_erase	Erase internal Flash
flash_user_system_data_erase	Erase user system data
flash_word_program	Flash word programming
flash_halfword_program	Flash half-word programming
flash_byte_program	Flash byte programming
flash_user_system_data_program	User system data programming
flash_epp_set	Erase/programming protection configuration
flash_epp_status_get	Get erase/programming protection status
flash_fap_enable	Flash low level access protection enable
flash_fap_status_get	Get Flash low level access protection status
flash_fap_high_level_enable	Flash high level access protection enable
flash_fap_high_level_status_get	Get Flash high level access protection status
flash_ssb_set	System configuration byte configuration
flash_ssb_status_get	Get system configuration byte configuration status
flash_interrupt_enable	Flash interrupt configuration
flash_slib_enable	sLib enable
flash_slib_disable	sLib disable
flash_slib_state_get	Get sLib states
flash_slib_start_sector_get	Get sLib start sector
flash_slib_datastart_sector_get	Get sLib data area start sector
flash_slib_end_sector_get	Get sLib end sector
flash_crc_calibrate	Flash CRC verify
flash_boot_memory_extension_mode_enable	Boot memory is used as an extended Flash memory
flash_extension_memory_slib_enable	Extended Flash memory is used as a security library
flash_extension_memory_slib_state_get	Get the status of extended Flash memory which is used as a security library
flash_em_slib_inststart_sector_get	Get the start page of instruction area of security library in the extended memory

5.10.1 flash_flag_get function

The table below describes the function flash_flag_get.

Table 247. flash_flag_get function

Name	Description
Function name	flash_flag_get
Function prototype	flag_status flash_flag_get(uint32_t flash_flag);
Function description	Get flag status
Input parameter	flash_flag: Flag selection
Output parameter	NA
Return value	flag_status: indicates the flag status Return RESET or SET
Required preconditions	NA
Called functions	NA

flash_flag

Flag selection.

FLASH_OBF_FLAG:	Flash operation busy (bank 1)
FLASH_ODF_FLAG:	Flash operation complete (bank 1)
FLASH_PRGMERR_FLAG:	Flash programming error (bank 1)
FLASH_EPPERR_FLAG:	Flash erase error (bank 1)
FLASH_USDERR_FLAG:	User system data area error

Example:

```
flag_status status;
status = flash_flag_get (FLASH_ODF_FLAG);
```

5.10.2 flash_flag_clear function

The table below describes the function flash_flag_clear.

Table 248. flash_flag_clear function

Name	Description
Function name	flash_flag_clear
Function prototype	void flash_flag_clear(uint32_t flash_flag);
Function description	Clear flag
Input parameter	flash_flag: flag selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flash_flag

Flag selection

FLASH_ODF_FLAG:	Flash operation complete
FLASH_PRGMERR_FLAG:	Flash programming error
FLASH_EPPERR_FLAG:	Flash erase error

Example:

```
flash_flag_clear(FLASH_ODF_FLAG);
```

5.10.3 flash_operation_status_get function

The table below describes the function flash_operation_status_get.

Table 249. flash_operation_status_get function

Name	Description
Function name	flash_operation_status_get
Function prototype	flash_status_type flash_operation_status_get(void);
Function description	Get operation status
Input parameter	NA
Output parameter	NA
Return value	Refer to the flash_status_type for details.
Required preconditions	NA
Called functions	NA

flash_status_type

FLASH_OPERATE_BUSY	Operate busy
FLASH_PROGRAM_ERROR	Programming error
FLASH_EPP_ERROR	Erase/program protection error
FLASH_OPERATE_DONE	Operation complete

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
/* check for the flash status */
status = flash_operation_status_get();
```

5.10.4 flash_operation_wait_for function

The table below describes the function flash_operation_wait_for.

Table 250. flash_operation_wait_for function

Name	Description
Function name	flash_operation_wait_for
Function prototype	flash_status_type flash_operation_wait_for(uint32_t time_out);
Function description	Wait for Flash operation
Input parameter	time_out: wait timeout The timeout value is defined in the flash.h file, refer to flash_time_out for details.
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	NA
Called functions	NA

flash_time_out

OPERATION_TIMEOUT	General operation timeout
-------------------	---------------------------

Example:

```
/* wait for operation to be completed */
```

```
status = flash_operation_wait_for(PROGRAMMING_TIMEOUT);
```

5.10.5 flash_unlock function

The table below describes the function flash_unlock.

Table 251. flash_unlock function

Name	Description
Function name	flash_unlock
Function prototype	void flash_unlock(void);
Function description	Unlock Flash memory controller
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_unlock();
```

5.10.6 flash_lock function

The table below describes the function flash_lock.

Table 252. flash_lock function

Name	Description
Function name	flash_lock
Function prototype	void flash_lock(void);
Function description	Lock Flash memory controller
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_lock();
```

5.10.7 flash_sector_erase function

The table below describes the function flash_sector_erase.

Table 253. flash_sector_erase function

Name	Description
Function name	flash_sector_erase
Function prototype	flash_status_type flash_sector_erase(uint32_t sector_address);
Function description	Erase data in the selected Flash sector address
Input parameter	sector_address: select the Flash sector address to be erased, usually Flash sector start address
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_sector_erase(0x08001000);
```

5.10.8 flash_internal_all_erase function

The table below describes the function flash_internal_all_erase.

Table 254. flash_internal_all_erase function

Name	Description
Function name	flash_internal_all_erase
Function prototype	flash_status_type flash_internal_all_erase(void);
Function description	Erase internal Flash data
Input parameter	NA
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_internal_all_erase();
```

5.10.9 flash_user_system_data_erase function

The table below describes the function flash_user_system_data_erase.

Table 255. flash_user_system_data_erase function

Name	Description
Function name	flash_user_system_data_erase
Function prototype	flash_status_type flash_user_system_data_erase(void);
Function description	Erase user system data
Input parameter	NA
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	NA
Called functions	NA

Note: As this function remains in FAP state, it only erases data except FAP in the user system data area.

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_user_system_data_erase();
```

5.10.10 flash_word_program function

The table below describes the function flash_word_program.

Table 256. flash_word_program function

Name	Description
Function name	flash_word_program
Function prototype	flash_status_type flash_word_program(uint32_t address, uint32_t data);
Function description	Write one word data to a given address
Input parameter 1	Address: programmed address, word-aligned
Input parameter 2	Data: programmed data
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	The programming operation can be allowed only when data in the address are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
uint32_t i;
flash_unlock();
status = flash_sector_erase(0x08001000);
if(status == FLASH_OPERATE_DONE)
{
    /* program 256 words */
    for(i = 0; i < 256; i++)
    {
        status = flash_word_program(0x08001000 + i*4, i);
    }
}
```

```
}
}
```

5.10.11 flash_halfword_program function

The table below describes the function flash_halfword_program.

Table 257. flash_halfword_program function

Name	Description
Function name	flash_halfword_program
Function prototype	flash_status_type flash_halfword_program(uint32_t address, uint16_t data);
Function description	Write a half-word data to a given address
Input parameter 1	Address: programmed address, half-word-aligned
Input parameter 2	Data: programmed data
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	The programming operation can be allowed only when data in the address are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
uint32_t i;
flash_unlock();
status = flash_sector_erase(0x08001000);
if(status == FLASH_OPERATE_DONE)
{
    /* program 256 halfwords */
    for(i = 0; i < 256; i++)
    {
        status = flash_halfword_program(0x08001000 + i*2, (uint16_t)i);
    }
}
```

5.10.12 flash_byte_program function

The table below describes the function flash_byte_program.

Table 258. flash_byte_program function

Name	Description
Function name	flash_byte_program
Function prototype	flash_status_type flash_byte_program(uint32_t address, uint8_t data);
Function description	Program a byte data to a given address
Input parameter 1	Address: programmed address
Input parameter 2	Data: programmed data
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	The programming operation can be allowed only when data in the address are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
uint32_t i;
flash_unlock();
status = flash_sector_erase(0x08001000);
if(status == FLASH_OPERATE_DONE)
{
    /* program 256 bytes */
    for(i = 0; i < 256; i++)
    {
        status = flash_byte_program(0x08001000 + i*2, (uint8_t)i);
    }
}
```

5.10.13 flash_user_system_data_program function

The table below describes the function flash_user_system_data_program.

Table 259. flash_user_system_data_program function

Name	Description
Function name	flash_user_system_data_program
Function prototype	flash_status_type flash_user_system_data_program (uint32_t address, uint8_t data);
Function description	Program a byte data to a given address in the user system data area
Input parameter 1	Address: programmed address
Input parameter 2	Data: programmed data
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	The programming operation can be allowed only when data and its inverse data in the user system data area are all 0xFF
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_user_system_data_erase();
if(status == FLASH_OPERATE_DONE)
{
    /* program user system data */
    status = flash_user_system_data_program(0x1FFFF804, 0x55);
}
```

5.10.14 flash_epp_set function

The table below describes the function flash_epp_set.

Table 260. flash_epp_set function

Name	Description
Function name	flash_epp_set
Function prototype	flash_status_type flash_epp_set(uint32_t *sector_bits);
Function description	Enable erase programming protection
Input parameter	*sector_bits: Erase programming protection sector address pointer. Each bit in bits 30~0 protects 4KB sectors, bit 31 protects 256KB of Flash memory from sector 62 to sector 63, as we;; as 128KB of Flash memory from sector 124 to sector 127, the extended Flash memory. Setting this bit to 1 enables sector protection.
Output parameter	NA
Return value	Return operation status. Refer to flash_status_type for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
uint32_t epp_val[1];
flash_unlock();
status = flash_user_system_data_erase();
if(status == FLASH_OPERATE_DONE)
{
    epp_val[0] = 0x00000001;
    /* program epp */
    status = flash_epp_set(epp_val);
}
```

5.10.15 flash_epp_status_get function

The table below describes the function flash_epp_status_get.

Table 261. flash_epp_status_get function

Name	Description
Function name	flash_epp_status_get
Function prototype	void flash_epp_status_get(uint32_t *sector_bits);
Function description	Get the status of erase programming protection
Input parameter	NA
Output parameter	*sector_bits: Erase programming protection sector address pointer. Each bit in bits 30~0 protects 4KB sectors, bit 31 protects 256KB of Flash memory from sector 62 to sector 63, as we;; as 128KB of Flash memory from sector 124 to sector 127, the extended Flash memory. Setting this bit to 1 enables sector protection.
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t epp_val[1];
/* get epp status */
flash_epp_status_get(epp_val);
```

5.10.16 flash_fap_enable function

The table below describes the function flash_fap_enable.

Table 262. flash_fap_enable function

Name	Description
Function name	flash_fap_enable
Function prototype	flash_status_type flash_fap_enable(confirm_state new_state);
Function description	Enable Flash low level access protection
Input parameter	new_state: Flash access protection status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	NA
Called functions	NA

Note: This function will erase the whole user system data area. If there were data programmed in the user system data area before calling this function, they have to be re-programmed after calling this function.

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_fap_enable(TRUE);
```

5.10.17 flash_fap_status_get function

The table below describes the function flash_fap_status_get.

Table 263. flash_fap_status_get function

Name	Description
Function name	flash_fap_status_get
Function prototype	flag_status flash_fap_status_get(void);
Function description	Get the status of Flash low level access protection
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status;
status = flash_fap_status_get();
```

5.10.18 flash_fap_high_level_enable

The table below describes the function flash_fap_high_level_enable.

Table 264. flash_fap_high_level_enable function

Name	Description
Function name	flash_fap_high_level_enable
Function prototype	flash_status_type flash_fap_high_level_enable (void);
Function description	Enable Flash high level access protection
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_fap_high_level_enable (void);
```

5.10.19 flash_fap_high_level_status_get

The table below describes the function flash_fap_high_level_status_get.

Table 265. flash_fap_high_level_status_get function

Name	Description
Function name	flash_fap_high_level_status_get
Function prototype	flash_status flash_fap_high_level_status_get (void);
Function description	Get Flash high level access protection status
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status;
status = flash_fap_high_level_status_get();
```

5.10.20 flash_ssb_set function

The table below describes the function flash_ssb_set.

Table 266. flash_ssb_set function

Name	Description
Function name	flash_ssb_set
Function prototype	flash_status_type flash_ssb_set(uint8_t usd_ssb);
Function description	Configure system setting bytes
Input parameter	usd_ssb: system setting byte value is a combination of the selected data from all data group, refer to ssb_data_define for details.
Output parameter	NA
Return value	Return operation status, refer to the flash_status_type for details.
Required preconditions	NA
Called functions	NA

ssb_data_define

type 1:

USD_WDT_ATO_DISABLE: Watchdog auto-start disabled

USD_WDT_ATO_ENABLE: Watchdog auto-start enabled

type 2:

USD_DEPSLP_NO_RST: No reset occurred when entering Deepsleep mode

USD_DEPSLP_RST: Reset occurred when entering Deepsleep mode

type 3:

USD_STDBY_NO_RST: No reset occurred when entering Standby mode

USD_STDBY_RST: Reset occurred when entering Standby mode

type 4:

USD_BOOT1_LOW BOOT1 LOW

USD_BOOT1_HIGH BOOT1 HIGH

type 5:

USD_DEPSLP_WDT_CONTINUE WDT continues counting in Deepsleep mode

USD_DEPSLP_WDT_STOP WDT stops counting in Deepsleep mode

type 6:

USD_STDBY_WDT_CONTINUE WDT continues counting in Standby mode

USD_STDBY_WDT_STOP WDT stops counting in Standby mode

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;
flash_unlock();
status = flash_user_system_data_erase();
if(status == FLASH_OPERATE_DONE)
{
    status = flash_ssb_set(USD_WDT_ATO_DISABLE | USD_DEPSLP_NO_RST | USD_STDBY_RST |
USD_BOOT1_LOW);
}
```

5.10.21 flash_ssb_status_get function

The table below describes the function flash_ssb_status_get.

Table 267. flash_ssb_status_get function

Name	Description
Function name	flash_ssb_status_get
Function prototype	uint8_t flash_ssb_status_get(void);
Function description	Get the status of system setting bytes
Input parameter	NA
Output parameter	NA
Return value	Return system setting byte value, refer to ssb_data_define for details.
Required preconditions	NA
Called functions	NA

Example:

```
uint8_t ssb_val;  
ssb_val = flash_ssb_status_get();
```

5.10.22 flash_interrupt_enable function

The table below describes the function flash_interrupt_enable.

Table 268. flash_interrupt_enable function

Name	Description
Function name	flash_interrupt_enable
Function prototype	void flash_interrupt_enable(uint32_t flash_int, confirm_state new_state);
Function description	Enable Flash interrupts
Input parameter 1	flash_int: Flash interrupt type. Refer to flash_interrupt_type for details.
Input parameter 2	new_state: interrupt status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flash_interrupt_type

FLASH_ERR_INT: Flash error interrupt

FLASH_ODF_INT: Flash operation complete interrupt

Example:

```
flash_interrupt_enable(FLASH_ERR_INT | FLASH_ODF_INT, TRUE);
```

5.10.23 flash_slrib_enable function

The table below describes the function flash_slrib_enable.

Table 269. flash_slrib_enable function

Name	Description
Function name	flash_slrib_enable
Function prototype	flash_status_type flash_slrib_enable(uint32_t pwd, uint16_t start_sector, uint16_t inst_start_sector, uint16_t end_sector);
Function description	Enable security library (sLib) and its address range
Input parameter 1	Pwd: sLib password. The sLib data are saved as ciphertext, associated with encrypted computing. A correct password is entered in order to unlock encryption.
Input parameter 2	start_sector: sLib start sector number
Input parameter 3	inst_start_sector: sLib data area instruction start sector number
Input parameter 4	end_sector: sLib end sector number
Output parameter	NA
Return value	Refer to flash_status_type for details.
Required preconditions	NA
Called functions	NA

Example:

```
flash_status_type status = FLASH_OPERATE_DONE;  
status = flash_slrib_enable(0x12345678, 0x04, 0x05, 0x06);
```

5.10.24 flash_slrib_disable function

The table below describes the function flash_slrib_disable.

Table 270. flash_slrib_disable function

Name	Description
Function name	flash_slrib_disable
Function prototype	error_status flash_slrib_disable(uint32_t pwd);
Function description	Disable security library (sLib)
Input parameter	Pwd: sLib password. it must be entered correctly, otherwise it is not allowed to enter until reset.
Output parameter	NA
Return value	Return error status This parameter can be ERROE or SUCCESS.
Required preconditions	NA
Called functions	NA

Note: Successful calling of this function will erase the whole internal Flash memory.

Example:

```
error_status status;  
status = flash_slrib_disable(0x12345678);
```

5.10.25 flash_slib_state_get function

The table below describes the function flash_slib_state_get.

Table 271. flash_slib_state_get function

Name	Description
Function name	flash_slib_state_get
Function prototype	flag_status flash_slib_state_get(void);
Function description	Get the status of sLib
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status;  
status = flash_slib_state_get();
```

5.10.26 flash_slib_start_sector_get function

The table below describes the function flash_slib_start_sector_get.

Table 272. flash_slib_start_sector_get function

Name	Description
Function name	flash_slib_start_sector_get
Function prototype	uint16_t flash_slib_start_sector_get(void);
Function description	Get the start sector number of sLib
Input parameter	NA
Output parameter	NA
Return value	Return the start sector number of sLib
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t num;  
num = flash_slib_start_sector_get();
```

5.10.27 flash_slib_inststart_sector_get function

The table below describes the function flash_slib_inststart_sector_get.

Table 273. flash_slib_inststart_sector_get function

Name	Description
Function name	flash_slib_inststart_sector_get
Function prototype	uint16_t flash_slib_inststart_sector_get(void);
Function description	Get the start sector number of sLib instruction area
Input parameter	NA
Output parameter	NA
Return value	Return the start sector number of sLib instruction area
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t num;  
num = flash_slib_inststart_sector_get();
```

5.10.28 flash_slib_end_sector_get function

The table below describes the function flash_slib_end_sector_get.

Table 274. flash_slib_end_sector_get function

Name	Description
Function name	flash_slib_end_sector_get
Function prototype	uint16_t flash_slib_end_sector_get(void);
Function description	Get the end sector number of sLib
Input parameter	NA
Output parameter	NA
Return value	Return the end sector number of sLib
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t num;  
num = flash_slib_end_sector_get();
```


5.10.29 flash_crc_calibrate function

The table below describes the function flash_crc_calibrate.

Table 275. flash_crc_calibrate function

Name	Description
Function name	flash_crc_calibrate
Function prototype	uint32_t flash_crc_calibrate(uint32_t start_sector, uint32_t sector_cnt);
Function description	Enable Flash CRC check
Input parameter 1	start_addr: CRC check start address
Input parameter 2	sector_cnt: CRC check sector count
Output parameter	NA
Return value	Return CRC calculation result
Required preconditions	NA
Called functions	NA

Note: The sector set to go through CRC check is only allowed to be on a single area, rather than on both security library and common area.

Example:

```
uint32_t crc_val;  
crc_val = flash_crc_calibrate(0, 10);
```

5.10.30 flash_boot_memory_extension_mode_enable

The table describes the flash_boot_memory_extension_mode_enable

Table 276. flash_boot_memory_extension_mode_enable

Name	Description
Function name	flash_boot_memory_extension_mode_enable
Function prototype	void flash_boot_memory_extension_mode_enable (void);
Function description	Boot memory is used as extended Flash memory
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
flash_boot_memory_extension_mode_enable ();
```

5.10.31 flash_extension_memory_slib_enable

The table describes the flash_extension_memory_slib_enable

Table 277. flash_extension_memory_slib_enable

Name	Description
Function name	flash_extension_memory_slib_enable
Function prototype	void flash_extension_memory_slib_enable (uint32_t pwd, uint16_t inst_start_sector);
Function description	Extended Flash memory is used as security library
Input parameter	Pwd: sLib password inst_start_sector: the start sector of security library instruction area when extended Flash memory is used as security library
Output parameter	NA
Return value	Operature status, see flash_status_type
Required preconditions	NA
Called functions	NA

Example:

```
flash_extension_memory_slib_enable(0x12345678, 0x04);
```

5.10.32 flash_extension_memory_slib_state_get

The table describes the flash_extension_memory_slib_state_get

Table 278. flash_extension_memory_slib_state_get

Name	Description
Function name	flash_extension_memory_slib_state_get
Function prototype	flag_status flash_extension_memory_slib_state_get (void);
Function description	Get the status of extended Flash memory which is used to store security library code
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status This value can be SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
flag_status status;  
status = flash_extension_memory_slib_state_get();
```

5.10.33 flash_em_slib_inststart_sector_get

The table describes the flash_em_slib_inststart_sector_get

Table 279. flash_em_slib_inststart_sector_get

Name	Description
Function name	flash_em_slib_inststart_sector_get
Function prototype	uint16_t flash_em_slib_inststart_sector_get (void);
Function description	Get the start sector of security library instruction area when an extended Flash memory is used as security library
Input parameter	NA
Output parameter	NA
Return value	Return the start sector of security library instruction area when the extended Flash memory is used as a security library
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t num;
num = flash_em_slib_inststart_sector_get();
```

5.11 General-purpose I/Os and multiplexed I/Os (GPIO/IOMUX)

The GPIO register structure `gpio_type` is defined in the “at32f402_405_gpio.h”:

```
/**
 * @brief type define gpio register all
 */
typedef struct
{

} gpio_type;
```

Table 299 gives a list of the GPIO registers

Table 280. Summary of GPIO registers

Register	Description
cfgr	GPIO configuration register
omode	GPIO output mode register
odrvr	GPIO drive capability switch control register
pull	GPIO pull-up/pull-down register
idt	GPIO input register
odt	GPIO output register
scr	GPIO set/clear register
wpr	GPIO write protection register
muxl	GPIO multiplexed function low register
muxh	GPIO multiplexed function high register
clr	GPIO port bit clear register
hdrv	GPIO huge current control register

Table 300 gives a list of GPIO and IOMUX library functions.

Table 281. GPIO and IOMUX library functions

Function name	Description
gpio_reset	GPIO is reset by CRM reset register
gpio_init	Initialize GPIO peripherals
gpio_default_para_init	Initialize GPIO default parameters
gpio_input_data_bit_read	Read GPIO input data bit
gpio_input_data_read	Read GPIO input data
gpio_output_data_bit_read	Read GPIO output data bit
gpio_output_data_read	Read GPIO output data
gpio_bits_set	Set GPIO bits
gpio_bits_reset	Reset GPIO bits
gpio_bits_toggle	Toggle GPIO bits
gpio_bits_write	Write GPIO bits
gpio_port_write	Write GPIO ports
gpio_pin_wp_config	Configure GPIO pin write protection

gpio_pins_huge_driven_config	Configure GPIO huge drive capability
gpio_pin_mux_config	Configure GPIO pin multiplexed function

5.11.1 gpio_reset function

The table below describes the function gpio_reset.

Table 282. gpio_reset function

Name	Description
Function name	gpio_reset
Function prototype	void gpio_reset(gpio_type *gpio_x);
Function description	GPIO is reset by CRM reset register
Input parameter	gpio_x: Select a GPIO peripheral. GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

gpio_reset(GPIOA);

5.11.2 gpio_init function

The table below describes the function gpio_init.

Table 283. gpio_init function

Name	Description
Function name	gpio_init
Function prototype	void gpio_init(gpio_type *gpio_x, gpio_init_type *gpio_init_struct);
Function description	Initialize GPIO peripherals
Input parameter 1	gpio_x: the selected GPIO peripheral GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Input parameter 2	gpio_init_struct: gpio_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

gpio_init_type structure

The gpio_init_type is defined in the at32f402_405_gpio.h:

typedef struct

```
{
    uint32_t          gpio_pins;
    gpio_output_type  gpio_out_type;
    gpio_pull_type    gpio_pull;
    gpio_mode_type    gpio_mode;
    gpio_drive_type   gpio_drive_strength;
```

```
} gpio_init_type;
```

gpio_pins

Select a GPIO pin.

```
GPIO_PINS_0:    GPIO pin 0
GPIO_PINS_1:    GPIO pin 1
GPIO_PINS_2:    GPIO pin 2
GPIO_PINS_3:    GPIO pin 3
GPIO_PINS_4:    GPIO pin 4
GPIO_PINS_5:    GPIO pin 5
GPIO_PINS_6:    GPIO pin 6
GPIO_PINS_7:    GPIO pin 7
GPIO_PINS_8:    GPIO pin 8
GPIO_PINS_9:    GPIO pin 9
GPIO_PINS_10:   GPIO pin 10
GPIO_PINS_11:   GPIO pin 11
GPIO_PINS_12:   GPIO pin 12
GPIO_PINS_13:   GPIO pin 13
GPIO_PINS_14:   GPIO pin 14
GPIO_PINS_15:   GPIO pin 15
```

gpio_out_type

Set GPIO output type.

```
GPIO_OUTPUT_PUSH_PULL:    GPIO push-pull
GPIO_OUTPUT_OPEN_DRAIN:   GPIO open drain
```

gpio_pull

Set GPIO pull-up or pull-down.

```
GPIO_PULL_NONE:           No GPIO pull-up/pull-down
GPIO_PULL_UP:             GPIO pull-up
GPIO_PULL_DOWN:           GPIO pull-down
```

gpio_mode

Set GPIO mode

```
GPIO_MODE_INPUT:          GPIO input mode
GPIO_MODE_OUTPUT:         GPIO output mode
GPIO_MODE_MUX:            GPIO multiplexed mode
GPIO_MODE_ANALOG:         GPIO analog mode
```

gpio_drive_strength

Set GPIO driver capability.

```
GPIO_DRIVE_STRENGTH_STRONGER: Strong drive strength
GPIO_DRIVE_STRENGTH_MODERATE: Moderate drive strength
```

Example:

```
gpio_init_type gpio_init_struct;
gpio_init_struct.gpio_pins = GPIO_PINS_0;
gpio_init_struct.gpio_mode = GPIO_MODE_MUX;
gpio_init_struct.gpio_out_type = GPIO_OUTPUT_PUSH_PULL;
gpio_init_struct.gpio_pull = GPIO_PULL_NONE;
gpio_init_struct.gpio_drive_strength = GPIO_DRIVE_STRENGTH_STRONGER;
```

```
gpio_init(GPIOA, &gpio_init_struct);
```

5.11.3 gpio_default_para_init function

The table below describes the function gpio_default_para_init.

Table 284. gpio_default_para_init function

Name	Description
Function name	gpio_default_para_init
Function prototype	void gpio_default_para_init(gpio_init_type *gpio_init_struct);
Function description	Initialize GPIO default parameters
Input parameter	gpio_init_struct: gpio_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of members of the gpio_init_struct.

Table 285. gpio_init_struct default values

Member	Default value
gpio_pins	GPIO_PINS_ALL
gpio_mode	GPIO_MODE_INPUT
gpio_out_type	GPIO_OUTPUT_PUSH_PULL
gpio_pull	GPIO_PULL_NONE
gpio_drive_strength	GPIO_DRIVE_STRENGTH_STRONGER

Example:

```
gpio_init_type gpio_init_struct;  
gpio_default_para_init(&gpio_init_struct);
```

5.11.4 gpio_input_data_bit_read function

The table below describes the function gpio_input_data_bit_read.

Table 286. gpio_input_data_bit_read function

Name	Description
Function name	gpio_input_data_bit_read
Function prototype	flag_status gpio_input_data_bit_read(gpio_type *gpio_x, uint16_t pins);
Function description	Read GPIO input port pins
Input parameter 1	gpio_x: the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Input parameter 2	Pins: indicates the GPIO pins; refer to “gpio_pins” for details.
Output parameter	NA
Return value	Return GPIO input pin status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_input_data_bit_read(GPIOA, GPIO_PINS_0);
```

5.11.5 gpio_input_data_read function

The table below describes the function gpio_input_data_read.

Table 287. gpio_input_data_read function

Name	Description
Function name	gpio_input_data_read
Function prototype	uint16_t gpio_input_data_read(gpio_type *gpio_x);
Function description	Read GPIO input ports
Input parameter	gpio_x: indicates the selected GPIO peripheral. This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Output parameter	NA
Return value	Return GPIO input port status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_input_data_read(GPIOA);
```


5.11.6 gpio_output_data_bit_read function

The table below describes the function `gpio_output_data_bit_read`.

Table 288. gpio_output_data_bit_read function

Name	Description
Function name	<code>gpio_output_data_bit_read</code>
Function prototype	<code>uint16_t gpio_output_data_bit_read(gpio_type *gpio_x);</code>
Function description	Read GPIO output port pin
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Input parameter 2	Pins: indicates the GPIO pins, refer to, refer to <i>gpio_pins</i> for details.
Output parameter	NA
Return value	Return GPIO output pin status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_output_data_bit_read(GPIOA, GPIO_PINS_0);
```

5.11.7 gpio_output_data_read function

The table below describes the function `gpio_output_data_read`.

Table 289. gpio_output_data_read function

Name	Description
Function name	<code>gpio_output_data_read</code>
Function prototype	<code>uint16_t gpio_output_data_read(gpio_type *gpio_x);</code>
Function description	Read GPIO output port
Input parameter	gpio_x: the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Output parameter	NA
Return value	Read GPIO output port status
Required preconditions	NA
Called functions	NA

Example:

```
gpio_output_data_read(GPIOA);
```

5.11.8 gpio_bits_set function

The table below describes the function gpio_bits_set.

Table 290. gpio_bits_set function

Name	Description
Function name	gpio_bits_set
Function prototype	void gpio_bits_set(gpio_type *gpio_x, uint16_t pins);
Function description	Set GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Input parameter 2	Pins: indicates the GPIO pins, refer to <i>gpio_pins</i> for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_bits_set(GPIOA, GPIO_PINS_0);
```

5.11.9 gpio_bits_reset function

The table below describes the function gpio_bits_reset.

Table 291. gpio_bits_reset function

Name	Description
Function name	gpio_bits_reset
Function prototype	void gpio_bits_reset(gpio_type *gpio_x, uint16_t pins);
Function description	Reset GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Input parameter 2	Pins: indicates the GPIO pins, refer to <i>gpio_pins</i> for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_bits_reset(GPIOA, GPIO_PINS_0);
```

5.11.10 gpio_bits_write function

The table below describes the function gpio_bits_write.

Table 292. gpio_bits_write function

Name	Description
Function name	gpio_bits_toggle
Function prototype	void gpio_bits_toggle (gpio_type *gpio_x, uint16_t pins);
Function description	Write GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Input parameter 2	Pins: indicates the GPIO pins, refer to <i>gpio_pins</i> for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_bits_toggle(GPIOA, GPIO_PINS_0);
```

5.11.11 gpio_bits_write function

The table below describes the function gpio_bits_write.

Table 293. gpio_bits_write function

Name	Description
Function name	gpio_bits_write
Function prototype	void gpio_bits_write(gpio_type *gpio_x, uint16_t pins, confirm_state bit_state);
Function description	Write GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF
Input parameter 2	Pins: indicates the GPIO pins, refer to <i>gpio_pins</i> for details.
Input parameter 3	bit_state: GPIO pin value to be written, it can be 1 (TRUE) or 0 (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_bits_write(GPIOA, GPIO_PINS_0, TRUE);
```

5.11.12 gpio_port_write function

The table below describes the function gpio_port_write.

Table 294. gpio_port_write function

Name	Description
Function name	gpio_port_write
Function prototype	void gpio_port_write(gpio_type *gpio_x, uint16_t port_value);
Function description	Write GPIO ports
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF, GPIOG, GPIOH
Input parameter 2	port_value: indicates the port value to write This parameter can be 0x0000~0xFFFF.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_port_write(GPIOA, 0xFFFF);
```

5.11.13 gpio_pin_wp_config function

The table below describes the function gpio_pin_wp_config.

Table 295. gpio_pin_wp_config function

Name	Description
Function name	gpio_pin_wp_config
Function prototype	void gpio_pin_wp_config(gpio_type *gpio_x, uint16_t pins);
Function description	Configure GPIO pin write protection
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF, GPIOG, GPIOH
Input parameter 2	Pins: indicates the GPIO pins, refer to <i>gpio_pins</i> for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_pin_wp_config(GPIOA, GPIO_PINS_0);
```

5.11.14 gpio_pins_huge_driven_config function

The table below describes the function `gpio_pins_huge_driven_config`.

Table 296. gpio_pins_huge_driven_config function

Name	Description
Function name	<code>gpio_pins_huge_driven_config</code>
Function prototype	<code>void gpio_pins_huge_driven_config(gpio_type *gpio_x, uint16_t pins, confirm_state new_state);</code>
Function description	Configure huge drive capability of GPIO pins
Input parameter 1	gpio_x: indicates the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF, GPIOG, GPIOH
Input parameter 2	Pins: refer to the <i>gpio_pins</i> for details
Input parameter 3	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
gpio_pins_huge_driven_config(GPIOA, GPIO_PINS_0, TRUE);
```

5.11.15 gpio_pin_mux_config function

The table below describes the function `gpio_pin_mux_config`.

Table 297. gpio_pin_mux_config function

Name	Description
Function name	<code>gpio_pin_mux_config</code>
Function prototype	<code>void gpio_pin_mux_config(gpio_type *gpio_x, gpio_pins_source_type gpio_pin_source, gpio_mux_sel_type gpio_mux);</code>
Function description	Configure GPIO pin multiplexed function
Input parameter 1	gpio_x: the selected GPIO peripheral This parameter can be GPIOA, GPIOB, GPIOC, GPIOD, GPIOE, GPIOF, GPIOG, GPIOH
Input parameter 2	gpio_pin_source: GPIO pin to be configured
Input parameter 3	gpio_mux: IOMUX index to be configured
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

gpio_pin_source

Set GPIO pins

GPIO_PINS_SOURCE0: GPIO pin 0

GPIO_PINS_SOURCE1: GPIO pin 1

GPIO_PINS_SOURCE2: GPIO pin 2

GPIO_PINS_SOURCE3:	GPIO pin 3
GPIO_PINS_SOURCE4:	GPIO pin 4
GPIO_PINS_SOURCE5:	GPIO pin 5
GPIO_PINS_SOURCE6:	GPIO pin 6
GPIO_PINS_SOURCE7:	GPIO pin 7
GPIO_PINS_SOURCE8:	GPIO pin 8
GPIO_PINS_SOURCE9:	GPIO pin 9
GPIO_PINS_SOURCE10:	GPIO pin 10
GPIO_PINS_SOURCE11:	GPIO pin 11
GPIO_PINS_SOURCE12:	GPIO pin 12
GPIO_PINS_SOURCE13:	GPIO pin 13
GPIO_PINS_SOURCE14:	GPIO pin 14
GPIO_PINS_SOURCE15:	GPIO pin 15

gpio_mux

Select IOMUX index

GPIO_MUX_0
 GPIO_MUX_1
 GPIO_MUX_2
 GPIO_MUX_3
 GPIO_MUX_4
 GPIO_MUX_5
 GPIO_MUX_6
 GPIO_MUX_7
 GPIO_MUX_8
 GPIO_MUX_9
 GPIO_MUX_10
 GPIO_MUX_11
 GPIO_MUX_12
 GPIO_MUX_13
 GPIO_MUX_14
 GPIO_MUX_15

Example:

```
gpio_pin_mux_config(GPIOA, GPIO_PINS_SOURCE0, GPIO_MUX_0);
```

5.12 I2C interfaces

The I2C register structure `i2c_type` is defined in the “at32f402_405_i2c.h”:

```
/**
 * @brief type define i2c register all
 */
typedef struct
{

} i2c_type;
```

The table below gives a list of the I2C registers

Table 298. Summary of I2C register

Register	Description
ctrl1	I2C Control register 1
ctrl2	I2C Control register 2
oaddr1	I2C Own address register 1
oaddr2	I2C Own address register 2
clkctrl	Timing register
timeout	Timeout register
sts	Status register
clr	Status clear register
pec	PEC register
rxdt	Receive data register
txdt	Transmit data register

The table below gives a list of I2C library functions.

Table 299. Summary of I2C library functions

Function name	Description
<code>i2c_reset</code>	I2C peripheral reset
<code>i2c_init</code>	Set I2C bus speed
<code>i2c_own_address1_set</code>	Set I2C own address 1
<code>i2c_own_address2_set</code>	Set I2C own address 2
<code>i2c_own_address2_enable</code>	Enable I2C own address 2
<code>i2c_smbus_enable</code>	Enable Smbus mode
<code>i2c_enable</code>	Enable I2C
<code>i2c_clock_stretch_enable</code>	Enable clock stretching capability
<code>i2c_ack_enable</code>	Enable ACK response
<code>i2c_addr10_mode_enable</code>	Enable master transmit 10-bit address mode
<code>i2c_transfer_addr_set</code>	Set master transfer address (slave address)
<code>i2c_transfer_addr_get</code>	Get slave address from master
<code>i2c_transfer_dir_set</code>	Set master data transfer direction
<code>i2c_transfer_dir_get</code>	Slave gets data transfer direction

i2c_matched_addr_get	Slave gets address match value
i2c_auto_stop_enable	Enable auto transmission stop conditions
i2c_reload_enable	Enable transmitted data reload mode
i2c_cnt_set	Set number of data to send/receive
i2c_addr10_header_enable	Enable 10-bit address header read timing
i2c_general_call_enable	Enable general call (broadcast address enable)
i2c_smbus_alert_set	Set SMBus alert pin level
i2c_slave_data_ctrl_enable	Enable slave single-byte receive control
i2c_pec_calculate_enable	Enable PEC calculation
i2c_pec_transmit_enable	Enable PEC transmit
i2c_pec_value_get	Get current PEC value
i2c_timeout_set	Set clock level timeout detection
i2c_timeout_detcet_set	Set clock level timeout detect mode
i2c_timeout_enable	Enable clock level timeout detect
i2c_ext_timeout_set	Set accumulated clock stretching timeout
i2c_ext_timeout_enable	Enable accumulated clock stretching timeout
i2c_interrupt_enable	I2C interrupt enable
i2c_interrupt_get	Get interrupt status
i2c_dma_enable	DMA transfer enable
i2c_transmit_set	Set master-initiated transfer
i2c_start_generate	Generate start conditions
i2c_stop_generate	Generate stop conditions
i2c_data_send	Send data
i2c_data_receive	Receive data
i2c_flag_get	Get flag
i2c_flag_clear	Clear flag

Table 300. I2C application-layer library functions

Function name	Description
i2c_config	I2C application initialization
i2c_lowlevel_init	I2C low-layer initialization
i2c_wait_end	I2C wait data transmit complete
i2c_wait_flag	I2C wait flag
i2c_master_transmit	I2C master transmits data (polling mode)
i2c_master_receive	I2C master receives data (polling mode)
i2c_slave_transmit	I2C slave transmits data (polling mode)
i2c_slave_receive	I2C slave receives data (polling mode)
i2c_master_transmit_int	I2C master transmits data (interrupt mode)
i2c_master_receive_int	I2C master receives data (interrupt mode)
i2c_slave_transmit_int	I2C slave transmits data (interrupt mode)
i2c_slave_receive_int	I2C slave receives data (interrupt mode)
i2c_master_transmit_dma	I2C master transmits data (DMA mode)
i2c_master_receive_dma	I2C master receives data (DMA mode)
i2c_slave_transmit_dma	I2C slave transmits data (DMA mode)

Function name	Description
i2c_slave_receive_dma	I2C slave receives data (DMA mode)
i2c_smbus_master_transmit	SMBus master sends data (polling mode)
i2c_smbus_master_receive	SMBus master receives data (polling mode)
i2c_smbus_slave_transmit	SMBus slave sends data (polling mode)
i2c_smbus_slave_receive	SMBus slave receives data (polling mode)
i2c_memory_write	I2C writes data to EEPROM (polling mode)
i2c_memory_write_int	I2C writes data to EEPROM (interrupt mode)
i2c_memory_write_dma	I2C writes data to EEPROM (DMA mode)
i2c_memory_read	I2C reads from EEPROM (polling mode)
i2c_memory_read_int	I2C reads from EEPROM (interrupt mode)
i2c_memory_read_dma	I2C reads from EEPROM (DMA mode)
i2c_evt_irq_handler	I2C event interrupt function
i2c_err_irq_handler	I2C error interrupt function
i2c_dma_tx_irq_handler	I2C DMA Tx interrupt function
i2c_dma_rx_irq_handler	I2C DMA Rx interrupt function

5.12.1 i2c_reset function

The table below describes the function i2c_reset.

Table 301. i2c_reset function

Name	Description
Function name	i2c_reset
Function prototype	void i2c_reset(i2c_type *i2c_x);
Function description	Reset all I2C registers to their initial values through CRM (Clock and reset management)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	void crm_periph_reset(crm_periph_reset_type value, confirm_state new_state)

Example:

i2c_reset(I2C1);

5.12.2 i2c_init function

The table below describes the function i2c_init.

Table 302. i2c_init function

Name	Description
Function name	i2c_init
Function prototype	void i2c_init(i2c_type *i2c_x, uint8_t dfilters, uint32_t clk);
Function description	Set I2C bus speed and digital filter
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3
Input parameter 2	Dfilters: digital filter, ranging from 0x00 to 0x0F When in use, it is recommended to program the digital filter with a maximum value to effectively filter disturbance
Input parameter 3	Clk: timing register (I2C_CLKCTRL) value used to control I2C communication speed. This value can be calculated through “Artery_I2C_Timing_Configuration” defined in the AN0091_AT32F402_405_I2C_Application_Note.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_init(I2C1, 0x0F, 0x80504C4E);
```

5.12.3 i2c_own_address1_set function

The table below describes the function i2c_own_address1_set.

Table 303. i2c_own_address1_set function

Name	Description
Function name	i2c_own_address1_set
Function prototype	void i2c_own_address1_set(i2c_type *i2c_x, i2c_address_mode_type mode, uint16_t address);
Function description	Set own address 1
Input parameter 1	Mode: Own address 1 address mode Refer to the “mode” description below for details.
Input parameter 2	Address: own address 1
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

mode

Own address 1 address mode

I2C_ADDRESS_MODE_7BIT: 7-bit address

I2C_ADDRESS_MODE_10BIT: 10-bit address

Example:

```
i2c_own_address1_set(I2C1, I2C_ADDRESS_MODE_7BIT, 0xA0);
```

5.12.4 i2c_own_address2_set function

The table below describes the function i2c_own_address2_set.

Table 304. i2c_own_address2_set function

Name	Description
Function name	i2c_own_address2_set
Function prototype	void i2c_own_address2_set(i2c_type *i2c_x, uint8_t address, i2c_addr2_mask_type mask);
Function description	Set own address 2. The address 2 becomes active only after it is enabled. Note: only 7-bit address is supported, not 10-bit address mode
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Address: own address 2
Input parameter 3	Mask: own address 2 bit mask Refer to the following “mask” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

mask

Own address 2 bit mask.

I2C_ADDR2_NOMASK:	match address bit [7:1]
I2C_ADDR2_MASK01:	match address bit [7:2] only
I2C_ADDR2_MASK02:	match address bit [7:3] only
I2C_ADDR2_MASK03:	match address bit [7:4] only
I2C_ADDR2_MASK04:	match address bit [7:5] only
I2C_ADDR2_MASK05:	match address bit [7:6] only
I2C_ADDR2_MASK06:	match address bit [7] only
I2C_ADDR2_MASK07:	All non-I2C reserved addresses would respond

Example:

```
i2c_own_address2_set(I2C1, 0xB0, I2C_ADDR2_NOMASK);
```

5.12.5 i2c_own_address2_enable function

The table below describes the function i2c_own_address2_enable.

Table 305. i2c_own_address2_enable function

Name	Description
Function name	i2c_own_address2_enable
Function prototype	void i2c_own_address2_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable own address 2. The address becomes active only after it is enabled. Note that this function should be used in conjunction with the i2c_own_address2_set.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates address 2 status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_own_address2_enable(I2C1, TRUE);
```

5.12.6 i2c_smbus_enable function

The table below describes the function i2c_smbus_enable.

Table 306. i2c_smbus_enable function

Name	Description
Function name	i2c_smbus_enable
Function prototype	void i2c_smbus_enable(i2c_type *i2c_x, i2c_smbus_mode_type mode, confirm_state new_state);
Function description	Enable SMBus mode. After power-on reset, the default mode is I2C mode.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Mode: SMBus mode selection Refer to the following “mode” descriptions for details.
Input parameter 3	new_state: indicates SMBus mode status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

mode

SMBus mode

I2C_SMBUS_MODE_DEVICE: SMBus device

I2C_SMBUS_MODE_HOST: SMBus host

Example:

```
i2c_smbus_enable(I2C1, I2C_SMBUS_MODE_DEVICE, TRUE);
```

5.12.7 i2c_enable function

The table below describes the function i2c_enable.

Table 307. i2c_enable function

Name	Description
Function name	i2c_enable
Function prototype	void i2c_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable I2C peripheral
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates I2C status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_enable(I2C1, TRUE);
```

5.12.8 i2c_clock_stretch_enable function

The table below describes the function i2c_clock_stretch_enable.

Table 308. i2c_clock_stretch_enable function

Name	Description
Function name	i2c_clock_stretch_enable
Function prototype	void i2c_clock_stretch_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable clock stretching capability. This function is applicable to slave mode only. In most cases, enabling the clock stretching mode is recommended in order to prevent slave from having no sufficient time to receive or send data due to slow process speed, which causes a loss of data. It should be noted that the host must be able to support clock stretching function before using this mode by slave. For example, some hosts based on IO analog are not equipped with the clock stretching capability.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates clock stretching status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_clock_stretch_enable(I2C1, TRUE);
```

5.12.9 i2c_ack_enable function

The table below describes the function i2c_ack_enable.

Table 309. i2c_ack_enable function

Name	Description
Function name	i2c_ack_enable
Function prototype	void i2c_ack_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable ACK and NACK. This function is used to enable ACK or NACK of each byte in master and slave mode. For ACK information on I2C communication protocol, refer to I2C protocol or AT32 reference manual.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: indicates ACK response status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_ack_enable(I2C1, TRUE);
```

5.12.10 i2c_addr10_mode_enable function

The table below describes the function i2c_addr10_mode_enable.

Table 310. i2c_addr10_mode_enable function

Name	Description
Function name	i2c_addr10_mode_enable
Function prototype	void i2c_addr10_mode_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable master transmit 10-bit address mode
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: 10-bit address mode enable state This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_addr10_mode_enable(I2C1, TRUE);
```

5.12.11 i2c_transfer_addr_set function

The table below describes the function i2c_transfer_addr_set.

Table 311. i2c_transfer_addr_set function

Name	Description
Function name	i2c_transfer_addr_set
Function prototype	void i2c_transfer_addr_set(i2c_type *i2c_x, uint16_t address);
Function description	Set master transfer address (slave address)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Address: slave address
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_transfer_addr_set(I2C1, 0xA0);
```

5.12.12 i2c_transfer_addr_get function

The table below describes the function i2c_transfer_addr_get.

Table 312. i2c_transfer_addr_get function

Name	Description
Function name	i2c_transfer_addr_get
Function prototype	uint16_t i2c_transfer_addr_get(i2c_type *i2c_x);
Function description	Get slave address sent from master
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	NA
Return value	uint16_t: slave address sent from master
Required preconditions	NA
Called functions	NA

Example:

```
i2c_transfer_addr_get(I2C1);
```

5.12.13 i2c_transfer_dir_set function

The table below describes the function i2c_transfer_dir_set.

Table 313. i2c_transfer_dir_set function

Name	Description
Function name	i2c_transfer_dir_set
Function prototype	void i2c_transfer_dir_set(i2c_type *i2c_x, i2c_transfer_dir_type i2c_direction);
Function description	Set master data transfer direction
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Direction: data transfer direction Refer to the following “direction” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

direction

Data transfer direction

I2C_DIR_TRANSMIT: Master sends data

I2C_DIR_RECEIVE: Master receives data

Example:

```
i2c_transfer_dir_set(I2C1, I2C_DIR_TRANSMIT);
```

5.12.14 i2c_transfer_dir_get function

The table below describes the function i2c_transfer_dir_get.

Table 314. i2c_transfer_dir_get function

Name	Description
Function name	i2c_transfer_dir_get
Function prototype	i2c_transfer_dir_type i2c_transfer_dir_get(i2c_type *i2c_x);
Function description	Get slave data transfer direction
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	NA
Return value	i2c_transfer_dir_type: slave data transfer direction Refer to the following “i2c_transfer_dir_type” descriptions for details.
Required preconditions	NA
Called functions	NA

i2c_transfer_dir_type

Data transfer direction.

I2C_DIR_TRANSMIT: master sends data, and slave receives data

I2C_DIR_RECEIVE: master receives data and slave sends data

Example:

```
i2c_transfer_dir_get(I2C1);
```


5.12.15 i2c_matched_addr_get function

The table below describes the function i2c_matched_addr_get.

Table 315. i2c_matched_addr_get function

Name	Description
Function name	i2c_matched_addr_get
Function prototype	uint8_t i2c_matched_addr_get(i2c_type *i2c_x);
Function description	Get slave address match value
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	NA
Return value	uint8_t: slave matched address
Required preconditions	NA
Called functions	NA

Example:

```
i2c_matched_addr_get(I2C1);
```

5.12.16 i2c_auto_stop_enable function

The table below describes the function i2c_auto_stop_enable.

Table 316. i2c_auto_stop_enable function

Name	Description
Function name	i2c_auto_stop_enable
Function prototype	void i2c_auto_stop_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable auto transmit stop conditions
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: auto transmit stop condition enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_auto_stop_enable(I2C1, TRUE);
```

5.12.17 i2c_reload_enable function

The table below describes the function i2c_reload_enable.

Table 317. i2c_reload_enable function

Name	Description
Function name	i2c_reload_enable
Function prototype	void i2c_reload_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable transmitted data reload mode
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: reload mode enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_reload_enable(I2C1, TRUE);
```

5.12.18 i2c_cnt_set function

The table below describes the function i2c_cnt_set.

Table 318. i2c_cnt_set function

Name	Description
Function name	i2c_cnt_set
Function prototype	void i2c_cnt_set(i2c_type *i2c_x, uint8_t cnt);
Function description	Set the number of data to send or receive, ranging from 1 to 255
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Cnt: number of data to send/receive
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_cnt_set(I2C1, 200);
```

5.12.19 i2c_addr10_header_enable function

The table below describes the function i2c_addr10_header_enable.

Table 319. i2c_addr10_header_enable function

Name	Description
Function name	i2c_addr10_header_enable
Function prototype	void i2c_addr10_header_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable 10-bit address header read timing
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: enable state of auto transmit stop conditions This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_addr10_header_enable(I2C1, TRUE);
```

5.12.20 i2c_general_call_enable function

The table below describes the function i2c_dma_enable.

Table 320. i2c_general_call_enable function

Name	Description
Function name	i2c_general_call_enable
Function prototype	void i2c_general_call_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable broadcast address. After enabled, broadcast address 0x00 is responded
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: Broadcast address enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_general_call_enable(I2C1, TRUE);
```

5.12.21 i2c_smbus_alert_set function

The table below describes the function i2c_smbus_alert_set.

Table 321. i2c_smbus_alert_set function

Name	Description
Function name	i2c_smbus_alert_set
Function prototype	void i2c_smbus_alert_set(i2c_type *i2c_x, i2c_smbus_alert_set_type level);
Function description	Set SMBus alert pin level (high or low)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	level: SMBus alert pin level Refer to the following “level” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

SMBus alert pin level

I2C_SMBUS_ALERT_LOW: SMBus alert pin output low

I2C_SMBUS_ALERT_HIGH: SMBus alert pin output high

Example:

```
i2c_smbus_alert_set(I2C1, I2C_SMBUS_ALERT_LOW);
```

5.12.22 i2c_slave_data_ctrl_enable function

The table below describes the function i2c_start_generate.

Table 322. i2c_start_generate function

Name	Description
Function name	i2c_slave_data_ctrl_enable
Function prototype	void i2c_slave_data_ctrl_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable slave data receive control. This function is used to control ACK or NACK response to each received byte when in slave receive mode. It is usually used for SMBus.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_data_ctrl_enable(I2C1, FALSE);
```

5.12.23 i2c_pec_calculate_enable function

The table below describes the function i2c_pec_calculate_enable

Table 323. i2c_pec_calculate_enable

Name	Description
Function name	i2c_pec_calculate_enable
Function prototype	void i2c_pec_calculate_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable PEC calculation
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: PEC calculation state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_pec_calculate_enable(I2C1, TRUE);
```

5.12.24 i2c_pec_transmit_enable function

The table below describes the function i2c_pec_transmit_enable.

Table 324. i2c_pec_transmit_enable function

Name	Description
Function name	i2c_pec_transmit_enable
Function prototype	void i2c_pec_transmit_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	PEC transmit enable (send/receive PEC)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: PEC transmit enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_pec_transmit_enable(I2C1, TRUE);
```

5.12.25 i2c_pec_value_get function

The table below describes the function i2c_pec_value_get

Table 325. i2c_pec_value_get function

Name	Description
Function name	i2c_pec_value_get
Function prototype	uint8_t i2c_pec_value_get(i2c_type *i2c_x);
Function description	Get current PEC value
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	uint8_t: current PEC value
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
pec_value = i2c_pec_value_get(I2C1);
```

5.12.26 i2c_timeout_set function

The table below describes the function i2c_timeout_set.

Table 326. i2c_timeout_set function

Name	Description
Function name	i2c_timeout_set
Function prototype	void i2c_timeout_set(i2c_type *i2c_x, uint16_t timeout);
Function description	Set SCL line level timeout detect time
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Timeout: timeout value, ranging from 0x0000 to 0x0FFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_timeout_set(I2C1, 0x0FFF);
```

5.12.27 i2c_timeout_detcet_set function

The table below describes the function i2c_timeout_detcet_set.

Table 327. i2c_timeout_detcet_set function

Name	Description
Function name	i2c_timeout_detcet_set
Function prototype	void i2c_timeout_detcet_set(i2c_type *i2c_x, i2c_timeout_detcet_type mode);
Function description	Set SCL line level timeout detect mode
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Mode: level detect mode Refer to the following “mode” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

mode

Level detection mode.

I2C_TIMEOUT_DETCET_HIGH: High level timeout detect

I2C_TIMEOUT_DETCET_LOW: Low level timeout detect

Example:

```
i2c_timeout_detcet_set(I2C1, I2C_TIMEOUT_DETCET_HIGH);
```

5.12.28 i2c_timeout_enable function

The table below describes the function i2c_timeout_enable.

Table 328. i2c_timeout_enable function

Name	Description
Function name	i2c_timeout_enable
Function prototype	void i2c_timeout_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable SCL line level timeout detect
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: level timeout detect enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_timeout_enable(I2C1, TRUE);
```

5.12.29 i2c_ext_timeout_set function

The table below describes the function i2c_ext_timeout_set.

Table 329. i2c_ext_timeout_set function

Name	Description
Function name	i2c_ext_timeout_set
Function prototype	void i2c_ext_timeout_set(i2c_type *i2c_x, uint16_t timeout);
Function description	Set SCL line cumulative clock stretching timeout value, usually used in SMBus mode
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Timeout: range from 0x0000 to 0x0FFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_ext_timeout_set(I2C1, 0x0FFF);
```

5.12.30 i2c_ext_timeout_enable function

The table below describes the function i2c_ext_timeout_enable.

Table 330. i2c_ext_timeout_enable function

Name	Description
Function name	i2c_ext_timeout_enable
Function prototype	void i2c_ext_timeout_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable SCL line cumulative clock stretching timeout
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	new_state: cumulative clock stretching timeout enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_ext_timeout_enable(I2C1, TRUE);
```


5.12.31 i2c_interrupt_enable function

The table below describes the function i2c_interrupt_enable.

Table 331. i2c_interrupt_enable function

Name	Description
Function name	i2c_interrupt_enable
Function prototype	void i2c_interrupt_enable(i2c_type *i2c_x, uint32_t source, confirm_state new_state);
Function description	I2C interrupt enable
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Source: interrupt sources Refer to the following “source” descriptions for details.
Input parameter 3	new_state: interrupt enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

Interrupt source.

I2C_TD_INT:	Data transmit interrupt
I2C_RD_INT:	Data receive interrupt
I2C_ADDR_INT:	Address match interrupt
I2C_ACKFIAL_INT:	Acknowledge failure interrupt
I2C_STOP_INT:	Stop condition generation complete interrupt
I2C_TDC_INT:	Data transfer complete interrupt
I2C_ERR_INT:	Error interrupt

Example:

```
i2c_interrupt_enable(I2C1, I2C_TD_INT, TRUE);
```

5.12.32 i2c_interrupt_get function

The table below describes the function i2c_interrupt_get.

Table 332. i2c_interrupt_get function

Name	Description
Function name	i2c_interrupt_get
Function prototype	flag_status i2c_interrupt_get(i2c_type *i2c_x, uint16_t source);
Function description	Get interrupt enable state
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Source: interrupt source Refer to the following “source” descriptions for details.
Output parameter	flag_status: flag status This parameter can be SET or RESET
Return value	NA
Required preconditions	NA
Called functions	NA

source

Interrupt source.

I2C_TD_INT:	Data transmit interrupt
I2C_RD_INT:	Data receive interrupt
I2C_ADDR_INT:	Address match interrupt
I2C_ACKFIAL_INT:	Acknowledge failure interrupt
I2C_STOP_INT:	Stop condition generation complete interrupt
I2C_TDC_INT:	Data transfer complete interrupt
I2C_ERR_INT:	Error interrupt

Example:

```
i2c_interrupt_get(I2C1, I2C_TD_INT, TRUE);
```

5.12.33 i2c_dma_enable function

The table below describes the function i2c_dma_enable.

Table 333. i2c_dma_enable function

Name	Description
Function name	i2c_dma_enable
Function prototype	void i2c_dma_enable(i2c_type *i2c_x, i2c_dma_request_type dma_req, confirm_state new_state);
Function description	DMA transfer enable
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	dma_req: DMA request Refer to the following “dma_req” descriptions for details.
Input parameter 3	new_state: DMA enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_req

I2C_DMA_REQUEST_TX: DMA data transmit enable

I2C_DMA_REQUEST_RX: DMA data receive enable

Example:

```
i2c_dma_enable(I2C1, I2C_DMA_REQUEST_TX, TRUE);
```

5.12.34 i2c_transmit_set function

The table below describes the function i2c_transmit_set.

Table 334. i2c_transmit_set function

Name	Description
Function name	i2c_transmit_set
Function prototype	void i2c_transmit_set(i2c_type *i2c_x, uint16_t address, uint8_t cnt, i2c_reload_stop_mode_type rld_stop, i2c_start_mode_type start);
Function description	Set master transmit. This function is used to start data transfer on bus.
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Address: slave address
Input parameter 3	Cnt: count of data to send/receive
Input parameter 4	rld_stop: reload mode and STOP condition generation mode Refer to the following “rld_stop” descriptions for details.
Input parameter 5	Start: set START condition generation mode Refer to the following “start” descriptions for details.
Output parameter	NA
Return value	NA

Name	Description
Required preconditions	NA
Called functions	NA

rd_stop

Reload mode and STOP condition generation mode.

I2C_AUTO_STOP_MODE: Auto stop mode (automatically sends STOP condition)

I2C_SOFT_STOP_MODE: Software stop mode (software sends STOP condition, usually RESTART condition)

I2C_RELOAD_MODE: Reload mode (when a single transfer >255)

start

START condition generation mode.

I2C_WITHOUT_START: Start sending data, without START, used in reload mode

I2C_GEN_START_READ: Start sending data, with START condition (for master receive data)

I2C_GEN_START_WRITE: Start sending data with START condition (for master transmit data)

Example:

```
i2c_transmit_set(I2C1, I2C_AUTO_STOP_MODE, I2C_GEN_START_WRITE);
```

5.12.35 i2c_start_generate function

The table below describes the function i2c_start_generate.

Table 335. i2c_slave_transmit function

Name	Description
Function name	i2c_start_generate
Function prototype	void i2c_start_generate(i2c_type *i2c_x);
Function description	Generate a START condition (for master)
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_start_generate(I2C1);
```

5.12.36 i2c_stop_generate function

The table below describes the function i2c_stop_generate.

Table 336. i2c_stop_generate function

Name	Description
Function name	i2c_stop_generate
Function prototype	void i2c_stop_generate(i2c_type *i2c_x);
Function description	Generate a STOP condition
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_stop_generate(I2C1);
```

5.12.37 i2c_data_send function

The table below describes the function i2c_data_send.

Table 337. i2c_data_send function

Name	Description
Function name	i2c_data_send
Function prototype	void i2c_data_send(i2c_type *i2c_x, uint8_t data);
Function description	Send data
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Data: data to be sent
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_data_send(I2C1, 0x55);
```

5.12.38 i2c_data_receive function

The table below describes the function i2c_data_receive

Table 338. i2c_data_receive function

Name	Description
Function name	i2c_data_receive
Function prototype	uint8_t i2c_data_receive(i2c_type *i2c_x);
Function description	Receive data
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Output parameter	uint8_t: data to be received
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
data_value = i2c_data_receive(I2C1);
```

5.12.39 i2c_flag_get function

The table below describes the function i2c_flag_get

Table 339. i2c_flag_get function

Name	Description
Function name	i2c_flag_get
Function prototype	flag_status i2c_flag_get(i2c_type *i2c_x, uint32_t flag);
Function description	Get flag status
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	flag: the selected flag Refer to the following “flag” descriptions for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This bit is used to select a flag to get its status. Optional parameters are below:

- I2C_TDBE_FLAG: Transmit data register empty flag
- I2C_TDIS_FLAG: Transmit interrupt status flag
- I2C_RDBF_FLAG: Receive data buffer full flag
- I2C_ADDRF_FLAG: Address match flag
- I2C_ACKFAIL_FLAG: Acknowledge failure flag
- I2C_STOPF_FLAG: STOP condition generation complete flag
- I2C_TDC_FLAG: Data transfer complete flag
- I2C_TCRLD_FLAG: Transfer complete to wait for loading data

I2C_BUSERR_FLAG: Bus error flag
 I2C_ARLOST_FLAG: Arbitration lost flag
 I2C_OUF_FLAG: Overflow or underflow flag
 I2C_PECERR_FLAG: PEC receive error flag
 I2C_TMOUT_FLAG: SMBus timeout flag
 I2C_ALERTF_FLAG: SMBus alert flag
 I2C_BUSYF_FLAG: Bus busy flag
 I2C_SDIR_FLAG: Slave data transfer direction

Example:

```
i2c_flag_get(I2C1, I2C_TDIS_FLAG);
```

5.12.40 i2c_interrupt_flag_get function

The table below describes the function i2c_interrupt_flag_get.

Table 340. i2c_interrupt_flag_get function

Name	Description
Function name	i2c_interrupt_flag_get
Function prototype	flag_status i2c_interrupt_flag_get(i2c_type *i2c_x, uint32_t flag);
Function description	Get flag status and judge the corresponding interrupt enable bit
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	flag: the selected flag Refer to the following “flag” descriptions for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This bit is used to select a flag to get its status. Optional parameters are below:

I2C_TDBE_FLAG: Transmit data register empty flag
 I2C_TDIS_FLAG: Transmit interrupt status flag
 I2C_RDBF_FLAG: Receive data buffer full flag
 I2C_ADDRF_FLAG: Address match flag
 I2C_ACKFAIL_FLAG: Acknowledge failure flag
 I2C_STOPF_FLAG: STOP condition generation complete flag
 I2C_TDC_FLAG: Data transfer complete flag
 I2C_TCRLD_FLAG: Transfer complete to wait for loading data
 I2C_BUSERR_FLAG: Bus error flag
 I2C_ARLOST_FLAG: Arbitration lost flag
 I2C_OUF_FLAG: Overflow or underflow flag
 I2C_PECERR_FLAG: PEC receive error flag
 I2C_TMOUT_FLAG: SMBus timeout flag
 I2C_ALERTF_FLAG: SMBus alert falg

Example

```
i2c_interrupt_flag_get(I2C1, I2C_TDIS_FLAG);
```

5.12.41 i2c_flag_clear function

The table below describes the function i2c_flag_clear.

Table 341. i2c_flag_clear function

Name	Description
Function name	i2c_flag_clear
Function prototype	void i2c_flag_clear(i2c_type *i2c_x, uint32_t flag);
Function description	Clear flag
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1, I2C2, I2C3.
Input parameter 2	Flag: the selected flag Refer to the following “flag” descriptions for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

This bit is used to select a flag, including:

- I2C_ADDRF_FLAG: Address match flag
- I2C_ACKFAIL_FLAG: Acknowledge failure flag
- I2C_STOPF_FLAG: STOP condition generation complete flag
- I2C_BUSERR_FLAG: Bus error flag
- I2C_ARLOST_FLAG: Arbitration lost flag
- I2C_OUF_FLAG: Overflow or underflow flag
- I2C_PECERR_FLAG: PEC receive error flag
- I2C_TMOUT_FLAG: SMBus timeout flag
- I2C_ALERTF_FLAG: SMBus alert flag

Example:

```
i2c_flag_clear(I2C1, I2C_ACKFAIL_FLAG);
```


5.12.42 i2c_wakeup_enable function

The table below describes the function i2c_wakeup_enable.

Table 342. i2c_wakeup_enable function

Name	Description
Function name	i2c_wakeup_enable
Function prototype	void i2c_wakeup_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable Deepsleep mode wakeup
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1
Input parameter 2	new_state: Deepsleep mode wakeup enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_wakeup_enable(I2C1, TRUE);
```

5.12.43 i2c_analog_filter_enable function

The table below describes the function i2c_analog_filter_enable.

Table 343. i2c_analog_filter_enable function

Name	Description
Function name	i2c_analog_filter_enable
Function prototype	void i2c_analog_filter_enable(i2c_type *i2c_x, confirm_state new_state);
Function description	Enable analog filter
Input parameter 1	i2c_x: indicates the selected I2C peripheral This parameter can be I2C1
Input parameter 2	new_state: enable state This parameter can be TRUE or FALSE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2c_analog_filter_enable(I2C1, TRUE);
```

5.12.44 i2c_config function

The table below describes the function i2c_config.

Table 344. i2c_config function

Name	Description
Function name	i2c_config
Function prototype	void i2c_config(i2c_handle_type* hi2c);
Function description	I2C initialization function used to initialize I2C. Call the function i2c_lowlevel_init() to initialize I2C peripherals, GPIO, DMA, interrupts and others.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

i2c_handle_type* hi2c

i2c_handle_type is defined in the i2c_application.h.

typedef struct

```
{
    i2c_type          *i2cx;
    uint8_t           *pbuff;
    __IO uint16_t      psize;
    __IO uint16_t      pcount;
    __IO uint32_t       mode;
    __IO uint32_t      timeout;
    __IO uint32_t      status;
    __IO i2c_status_type error_code;
    dma_channel_type   *dma_tx_channel;
    dma_channel_type   *dma_rx_channel;
    dma_init_type      dma_init_struct;
}i2c_handle_type;
```

i2cx

Select an I2C peripheral from I2C1, I2C2 or I2C3

pbuff

An array of data to be sent or received.

psize

This bit is used to count the size of bytes in a single transfer when the transfer size is over 255. It is used in internal state machine. Users don't care.

pcount

The number of data to be sent or received.

mode

I2C communication mode. It is used in internal state machine. Users don't care.

timeout

Communications timeout

status

Transfer status. It is used in internal state machine. Users don't care.

error_code

This bit is used to enumerate error code in the `i2c_status_type`. When a communication error occurred, it logs the corresponding error code.

<code>I2C_OK:</code>	Communication OK
<code>I2C_ERR_STEP_1:</code>	Step 1 error
<code>I2C_ERR_STEP_2:</code>	Step 2 error
<code>I2C_ERR_STEP_3:</code>	Step 3 error
<code>I2C_ERR_STEP_4:</code>	Step 4 error
<code>I2C_ERR_STEP_5:</code>	Step 5 error
<code>I2C_ERR_STEP_6:</code>	Step 6 error
<code>I2C_ERR_STEP_7:</code>	Step 7 error
<code>I2C_ERR_STEP_8:</code>	Step 8 error
<code>I2C_ERR_STEP_9:</code>	Step 9 error
<code>I2C_ERR_STEP_10:</code>	Step 10 error
<code>I2C_ERR_STEP_11:</code>	Step 11 error
<code>I2C_ERR_STEP_12:</code>	Step 12 error
<code>I2C_ERR_TCRLD:</code>	Wait for TCRLD timeout
<code>I2C_ERR_TDC:</code>	Wait for TDC timeout
<code>I2C_ERR_ADDR:</code>	Address send error
<code>I2C_ERR_STOP:</code>	STOP condition send error
<code>I2C_ERR_ACKFAIL:</code>	Acknowledge error
<code>I2C_ERR_TIMEOUT:</code>	Timeout error
<code>I2C_ERR_INTERRUPT:</code>	Enter an interrupt when an error event occurred

dma_tx_channel

I2C transmit DMA channel

dma_rx_channel

I2C receive DMA channel

dma_init_struct

DMA initialization structure

Example:

```
i2c_handle_type hi2c;
hi2c.i2cx = I2C1;
i2c_config(&hi2c);
```

5.12.45 i2c_lowlevel_init function

The table below describes the function i2c_lowlevel_init.

Table 345. i2c_lowlevel_init function

Name	Description
Function name	i2c_lowlevel_init
Function prototype	void i2c_lowlevel_init(i2c_handle_type* hi2c);
Function description	I2C lower-level initialization callback function. It is called in the i2c_config to initialize I2C peripherals, GPIO, DMA, interrupts, etc. It requires users to implement I2C initialization inside the function.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void i2c_lowlevel_init(i2c_handle_type* hi2c)
{
    if(hi2c->i2cx == I2C1)
    {
        Implement I2C1 initialization
    }
    else if(hi2c->i2cx == I2C2)
    {
        Implement I2C1 initialization
    }
}
```

5.12.46 i2c_wait_end function

The table below describes the function i2c_wait_end.

Table 346. i2c_wait_end function

Name	Description
Function name	i2c_wait_end
Function prototype	i2c_status_type i2c_wait_end(i2c_handle_type* hi2c, uint32_t timeout);
Function description	Wait for the end of communications. This function is used in DMA and interrupt transfer modes as they are non-blocking functions and can thus be used to wait for the end of transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```

if (i2c_master_transmit_dma(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF) != I2C_OK)
{
    error_handler(i2c_status);
}

/* wait for the end of transfer*/
if(i2c_wait_end(&hi2c, 0xFFFFFFFF) != I2C_OK)
{
    error_handler(i2c_status);
}
    
```

5.12.47 i2c_wait_flag function

The table below describes the function i2c_wait_flag.

Table 347. i2c_wait_flag function

Name	Description
Function name	i2c_wait_flag
Function prototype	i2c_status_type i2c_wait_flag(i2c_handle_type* hi2c, uint32_t flag, uint32_t event_check, uint32_t timeout)
Function description	Wait for a flag to be set or reset Only BUSFY flag is “wait for a flag to be reset”, and others are “wait for a flag to be set”
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Flag: the selected flag Refer to the following “flag” descriptions for details.
Input parameter 3	event_check: check if the event has occurred or not while waiting for a flag Refer to the “event_check” descriptions below for details.
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

flag

Select a flag to wait for.

I2C_TDBE_FLAG:	Transmit data register empty flag
I2C_TDIS_FLAG:	Transmit interrupt status flag
I2C_RDBF_FLAG:	Receive data buffer full flag
I2C_ADDRF_FLAG:	Address match flag
I2C_ACKFAIL_FLAG:	Acknowledge failure flag
I2C_STOPF_FLAG:	STOP condition generation complete flag
I2C_TDC_FLAG:	Data transfer complete flag
I2C_TCRLD_FLAG:	Transfer complete to wait for loading data
I2C_BUSERR_FLAG:	Bus error flag
I2C_ARLOST_FLAG:	Arbitration lost flag
I2C_OUF_FLAG:	Overflow or underflow flag
I2C_PECERR_FLAG:	PEC receive error flag
I2C_TMOUT_FLAG:	SMBus timeout flag
I2C_ALERTF_FLAG:	SMBus alert flag
I2C_BUSYF_FLAG:	Bus busy flag
I2C_SDIR_FLAG:	Slave data transfer direction

event_check

Check if the event has occurred or not while waiting for a flag.

I2C_EVENT_CHECK_NONE:	None
I2C_EVENT_CHECK_ACKFAIL:	Check ACKFAIL event

I2C_EVENT_CHECK_STOP: Check STOP event

Example:

```
i2c_wait_flag(&hi2c, I2C_BUSYF_FLAG, I2C_EVENT_CHECK_NONE, 0xFFFFFFFF);
```

5.12.48 i2c_master_transmit function

The table below describes the function i2c_master_transmit.

Table 348. i2c_master_transmit function

Name	Description
Function name	i2c_master_transmit
Function prototype	i2c_status_type i2c_master_transmit(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master sends data (polling mode). This is a blocking function, and so I2C transfer ends after the function is executed
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address of to-be-sent data
Input parameter 4	Size: the size of data to be sent
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_transmit(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF);
```

5.12.49 i2c_master_receive function

The table below describes the function i2c_master_receive.

Table 349. i2c_master_receivefunction

Name	Description
Function name	i2c_master_receive
Function prototype	i2c_status_type i2c_master_receive(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master receives data (polling mode). This function is a blocking type. After the execution is done, so does I2C transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address to receive data
Input parameter 4	Size: number of data to receive
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_receive(&hi2c, 0xB0, rx_buf, 8, 0xFFFFFFFF);
```


5.12.50 i2c_slave_transmit function

The table below describes the function i2c_slave_transmit.

Table 350. i2c_slave_transmit function

Name	Description
Function name	i2c_slave_transmit
Function prototype	i2c_status_type i2c_slave_transmit(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave sends data (polling mode). This function is a blocking type. In other words, after the function execution is done, so is I2C transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address of data to be sent
Input parameter 3	Size: number of data to be sent
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_transmit(&hi2c, tx_buf, 8, 0xFFFFFFFF);
```

5.12.51 i2c_slave_receive function

The table below describes the function i2c_slave_receive.

Table 351. i2c_slave_receive function

Name	Description
Function name	i2c_slave_receive
Function prototype	i2c_status_type i2c_slave_receive(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave receives data (polling mode). This function is a blocking type. In other words, after the function execution is done, so is I2C transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address to receive data
Input parameter 3	Size: number of data to be received
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_receive(&hi2c, rx_buf, 8, 0xFFFFFFFF);
```

5.12.52 i2c_master_transmit_int function

The table below describes the function i2c_master_transmit_int.

Table 352. i2c_master_transmit_int function

Name	Description
Function name	i2c_master_transmit_int
Function prototype	i2c_status_type i2c_master_transmit_int(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master sends data (interrupt mode). This function is a non-blocking type. In other words, after the function execution is done, I2C transfer has not completed yet. In this case, it is possible to call the i2c_wait_end() to wait for the completion of communication.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address of data to be sent
Input parameter 4	Size: number of data to be sent
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_transmit_int(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF);
```

5.12.53 i2c_master_receive_int function

The table below describes the function i2c_master_receive_int.

Table 353. i2c_master_receive_int function

Name	Description
Function name	i2c_master_receive_int
Function prototype	i2c_status_type i2c_master_receive_int(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master receives data (through interrupt mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the i2c_wait_end() to wait for the end of transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address to receive data
Input parameter 4	Size: number of data to be received
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_receive_int(&hi2c, 0xB0, rx_buf, 8, 0xFFFFFFFF);
```

5.12.54 i2c_slave_transmit_int function

The table below describes the function `i2c_master_receive_int`.

Table 354. i2c_master_receive_int function

Name	Description
Function name	<code>i2c_slave_transmit_int</code>
Function prototype	<code>i2c_status_type i2c_slave_transmit_int(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	Slave sends data (through interrupt mode). This function operates in non-blocking mode. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the <code>i2c_wait_end()</code> to wait for the end of transfer.
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address of data to be sent
Input parameter 3	Size: number of data to be sent
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	<code>i2c_status_type</code> : error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_transmit_int(&hi2c, tx_buf, 8, 0xFFFFFFFF);
```

5.12.55 i2c_slave_receive_int function

The table below describes the function `i2c_slave_receive_int`

Table 355. i2c_master_receive_int function

Name	Description
Function name	<code>i2c_slave_receive_int</code>
Function prototype	<code>i2c_status_type i2c_slave_receive_int(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	Slave receives data (through interrupt mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the <code>i2c_wait_end()</code> to wait for the end of transfer.
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address to receive data
Input parameter 3	Size: number of data to be received
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	<code>i2c_status_type</code> : error code

Name	Description
	Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_receive_int(&hi2c, rx_buf, 8, 0xFFFFFFFF);
```

5.12.56 i2c_master_transmit_dma function

The table below describes the function i2c_master_transmit_dma.

Table 356. i2c_master_transmit_dma function

Name	Description
Function name	i2c_master_transmit_dma
Function prototype	i2c_status_type i2c_master_transmit_dma(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Master sends data (through DMA mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the i2c_wait_end() to wait for the end of transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address of data to be sent
Input parameter 4	Size: number of data to send
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_transmit_dma(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF);
```

5.12.57 i2c_master_receive_dma function

The table below describes the function `i2c_master_receive_dma`.

Table 357. i2c_master_receive_dma function

Name	Description
Function name	<code>i2c_master_receive_dma</code>
Function prototype	<code>i2c_status_type i2c_master_receive_dma(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	Master receives data (through DMA mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the <code>i2c_wait_end()</code> to wait for the end of transfer.
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address to receive data
Input parameter 4	Size: number of data to be received
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	<code>i2c_status_type</code> : error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_master_receive_dma(&hi2c, 0xB0, rx_buf, 8, 0xFFFFFFFF);
```

5.12.58 i2c_slave_transmit_dma function

The table below describes the function `i2c_slave_transmit_dma`.

Table 358. i2c_slave_transmit_dma function

Name	Description
Function name	<code>i2c_slave_transmit_dma</code>
Function prototype	<code>i2c_status_type i2c_slave_transmit_dma(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	Slave sends data (through DMA mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the <code>i2c_wait_end()</code> to wait for the end of transfer.
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address of data to be sent
Input parameter 3	Size: number of data to be sent
Input parameter 4	Timeout: wait timeout
Output parameter	NA

Name	Description
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_transmit_dma(&hi2c, tx_buf, 8, 0xFFFFFFFF);
```

5.12.59 i2c_slave_receive_dma function

The table below describes the function i2c_slave_receive_dma.

Table 359. i2c_slave_receive_dma function

Name	Description
Function name	i2c_slave_receive_dma
Function prototype	i2c_status_type i2c_slave_receive_dma(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Slave receives data (through DMA mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the i2c_wait_end() to wait for the end of transfer.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address to receive data
Input parameter 3	Size: number of data to be received
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_slave_receive_dma(&hi2c, rx_buf, 8, 0xFFFFFFFF);
```

5.12.60 i2c_smbus_master_transmit function

The table below describes the function i2c_smbus_master_transmit

Table 360. i2c_smbus_master_transmit function

Name	Description
Function name	i2c_smbus_master_transmit
Function prototype	i2c_status_type i2c_smbus_master_transmit(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	SMBus master sends data (through polling mode). This function is a blocking type. In other words, after the function execution is done, so is data transfer. It is mainly used for PEC transmission and reception.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address of data to be sent
Input parameter 4	Size: number of data to be sent
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_smbus_master_transmit(&hi2c, 0xB0, tx_buf, 8, 0xFFFFFFFF);
```


5.12.61 i2c_smbus_master_receive function

The table below describes the function i2c_smbus_master_receive

Table 361. i2c_smbus_master_receive function

Name	Description
Function name	i2c_smbus_master_receive
Function prototype	i2c_status_type i2c_smbus_master_receive(i2c_handle_type* hi2c, uint16_t address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	SMBus master receives data (through polling mode). This function is a blocking type. In other words, after the function execution is done, so is data transfer. It is mainly used for PEC transmission and reception.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Address: slave address
Input parameter 3	Pdata: array address to receive data
Input parameter 4	Size: number of data to be received
Input parameter 5	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_smbus_master_receive(&hi2c, 0xB0, rx_buf, 8, 0xFFFFFFFF);
```

5.12.62 i2c_smbus_slave_transmit function

The table below describes the function i2c_smbus_slave_transmit

Table 362. i2c_smbus_slave_transmit function

Name	Description
Function name	i2c_smbus_slave_transmit
Function prototype	i2c_status_type i2c_smbus_slave_transmit(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	SMBus slave sends data (through polling mode). This function is a blocking type. In other words, after the function execution is done, so is data transfer. It is mainly used for PEC transmission and reception.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address of data to be sent
Input parameter 3	Size: number of data to be sent
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_smbus_slave_transmit(&hi2c, tx_buf, 8, 0xFFFFFFFF);
```

5.12.63 i2c_smbus_slave_receive function

The table below describes the function `i2c_smbus_slave_receive`

Table 363. i2c_smbus_slave_receive function

Name	Description
Function name	<code>i2c_smbus_slave_receive</code>
Function prototype	<code>i2c_status_type i2c_smbus_slave_receive(i2c_handle_type* hi2c, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	SMBus slave receives data (through polling mode). This function is a blocking type. In other words, after the function execution is done, so is data transfer. It is mainly used for PEC transmission and reception.
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	Pdata: array address to receive data
Input parameter 3	Size: number of data to be received
Input parameter 4	Timeout: wait timeout
Output parameter	NA
Return value	<code>i2c_status_type</code> : error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

Example:

```
i2c_smbus_slave_receive(&hi2c, rx_buf, 8, 0xFFFFFFFF);
```

5.12.64 i2c_memory_write function

The table below describes the function `i2c_memory_write`

Table 364. i2c_memory_write function

Name	Description
Function name	<code>i2c_memory_write</code>
Function prototype	<code>i2c_status_type i2c_memory_write(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	Write data to EEPROM (through polling mode). This function is a blocking type. In other words, after the function execution is done, so is I2C transfer.
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” below for details.
Input parameter 3	address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	Pdata: array address of data to be sent
Input parameter 6	Size: number of data to be sent
Input parameter 7	Timeout: wait timeout

Name	Description
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width

I2C_MEM_ADDR_WIDIH_8: 8-bit address width

I2C_MEM_ADDR_WIDIH_16: 16-bit address width

Example:

```
i2c_memory_write(&hi2c, 0xA0, 0x05, tx_buf, 8, 0xFFFFFFFF);
```

5.12.65 i2c_memory_write_int function

The table below describes the function i2c_memory_write_int

Table 365. i2c_memory_write_int function

Name	Description
Function name	i2c_memory_write_int
Function prototype	i2c_status_type i2c_memory_write_int(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Write EEPROM (through interrupt mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the i2c_wait_end() to wait for the end of transfer
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” below for details.
Input parameter 3	address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	pdata: array address of data to be sent
Input parameter 6	size: number of data to be sent
Input parameter 7	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width

I2C_MEM_ADDR_WIDIH_8: 8-bit address width

I2C_MEM_ADDR_WIDIH_16: 16-bit address width

Example:

```
i2c_memory_write_int(&hi2c, 0xA0, 0x05, tx_buf, 8, 0xFFFFFFFF);
```

5.12.66 i2c_memory_write_dma function

The table below describes the function `i2c_memory_write_dma`

Table 366. i2c_memory_write_dma function

Name	Description
Function name	<code>i2c_memory_write_dma</code>
Function prototype	<code>i2c_status_type i2c_memory_write_dma(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	Write EEPROM (through DMA mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the <code>i2c_wait_end()</code> to wait for the end of transfer
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” below for details.
Input parameter 3	address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	pdata: array address of data to be sent
Input parameter 6	size: number of data to be sent
Input parameter 7	Timeout: wait timeout
Output parameter	NA
Return value	<code>i2c_status_type</code> : error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width

I2C_MEM_ADDR_WIDIH_8: 8-bit address width

I2C_MEM_ADDR_WIDIH_16: 16-bit address width

Example:

```
i2c_memory_write_dma(&hi2c, 0xA0, 0x05, tx_buf, 8, 0xFFFFFFFF);
```

5.12.67 i2c_memory_read function

The table below describes the function i2c_memory_write_dma

Table 367. i2c_memory_write_dma function

Name	Description
Function name	i2c_memory_read
Function prototype	i2c_status_type i2c_memory_read(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Read EEPROM (through DMA mode). This function is a blocking type. In other words, after the function execution is done, so is data transfer. It is mainly used for PEC transmission and reception.
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” below for details.
Input parameter 3	address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	pdata: array address of data to be read
Input parameter 6	size: number of data to be read
Input parameter 7	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width

I2C_MEM_ADDR_WIDIH_8: 8-bit address width

I2C_MEM_ADDR_WIDIH_16: 16-bit address width

Example:

```
i2c_memory_read(&hi2c, 0xA0, 0x05, rx_buf, 8, 0xFFFFFFFF);
```

5.12.68 i2c_memory_read_int function

The table below describes the function i2c_memory_read_int

Table 368. i2c_memory_write_dma function

Name	Description
Function name	i2c_memory_read_int
Function prototype	i2c_status_type i2c_memory_read_int(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);
Function description	Read EEPROM (through interrupt mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the i2c_wait_end() to wait for the end of transfer
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” below for details.
Input parameter 3	address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	pdata: array address of data to be read
Input parameter 6	size: number of data to be read
Input parameter 7	Timeout: wait timeout
Output parameter	NA
Return value	i2c_status_type: error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width

I2C_MEM_ADDR_WIDIH_8: 8-bit address width

I2C_MEM_ADDR_WIDIH_16: 16-bit address width

Example:

```
i2c_memory_read_int(&hi2c, 0xA0, 0x05, rx_buf, 8, 0xFFFFFFFF);
```

5.12.69 i2c_memory_read_dma function

The table below describes the function `i2c_memory_read_dma`

Table 369. i2c_memory_write_dma function

Name	Description
Function name	<code>i2c_memory_read_dma</code>
Function prototype	<code>i2c_status_type i2c_memory_read_dma(i2c_handle_type* hi2c, i2c_mem_address_width_type mem_address_width, uint16_t address, uint16_t mem_address, uint8_t* pdata, uint16_t size, uint32_t timeout);</code>
Function description	Read EEPROM (through DMA mode). This function is a non-blocking type. In other words, after the function is executed, the I2C transfer has not completed yet. So in this case, it is possible to call the <code>i2c_wait_end()</code> to wait for the end of transfer
Input parameter 1	hi2c: <code>i2c_handle_type</code> pointer Refer to i2c_handle_type
Input parameter 2	mem_address_width: EEPROM memory address width Refer to the “mem_address_width” below for details.
Input parameter 3	address: EEPROM address
Input parameter 4	mem_address: EEPROM data memory address
Input parameter 5	pdata: array address of data to be read
Input parameter 6	size: number of data to be read
Input parameter 7	Timeout: wait timeout
Output parameter	NA
Return value	<code>i2c_status_type</code> : error code Refer to error_code for details.
Required preconditions	NA
Called functions	NA

mem_address_width

EEPROM memory address width

I2C_MEM_ADDR_WIDIH_8: 8-bit address width

I2C_MEM_ADDR_WIDIH_16: 16-bit address width

Example:

```
i2c_memory_read_dma(&hi2c, 0xA0, 0x05, rx_buf, 8, 0xFFFFFFFF);
```


5.12.70 i2c_evt_irq_handler function

The table below describes the function i2c_evt_irq_handler

Table 370. i2c_evt_irq_handler function

Name	Description
Function name	i2c_evt_irq_handler
Function prototype	void i2c_evt_irq_handler(i2c_handle_type* hi2c);
Function description	Event interrupt function. It is used to handle I2C event interrupt
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void I2C1_EVT_IRQHandler(void)
{
    i2c_evt_irq_handler(&hi2c);
}
```

5.12.71 i2c_err_irq_handler function

The table below describes the function i2c_err_irq_handler

Table 371. i2c_err_irq_handler function

Name	Description
Function name	i2c_err_irq_handler
Function prototype	void i2c_err_irq_handler(i2c_handle_type* hi2c);
Function description	Error interrupt function. It is used to handle I2C error interrupt
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void I2C1_ERR_IRQHandler(void)
{
    i2c_err_irq_handler(&hi2c);
}
```

5.12.72 i2c_dma_tx_irq_handler function

The table below describes the function i2c_dma_tx_irq_handler

Table 372. i2c_dma_tx_irq_handler function

Name	Description
Function name	i2c_dma_tx_irq_handler
Function prototype	void i2c_dma_tx_irq_handler(i2c_handle_type* hi2c);
Function description	DMA transmit interrupt function. It is used to handle DMA transmit interrupt
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void DMA1_Channel6_IRQHandler(void)
{
    i2c_dma_tx_irq_handler(&hi2c);
}
```

5.12.73 i2c_dma_rx_irq_handler function

The table below describes the function i2c_dma_rx_irq_handler

Table 373. i2c_dma_rx_irq_handler function

Name	Description
Function name	i2c_dma_rx_irq_handler
Function prototype	void i2c_dma_rx_irq_handler(i2c_handle_type* hi2c);
Function description	DMA receive interrupt function. It is used to handle DMA receive interrupt
Input parameter 1	hi2c: i2c_handle_type pointer Refer to i2c_handle_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
void DMA1_Channel7_IRQHandler(void)
{
    i2c_dma_rx_irq_handler(&hi2c);
}
```

5.13 Nested vectored interrupt controller (NVIC)

The NVIC register structure NVIC_Type is defined in the “core_cm4.h”:

```
/**
 * @brief Structure type to access the Nested Vectored Interrupt Controller (NVIC).
 */
typedef struct
{
    .....
} NVIC_Type;
```

The table below gives a list of the NVIC registers

Table 374. Summary of NVIC registers

Register	Description
iser	Interrupt enable set register
icer	Interrupt enable clear register
ispr	Interrupt suspend set register
icpr	Interrupt suspend clear register
iabr	Interrupt activate bit register
ip	Interrupt priority register
stir	Software trigger interrupt register

The table below gives a list of NVIC library functions.

Table 375. Summary of NVIC library functions

Function name	Description
nvic_system_reset	System software reset
nvic_irq_enable	NVIC interrupt enable and priority enable
nvic_irq_disable	NVIC interrupt disable
nvic_priority_group_config	NVIC interrupt priority grouping configuration
nvic_vector_table_set	NVIC interrupt vector table base address and offset address configuration
nvic_lowpower_mode_config	NVIC low-power mode configuration

5.13.1 nvic_system_reset function

The table below describes the function nvic_system_reset.

Table 376. nvic_system_reset function

Name	Description
Function name	nvic_system_reset
Function prototype	void nvic_system_reset(void)
Function description	System software reset
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_SystemReset()

Example:

<pre>/* system reset */ nvic_system_reset();</pre>
--

5.13.2 nvic_irq_enable function

The table below describes the function nvic_irq_enable.

Table 377. nvic_irq_enable function

Name	Description
Function name	nvic_irq_enable
Function prototype	void nvic_irq_enable(IRQn_Type irqn, uint32_t preempt_priority, uint32_t sub_priority)
Function description	NVIC interrupt enable and priority configuration
Input parameter 1	Irqn: interrupt vector selection Refer to the “irqn” descriptions below for details.
Input parameter 2	preempt_priority: set preemption priority This parameter cannot be greater than the highest preemption priority defined in the NVIC_PRIORITY_GROUP_x
Input parameter 3	sub_priority: set response priority This parameter cannot be greater than the highest response priority defined in the NVIC_PRIORITY_GROUP_x
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_SetPriority() NVIC_EnableIRQ()

irqn

irqn is used to select interrupt vectors, including:

WWDT_IRQn: Window timer interrupt

PVM_IRQn: PVM interrupt linked to EXINT

.....

DMA2_Channel6_IRQn: DMA2 channel 6 global interrupt

DMA2_Channel7_IRQn: DMA2 channel 7 global interrupt

Example:

```
/* enable nvic irq */
nvic_irq_enable(ADC1_2_3_IRQn, 0, 0);
```

5.13.3 nvic_irq_disable function

The table below describes the function nvic_irq_disable.

Table 378. nvic_irq_disable function

Name	Description
Function name	nvic_irq_disable
Function prototype	void nvic_irq_disable(IRQn_Type irqn)
Function description	NVIC interrupt enable
Input parameter	Irqn: select interrupt vector. Refer to irqn for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_DisableIRQ()

Example:

```
/* disable nvic irq */
nvic_irq_disable(ADC1_2_3_IRQn);
```

5.13.4 nvic_priority_group_config function

The table below describes the function nvic_priority_group_config.

Table 379. nvic_priority_group_config function

Name	Description
Function name	nvic_priority_group_config
Function prototype	void nvic_priority_group_config(nvic_priority_group_type priority_group)
Function description	NVIC interrupt priority grouping configuration
Input parameter	priority_group: select interrupt priority group This parameter can be any enumerated value in the nvic_priority_group_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NVIC_SetPriorityGrouping()

priority_group

priority_group is used to select priority group from the parameters below

NVIC_PRIORITY_GROUP_0:

Priority group 0 (0 bit for preemption priority, and 4 bits for response priority)

NVIC_PRIORITY_GROUP_1:

Priority group 1 (1 bit for preemption priority, and 3 bits for response priority)

NVIC_PRIORITY_GROUP_2:

Priority group 2 (2 bits for preemption priority, and 2 bits for response priority)

NVIC_PRIORITY_GROUP_3:

Priority group 3 (3 bits for preemption priority, and 1 bit for response priority)

NVIC_PRIORITY_GROUP_4:

Priority group 4 (4 bits for preemption priority, and 0 bit for response priority)

Example:

```
/* config nvic priority group */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
```

5.13.5 nvic_vector_table_set function

The table below describes the function nvic_vector_table_set.

Table 380. nvic_vector_table_set function

Name	Description
Function name	nvic_vector_table_set
Function prototype	void nvic_vector_table_set(uint32_t base, uint32_t offset)
Function description	Set NVIC interrupt vector table base address and offset address
Input parameter 1	Base: base address of interrupt vector table The base address can be set in RAM or FLASH
Input parameter 2	Offset: offset address of interrupt vector table This parameter defines the start address of interrupt vector table, so it must be set to a multiple of 0x200.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

base

base is used to select the base address of interrupt vector table, including:

NVIC_VECTTAB_RAM: Interrupt vector table base address is located in RAM

NVIC_VECTTAB_FLASH: Interrupt vector table base address is located in FLASH

Example:

```
/* config vector table offset */
nvic_vector_table_set(NVIC_VECTTAB_FLASH, 0x4000);
```

5.13.6 nvic_lowpower_mode_config function

The table below describes the function `nvic_lowpower_mode_config`.

Table 381. nvic_lowpower_mode_config function

Name	Description
Function name	<code>nvic_lowpower_mode_config</code>
Function prototype	<code>void nvic_lowpower_mode_config(nvic_lowpower_mode_type lp_mode, confirm_state new_state)</code>
Function description	Configure NVIC low-power mode
Input parameter 1	<code>lp_mode</code> : select low-power modes This parameter can be any enumerated value in the <code>nvic_lowpower_mode_type</code> .
Input parameter 2	<code>new_state</code> : indicates the pre-configured status of battery powered domain This parameter can be <code>TRUE</code> or <code>FALSE</code> .
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

lp_mode

`lp_mode` is used to select low-power modes, including:

`NVIC_LP_SEVONPEND`:

Send wakeup event upon interrupt suspend (this option is usually used in conjunction with `WFE`)

`NVIC_LP_SLEEPDEEP`:

Deepsleep mode control bit (enable or disable core clock)

`NVIC_LP_SLEEPONEXIT`: Sleep mode entry when system leaves the lowest-priority interrupt

Example:

```
/* enable sleep-on-exit feature */  
nvic_lowpower_mode_config(NVIC_LP_SLEEPONEXIT, TRUE);
```

5.14 Power controller (PWC)

The PWC register structure pwc_type is defined in the “at32f402_405_pwc.h”:

```
/**
 * @brief type define pwc register all
 */
typedef struct
{
    .....
} pwc_type;
```

The table below gives a list of the PWC registers

Table 382. Summary of PWC registers

Register	Description
ctrl	Power control register
ctrlsts	Power control/status register
ldoov	LDO calibration register

The table below gives a list of PWC library functions.

Table 383. Summary of PWC library functions

Function name	Description
pwc_reset	Reset PWC registers to their reset values.
pwc_battery_powered_domain_access	Enable battery powered domain access
pwc_pvm_level_select	Select PVM threshold
pwc_power_voltage_monitor_enable	Enable Voltage monitor
pwc_wakeup_pin_enable	Enable standby-mode wakeup pin
pwc_flag_clear	Clear flag
pwc_flag_get	Get flag status
pwc_sleep_mode_enter	Enter Sleep mode
pwc_deep_sleep_mode_enter	Enter Deepsleep mode
pwc_voltage_regulate_set	Select voltage regulator status in Deepsleep mode
pwc_standby_mode_enter	Enter Standby mode
pwc_ldo_output_voltage_set	Set LDO output voltage

5.14.1 pwc_reset function

The table below describes the function pwc_reset.

Table 384. pwc_reset function

Name	Description
Function name	pwc_reset
Function prototype	void pwc_reset(void)
Function description	Reset all PWC registers to their reset values.
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset()

Example:

```
/* deinitialize pwc */
pwc_reset();
```

5.14.2 pwc_battery_powered_domain_access function

The table below describes the function pwc_battery_powered_domain_access.

Table 385. pwc_battery_powered_domain_access function

Name	Description
Function name	pwc_battery_powered_domain_access
Function prototype	void pwc_battery_powered_domain_access(confirm_state new_state)
Function description	Battery powered domain access enable
Input parameter	new_state: indicates the pre-configured status of battery powered domain This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the battery-powered domain write operations */
pwc_battery_powered_domain_access(TRUE);
```

Note: Access to battery powered domain (such as, RTC) is allowed only after enabling it through this function.

5.14.3 pwc_pvm_level_select function

The table below describes the function pwc_pvm_level_select.

Table 386. pwc_pvm_level_select function

Name	Description
Function name	pwc_pvm_level_select
Function prototype	void pwc_pvm_level_select(pwc_pvm_voltage_type pvm_voltage)
Function description	Select PVM threshold
Input parameter	pvm_voltage: indicates the selected PVM threshold This parameter can be any enumerated value in the pwc_pvm_voltage_type.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pvm_voltage

pvm_voltage is used to select a PVM threshold from the optional parameters below:

PWC_PVM_VOLTAGE_2V3: PVM threshold is 2.3V

PWC_PVM_VOLTAGE_2V4: PVM threshold is 2.4V

PWC_PVM_VOLTAGE_2V5: PVM threshold is 2.5V

PWC_PVM_VOLTAGE_2V6: PVM threshold is 2.6V

PWC_PVM_VOLTAGE_2V7: PVM threshold is 2.7V

PWC_PVM_VOLTAGE_2V8: PVM threshold is 2.8V

PWC_PVM_VOLTAGE_2V9: PVM threshold is 2.9V

Example:

```
/* set the threshold voltage to 2.9v */
pwc_pvm_level_select(PWC_PVM_VOLTAGE_2V9);
```

5.14.4 pwc_power_voltage_monitor_enable function

The table below describes the function pwc_power_voltage_monitor_enable.

Table 387. pwc_power_voltage_monitor_enable function

Name	Description
Function name	pwc_power_voltage_monitor_enable
Function prototype	void pwc_power_voltage_monitor_enable(confirm_state new_state)
Function description	Enable power voltage monitor (PVM)
Input parameter	new_state: indicates the pre-configured status of PVM This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable power voltage monitor */
pwc_power_voltage_monitor_enable(TRUE);
```

5.14.5 pwc_wakeup_pin_enable function

The table below describes the function pwc_wakeup_pin_enable.

Table 388. pwc_wakeup_pin_enable function

Name	Description
Function name	pwc_wakeup_pin_enable
Function prototype	void pwc_wakeup_pin_enable(uint32_t pin_num, confirm_state new_state)
Function description	Enable Standby wakeup pin
Input parameter 1	pin_num: select a standby wakeup pin This parameter can be any pin that is capable of waking up from Standby mode.
Input parameter 2	new_state: indicates the pre-configured status of Standby wakeup pins This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pin_num

pin_num is used to select Standby-mode wakeup pin, including:

PWC_WAKEUP_PIN_1: Standby wakeup pin 1 (corresponding GPIO is PA0)

PWC_WAKEUP_PIN_2: Standby wakeup pin 1 (corresponding GPIO is PC13)

Example:

```
/* enable wakeup pin - pa0 */
pwc_wakeup_pin_enable(PWC_WAKEUP_PIN_1, TRUE);
```

5.14.6 pwc_flag_clear function

The table below describes the function pwc_flag_clear.

Table 389. pwc_flag_clear function

Name	Description
Function name	pwc_flag_clear
Function prototype	void pwc_flag_clear(uint32_t pwc_flag)
Function description	Clear flag
Input parameter	pwc_flag: to-be-cleared flag Refer to the "pwc_flag" description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_flag

pwc_flag is used to select a flag from the optional parameters below:

PWC_WAKEUP_FLAG: Standby wakeup event

PWC_STANDBY_FLAG: Standby mode entry

PWC_PVM_OUTPUT_FLAG: PVM output (this parameter cannot be cleared by software)

Example:

```
/* wakeup event flag clear */  
pwc_flag_clear(PWC_WAKEUP_FLAG);
```

5.14.7 pwc_flag_get function

The table below describes the function pwc_flag_get.

Table 390. pwc_flag_get function

Name	Description
Function name	pwc_flag_get
Function prototype	flag_status pwc_flag_get(uint32_t pwc_flag)
Function description	Get flag status
Input parameter	pwc_flag: select a flag. Refer to pwc_flag for details.
Output parameter	NA
Return value	flag_status: indicates flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

Example:

```
/* check if wakeup event flag is set */  
if(pwc_flag_get(PWC_WAKEUP_FLAG) != RESET)
```

5.14.8 pwc_sleep_mode_enter function

The table below describes the function pwc_sleep_mode_enter.

Table 391. pwc_sleep_mode_enter function

Name	Description
Function name	pwc_sleep_mode_enter
Function prototype	void pwc_sleep_mode_enter(pwc_sleep_enter_type pwc_sleep_enter)
Function description	Enter Sleep mode
Input parameter	pwc_sleep_enter: select a command to enter Sleep mode This parameter can be any enumerated value in the pwc_sleep_enter_type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_sleep_enter

pwc_sleep_enter is used to select a command to enter Sleep mode from the optional parameters below:

PWC_SLEEP_ENTER_WFI: Enter Sleep mode by WFI

PWC_SLEEP_ENTER_WFE: Enter Sleep mode by WFE

Example:

```
/* enter sleep mode */  
pwc_sleep_mode_enter(PWC_SLEEP_ENTER_WFI);
```

5.14.9 pwc_deep_sleep_mode_enter function

The table below describes the function `pwc_deep_sleep_mode_enter`.

Table 392. pwc_deep_sleep_mode_enter function

Name	Description
Function name	<code>pwc_deep_sleep_mode_enter</code>
Function prototype	<code>void pwc_deep_sleep_mode_enter(pwc_deep_sleep_enter_type pwc_deep_sleep_enter)</code>
Function description	Enter Deepsleep mode
Input parameter	<code>pwc_deep_sleep_enter</code> : select a command to enter Deepsleep mode This parameter can be any enumerated value in the <code>pwc_deep_sleep_enter_type</code>
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_deep_sleep_enter

`pwc_deep_sleep_enter` is used to select a command to enter Deepsleep mode, including:

`PWC_DEEP_SLEEP_ENTER_WFI`: Enter Deepsleep mode by WFI

`PWC_DEEP_SLEEP_ENTER_WFE`: Enter Deepsleep mode by WFE

Example:

```
/* enter deep sleep mode */  
pwc_deep_sleep_mode_enter(PWC_DEEP_SLEEP_ENTER_WFI);
```

5.14.10 pwc_voltage_regulate_set function

The table below describes the function `pwc_voltage_regulate_set`.

Table 393. pwc_voltage_regulate_set function

Name	Description
Function name	<code>pwc_voltage_regulate_set</code>
Function prototype	<code>void pwc_voltage_regulate_set(pwc_regulator_type pwc_regulator)</code>
Function description	Select the status of voltage regulator in Deepsleep mode
Input parameter	<code>pwc_regulator</code> : select voltage regulator status This parameter can be any enumerated value in the <code>pwc_regulator_type</code>
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

pwc_regulator

`pwc_regulator` is used to select the status of voltage regulator from the optional parameters below:

`PWC_REGULATOR_ON`: Voltage regulator ON in Deepsleep mode

`PWC_REGULATOR_LOW_POWER`: Voltage regulator low-power mode in Deepsleep mode

Example:

```
/* config the voltage regulator mode */  
pwc_voltage_regulate_set(PWC_REGULATOR_LOW_POWER);
```

5.14.11 pwc_standby_mode_enter function

The table below describes the function pwc_standby_mode_enter

Table 394. pwc_standby_mode_enter function

Name	Description
Function name	pwc_standby_mode_enter
Function prototype	void pwc_standby_mode_enter(void)
Function description	Enter Standby mode
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enter standby mode */  
pwc_standby_mode_enter();
```

pwc_ldo_output_voltage_set

The table below describes the function pwc_ldo_output_voltage_set.

Table 395. pwc_ldo_output_voltage_set function

Name	Description
Function name	pwc_ldo_output_voltage_set
Function prototype	pwc_ldo_output_voltage_set(val)
Function description	Set LDO output voltage
Input parameter	Val: LDO output voltage value This parameter can be any enumerated value in the pwc_ldo_output_voltage_type.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

val

It is used to select the LDO output voltage value, as shown below.

PWC_LDO_OUTPUT_1V0: LDO output 1.0 V

PWC_LDO_OUTPUT_1V2: LDO output 1.2 V

PWC_LDO_OUTPUT_1V3: LDO output 1.3 V

Example:

```
/* reduce ldo before enter deepsleep mode */  
pwc_ldo_output_voltage_set(PWC_LDO_OUTPUT_1V0);
```

5.15 System configuration controller (SCFG)

The SCFG register structure `scfg_type` is defined in the “at32f402_405_scfg.h”

```
/**
 * @brief type define scfg register all
 */
typedef struct
{
    ...
} scfg_type;
```

The table below gives a list of the SCFG registers

Table 396. Summary of SCFG registers

Register	Description
<code>scfg_cfg1</code>	SCFG configuration register 1
<code>scfg_cfg2</code>	SCFG configuration register 2
<code>scfg_exintc1</code>	SCFG external interrupt configuration register 1
<code>scfg_exintc2</code>	SCFG external interrupt configuration register 2
<code>scfg_exintc3</code>	SCFG external interrupt configuration register 3
<code>scfg_exintc4</code>	SCFG external interrupt configuration register 4
<code>scfg_uhdrv</code>	SCFG ultra-high drive capability register

The table below gives a list of SCFG library functions.

Table 397. Summary of SCFG library functions

Function name	Description
<code>scfg_reset</code>	SCFG reset
<code>scfg_infrared_config</code>	infrared configuration
<code>scfg_mem_map_get</code>	Get memory address map
<code>scfg_i2s_full_duplex_config</code>	I2S full-duplex mode configuration
<code>scfg_pvm_lock_enable</code>	PVM Lock enable
<code>scfg_sram_operr_status_get</code>	Get SRAM parity check error status
<code>scfg_sram_operr_lock_enable</code>	Enable SRAM parity check error lock
<code>scfg_lockup_enable</code>	Lockup lock enable
<code>scfg_exint_line_config</code>	External interrupt line configuration
<code>scfg_pins_ultra_driven_enable</code>	Pin ultra-high current sinking capability enable

5.15.1 scfg_reset function

The table below describes the function scfg_reset.

Table 398. scfg_reset function

Name	Description
Function name	scfg_reset
Function prototype	void scfg_reset(void);
Function description	Reset SCFG
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
scfg_reset();
```

5.15.2 scfg_infrared_config function

The table below describes the function scfg_infrared_config.

Table 399. scfg_infrared_config function

Name	Description
Function name	scfg_infrared_config
Function prototype	void scfg_infrared_config(scfg_ir_source_type source, scfg_ir_polarity_type polarity);
Function description	Infrared configuration
Input parameter 1	Source: infrared modulation signal source
Input parameter 2	Polarity: output signal polarity
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

scfg_ir_source_type

Select infrared signal source.

SCFG_IR_SOURCE_TMR10: Infrared signal source is TMR10

SCFG_IR_SOURCE_USART1: Infrared signal source is USART1

SCFG_IR_SOURCE_USART2: Infrared signal source is USART2

scfg_ir_polarity_type

Select infrared signal polarity.

SCFG_IR_POLARITY_NO_AFFECTE: Infrared output signal not inverted

SCFG_IR_POLARITY_REVERSE: Infrared output signal inverted

Example:

```
scfg_infrared_config(SCFG_IR_SOURCE_TMR10, SCFG_IR_POLARITY_NO_AFFECTE);
```


5.15.3 scfg_mem_map_get function

The table below describes the function `scfg_mem_map_get`.

Table 400. scfg_mem_map_get function

Name	Description
Function name	<code>scfg_mem_map_set</code>
Function prototype	<code>scfg_mem_map_type scfg_mem_map_get(void);</code>
Function description	Get the status on a memory being mapped on the address 0x00000000
Input parameter	NA
Output parameter	NA
Return value	<code>scfg_mem_map_type</code> : memory address map type
Required preconditions	NA
Called functions	NA

scfg_mem_map_type

Select a memory to be mapped at address 0x00000000

SCFG_MEM_MAP_MAIN_MEMORY: Main memory is mapped to 0x00000000

SCFG_MEM_MAP_BOOT_MEMORY: Boot memory is mapped to 0x00000000

SCFG_MEM_MAP_INTERNAL_SRAM: Internal memory is mapped to 0x00000000

Example:

```
scfg_mem_map_type value;
value = scfg_mem_map_get();
```

5.15.4 scfg_i2s_full_duplex_config function

The table below describes the function `scfg_i2s_full_duplex_config`.

Table 401. scfg_i2s_full_duplex_config function

Name	Description
Function name	<code>scfg_i2s_full_duplex_config</code>
Function prototype	<code>void scfg_i2s_full_duplex_config(scfg_i2s_type i2s_full_duplex);</code>
Function description	I2S full-duplex mode configuration
Input parameter	<code>i2s_full_duplex</code> : full-duplex mode selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

scfg_i2s_type

Set full-duplex mode.

SCFG_FULL_DUPLEX_I2S_NONE: None (SPI/I2S1~3 run independently)

SCFG_FULL_DUPLEX_I2S1_I2S3: Combine I2S1 and I2S3 as full-duplex mode

SCFG_FULL_DUPLEX_I2S2_I2S3: Combine I2S2 and I2S3 as full-duplex mode

SCFG_FULL_DUPLEX_I2S1_I2S2: Combine I2S1 and I2S2 as full-duplex mode

Example:

```
scfg_i2s_full_duplex_config(SCFG_FULL_DUPLEX_I2S1_I2S3);
```

5.15.5 scfg_pvm_lock_enable function

The table below describes the function `scfg_adc_dma_channel_remap`.

Table 402. scfg_adc_dma_channel_remap function

Name	Description
Function name	<code>scfg_pvm_lock_enable</code>
Function prototype	<code>void scfg_pvm_lock_enable(confirm_state new_state);</code>
Function description	PVM Lock enable
Input parameter	<code>new_state</code> : enabled or disables This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
scfg_pvm_lock_enable(TRUE);
```

5.15.6 scfg_sram_operr_status_get function

The table below describes the function `scfg_sram_operr_status_get`.

Table 403. scfg_sram_operr_status_get function

Name	Description
Function name	<code>scfg_sram_operr_status_get</code>
Function prototype	<code>error_status scfg_sram_operr_status_get(void);</code>
Function description	Get parity check error status
Input parameter	<code>new_state</code> : enabled or disables This parameter can be FALSE or TRUE
Output parameter	NA
Return value	<code>error_status</code> : return SRAM parity check error status Normal (SUCCESS), Error (ERROR)
Required preconditions	NA
Called functions	NA

Example:

```
error_status status;
status = scfg_sram_operr_status_get();
```

5.15.7 scfg_sram_operr_lock_enable function

The table below describes the function `scfg_sram_operr_lock_enable`.

Table 404. scfg_sram_operr_lock_enable function

Name	Description
Function name	<code>scfg_sram_operr_lock_enable</code>
Function prototype	<code>void scfg_sram_operr_lock_enable(confirm_state new_state);</code>
Function description	Enable SRAM parity check error lock
Input parameter	<code>new_state</code> : enabled or disabled This parameter can be FALSE or TRUE
Output parameter	NA
Return value	<code>error_status</code> : return SRAM parity check error status Normal (SUCCESS), Error (ERROR)
Required preconditions	NA
Called functions	NA

Example:

```
scfg_sram_operr_lock_enable(TRUE);
```

5.15.8 scfg_lockup_enable function

The table below describes the function `scfg_usart1_tx_dma_channel_remap`.

Table 405. scfg_usart1_tx_dma_channel_remap function

Name	Description
Function name	<code>scfg_lockup_enable</code>
Function prototype	<code>void scfg_lockup_enable(confirm_state new_state);</code>
Function description	Lockup lock enable
Input parameter	<code>new_state</code> : enabled or disables This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
scfg_lockup_enable(TRUE);
```

5.15.9 scfg_exint_line_config function

The table below describes the function `scfg_exint_line_config`.

Table 406. scfg_exint_line_config function

Name	Description
Function name	<code>scfg_exint_line_config</code>
Function prototype	<code>void scfg_exint_line_config(scfg_port_source_type port_source, scfg_pins_source_type pin_source);</code>
Function description	External interrupt line configuration
Input parameter 1	<code>port_source</code> : port source
Input parameter 2	<code>pin_source</code> : pin source
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

scfg_port_source_type

SCFG_PORT_SOURCE_GPIOA: port A

SCFG_PORT_SOURCE_GPIOB: port B

SCFG_PORT_SOURCE_GPIOC: port C

SCFG_PORT_SOURCE_GPIOD: port D

SCFG_PORT_SOURCE_GPIOF: port F

scfg_pins_source_type

SCFG_PINS_SOURCE0: pin 0

SCFG_PINS_SOURCE1: pin 1

SCFG_PINS_SOURCE2: pin 2

.....

SCFG_PINS_SOURCE13: pin 13

SCFG_PINS_SOURCE14: pin 14

SCFG_PINS_SOURCE15: pin15

Example:

```
scfg_exint_line_config(SCFG_PORT_SOURCE_GPIOA, SCFG_PINS_SOURCE1);
```

5.15.10 scfg_pins_ultra_driven_enable function

The table below describes the function `scfg_pins_ultra_driven_enable`.

Table 407. scfg_pins_ultra_driven_enable function

Name	Description
Function name	<code>scfg_pins_ultra_driven_enable</code>
Function prototype	<code>void scfg_pins_ultra_driven_enable(scfg_ultra_driven_pins_type value, confirm_state new_state);</code>
Function description	Enable pin ultra-high current sinking capability
Input parameter 1	Value: pin
Input parameter 2	<code>new_state</code> : enabled or disabled This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

scfg_ultra_driven_pins_type

SCFG_ULTRA_DRIVEN_PB3: PB3

SCFG_ULTRA_DRIVEN_PB9: PB9

SCFG_ULTRA_DRIVEN_PB10: PB10

Example:

```
scfg_pins_ultra_driven_enable(SCFG_ULTRA_DRIVEN_PB8, TRUE);
```

5.16 Qud-SPI interface (QSPI)

The QSPI register structure `qspi_type` is defined in the “at32f402_405_qspi.h”.

```
/**
 * @brief type define qspi register all
 */
typedef struct
{
    ...
} qspi_type;
```

The table below gives a list of the QSPI registers.

Table 408. Summary of QSPI registers

Register	Description
cmd_w0	Command word 0
cmd_w1	Command word 1
cmd_w2	Command word 2
cmd_w3	Command word 3
ctrl	Control register
actr	AC timing register
fifosts	FIFO status register
ctrl2	Control register 2
cmdsts	Command status register
rsts	Read status register
fsize	Flash size register
xip cmd_w0	XIP command word 0
xip cmd_w1	XIP command word 1
xip cmd_w2	XIP command word 2
xip cmd_w3	XIP command word 3
rev	Revision register
dt	Data port register

The table below gives a list of QSPI library functions.

Table 409. Summary of QSPI library functions

Function name	Description
qspi_encryption_enable	QSPI encryption enable
qspi_sck_mode_set	Configure QSPI clock mode
qspi_clk_division_set	Configure QSPI clock division
qspi_xip_cache_bypass_set	Enable QSPI XIP port cache bypass
qspi_interrupt_enable	Enable QSPI interrupt
qspi_flag_get	Get QSPI flag
qspi_flag_clear	Clear QSPI flag
qspi_dma_rx_threshold_set	Configure QSPI DMA receive FIFO threshold

Function name	Description
qspi_dma_tx_threshold_set	Configure QSPI DMA transmit FIFO threshold
qspi_dma_enable	Enable QSPI DMA
qspi_busy_config	Configure the offset of QSPI busy bit in the status register
qspi_xip_enable	QSPI XIP port enable
qspi_cmd_operation_kick	Start QSPI command operation
qspi_xip_init	QSPI XIP port initialization
qspi_byte_read	QSPI byte read
qspi_half_word_read	QSPI half-word read
qspi_word_read	QSPI word read
qspi_word_write	QSPI word write
qspi_half_word_write	QSPI half-word write
qspi_byte_write	QSPI byte write

5.16.1 qspi_encryption_enable function

The table below describes the function qspi_encryption_enable

Table 410. qspi_encryption_enable function function

Name	Description
Function name	qspi_encryption_enable
Function prototype	void qspi_encryption_enable(qspi_type* qspi_x, confirm_state new_state);
Function description	Enable QSPI encryption. This function is called only when QSPI is in the command port.
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	new_state: encryption status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_encryption_enable(QSPI1, TRUE);
```

5.16.2 qspi_sck_mode_set function

The table below describes the function qspi_sck_mode_set.

Table 411. qspi_sck_mode_set function

Name	Description
Function name	qspi_sck_mode_set
Function prototype	void qspi_sck_mode_set(qspi_type* qspi_x, qspi_clk_mode_type new_mode);
Function description	Configure QSPI clock mode
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	new_mode: clock mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

qspi_clk_mode_type

QSPI_SCK_MODE_0: QSPI clock mode 0

QSPI_SCK_MODE_3: QSPI clock mode 3

Example:

```
qspi_sck_mode_set(QSPI1, QSPI_SCK_MODE_0);
```

5.16.3 qspi_clk_division_set function

The table below describes the function qspi_clk_division_set.

Table 412. qspi_clk_division_set function

Name	Description
Function name	qspi_clk_division_set
Function prototype	void qspi_clk_division_set (qspi_type* qspi_x, qspi_clk_div_type new_clkdiv);
Function description	Configure QSPI clock division
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	new_clkdiv: clock division
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

qspi_clk_div_type

QSPI_CLK_DIV_2: Divided by 2

QSPI_CLK_DIV_4: Divided by 4

QSPI_CLK_DIV_6: Divided by 6

QSPI_CLK_DIV_8: Divided by 8

QSPI_CLK_DIV_3: Divided by 3

QSPI_CLK_DIV_5: Divided by 5

QSPI_CLK_DIV_10: Divided by 10

QSPI_CLK_DIV_12: Divided by 12

Example:

```
qspi_clk_division_set(QSPI1, QSPI_CLK_DIV_2);
```

5.16.4 qspi_xip_cache_bypass_set function

The table below describes the function qspi_xip_cache_bypass_set.

Table 413. qspi_xip_cache_bypass_set function

Name	Description
Function name	qspi_xip_cache_bypass_set
Function prototype	void qspi_xip_cache_bypass_set(qspi_type* qspi_x, confirm_state new_state);
Function description	Configure QSPI XIP port cache bypass
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	new_state: bypass status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_xip_cache_bypass_set(QSPI1, TRUE);
```

5.16.5 qspi_interrupt_enable function

The table below describes the function qspi_interrupt_enable.

Table 414. qspi_interrupt_enable function

Name	Description
Function name	qspi_interrupt_enable
Function prototype	void qspi_interrupt_enable(qspi_type* qspi_x, confirm_state new_state);
Function description	Configure QSPI interrupts
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	new_state: interrupt status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_interrupt_enable(QSPI1, TRUE);
```

5.16.6 qspi_interrupt_flag_get function

The table below describes the function qspi_interrupt_flag_get.

Table 415. qspi_interrupt_flag_get function

Name	Description
Function name	qspi_interrupt_flag_get
Function prototype	flag_status qspi_interrupt_flag_get(qspi_type* qspi_x, uint32_t flag);
Function description	Get flag status
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1.
Input parameter 2	flag: QSPI_CMDSTS_FLAG valid only
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

qspi_flag

Get a QSPI status flag

QSPI_CMDSTS_FLAG: QSPI command complete flag

Example

```
flag_status status;
status = qspi_flag_get(QSPI_CMDSTS_FLAG);
```

5.16.7 qspi_flag_get function

The table below describes the function qspi_flag_get.

Table 416. qspi_flag_get function

Name	Description
Function name	qspi_flag_get
Function prototype	flag_status qspi_flag_get(qspi_type* qspi_x, uint32_t flag);
Function description	Get flag status
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	flag: flag selection
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

qspi_flag

Get a QSPI status flag

QSPI_CMDSTS_FLAG: QSPI command complete flag

QSPI_RXFIFORDY_FLAG: QSPI receive FIFO ready flag

QSPI_TXFIFORDY_FLAG: QSPI transmit FIFO ready flag

Example:

```
flag_status status;
status = qspi_flag_get(QSPI_CMDSTS_FLAG);
```

5.16.8 qspi_flag_clear function

The table below describes the function qspi_flag_clear.

Table 417. qspi_flag_clear function

Name	Description
Function name	qspi_flag_clear
Function prototype	void qspi_flag_clear(qspi_type* qspi_x, uint32_t flag);
Function description	Clear flag
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	flag: to-be-cleared flag
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

qspi_flag

Clear the QSPI status flag

QSPI_CMDSTS_FLAG: QSPI command complete flag

Example:

```
qspi_flag_clear(QSPI_CMDSTS_FLAG);
```

5.16.9 qspi_dma_rx_threshold_set function

The table below describes the function qspi_dma_rx_threshold_set.

Table 418. qspi_dma_rx_threshold_set function

Name	Description
Function name	qspi_dma_rx_threshold_set
Function prototype	void qspi_dma_rx_threshold_set(qspi_type* qspi_x, qspi_dma_fifo_thod_type new_threshold);
Function description	Set QSPI DMA receive FIFO threshold
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	new_threshold: threshold
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

qspi_dma_fifo_thod_type

Configure a QSPI DMA FIFO threshold

QSPI_DMA_FIFO_THOD_WORD08: QSPI DMA FIFO 8 WORDs

QSPI_DMA_FIFO_THOD_WORD16: QSPI DMA FIFO 16 WORDs

QSPI_DMA_FIFO_THOD_WORD32: QSPI DMA FIFO 32 WORDs

Example:

```
qspi_dma_rx_threshold_set(QSPI_DMA_FIFO_THOD_WORD08);
```

5.16.10 qspi_dma_tx_threshold_set function

The table below describes the function `qspi_dma_tx_threshold_set`.

Table 419. qspi_dma_tx_threshold_set function

Name	Description
Function name	<code>qspi_dma_tx_threshold_set</code>
Function prototype	<code>void qspi_dma_tx_threshold_set(qspi_type* qspi_x, qspi_dma_fifo_thod_type new_threshold);</code>
Function description	Configure QSPI DMA transmit FIFO threshold
Input parameter 1	<code>qspi_x</code> : selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	<code>new_threshold</code> : threshold Refer to qspi_dma_fifo_thod_type .
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_dma_tx_threshold_set(QSPI_DMA_FIFO_THOD_WORD08);
```

5.16.11 qspi_dma_enable function

The table below describes the function `qspi_dma_enable`.

Table 420. qspi_dma_enable function

Name	Description
Function name	<code>qspi_dma_enable</code>
Function prototype	<code>void qspi_dma_enable(qspi_type* qspi_x, confirm_state new_state);</code>
Function description	Enable QSPI interrupts
Input parameter 1	<code>qspi_x</code> : selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	<code>new_state</code> : DMA status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_dma_enable(QSPI1, TRUE);
```

5.16.12 qspi_busy_config function

The table below describes the function qspi_busy_config.

Table 421. qspi_busy_config function

Name	Description
Function name	qspi_busy_config
Function prototype	void qspi_busy_config(qspi_type* qspi_x, qspi_busy_pos_type busy_pos);
Function description	Configure the offset of QSPI busy bit in the status register
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	busy_pos: offset
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

qspi_busy_pos_type

QSPI_BUSY_OFFSET_0:	Busy bit offset 0
QSPI_BUSY_OFFSET_1:	Busy bit offset 1
QSPI_BUSY_OFFSET_2:	Busy bit offset 2
QSPI_BUSY_OFFSET_3:	Busy bit offset 3
QSPI_BUSY_OFFSET_4:	Busy bit offset 4
QSPI_BUSY_OFFSET_5:	Busy bit offset 5
QSPI_BUSY_OFFSET_6:	Busy bit offset 6
QSPI_BUSY_OFFSET_7:	Busy bit offset 7

Example:

```
qspi_busy_config(QSPI1, QSPI_BUSY_OFFSET_0);
```

5.16.13 qspi_xip_enable function

The table below describes the function qspi_xip_enable.

Table 422. qspi_xip_enable function

Name	Description
Function name	qspi_xip_enable
Function prototype	void qspi_xip_enable(qspi_type* qspi_x, confirm_state new_state);
Function description	Enable QSPI XIP port
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	new_state: XIP port status This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_xip_enable(QSPI1, TRUE);
```

5.16.14 qspi_cmd_operation_kick function

The table below describes the function qspi_cmd_operation_kick.

Table 423. qspi_cmd_operation_kick function

Name	Description
Function name	qspi_cmd_operation_kick
Function prototype	void qspi_cmd_operation_kick(qspi_type* qspi_x, qspi_cmd_type* qspi_cmd_struct);
Function description	Start QSPI command operation
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	qspi_cmd_struct: command structure pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

qspi_cmd_type structure

The qspi_cmd_type is defined in the at32f402_405_qspi.h.

typedef struct

```
{
    confirm_state          pe_mode_enable;
    uint8_t                pe_mode_operate_code;
    uint8_t                instruction_code;
    qspi_cmd_inslen_type    instruction_length;
    uint32_t               address_code;
    qspi_cmd_adrlen_type    address_length;
    uint32_t               data_counter;
    uint8_t                second_dummy_cycle_num;
    qspi_operate_mode_type  operation_mode;
    qspi_read_status_conf_type read_status_config;
    confirm_state          read_status_enable;
    confirm_state          write_data_enable;
} qspi_cmd_type;
```

pe_mode_enable

Enable performance enhance mode according to the selected QSPI peripheral.

TRUE: Performance enhance mode enabled

FALSE: Performance enhance mode disabled

pe_mode_operate_code

Performance enhance mode operate code, dependent on the selected QSPI peripheral.

instruction_code

Command instruction code

instruction_length

Command instruction code length

QSPI_CMD_INSLEN_0_BYTE: No instruction code
 QSPI_CMD_INSLEN_1_BYTE: 1-byte instruction code
 QSPI_CMD_INSLEN_2_BYTE: 2-byte instruction code

address_code

Address code

address_length

Address length

QSPI_CMD_ADRLen_0_BYTE: No address
 QSPI_CMD_ADRLen_1_BYTE: 1-byte address
 QSPI_CMD_ADRLen_2_BYTE: 2-byte address
 QSPI_CMD_ADRLen_3_BYTE: 3-byte address
 QSPI_CMD_ADRLen_4_BYTE: 4-byte address

data_counter

Number of data

second_dummy_cycle_num

Number of the second dummy cycle, ranging from 0 to 32

operation_mode

Operation mode

QSPI_OPERATE_MODE_111: qspi serial mode
 QSPI_OPERATE_MODE_112: qspi dual mode
 QSPI_OPERATE_MODE_114: qspi quad mode
 QSPI_OPERATE_MODE_122: qspi dual i/o mode
 QSPI_OPERATE_MODE_144: qspi quad i/o mode
 QSPI_OPERATE_MODE_222: qspi instruction 2-bit mode
 QSPI_OPERATE_MODE_444: qspi instruction 4-bit mode(qpi)

read_status_config

QSPI_RSTSC_HW_AUTO: Read by hardware automatically
 QSPI_RSTSC_SW_ONCE: Read by software for once

read_status_enable

Enable read status

TRUE: Enable

FALSE: Disbale

write_data_enable

Enable write data

TRUE: Enable

FALSE: Disable

Example:

```
esmt32m_cmd_erase_config(&esmt32m_cmd_config, sec_addr);
qspi_cmd_operation_kick(QSPI1, &esmt32m_cmd_config);
```

5.16.15 qspi_xip_init function

The table below describes the function qspi_xip_init.

Table 424. qspi_xip_init function

Name	Description
Function name	qspi_xip_init
Function prototype	void qspi_xip_init(qspi_type* qspi_x, qspi_xip_type* xip_init_struct);
Function description	Initialize QSPI XIP port
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	xip_init_struct: initialization structure pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

qspi_xip_type structure

The qspi_xip_type is defined in the at32f402_405_qspi.h.

typedef struct

```
{
    uint8_t                read_instruction_code;
    qspi_xip_addrlen_type  read_address_length;
    qspi_operate_mode_type read_operation_mode;
    uint8_t                read_second_dummy_cycle_num;
    uint8_t                write_instruction_code;
    qspi_xip_addrlen_type  write_address_length;
    qspi_operate_mode_type write_operation_mode;
    uint8_t                write_second_dummy_cycle_num;
    qspi_xip_write_sel_type write_select_mode;
    uint8_t                write_time_counter;
    uint8_t                write_data_counter;
    qspi_xip_read_sel_type read_select_mode;
    uint8_t                read_time_counter;
    uint8_t                read_data_counter;
} qspi_xip_type;
```

read_instruction_code

Read command instruction code

read_address_length

Read command address length

QSPI_XIP_ADDRLEN_3_BYTE: 3-byte address

QSPI_XIP_ADDRLEN_4_BYTE: 4-byte address

read_operation_mode

Read command operation mode

QSPI_OPERATE_MODE_111: qspi serial mode

QSPI_OPERATE_MODE_112: qspi dual mode

QSPI_OPERATE_MODE_114: qspi quad mode

QSPI_OPERATE_MODE_122: qspi dual i/o mode
 QSPI_OPERATE_MODE_144: qspi quad i/o mode
 QSPI_OPERATE_MODE_222: qspi instruction 2-bit mode
 QSPI_OPERATE_MODE_444: qspi instruction 4-bit mode(qpi)

read_second_dummy_cycle_num

Number of read command second dummy cycle, ranging from 0 to 32

write_instruction_code

Write command instruction code

write_address_length

Write command address length

QSPI_XIP_ADDRLEN_3_BYTE: 3-byte address

QSPI_XIP_ADDRLEN_4_BYTE: 4-byte address

write_operation_mode

Write command operation mode

QSPI_OPERATE_MODE_111: qspi serial mode

QSPI_OPERATE_MODE_112: qspi dual mode

QSPI_OPERATE_MODE_114: qspi quad mode

QSPI_OPERATE_MODE_122: qspi dual i/o mode

QSPI_OPERATE_MODE_144: qspi quad i/o mode

QSPI_OPERATE_MODE_222: qspi instruction 2-bit mode

QSPI_OPERATE_MODE_444: qspi instruction 4-bit mode(qpi)

write_second_dummy_cycle_num

Number of write command second dummy cycle, ranging from 0 to 32

write_select_mode

Write command mode selection

QSPI_XIPW_SEL_MODED: Mode D

QSPI_XIPW_SEL_MODET: Mode T

write_time_counter

Number of clock in write command mode T

write_data_counter

Number of data in write command mode D

read_select_mode

Read command mode selection

QSPI_XIPW_SEL_MODED: Mode D

QSPI_XIPW_SEL_MODET: Mode T

read_time_counter

Number of clock in read command mode T

read_data_counter

Number of data in read command mode T

Example:

```

/* initial xip */
xip_init_ly68l6400_config(&ly68l6400_xip_init);
qspi_xip_init(QSPI1, &ly68l6400_xip_init);
  
```

5.16.16 qspi_byte_read function

The table below describes the function qspi_byte_read.

Table 425. qspi_byte_read function

Name	Description
Function name	qspi_byte_read
Function prototype	uint8_t qspi_byte_read(qspi_type* qspi_x);
Function description	Read data in bytes
Input parameter	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Output parameter	NA
Return value	Return the data value.
Required preconditions	NA
Called functions	NA

Example:

```
uint8_t data;  
data = qspi_byte_read(QSPI1);
```

5.16.17 qspi_half_word_read function

The table below describes the function qspi_half_word_read.

Table 426. qspi_half_word_read function

Name	Description
Function name	qspi_half_word_read
Function prototype	uint16_t qspi_half_word_read(qspi_type* qspi_x);
Function description	Read data in half-words.
Input parameter	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Output parameter	NA
Return value	Return the data value.
Required preconditions	NA
Called functions	NA

Example:

```
uint16_t data;  
data = qspi_half_word_read(QSPI1);
```

5.16.18 qspi_word_read function

The table below describes the function qspi_word_read

Table 427. qspi_word_read function

Name	Description
Function name	qspi_word_read
Function prototype	uint32_t qspi_word_read(qspi_type* qspi_x);
Function description	Read data in words.
Input parameter	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Output parameter	NA
Return value	Return the data value.
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t data;  
data = qspi_word_read(QSPI1);
```

5.16.19 qspi_word_write function

The table below describes the function qspi_word_write.

Table 428. qspi_word_write function

Name	Description
Function name	qspi_word_write
Function prototype	void qspi_word_write(qspi_type* qspi_x, uint32_t value);
Function description	Write data in words.
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	value: the to-be-written data
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_word_write(QSPI1, 0x12345678);
```

5.16.20 qspi_half_word_write function

The table below describes the function qspi_half_word_write.

Table 429. qspi_half_word_write function

Name	Description
Function name	qspi_half_word_write
Function prototype	void qspi_half_word_write(qspi_type* qspi_x, uint16_t value);
Function description	Write data in half-words.
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	value: the to-be-written data
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_half_word_write(QSPI1, 0x1234);
```

5.16.21 qspi_byte_write function

The table below describes the function qspi_byte_write.

Table 430. qspi_byte_write function

Name	Description
Function name	qspi_byte_write
Function prototype	void qspi_byte_write(qspi_type* qspi_x, uint8_t value);
Function description	Write data in bytes.
Input parameter 1	qspi_x: selected QSPI peripheral This parameter can be QSPI1 or QSPI2.
Input parameter 2	value: the to-be-written data
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
qspi_byte_write(QSPI1, 0x12);
```

5.17 Serial peripheral interface (SPI)/ I²S

The SPI register structure spi_type is defined in the “at32f402_405_spi.h”:

```
/**
 * @brief type define spi register all
 */
typedef struct
{
    ...
} spi_type;
```

The table below gives a list of the SPI registers

Table 431. Summary of SPI registers

Register	Description
ctrl1	SPI control register 1
ctrl2	SPI control register 2
sts	SPI status register
dt	SPI data register
cpoly	SPI CRC register
rcrc	SPI RxCRC register
tcrc	SPI TxCRC register
i2sctrl	SPI_I2S configuration register
i2sclkp	SPI_I2S prescaler register

The table below gives a list of SPI library functions.

Table 432. Summary of SPI library functions

Function name	Description
spi_i2s_reset	Reset SPI/I ² S registers to their reset values
spi_default_para_init	Configure the SPI initialization structure with an initial value
spi_init	Initialize SPI
spi_ti_mode_enable	SPI TI mode enable
spi_crc_next_transmit	Next data transfer is CRC command
spi_crc_polynomial_set	SPI CRC polynomial configuration
spi_crc_polynomial_get	Get SPI CRC polynomial
spi_crc_enable	Enable SPI CRC
spi_crc_value_get	Get CRC result of SPI receive/transmit
spi_hardware_cs_output_enable	Enable hardware CS output
spi_software_cs_internal_level_set	Set software CS internal level
spi_frame_bit_num_set	Set the number of frame bits
spi_half_duplex_direction_set	Set transfer direction of single-wire bidirectional half-duplex mode
spi_enable	Enable SPI
i2s_default_para_init	Set an initial value for the I ² S initialization structure
i2s_init	Initialize I ² S
i2s_enable	Enable I ² S

spi_i2s_interrupt_enable	Enable SPI/I ² S interrupts
spi_i2s_dma_transmitter_enable	Enable SPI/I ² S DMA transmit
spi_i2s_dma_receiver_enable	Enable SPI/I ² S DMA receive
spi_i2s_data_transmit	SPI/I ² S transmits data
spi_i2s_data_receive	SPI/I ² S receives data
spi_i2s_flag_get	Get SPI/I ² S flags
spi_i2s_flag_clear	Clear SPI/I ² S flags

5.17.1 spi_i2s_reset function

The table below describes the function spi_i2s_reset.

Table 433. spi_i2s_reset function

Name	Description
Function name	spi_i2s_reset
Function prototype	void spi_i2s_reset(spi_type *spi_x);
Function description	Reset SPI/I ² S registers to their reset values.
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

```
spi_i2s_reset (SPI1);
```

5.17.2 spi_default_para_init function

The table below describes the function spi_default_para_init.

Table 434. spi_default_para_init function

Name	Description
Function name	spi_default_para_init
Function prototype	void spi_default_para_init(spi_init_type* spi_init_struct);
Function description	Set an initial value for the SPI initialization structure
Input parameter 1	spi_init_struct: spi_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of spi_init_type before starting.
Called functions	NA

Example:

```
spi_init_type spi_init_struct;
spi_default_para_init (&spi_init_struct);
```

5.17.3 spi_init function

The table below describes the function spi_init.

Table 435. spi_init function

Name	Description
Function name	spi_init
Function prototype	void spi_init(spi_type* spi_x, spi_init_type* spi_init_struct);
Function description	Initialize SPI
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	spi_init_struct: spi_init_type pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of spi_init_type before starting.
Called functions	NA

spi_init_type is defined in the at32f402_405_spi.h:

typedef struct

```
{
    spi_transmission_mode_type    transmission_mode;
    spi_master_slave_mode_type    master_slave_mode;
    spi_mclk_freq_div_type        mclk_freq_division;
    spi_first_bit_type            first_bit_transmission;
    spi_frame_bit_num_type        frame_bit_num;
    spi_clock_polarity_type       clock_polarity;
    spi_clock_phase_type          clock_phase;
    spi_cs_mode_type              cs_mode_selection;
}
```

} spi_init_type;

spi_transmission_mode

SPI transmission mode.

SPI_TRANSMIT_FULL_DUPLEX: Two-wire unidirectional full-duplex mode
 SPI_TRANSMIT_SIMPLEX_RX: Two-wire unidirectional receive-only mode
 SPI_TRANSMIT_HALF_DUPLEX_RX: Single-wire bidirectional receive-only mode
 SPI_TRANSMIT_HALF_DUPLEX_TX: Single-wire bidirectional transmit-only mode

master_slave_mode

Master/slave mode selection.

SPI_MODE_SLAVE: Slave mode
 SPI_MODE_MASTER: Master mode

mclk_freq_division

Frequency division factor selection.

SPI_MCLK_DIV_2: Divided by 2
 SPI_MCLK_DIV_4: Divided by 4
 SPI_MCLK_DIV_8: Divided by 8
 SPI_MCLK_DIV_16: Divided by 16
 SPI_MCLK_DIV_32: Divided by 32
 SPI_MCLK_DIV_64: Divided by 64

SPI_MCLK_DIV_128: Divided by 128
 SPI_MCLK_DIV_256: Divided by 256
 SPI_MCLK_DIV_512: Divided by 512
 SPI_MCLK_DIV_1024: Divided by 1024

first_bit_transmission

SPI MSB-first/LSB-first selection

SPI_FIRST_BIT_MSB: MSB-first

SPI_FIRST_BIT_LSB: LSB-first

frame_bit_num

Set the number of bits in a frame

SPI_FRAME_8BIT: 8-bit data in a frame

SPI_FRAME_16BIT: 16-bit data in a frame

clock_polarity

Select Clock polarity.

SPI_CLOCK_POLARITY_LOW: Clock output low in idle state

SPI_CLOCK_POLARITY_HIGH: Clock output high in idle state

clock_phase

Select clock phase.

SPI_CLOCK_PHASE_1EDGE: Sample on the first clock edge

SPI_CLOCK_PHASE_2EDGE: Sample on the second clock edge

cs_mode_selection

Select CS mode.

SPI_CS_HARDWARE_MODE: Hardware CS mode

SPI_CS_SOFTWARE_MODE: Software CS mode

Example:

```
spi_init_type spi_init_struct;
spi_default_para_init(&spi_init_struct);
spi_init_struct.transmission_mode = SPI_TRANSMIT_FULL_DUPLEX;
spi_init_struct.master_slave_mode = SPI_MODE_MASTER;
spi_init_struct.mclk_freq_division = SPI_MCLK_DIV_8;
spi_init_struct.first_bit_transmission = SPI_FIRST_BIT_MSB;
spi_init_struct.frame_bit_num = SPI_FRAME_16BIT;
spi_init_struct.clock_polarity = SPI_CLOCK_POLARITY_LOW;
spi_init_struct.clock_phase = SPI_CLOCK_PHASE_2EDGE;
spi_init_struct.cs_mode_selection = SPI_CS_SOFTWARE_MODE;
spi_init(SPI1, &spi_init_struct);
```


5.17.4 spi_ti_mode_enable function

The table below describes the function spi_crc_next_transmit.

Table 436. spi_ti_mode_enable function

Name	Description
Function name	spi_ti_mode_enable
Function prototype	void spi_ti_mode_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI TI mode
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: enabled or disabled This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* spi ti mode enable */
spi_ti_mode_enable (SPI1, TRUE);
```

5.17.5 spi_crc_next_transmit function

The table below describes the function spi_crc_next_transmit.

Table 437. spi_crc_next_transmit function

Name	Description
Function name	spi_crc_next_transmit
Function prototype	void spi_crc_next_transmit(spi_type* spi_x);
Function description	The next data to be sent is CRC command
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
spi_crc_next_transmit (SPI1);
```

5.17.6 spi_crc_polynomial_set function

The table below describes the function spi_crc_polynomial_set.

Table 438. spi_crc_polynomial_set function

Name	Description
Function name	spi_crc_polynomial_set
Function prototype	void spi_crc_polynomial_set(spi_type* spi_x, uint16_t crc_poly);
Function description	Set SPI CRC polynomial
Input parameter 1	spi_x: select SPI peripherals This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	crc_poly: CRC polynomial Value is 0x0000~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/*set spi crc polynomial value */
spi_crc_polynomial_set (SPI1, 0x07);
```

5.17.7 spi_crc_polynomial_get function

The table below describes the function spi_crc_polynomial_get.

Table 439. spi_crc_polynomial_get function

Name	Description
Function name	spi_crc_polynomial_get
Function prototype	uint16_t spi_crc_polynomial_get(spi_type* spi_x);
Function description	Get SPI CRC polynomial
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Output parameter	NA
Return value	CRC polynomial Value is 0x0000~0xFFFF
Required preconditions	NA
Called functions	NA

Example:

```
/*get spi crc polynomial value */
uint16_t crc_poly;
crc_poly = spi_crc_polynomial_get (SPI1);
```

5.17.8 spi_crc_enable function

The table below describes the function spi_crc_enable.

Table 440. spi_crc_enable function

Name	Description
Function name	spi_crc_enable
Function prototype	void spi_crc_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI CRC
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: enabled or disabled This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* spi crc enable */
spi_crc_enable (SPI1, TRUE);
```

5.17.9 spi_crc_value_get function

The table below describes the function spi_crc_value_get.

Table 441. spi_crc_value_get function

Name	Description
Function name	spi_crc_value_get
Function prototype	uint16_t spi_crc_value_get(spi_type* spi_x, spi_crc_direction_type crc_direction);
Function description	Get SPI receive/transmit CRC result
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	crc_direction : Select receive/transmit CRC
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

crc_direction

Select receive/transmit CRC

SPI_CRC_RX: Receive CRC

SPI_CRC_TX: Transmit CRC

Example:

```
/* get spi rx & tx crc enable */
uint16_t spi_rx_crc, spi_tx_crc;
spi_rx_crc = spi_crc_value_get (SPI1, SPI_CRC_RX);
spi_tx_crc = spi_crc_value_get (SPI1, SPI_CRC_TX);
```

5.17.10 spi hardware_cs_output_enable function

The table below describes the function spi hardware_cs_output_enable.

Table 442. spi hardware_cs_output_enable function

Name	Description
Function name	spi hardware_cs_output_enable
Function prototype	void spi hardware_cs_output_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable hardware CS output
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: enabled or disabled This parameter can FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	This setting is applicable to SPI master mode only.
Called functions	NA

Example:

```
/* enable the hardware cs output */
spi hardware_cs_output_enable (SPI1, TRUE);
```

5.17.11 spi software_cs_internal_level_set function

The table below describes the function spi software_cs_internal_level_set.

Table 443. spi software_cs_internal_level_set function

Name	Description
Function name	spi software_cs_internal_level_set
Function prototype	void spi software_cs_internal_level_set(spi_type* spi_x, spi software_cs_level_type level);
Function description	Set software CS internal level
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	<i>level</i> : set software CS internal level
Output parameter	NA
Return value	NA
Required preconditions	1. This setting is applicable to software CS mode only; 2. In master mode, the "level" value must be "SPI_SWCS_INTERNAL_LEVEL_HIGHT".
Called functions	NA

level

Set software CS internal level

SPI_SWCS_INTERNAL_LEVEL_LOW: Software CS internal low level

SPI_SWCS_INTERNAL_LEVEL_HIGHT: Software CS internal high level

Example:

```
/* set the internal level high */
```

```
spi_software_cs_internal_level_set (SPI1, SPI_SWCS_INTERNAL_LEVEL_HIGHT);
```

5.17.12 spi_frame_bit_num_set function

The table below describes the function spi_frame_bit_num_set.

Table 444. spi_frame_bit_num_set function

Name	Description
Function name	spi_frame_bit_num_set
Function prototype	void spi_frame_bit_num_set(spi_type* spi_x, spi_frame_bit_num_type bit_num);
Function description	Set the number of bits in a frame
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	<i>bit_num</i> : Set the number of bits in a frame
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

bit_num

Set the number of bits in a frame

SPI_FRAME_8BIT: 8-bit data in a frame

SPI_FRAME_16BIT: 16-bit data in a frame

Example:

```
/* set the data frame bit num as 8 */
spi_frame_bit_num_set (SPI1, SPI_FRAME_8BIT);
```

5.17.13 spi_half_duplex_direction_set function

The table below describes the function spi_half_duplex_direction_set.

Table 445. spi_half_duplex_direction_set function

Name	Description
Function name	spi_half_duplex_direction_set
Function prototype	void spi_half_duplex_direction_set(spi_type* spi_x, spi_half_duplex_direction_type direction);
Function description	Set the transfer direction of single-wire bidirectional half-duplex mode
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	<i>direction</i> : transfer direction
Output parameter	NA
Return value	NA
Required preconditions	This setting is applicable to the single-wire bidirectional half-duplex mode only.
Called functions	NA

direction

Transfer direction

SPI_HALF_DUPLEX_DIRECTION_RX: Receive

SPI_HALF_DUPLEX_DIRECTION_TX: Transmit

Example:

```
/* set the data transmission direction as transmit */  
spi_half_duplex_direction_set (SPI1, SPI_HALF_DUPLEX_DIRECTION_TX);
```

5.17.14 spi_enable function

The table below describes the function spi_enable.

Table 446. spi_enable function

Name	Description
Function name	spi_enable
Function prototype	void spi_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: enabled or disabled This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable spi */  
spi_enable (SPI1, TRUE);
```

5.17.15 i2s_default_para_init function

The table below describes the function i2s_default_para_init.

Table 447. i2s_default_para_init function

Name	Description
Function name	i2s_default_para_init
Function prototype	void i2s_default_para_init(i2s_init_type* i2s_init_struct);
Function description	Set an initial value for the I ² S initialization structure
Input parameter 1	i2s_init_struct: spi_i2s_flag pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of i2s_init_type before starting.
Called functions	NA

Example:

```
i2s_init_type i2s_init_struct;  
i2s_default_para_init (&i2s_init_struct);
```

5.17.16 i2s_init function

The table below describes the function i2s_init.

Table 448. i2s_init function

Name	Description
Function name	i2s_init
Function prototype	void i2s_init(spi_type* spi_x, i2s_init_type* i2s_init_struct);
Function description	Initialize I ² S
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	i2s_init_struct: spi_i2s_flag pointer
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define a variable of i2s_init_type before starting.
Called functions	NA

i2s_init_type is defined in the at32f405_405_spi.h:

typedef struct

```
{
    i2s_operation_mode_type      operation_mode;
    i2s_audio_protocol_type      audio_protocol;
    i2s_audio_sampling_freq_type audio_sampling_freq;
    i2s_data_channel_format_type data_channel_format;
    i2s_clock_polarity_type      clock_polarity;
    confirm_state                mclk_output_enable;
    uint32_t                     i2s_ckin_value;
    i2s_pcm_sample_clock_type     pcm_sample_clock_selection;
} i2s_init_type;
```

operation_mode

I²S transfer mode

I2S_MODE_SLAVE_TX:	I2S slave transmit
I2S_MODE_SLAVE_RX:	I2S slave receive
I2S_MODE_MASTER_TX:	I2S master transmit
I2S_MODE_MASTER_RX:	I2S master receive

audio_protocol

I²S audio protocol standards

I2S_AUDIO_PROTOCOL_PHILLIPS:	Phillips
I2S_AUDIO_PROTOCOL_MSB:	MSB aligned (left-aligned)
I2S_AUDIO_PROTOCOL_LSB:	LSB aligned (right-aligned)
I2S_AUDIO_PROTOCOL_PCM_SHORT:	PCM short frame synchronization
I2S_AUDIO_PROTOCOL_PCM_LONG:	PCM long frame synchronization

audio_sampling_freq

I²S audio sampling frequency.

I2S_AUDIO_FREQUENCY_DEFAULT:

Kept at its reset value (sampling frequency changes with SCLK)

I2S_AUDIO_FREQUENCY_8K:	I2S sampling frequency 8K
I2S_AUDIO_FREQUENCY_11_025K:	I2S sampling frequency 11.025K

I2S_AUDIO_FREQUENCY_16K:	I2S sampling frequency 16K
I2S_AUDIO_FREQUENCY_22_05K:	I2S sampling frequency 22.05K
I2S_AUDIO_FREQUENCY_32K:	I2S sampling frequency 32K
I2S_AUDIO_FREQUENCY_44_1K:	I2S sampling frequency 44.1K
I2S_AUDIO_FREQUENCY_48K:	I2S sampling frequency 48K
I2S_AUDIO_FREQUENCY_96K:	I2S sampling frequency 96K
I2S_AUDIO_FREQUENCY_192K:	I2S sampling frequency 192K

data_channel_format

I²S data/channel bits format

I2S_DATA_16BIT_CHANNEL_16BIT: 16-bit data, 16-bit channel

I2S_DATA_16BIT_CHANNEL_32BIT: 16-bit data, 32-bit channel

I2S_DATA_24BIT_CHANNEL_32BIT: 24-bit data, 32-bit channel

I2S_DATA_32BIT_CHANNEL_32BIT: 32-bit data, 32-bit channel

clock_polarity

I²S clock polarity

I2S_CLOCK_POLARITY_LOW: Clock output low in idle state

I2S_CLOCK_POLARITY_HIGH: Clock output high in idle state

mclk_output_enable

Enable mclk clock output

This parameter can be FALSE or TURE.

i2s_ckin_value

Set I2SF input clock frequency, for example, i2s_init_struct.i2s_ckin_value = 8000000; represents 8MHz input clock

pcm_sample_clock_selection

Select I2SF PCM mode clock sampling edge

I2S_PCM_SAMPLE_CLOCK_FALLING: Falling edge

I2S_PCM_SAMPLE_CLOCK_RISING: Rising edge

Example:

```
i2s_init_type i2s_init_struct;
i2s_default_para_init(&i2s_init_struct);
i2s_init_struct.audio_protocol = I2S_AUDIO_PROTOCOL_PHILLIPS;
i2s_init_struct.data_channel_format = I2S_DATA_16BIT_CHANNEL_32BIT;
i2s_init_struct.mclk_output_enable = FALSE;
i2s_init_struct.audio_sampling_freq = I2S_AUDIO_FREQUENCY_48K;
i2s_init_struct.clock_polarity = I2S_CLOCK_POLARITY_LOW;
i2s_init_struct.operation_mode = I2S_MODE_MASTER_TX;
i2s_init(SPI2, &i2s_init_struct);
```


5.17.17 i2s_enable function

The table below describes the function i2s_enable.

Table 449. i2s_enable function

Name	Description
Function name	i2s_enable
Function prototype	void i2s_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable I ² S
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4.
Input parameter 2	new_state: Enable or disable This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable i2s*/
i2s_enable (SPI1, TRUE);
```

5.17.18 spi_i2s_interrupt_enable function

The table below describes the function spi_i2s_interrupt_enable.

Table 450. spi_i2s_interrupt_enable function

Name	Description
Function name	spi_i2s_interrupt_enable
Function prototype	void spi_i2s_interrupt_enable(spi_type* spi_x, uint32_t spi_i2s_int, confirm_state new_state);
Function description	Enable SPI/I ² S interrupts
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	spi_i2s_int : select SPI interrupts
Input parameter 3	new_state: Enable or disable This parameter can be FALSE or TURE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

spi_i2s_int

Select SPI/I²S interrupt selection.

SPI_I2S_ERROR_INT: SPI/I²S error interrupts (including CRC error, overflow error, underflow error and mode error)

SPI_I2S_RDBF_INT: Receive data buffer full

SPI_I2S_TDBE_INT: Transmit data buffer empty

Example:

```

/* enable the specified spi/i2s interrupts */
spi_i2s_interrupt_enable (SPI1, SPI_I2S_ERROR_INT);
spi_i2s_interrupt_enable (SPI1, SPI_I2S_RDBF_INT);
spi_i2s_interrupt_enable (SPI1, SPI_I2S_TDBE_INT);

```

5.17.19 spi_i2s_dma_transmitter_enable function

The table below describes the function spi_i2s_dma_transmitter_enable.

Table 451. spi_i2s_dma_transmitter_enable function

Name	Description
Function name	spi_i2s_dma_transmitter_enable
Function prototype	void spi_i2s_dma_transmitter_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI/I ² S DMA transmitter
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	new_state: enabled or disabled This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```

/* enable spi transmitter dma */
spi_i2s_dma_transmitter_enable (SPI1, TRUE);

```

5.17.20 spi_i2s_dma_receiver_enable function

The table below describes the function spi_i2s_dma_receiver_enable.

Table 452. spi_i2s_dma_receiver_enable function

Name	Description
Function name	spi_i2s_dma_receiver_enable
Function prototype	void spi_i2s_dma_receiver_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable SPI/I ² S DMA receiver
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	new_state: enabled or disabled This parameter can be FALSE or TRUE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```

/* enable spi dma transmitter */
spi_i2s_dma_transmitter_enable (SPI1, TRUE);

```

5.17.21 spi_i2s_data_transmit function

The table below describes the function spi_i2s_data_transmit.

Table 453. spi_i2s_data_transmit function

Name	Description
Function name	spi_i2s_data_transmit
Function prototype	void spi_i2s_data_transmit(spi_type* spi_x, uint16_t tx_data);
Function description	SPI/I ² S sends data
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	tx_data: data to send Value range (for 8-bit bit in a frame): 0x00~0xFF Value range (for16-bit in a frame): 0x0000~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* spi data transmit */
uint16_t tx_data = 0x6666;
spi_i2s_data_transmit (SPI1, tx_data);
```

5.17.22 spi_i2s_data_receive function

The table below describes the function spi_i2s_data_receive.

Table 454. spi_i2s_data_receive function

Name	Description
Function name	spi_i2s_data_receive
Function prototype	uint16_t spi_i2s_data_receive(spi_type* spi_x);
Function description	SPI/I ² S receives data
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Output parameter	rx_data: data to receive Value range (for 8-bit bit in a frame): 0x00~0xFF Value range (for16-bit in a frame): 0x0000~0xFFFF
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* spi data receive */
uint16_t rx_data = 0;
rx_data = spi_i2s_data_receive (SPI1);
```

5.17.23 spi_i2s_flag_get function

The table below describes the function spi_i2s_flag_get.

Table 455. spi_i2s_flag_get function

Name	Description
Function name	spi_i2s_flag_get
Function prototype	flag_status spi_i2s_flag_get(spi_type* spi_x, uint32_t spi_i2s_flag);
Function description	Get SPI/I ² S flags
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	spi_i2s_flag: flag selection Refer to the “spi_i2s_flag” description below for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

spi_i2s_flag

SPI/I²S is used to select a flag from the optional parameters below:

SPI_I2S_RDBF_FLAG:	SPI/I ² S receive data buffer full
SPI_I2S_TDBE_FLAG:	SPI/I ² S transmit data buffer empty
I2S_ACS_FLAG:	I2S audio channel state (indicating left/right channel)
I2S_TUERR_FLAG:	I2S transmitter underload error
SPI_CCERR_FLAG:	SPI CRC error
SPI_MMERR_FLAG:	SPI master mode error
SPI_I2S_ROERR_FLAG:	SPI/I ² S receive overflow error
SPI_I2S_BF_FLAG:	SPI/I ² S busy
SPI_CSPAS_FLAG:	SPI CS pulse error

Example:

```
/* get receive data buffer full flag */
flag_status status;
status = spi_i2s_flag_get(SPI1, SPI_I2S_RDBF_FLAG);
```

5.17.24 spi_i2s_interrupt_flag_get function

The table below describes the function spi_i2s_interrupt_flag_get.

Table 456. spi_i2s_interrupt_flag_get function

Name	Description
Function name	spi_i2s_interrupt_flag_get
Function prototype	flag_status spi_i2s_interrupt_flag_get(spi_type* spi_x, uint32_t spi_i2s_flag);
Function description	Get SPI/I ² S flags
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT or I2S3EXT.
Input parameter 2	spi_i2s_flag: flag selection Refer to the “spi_i2s_flag” description below for details.
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

spi_i2s_flag

SPI/I²S is used to select a flag from the optional parameters below:

SPI_I2S_RDBF_FLAG:	SPI/I ² S receive data buffer full
SPI_I2S_TDBE_FLAG:	SPI/I ² S transmit data buffer empty
I2S_TUERR_FLAG:	I2S transmitter underload error
SPI_CCERR_FLAG:	SPI CRC error
SPI_MMERR_FLAG:	SPI master mode error
SPI_I2S_ROERR_FLAG:	SPI/I ² S receive overflow error
SPI_CSPAS_FLAG:	SPI CS pulse error

Example

```
/* get receive data buffer full flag */
flag_status status;
status = spi_i2s_interrupt_flag_get(SPI1, SPI_I2S_RDBF_FLAG);
```

5.17.25 spi_i2s_flag_clear function

The table below describes the function spi_i2s_flag_clear.

Table 457. spi_i2s_flag_clear function

Name	Description
Function name	spi_i2s_flag_clear
Function prototype	void spi_i2s_flag_clear(spi_type* spi_x, uint32_t spi_i2s_flag)
Function description	Clear SPI/I ² S flags
Input parameter 1	spi_x: select SPI peripheral This parameter can be SPI1, SPI2, SPI3, SPI4, I2S2EXT, I2S3EXT.
Input parameter 2	spi_i2s_flag : select a flag to clear Refer to the “spi_i2s_flag” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

spi_i2s_flag:

SPI/I²S is used for flag selection, including:

SPI_I2S_RDBF_FLAG: SPI/I²S receive data buffer full
I2S_TUERR_FLAG: I2S transmitter underload error
SPI_CCERR_FLAG: SPI CRC error
SPI_MMERR_FLAG: SPI master mode error
SPI_I2S_ROERR_FLAG: SPI/I²S receive overflow error
SPI_CSPAS_FLAG: SPI CS pulse error

Note: the SPI_I2S_TDBE_FLAG (SPI/I²S transmit data buffer empty), the I2S_ACS_FLAG (Audio channel state) and the SPI_I2S_BF_FLAG (SPI/I²S busy) are all set and cleared by hardware to indicate communication state, without the intervention of software.

Example:

```
/* clear receive data buffer full flag */  
spi_i2s_flag_clear (SPI1, SPI_I2S_RDBF_FLAG);
```

5.18 Full-duplex I²S interface I²SF

The I2SF register structure i2sf_type is defined in the “at32f402_405_i2sf.h”:

```
/**
 * @brief type define spi register all
 */
typedef struct
{
    ...
} i2sf_type;
```

The table below gives a list of the I2SF registers

Table 458. Summary of I2SF registers

Register	Description
ctrl2	I2SF control register 2
sts	I2SF status register
dt	I2SF data register
i2sctrl	I2SF configuration register
i2sclkp	I2SF prescaler register
misc1	I2SF extra register 1

The table below gives a list of SPI library functions.

Table 459. Summary of I2SF library functions

Function name	Description
spi_i2s_reset	Reset SPI/I ² S registers to their reset values
i2s_default_para_init	Configure the SPI initialization structure with an initial value
i2s_init	Initialize I2SF
i2s_enable	Enable I2SF CRC
spi_i2s_interrupt_enable	Enable SPI/I ² S interrupts
spi_i2s_dma_transmitter_enable	Enable SPI/I ² S DMA transmit
spi_i2s_dma_receiver_enable	Enable SPI/I ² S DMA receive
spi_i2s_data_transmit	SPI/I ² S transmits data
spi_i2s_data_receive	SPI/I ² S receives data
spi_i2s_flag_get	Get SPI/I ² S flags
spi_i2s_flag_clear	Clear SPI/I ² S flags
i2sf_full_duplex_mode_enable	Enable I2SF full-duplex mode
i2sf_pcm_sample_clock_set	Select I2SF_PCM clock sampling edge

I2SF library functions are the same as SPI/I²S except for the following two functions:

5.18.1 i2sf_full_duplex_mode_enable function

The table below describes the function i2sf_full_duplex_mode_enable.

Table 460. i2sf_full_duplex_mode_enable function

Name	Description
Function name	i2sf_full_duplex_mode_enable
Function prototype	void i2sf_full_duplex_mode_enable(spi_type* spi_x, confirm_state new_state);
Function description	Enable I2SF full-duplex mode
Input parameter 1	spi_x: the selected SPI peripheral This parameter can be I2SF5.
Input parameter 2	new_state: Enable or Disable This parameter can be FALSE or TRUE
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
i2sf_full_duplex_mode_enable(I2SF5, TRUE);
```

5.18.2 i2sf_pcm_sample_clock_set function

The table below describes the function i2sf_pcm_sample_clock_set.

Table 461. i2sf_pcm_sample_clock_set function

Name	Description
Function name	i2sf_pcm_sample_clock_set
Function prototype	void i2sf_pcm_sample_clock_set(spi_type* spi_x, i2s_pcm_sample_clock_type pcm_sample_clock);
Function description	Set I2SF PCM mode clock sampling edge
Input parameter 1	spi_x: the selected SPI peripheral This parameter can be I2SF5.
Input parameter 2	pcm_sample_clock: I2SF PCM mode clock sampling edge
Output parameter	NA
Return value	NA
Required preconditions	It is necessary to define the variables of spi_init_type before starting.
Called functions	NA

pcm_sample_clock

Set I2SF PCM mode clock sampling edge.

I2S_PCM_SAMPLE_CLOCK_FALLING: Falling edge

I2S_PCM_SAMPLE_CLOCK_RISING: Rising edge

Example:

```
i2sf_pcm_sample_clock_set(I2SF5, I2S_PCM_SAMPLE_CLOCK_FALLING);
```


5.19 SysTick

The SysTick register structure SysTick_Type is defined in the “core_cm4.h”:

```
typedef struct
{
    ...

} SysTick_Type;
```

The table below gives a list of the SysTick registers

Table 462. Summary of SysTick registers

Register	Description
ctrl	Controls status register
load	Reload value register
val	Current counter value register
calib	Calibration register

The table below gives a list of SysTick library functions.

Table 463. Summary of SysTick library functions

Function name	Description
systick_clock_source_config	Configure SysTick clock sources
SysTick_Config	Configure SysTick counter reload value and interrupts

5.19.1 systick_clock_source_config function

The table below describes the function systick_clock_source_config.

Table 464. systick_clock_source_config function

Name	Description
Function name	systick_clock_source_config
Function prototype	void systick_clock_source_config(systick_clock_source_type source);
Function description	Configure SysTick clock source
Input parameter 1	Source: systick clock source
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

source

SYSTICK_CLOCK_SOURCE_AHBCLK_DIV8: AHB/8 as SysTick clock

SYSTICK_CLOCK_SOURCE_AHBCLK_NODIV: AHB as SysTick clock

Example:

```
/* config systick clock source */
systick_clock_source_config(SYSTICK_CLOCK_SOURCE_AHBCLK_NODIV);
```

5.19.2 SysTick_Config function

The table below describes the function SysTick_Config

Table 465. SysTick_Config function

Name	Description
Function name	SysTick_Config
Function prototype	uint32_t SysTick_Config(uint32_t ticks);
Function description	Configure SysTick counter reload value and enable interrupt
Input parameter 1	Ticks: SysTick counter interrupt reload value
Output parameter	NA
Return value	Return the setting status of this function, success (0) or failure (1)
Required preconditions	NA
Called functions	NA

Example:

```
/* config systick reload value and enable interrupt */  
SysTick_Config(1000);
```

5.20 TMR

The TMR register structure tmr_type is defined in the "at32f402_405_tmr.h":

```
/**
 * @brief type define tmr register all
 */
typedef struct
{

} tmr_type;
```

The table below gives a list of the TMR registers.

Table 466. Summary of TMR registers

Register	Description
ctrl1	TMR control register 1
ctrl2	TMR control register 2
stctrl	TMR slave timer control register
iden	TMR DMA/ interrupt enable register
ists	TMR interrupt status register
swevt	TMR software event register
cm1	TMR channel mode register 1
cm2	TMR channel mode register 2
cctrl	TMR channel control register
cval	TMR counter value register
div	TMR division register
pr	TMR period register
rpr	TMR repetition period channel
c1dt	TMR channel 1 data register
c2dt	TMR channel 2 data register
c3dt	TMR channel 3 data register
c4dt	TMR channel 4 data register
brk	TMR break register
dmactrl	TMR DMA control register
dmadt	TMR DMA data register
rmp	TMR channel input remap register
cm3	TMR channel mode register 3
c5dt	TMR channel 5 data register

The table below gives a list of TMR library functions.

Table 467. Summary of TMR library functions

Function name	Description
tmr_reset	TMR is reset by CRM reset register
tmr_counter_enable	Enable or disable TMR
tmr_output_default_para_init	Initialize TMR output default parameters
tmr_input_default_para_init	Initialize TMR input default parameters
tmr_brkdt_default_para_init	Initialize TMR brkdt default parameters
tmr_base_init	Initialize TMR period and division
tmr_clock_source_div_set	Set TMR clock source frequency division factor
tmr_cnt_dir_set	Set TMR counter direction
tmr_repetition_counter_set	Set repetition period register
tmr_counter_value_set	Set TMR counter value
tmr_counter_value_get	Get TMR counter value
tmr_div_value_set	Set TMR division value
tmr_div_value_get	Get TMR division value
tmr_output_channel_config	Configure TMR output channels
tmr_output_channel_mode_select	Select TMR output channel mode
tmr_period_value_set	Set TMR period value
tmr_period_value_get	Get TMR period value
tmr_channel_value_set	Set TMR channel value
tmr_channel_value_get	Get TMR channel value
tmr_period_buffer_enable	Enable or disable TMR periodic buffer
tmr_output_channel_buffer_enable	Enable or disable TMR output channel buffer
tmr_output_channel_immediately_set	TMR output channel enable immediately
tmr_output_channel_switch_set	Set TMR output channel switch
tmr_one_cycle_mode_enable	Enable or disable TMR one-cycle mode
tmr_32_bit_function_enable	Enable or disable TMR 32-bit function (plus mode)
tmr_overflow_request_source_set	Select TMR overflow event source
tmr_overflow_event_disable	Enable or disable TMR overflow event generation
tmr_channel_enable	Enable or disable TMR channel
tmr_input_channel_filter_set	Set TMR input channel filter
tmr_pwm_input_config	Configure TMR pwm input
tmr_channel1_input_select	Select TMR channel 1 input
tmr_input_channel_divider_set	Set TMR input channel divider
tmr_primary_mode_select	Select TMR master mode
tmr_sub_mode_select	Select TMR slave timer mode
tmr_channel_dma_select	Select TMR channel DMA request source
tmr_hall_select	Select TMR hall mode
tmr_channel_buffer_enable	Enable or disable TMR channel buffer
tmr_trigger_input_select	Select TMR slave timer trigger input
tmr_sub_sync_mode_set	Set TMR slave timer synchronization mode
tmr_dma_request_enable	Enable or disable TMR DMA request
tmr_interrupt_enable	Enable or disable TMR interrupt
tmr_flag_get	Get TMR flags

tmr_flag_clear	Clear TMR flags
tmr_event_sw_trigger	Software trigger TMR event
tmr_output_enable	Enable or disable TMR output
tmr_internal_clock_set	Set TMR internal clock
tmr_output_channel_polarity_set	Set TMR output channel polarity
tmr_external_clock_config	Set TMR external clock
tmr_external_clock_mode1_config	Set TMR external clock mode 1
tmr_external_clock_mode2_config	Set TMR external clock mode 2
tmr_encoder_mode_config	Set TMR encode mode
tmr_force_output_set	Set TMR forced output
tmr_dma_control_config	Set TMR DMA control
tmr_brkdt_config	Set TMR break mode and dead-time
tmr_iremap_config	Set TMR internal remap

5.20.1 tmr_reset function

The table below describes the function tmr_reset.

Table 468. tmr_reset function

Name	Description
Function name	tmr_reset
Function prototype	void tmr_reset(tmr_type *tmr_x);
Function description	TMR is reset by CRM reset register.
Input parameter	tmr_x: select TMR peripheral, including: TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset();

Example:

tmr_reset(TMR1);

5.20.2 tmr_counter_enable function

The table below describes the function tmr_counter_enable.

Table 469. tmr_counter_enable function

Name	Description
Function name	tmr_counter_enable
Function prototype	void tmr_counter_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR
Input parameter 1	tmr_x: select TMR peripheral, including: TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	new_state: indicates counter status, ON (TRUE) or OFF (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_counter_enable(TMR1, TRUE);
```

5.20.3 tmr_output_default_para_init function

The table below describes the function tmr_output_default_para_init.

Table 470. tmr_output_default_para_init function

Name	Description
Function name	tmr_output_default_para_init
Function prototype	void tmr_output_default_para_init(tmr_output_config_type *tmr_output_struct);
Function description	Initialize tmr output default parameters
Input parameter	tmr_output_struct: tmr_output_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of members of the function tmr_output_struct.

Table 471. tmr_output_struct default values

Member	Default values
oc_mode	TMR_OUTPUT_CONTROL_OFF
oc_idle_state	FALSE
occ_idle_state	FALSE
oc_polarity	TMR_OUTPUT_ACTIVE_HIGH
occ_polarity	TMR_OUTPUT_ACTIVE_HIGH
oc_output_state	FALSE
occ_output_state	FALSE

Example:

```
tmr_output_config_type tmr_output_struct;
```

```
tmr_output_default_para_init(&tmr_output_struct);
```

5.20.4 tmr_input_default_para_init function

The table below describes the function `tmr_input_default_para_init`.

Table 472. tmr_input_default_para_init function

Name	Description
Function name	<code>tmr_input_default_para_init</code>
Function prototype	<code>void tmr_input_default_para_init(tmr_input_config_type *tmr_input_struct);</code>
Function description	Initialize TMR input default parameters
Input parameter	<code>tmr_input_struct</code> : <code>tmr_input_config_type</code> pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Table 470 describes the default values of members of the function `tmr_input_struct`.

Table 473. tmr_input_struct default values

Member	Default values
<code>input_channel_select</code>	<code>TMR_SELECT_CHANNEL_1</code>
<code>input_polarity_select</code>	<code>TMR_INPUT_RISING_EDGE</code>
<code>input_mapped_select</code>	<code>TMR_CC_CHANNEL_MAPPED_DIRECT</code>
<code>input_filter_value</code>	<code>0x0</code>

Example:

```
tmr_input_config_type tmr_input_struct;  
tmr_input_default_para_init(&tmr_input_struct);
```

5.20.5 tmr_brkdt_default_para_init function

The table below describes the function `tmr_brkdt_default_para_init`.

Table 474. tmr_brkdt_default_para_init function

Name	Description
Function name	<code>tmr_brkdt_default_para_init</code>
Function prototype	<code>void tmr_brkdt_default_para_init(tmr_brkdt_config_type *tmr_brkdt_struct);</code>
Function description	Initialize TMR brkdt default parameters
Input parameter	<code>tmr_brkdt_struct</code> : <code>tmr_brkdt_config_type</code> pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

The table below describes the default values of members of the function `tmr_brkdt_struct`.

Table 475. tmr_brkdt_struct default values

Member	Default values
brk_filter_value	0x0
deadtime	0x0
brk_polarity	TMR_BRK_INPUT_ACTIVE_LOW
wp_level	TMR_WP_OFF
auto_output_enable	FALSE
fcsoen_state	FALSE
fcsodis_state	FALSE
brk_enable	FALSE

Example:

```
tmr_brkdt_config_type tmr_brkdt_struct;
tmr_brkdt_default_para_init(&tmr_brkdt_struct);
```

5.20.6 tmr_base_init function

The table below describes the function tmr_base_init.

Table 476. tmr_base_init function

Name	Description
Function name	tmr_base_init
Function prototype	void tmr_base_init(tmr_type* tmr_x, uint32_t tmr_pr, uint32_t tmr_div);
Function description	Initialize TMR period and division
Input parameter 1	tmr_x: TMR peripheral including: TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR12, TMR13, TMR14
Input parameter 2	tmr_pr: timer period value, 0x0000~0xFFFF for 16-bit timer, and 0x0000_0000~0xFFFF_FFFF for 32-bit timer,
Input parameter 3	tmr_div: timer division value, 0x0000~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_base_init(TMR1, 0xFFFF, 0xFFFF);
```


5.20.7 tmr_clock_source_div_set function

The table below describes the function `tmr_clock_source_div_set`.

Table 477. tmr_clock_source_div_set function

Name	Description
Function name	<code>tmr_clock_source_div_set</code>
Function prototype	<code>void tmr_clock_source_div_set(tmr_type *tmr_x, tmr_clock_division_type tmr_clock_div);</code>
Function description	Set TMR clock source division
Input parameter 1	<code>tmr_x</code> : TMR peripheral, including: TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_clock_div</code> : timer clock source frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_clock_div

Select TMR clock source frequency division factor

TMR_CLOCK_DIV1: Divided by 1

TMR_CLOCK_DIV2: Divided by 2

TMR_CLOCK_DIV4: Divided by 4

Example:

```
tmr_clock_source_div_set(TMR1, TMR_CLOCK_DIV4);
```

5.20.8 tmr_cnt_dir_set function

The table below describes the function `tmr_cnt_dir_set`.

Table 478. tmr_cnt_dir_set function

Name	Description
Function name	<code>tmr_cnt_dir_set</code>
Function prototype	<code>void tmr_cnt_dir_set(tmr_type *tmr_x, tmr_count_mode_type tmr_cnt_dir);</code>
Function description	Set TMR counter direction
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, including: TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_cnt_dir</code> : timer counting direction
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_cnt_dir

Select timer counting direction.

TMR_COUNT_UP: Up counting

TMR_COUNT_DOWN: Down counting

TMR_COUNT_TWO_WAY_1: Center-aligned mode (up/down counting) 1

TMR_COUNT_TWO_WAY_2: Center-aligned mode (up/down counting) 2

TMR_COUNT_TWO_WAY_3: Center-aligned mode (up/down counting) 3

Example:

```
tmr_cnt_dir_set(TMR1, TMR_COUNT_UP);
```

5.20.9 tmr_repetition_counter_set function

The table below describes the function tmr_repetition_counter_set.

Table 479. tmr_repetition_counter_set function

Name	Description
Function name	tmr_repetition_counter_set
Function prototype	void tmr_repetition_counter_set(tmr_type *tmr_x, uint16_t tmr_rpr_value);
Function description	Set repetition period register (rpr)
Input parameter 1	tmr_x: TMR peripheral, it includes: TMR1, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_rpr_value: timer repetition period value, it can be 0x0000~0xFFFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_repetition_counter_set(TMR1, 0x1000);
```

5.20.10 tmr_counter_value_set function

The table below describes the function tmr_counter_value_set.

Table 480. tmr_counter_value_set function

Name	Description
Function name	tmr_counter_value_set
Function prototype	void tmr_counter_value_set(tmr_type *tmr_x, uint32_t tmr_cnt_value);
Function description	Set TMR counter value
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_cnt_value: timer counter value, 0x0000~0xFFFF for 16-bit timer; 0x0000_0000~0xFFFF_FFFF 32-bit timer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_counter_value_set(TMR1, 0xFFFF);
```

5.20.11 tmr_counter_value_get function

The table below describes the function tmr_counter_value_get.

Table 481. tmr_counter_value_get function

Name	Description
Function name	tmr_counter_value_get
Function prototype	uint32_t tmr_counter_value_get(tmr_type *tmr_x);
Function description	Get TMR counter value
Input parameter	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Output parameter	NA
Return value	Timer counter value
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t counter_value;
counter_value = tmr_counter_value_get(TMR1);
```

5.20.12 tmr_div_value_set function

The table below describes the function tmr_div_value_set.

Table 482. tmr_div_value_set function

Name	Description
Function name	tmr_div_value_set
Function prototype	void tmr_div_value_set(tmr_type *tmr_x, uint32_t tmr_div_value);
Function description	Set TMR frequency division value
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_div_value: timer frequency division value. 0x0000~0xFFFF for 16-bit timer; 0x0000_0000~0xFFFF_FFFF for 32-bit timer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_div_value_set(TMR1, 0xFFFF);
```

5.20.13 tmr_div_value_get function

The table below describes the function tmr_div_value_get.

Table 483. tmr_div_value_get function

Name	Description
Function name	tmr_div_value_get
Function prototype	uint32_t tmr_div_value_get(tmr_type *tmr_x);
Function description	Get TMR frequency division value
Input parameter	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Output parameter	NA
Return value	Timer frequency division value
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t div_value;
div_value = tmr_div_value_get(TMR1);
```

5.20.14 tmr_output_channel_config function

The table below describes the function tmr_output_channel_config.

Table 484. tmr_output_channel_config function

Name	Description
Function name	tmr_output_channel_config
Function prototype	void tmr_output_channel_config(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_output_config_type *tmr_output_struct);
Function description	Configure TMR output channels
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel
Input parameter 3	tmr_output_struct: tmr_output_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Channel 1

TMR_SELECT_CHANNEL_2: Channel 2

TMR_SELECT_CHANNEL_3: Channel 3

TMR_SELECT_CHANNEL_4: Channel 4

TMR_SELECT_CHANNEL_5: Channel 5

tmr_output_config_type structure

tmr_output_config_type is defined in the at32f402_405_tmr.h:

typedef struct

```
{
    tmr_output_control_mode_type    oc_mode;
    confirm_state                   oc_idle_state;
    confirm_state                   occ_idle_state;
    tmr_output_polarity_type        oc_polarity;
    tmr_output_polarity_type        occ_polarity;
    confirm_state                   oc_output_state;
    confirm_state                   occ_output_state;
} tmr_output_config_type;
```

oc_mode

Set output channel mode, that is, to configure channel original signals (CxORAW).

TMR_OUTPUT_CONTROL_OFF: Disconnect channel output (CxOUT) from CxORAW

TMR_OUTPUT_CONTROL_HIGH: CxORAW high

TMR_OUTPUT_CONTROL_LOW: CxORAW low

TMR_OUTPUT_CONTROL_SWITCH: Switch CxORAW level

TMR_OUTPUT_CONTROL_FORCE_LOW: CxORAW forced low

TMR_OUTPUT_CONTROL_FORCE_HIGH: CxORAW forced high

TMR_OUTPUT_CONTROL_PWM_MODE_A: PWM A mode

TMR_OUTPUT_CONTROL_PWM_MODE_B: PWM B mode

oc_idle_state

Set output channel idle state.

FALSE: Output channel idle state is 0

TRUE: Output channel idle state is 1

occ_idle_state

Set complementary output channel idle state.

FALSE: Complementary output channel idle state is 0

TRUE: Complementary output channel idle state is 1

oc_polarity

Set the polarity of output channels.

TMR_OUTPUT_ACTIVE_HIGH: Active high

TMR_OUTPUT_ACTIVE_LOW: Active low

occ_polarity

Set the polarity of complementary output channels.

TMR_OUTPUT_ACTIVE_HIGH: Active high

TMR_OUTPUT_ACTIVE_LOW: Active low

oc_output_state

Set the state of output channels.

FALSE: Output channel OFF

TRUE: Output channel ON

occ_output_state

Set the state of complementary output channels.

FALSE: Complementary output channel OFF

TRUE: Complementary output channel ON

Example:

```

tmr_output_config_type tmr_output_struct;
tmr_output_struct.oc_mode = TMR_OUTPUT_CONTROL_OFF;
tmr_output_struct.oc_output_state = TRUE;
tmr_output_struct.oc_polarity = TMR_OUTPUT_ACTIVE_HIGH;
tmr_output_struct.oc_idle_state = TRUE;
tmr_output_struct.occ_output_state = TRUE;
tmr_output_struct.occ_polarity = TMR_OUTPUT_ACTIVE_HIGH;
tmr_output_struct.occ_idle_state = TRUE;
tmr_output_channel_config(TMR1, TMR_SELECT_CHANNEL_1, &tmr_output_struct);

```

5.20.15 tmr_output_channel_mode_select function

The table below describes the function `tmr_output_channel_mode_select`.

Table 485. tmr_output_channel_mode_select function

Name	Description
Function name	<code>tmr_output_channel_mode_select</code>
Function prototype	<code>void tmr_output_channel_mode_select(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_output_control_mode_type oc_mode);</code>
Function description	Select TMR output channel mode
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_channel</code> : refer to the “ tmr_channel ” descriptions below for details
Input parameter 3	<code>oc_mode</code> : refer to the “ oc_mode ” descriptions below for details
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

TMR_SELECT_CHANNEL_5: Timer channel 5

oc_mode

Set output channel mode, that is, to configure channel original signals (CxORAW).

TMR_OUTPUT_CONTROL_OFF: Disconnect channel output (CxOUT) from CxORAW

TMR_OUTPUT_CONTROL_HIGH: CxORAW high

TMR_OUTPUT_CONTROL_LOW: CxORAW low

TMR_OUTPUT_CONTROL_SWITCH: Switch CxORAW level

TMR_OUTPUT_CONTROL_FORCE_LOW: CxORAW forced low

TMR_OUTPUT_CONTROL_FORCE_HIGH: CxORAW forced high

TMR_OUTPUT_CONTROL_PWM_MODE_A: PWM A mode

TMR_OUTPUT_CONTROL_PWM_MODE_B: PWM B mode

Example:

```
tmr_output_channel_mode_select(TMR1, TMR_SELECT_CHANNEL_1, TMR_OUTPUT_CONTROL_SWITCH);
```

5.20.16 tmr_period_value_set function

The table below describes the function `tmr_period_value_set`.

Table 486. tmr_period_value_set function

Name	Description
Function name	<code>tmr_period_value_set</code>
Function prototype	<code>void tmr_period_value_set(tmr_type *tmr_x, uint32_t tmr_pr_value);</code>
Function description	Set TMR period value
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_pr_value</code> : timer period value., 0x0000~0xFFFF for 16-bit timer; 0x0000_0000~0xFFFF_FFFF for 32-bit timer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_period_value_set(TMR1, 0xFFFF);
```

5.20.17 tmr_period_value_get function

The table below describes the function `tmr_period_value_get`.

Table 487. tmr_period_value_get function

Name	Description
Function name	<code>tmr_period_value_get</code>
Function prototype	<code>uint32_t tmr_period_value_get(tmr_type *tmr_x);</code>
Function description	Get TMR period value
Input parameter	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Output parameter	NA
Return value	Timer period value
Required preconditions	NA
Called functions	NA

Example:

```
uint32_t pr_value;
pr_value = tmr_period_value_get(TMR1);
```

5.20.18 tmr_channel_value_set function

The table below describes the function tmr_channel_value_set.

Table 488. tmr_channel_value_set function

Name	Description
Function name	tmr_channel_value_set
Function prototype	void tmr_channel_value_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, uint32_t tmr_channel_value);
Function description	Set TMR channel value
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel
Input parameter 3	tmr_channel_value: timer channel value. 0x0000~0xFFFF for 16-bit timer; 0x0000_0000~0xFFFF_FFFF for 32-bit timer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1:	Channel 1
TMR_SELECT_CHANNEL_2:	Channel 2
TMR_SELECT_CHANNEL_3:	Channel 3
TMR_SELECT_CHANNEL_4:	Channel 4
TMR_SELECT_CHANNEL_5:	Channel 5

Example:

```
tmr_channel_value_set(TMR1, TMR_SELECT_CHANNEL_1, 0xFFFF);
```


5.20.19 tmr_channel_value_get function

The table below describes the function `tmr_channel_value_get`.

Table 489. tmr_channel_value_get function

Name	Description
Function name	<code>tmr_channel_value_get</code>
Function prototype	<code>uint32_t tmr_channel_value_get(tmr_type *tmr_x, tmr_channel_select_type tmr_channel);</code>
Function description	Get TMR channel value
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_channel</code> : timer channel
Output parameter	Timer channel value
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: TMR channel 1

TMR_SELECT_CHANNEL_2: TMR channel 2

TMR_SELECT_CHANNEL_3: TMR channel 3

TMR_SELECT_CHANNEL_4: TMR channel 4

TMR_SELECT_CHANNEL_5: TMR channel 5

Example:

```
uint32_t ch_value;  
ch_value = tmr_channel_value_get(TMR1, TMR_SELECT_CHANNEL_1);
```

5.20.20 tmr_period_buffer_enable function

The table below describes the function `tmr_period_buffer_enable`.

Table 490. tmr_period_buffer_enable function

Name	Description
Function name	<code>tmr_period_buffer_enable</code>
Function prototype	<code>void tmr_period_buffer_enable(tmr_type *tmr_x, confirm_state new_state);</code>
Function description	Enable or disable TMR period buffer
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>new_state</code> : indicates the status of period buffer. It can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_period_buffer_enable(TMR1, TRUE);
```

5.20.21 tmr_output_channel_buffer_enable function

The table below describes the function `tmr_output_channel_buffer_enable`.

Table 491. tmr_output_channel_buffer_enable function

Name	Description
Function name	<code>tmr_output_channel_buffer_enable</code>
Function prototype	<code>void tmr_output_channel_buffer_enable(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);</code>
Function description	Enable or disable TMR output channel buffer
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_channel</code> : timer channel
Input parameter 3	<code>new_state</code> : indicates the status of output channel buffer. It can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

TMR_SELECT_CHANNEL_5: Timer channel 5

Example:

```
tmr_output_channel_buffer_enable(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```

5.20.22 tmr_output_channel_immediately_set function

The table below describes the function `tmr_output_channel_immediately_set`.

Table 492. tmr_output_channel_immediately_set function

Name	Description
Function name	<code>tmr_output_channel_immediately_set</code>
Function prototype	<code>void tmr_output_channel_immediately_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);</code>
Function description	Enable TMR output channel immediately
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_channel</code> : timer channel
Input parameter 3	<code>new_state</code> : indicates the status of output channel enable. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

TMR_SELECT_CHANNEL_5: Timer channel 5

Example:

```
tmr_output_channel_immediately_set(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```

5.20.23 tmr_output_channel_switch_set function

The table below describes the function `tmr_output_channel_switch_set`.

Table 493. tmr_output_channel_switch_set function

Name	Description
Function name	<code>tmr_output_channel_switch_set</code>
Function prototype	<code>void tmr_output_channel_switch_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);</code>
Function description	Set TMR output channel switch
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR12, TMR14
Input parameter 2	<code>tmr_channel</code> : timer channel
Input parameter 3	<code>new_state</code> : indicates the status of output channel switch. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

TMR_SELECT_CHANNEL_5: Timer channel 5

Example:

```
tmr_output_channel_switch_set(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```

5.20.24 tmr_one_cycle_mode_enable function

The table below describes the function `tmr_one_cycle_mode_enable`.

Table 494. tmr_one_cycle_mode_enable function

Name	Description
Function name	<code>tmr_one_cycle_mode_enable</code>
Function prototype	<code>void tmr_one_cycle_mode_enable(tmr_type *tmr_x, confirm_state new_state);</code>
Function description	Enable or disable TMR one-cycle mode
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>new_state</code> : indicates the status of one-cycle mode. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_one_cycle_mode_enable(TMR1, TRUE);
```

5.20.25 tmr_32_bit_function_enable function

The table below describes the function tmr_32_bit_function_enable.

Table 495. tmr_32_bit_function_enable function

Name	Description
Function name	tmr_32_bit_function_enable
Function prototype	void tmr_32_bit_function_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR 32-bit feature (plus mode)
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR2
Input parameter 2	new_state: the status of 32-bit mode This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_32_bit_function_enable(TMR2, TRUE);
```

5.20.26 tmr_overflow_request_source_set function

The table below describes the function tmr_overflow_request_source_set.

Table 496. tmr_overflow_request_source_set function

Name	Description
Function name	tmr_overflow_request_source_set
Function prototype	void tmr_overflow_request_source_set(tmr_type *tmr_x, confirm_state new_state);
Function description	Select TMR overflow event sources
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	new_state: indicates the overflow event source.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Select an overflow event source.

FALSE: Counter overflow, OVFSWTR being set, overflow event from slave mode timer controller

TRUE: Counter overflow only.

Example:

```
tmr_overflow_request_source_set(TMR1, TRUE);
```

5.20.27 tmr_overflow_event_disable function

The table below describes the function tmr_overflow_event_disable.

Table 497. tmr_overflow_event_disable function

Name	Description
Function name	tmr_overflow_event_disable
Function prototype	void tmr_overflow_event_disable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR overflow event generation
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	new_state: indicates the status of overflow event generation.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Select the status of overflow event generation.

FALSE: Enable overflow event generation, which can be generated from the following:

- Counter overflow
- Set OVFSWTR=1
- Overflow event from slave mode timer controller

TRUE: Disable overflow event generation

Example:

```
tmr_overflow_event_disable(TMR1, TRUE);
```

5.20.28 tmr_input_channel_init function

The table below describes the function tmr_input_channel_init.

Table 498. tmr_input_channel_init function

Name	Description
Function name	tmr_input_channel_init
Function prototype	void tmr_input_channel_init(tmr_type *tmr_x, tmr_input_config_type *input_struct, tmr_channel_input_divider_type divider_factor);
Function description	Initialize TMR input channels
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	input_struct: tmr_input_config_type pointer
Input parameter 3	divider_factor: input channel frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_input_config_type structure

tmr_input_config_type is defined in the at32f402_405_tmr.h:

typedef struct

```
{
    tmr_channel_select_type      input_channel_select;
    tmr_input_polarity_type      input_polarity_select;
    tmr_input_direction_mapped_type  input_mapped_select;
    uint8_t                      input_filter_value;
}
```

} tmr_input_config_type;

input_channel_select

Select a TMR input channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

input_polarity_select

Select the polarity of input channels.

TMR_INPUT_RISING_EDGE: Rising edge

TMR_INPUT_FALLING_EDGE: Falling edge

TMR_INPUT_BOTH_EDGE: Both edges (Rising edge and Falling edge)

input_mapped_select

Select input channel mapping.

TMR_CC_CHANNEL_MAPPED_DIRECT:

TMR input channel 1,2,3 and 4 is linked to C1IRAW, C2IRAW, C3IRAW and C4IRAW respectively.

TMR_CC_CHANNEL_MAPPED_INDIRECT:

TMR input channel 1,2,3 and 4 is linked to C2IRAW, C1IRAW, C4IRAW and C3IRAW respectively.

TMR_CC_CHANNEL_MAPPED_STI:

TMR input channel is mapped on STI

input_filter_value

Select an input channel filter value, between 0x00~0x0F

divider_factor

Select input channel frequency division factor.

TMR_CHANNEL_INPUT_DIV_1: Divided by 1

TMR_CHANNEL_INPUT_DIV_2: Divided by 2

TMR_CHANNEL_INPUT_DIV_4: Divided by 4

TMR_CHANNEL_INPUT_DIV_8: Divided by 8

Example:

```
tmr_input_config_type tmr_input_config_struct;
tmr_input_config_struct.input_channel_select = TMR_SELECT_CHANNEL_2;
tmr_input_config_struct.input_mapped_select = TMR_CC_CHANNEL_MAPPED_DIRECT;
tmr_input_config_struct.input_polarity_select = TMR_INPUT_RISING_EDGE;
tmr_input_config_struct.input_filter_value = 0x00;
tmr_input_channel_init(TMR1, &tmr_input_config_struct, TMR_CHANNEL_INPUT_DIV_1);
```

5.20.29 tmr_channel_enable function

The table below describes the function tmr_channel_enable.

Table 499. tmr_channel_enable function

Name	Description
Function name	tmr_channel_enable
Function prototype	void tmr_channel_enable(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, confirm_state new_state);
Function description	Enable or disable TMR channels
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel
Input parameter 3	new_state: indicates the status of timer channels. This parameter can be Enable (TRUE) or Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1:	Timer channel 1
TMR_SELECT_CHANNEL_1C:	Complementary channel 1
TMR_SELECT_CHANNEL_2:	Timer channel 2
TMR_SELECT_CHANNEL_2C:	Complementary channel 2
TMR_SELECT_CHANNEL_3:	Timer channel 3
TMR_SELECT_CHANNEL_3C:	Complementary channel 3
TMR_SELECT_CHANNEL_4:	Timer channel 4

Example:

```
tmr_channel_enable(TMR1, TMR_SELECT_CHANNEL_1, TRUE);
```


5.20.30 tmr_input_channel_filter_set function

The table below describes the function tmr_input_channel_filter_set.

Table 500. tmr_input_channel_filter_set function

Name	Description
Function name	tmr_input_channel_filter_set
Function prototype	void tmr_input_channel_filter_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, uint16_t filter_value);
Function description	Set TMR input channel filter
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel
Input parameter 3	filter_value: set channel filter value, 0x00~0x0F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

Example:

```
tmr_input_channel_filter_set(TMR1, TMR_SELECT_CHANNEL_1, 0x0F);
```

5.20.31 tmr_pwm_input_config function

The table below describes the function tmr_pwm_input_config.

Table 501. tmr_pwm_input_config function

Name	Description
Function name	tmr_pwm_input_config
Function prototype	void tmr_pwm_input_config(tmr_type *tmr_x, tmr_input_config_type *input_struct, tmr_channel_input_divider_type divider_factor);
Function description	Configure TMR pwm input
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	input_struct: tmr_input_config_type pointer
Input parameter 3	divider_factor: input channel frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

input_struct

Point to the tmr_input_config_type, see [tmr_input_config_type](#) for details.

divider_factor

Select input channel frequency division factor

TMR_CHANNEL_INPUT_DIV_1: Divided by 1

TMR_CHANNEL_INPUT_DIV_2: Divided by 2

TMR_CHANNEL_INPUT_DIV_4: Divided by 4

TMR_CHANNEL_INPUT_DIV_8: Divided by 8

Example:

```
tmr_input_config_type tmr_ic_init_structure;
tmr_ic_init_structure.input_filter_value = 0;
tmr_ic_init_structure.input_channel_select = TMR_SELECT_CHANNEL_2;
tmr_ic_init_structure.input_mapped_select = TMR_CC_CHANNEL_MAPPED_DIRECT;
tmr_ic_init_structure.input_polarity_select = TMR_INPUT_RISING_EDGE;
tmr_pwm_input_config(TMR1, &tmr_ic_init_structure, TMR_CHANNEL_INPUT_DIV_1);
```

5.20.32 tmr_channel1_input_select function

The table below describes the function tmr_channel1_input_select.

Table 502. tmr_channel1_input_select function

Name	Description
Function name	tmr_channel1_input_select
Function prototype	void tmr_channel1_input_select(tmr_type *tmr_x, tmr_channel1_input_connected_type ch1_connect);
Function description	Select TMR channel 1 input
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4
Input parameter 2	ch1_connect: channel 1 input selection
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

ch1_connect

Select channel 1 input.

TMR_CHANNEL1_CONNECTED_C1IRAW: CH1 pin is connected to C1IRAW

TMR_CHANNEL1_2_3_CONNECTED_C1IRAW_XOR: Connect the XOR results of CH1, CH2 and CH3 pins to C1IRAW

Example:

```
tmr_channel1_input_select(TMR1, TMR_CHANNEL1_2_3_CONNECTED_C1IRAW_XOR);
```

5.20.33 tmr_input_channel_divider_set function

The table below describes the function tmr_input_channel_divider_set.

Table 503. tmr_input_channel_divider_set function

Name	Description
Function name	tmr_input_channel_divider_set
Function prototype	void tmr_input_channel_divider_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_channel_input_divider_type divider_factor);
Function description	Set TMR input channel divider
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel
Input parameter 3	divider_factor: input channel frequency division factor
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

divider_factor

Select input channel frequency division factor

TMR_CHANNEL_INPUT_DIV_1: Divided by 1

TMR_CHANNEL_INPUT_DIV_2: Divided by 2

TMR_CHANNEL_INPUT_DIV_4: Divided by 4

TMR_CHANNEL_INPUT_DIV_8: Divided by 8

Example:

```
tmr_input_channel_divider_set(TMR1, TMR_SELECT_CHANNEL_1, TMR_CHANNEL_INPUT_DIV_2);
```

5.20.34 tmr_primary_mode_select function

The table below describes the function tmr_primary_mode_select.

Table 504. tmr_primary_mode_select function

Name	Description
Function name	tmr_primary_mode_select
Function prototype	void tmr_primary_mode_select(tmr_type *tmr_x, tmr_primary_select_type primary_mode);
Function description	Select TMR primary (master) mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9
Input parameter 2	primary_mode: master mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

primary_mode

Select primary mode, that is, master timer output signal selection.

TMR_PRIMARY_SEL_RESET:	Reset
TMR_PRIMARY_SEL_ENABLE:	Enable
TMR_PRIMARY_SEL_OVERFLOW:	Overflow
TMR_PRIMARY_SEL_COMPARE:	Compare pulse
TMR_PRIMARY_SEL_C1ORAW:	C1ORAW
TMR_PRIMARY_SEL_C2ORAW:	C2ORAW
TMR_PRIMARY_SEL_C3ORAW:	C3ORAW
TMR_PRIMARY_SEL_C4ORAW:	C4ORAW

Example:

```
tmr_primary_mode_select(TMR1, TMR_PRIMARY_SEL_RESET);
```

5.20.35 tmr_sub_mode_select function

The table below describes the function tmr_sub_mode_select.

Table 505. tmr_sub_mode_select function

Name	Description
Function name	tmr_sub_mode_select
Function prototype	void tmr_sub_mode_select(tmr_type *tmr_x, tmr_sub_mode_select_type sub_mode);
Function description	Select TMR slave timer mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR5, TMR8, TMR9
Input parameter 2	sub_mode: slave timer mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

primary_mode

Select slave timer modes.

TMR_SUB_MODE_DISABLE:	Disable
TMR_SUB_ENCODER_MODE_A:	Encoder mode A
TMR_SUB_ENCODER_MODE_B:	Encoder mode B
TMR_SUB_ENCODER_MODE_C:	Encoder mode C
TMR_SUB_RESET_MODE:	Reset
TMR_SUB_HANG_MODE:	Suspend
TMR_SUB_TRIGGER_MODE:	Trigger
TMR_SUB_EXTERNAL_CLOCK_MODE_A:	External clock A

Example:

```
tmr_sub_mode_select(TMR1, TMR_SUB_HANG_MODE);
```

5.20.36 tmr_channel_dma_select function

The table below describes the function tmr_channel_dma_select.

Table 506. tmr_channel_dma_select function

Name	Description
Function name	tmr_channel_dma_select
Function prototype	void tmr_channel_dma_select(tmr_type *tmr_x, tmr_dma_request_source_type cc_dma_select);
Function description	Select TMR channel DMA request source
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	cc_dma_select: TMR channel DMA request source
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

cc_dma_select

Select DMA request source for TMR channels.

TMR_DMA_REQUEST_BY_CHANNEL: DMA request upon a channel event (CxIF = 1)

TMR_DMA_REQUEST_BY_OVERFLOW: DMA request upon an overflow event (OVFIF = 1)

Example:

```
tmr_channel_dma_select(TMR1, TMR_DMA_REQUEST_BY_OVERFLOW);
```

5.20.37 tmr_hall_select function

The table below describes the function tmr_hall_select

Table 507. tmr_hall_select function

Name	Description
Function name	tmr_hall_select
Function prototype	void tmr_hall_select(tmr_type *tmr_x, confirm_state new_state);
Function description	Select TMR hall mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	new_state: indicates the status of TMR hall mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

new_state

Select the status of TMR hall mode in order to refresh channel control bit.

FALSE: Refresh channel control bit through HALL

TRUE: Refresh channel control bit through HALL or the rising edge of TRGIN

Example:

```
tmr_hall_select(TMR1, TRUE);
```

5.20.38 tmr_channel_buffer_enable function

The table below describes the function tmr_channel_buffer_enable.

Table 508. tmr_channel_buffer_enable function

Name	Description
Function name	tmr_channel_buffer_enable
Function prototype	void tmr_channel_buffer_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR channel buffer
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	new_state: indicates the status of TMR channel buffer. This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_channel_buffer_enable(TMR1, TRUE);
```

5.20.39 tmr_trgout2_enable function

The table below describes the function tmr_trgout2_enable.

Table 509. tmr_trgout2_enable function

Name	Description
Function name	tmr_trgout2_enable
Function prototype	void tmr_trgout2_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR trigger output 2 signal
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1
Input parameter 2	new_state: indicates the status of TMR trigger output 2 signal. It can be enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_trgout2_enable(TMR1, TRUE);
```

5.20.40 tmr_trigger_input_select function

The table below describes the function tmr_trigger_input_select.

Table 510. tmr_trigger_input_select function

Name	Description
Function name	tmr_trigger_input_select
Function prototype	void tmr_trigger_input_select(tmr_type *tmr_x, sub_tmr_input_sel_type trigger_select);
Function description	Select TMR slave timer trigger input
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9
Input parameter 2	trigger_select: select TMR slave timer trigger input
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

trigger_select

Select TMR slave timer trigger input.

TMR_SUB_INPUT_SEL_IS0: Internal input 0
 TMR_SUB_INPUT_SEL_IS1: Internal input 1
 TMR_SUB_INPUT_SEL_IS2: Internal input 2
 TMR_SUB_INPUT_SEL_IS3: Internal input 3
 TMR_SUB_INPUT_SEL_C1INC: C1IRAW input detection
 TMR_SUB_INPUT_SEL_C1DF1: Filter input channel 1
 TMR_SUB_INPUT_SEL_C2DF2: Filter input channel 2
 TMR_SUB_INPUT_SEL_EXTIN: External input channel EXT

Example:

```
tmr_trigger_input_select(TMR1, TMR_SUB_INPUT_SEL_IS0);
```

5.20.41 tmr_sub_sync_mode_set function

The table below describes the function tmr_sub_sync_mode_set.

Table 511. tmr_sub_sync_mode_set function

Name	Description
Function name	tmr_sub_sync_mode_set
Function prototype	void tmr_sub_sync_mode_set(tmr_type *tmr_x, confirm_state new_state);
Function description	Set TMR slave timer synchronization mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9
Input parameter 2	new_state: indicates the status of TMR slave timer synchronization mode This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_sub_sync_mode_set(TMR1, TRUE);
```

5.20.42 tmr_dma_request_enable function

The table below describes the function `tmr_dma_request_enable`.

Table 512. tmr_dma_request_enable function

Name	Description
Function name	<code>tmr_dma_request_enable</code>
Function prototype	<code>void tmr_dma_request_enable(tmr_type *tmr_x, tmr_dma_request_type dma_request, confirm_state new_state);</code>
Function description	Enable or disable TMR DMA request
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>dma_request</code> : DMA request
Input parameter 3	<code>new_state</code> : indicates the status of DMA request. This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_request

Select a DMA request.

TMR_OVERFLOW_DMA_REQUEST: Overflow event DMA request
TMR_C1_DMA_REQUEST: Channel 1 DMA request
TMR_C2_DMA_REQUEST: Channel 2 DMA request
TMR_C3_DMA_REQUEST: Channel 3 DMA request
TMR_C4_DMA_REQUEST: Channel 4 DMA request
TMR_HALL_DMA_REQUEST: HALL event DMA request
TMR_TRIGGER_DMA_REQUEST: Trigger event DMA request

Example:

```
tmr_dma_request_enable(TMR1, TMR_OVERFLOW_DMA_REQUEST, TRUE);
```

5.20.43 tmr_interrupt_enable function

The table below describes the function tmr_interrupt_enable.

Table 513. tmr_interrupt_enable function

Name	Description
Function name	tmr_interrupt_enable
Function prototype	void tmr_interrupt_enable(tmr_type *tmr_x, uint32_t tmr_interrupt, confirm_state new_state);
Function description	Enable or disable TMR interrupts
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_interrupt: TMR interrupts
Input parameter 3	new_state: indicates the status of TMR interrupts. This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_interrupt

Select a TMR interrupt.

TMR_OVF_INT:	Overflow event interrupt
TMR_C1_INT:	Channel 1 event interrupt
TMR_C2_INT:	Channel 2 event interrupt
TMR_C3_INT:	Channel 3 event interrupt
TMR_C4_INT:	Channel 4 event interrupt
TMR_HALL_INT:	HALL event interrupt
TMR_TRIGGER_INT:	Trigger event interrupt
TMR_BRK_INT:	Break event interrupt

Example:

```
tmr_interrupt_enable(TMR1, TMR_OVF_INT, TRUE);
```

5.20.44 tmr_interrupt_flag_get function

The table below describes the function tmr_interrupt_flag_get.

Table 514. tmr_interrupt_flag_get function

Name	Description
Function name	tmr_interrupt_flag_get
Function prototype	flag_status tmr_interrupt_flag_get (tmr_type *tmr_x, uint32_t tmr_flag);
Function description	Get interrupt flag status
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14.
Input parameter 2	tmr_flag: Flag selection Refer to the “tmr_flag” description below for details.
Output parameter	NA
Return value	flag_status: Flas status Return SET or RESET.
Required preconditions	NA
Called functions	NA

tmr_flag

This is used for flag selection, including:

TMR_OVF_FLAG:	Overflow interrupt flag
TMR_C1_FLAG:	Channel 1 interrupt flag
TMR_C2_FLAG:	Channel 2 interrupt flag
TMR_C3_FLAG:	Channel 3 interrupt flag
TMR_C4_FLAG:	Channel 4 interrupt flag
TMR_HALL_FLAG:	HALL interrupt flag
TMR_TRIGGER_FLAG:	Trigger interrupt flag
TMR_BRK_FLAG:	Break interrupt flag

Example

```
if(tmr_interrupt_flag_get (TMR1, TMR_OVF_FLAG) != RESET)
```

5.20.45 tmr_flag_get function

The table below describes the function tmr_flag_get.

Table 515. tmr_flag_get function

Name	Description
Function name	tmr_flag_get
Function prototype	flag_status tmr_flag_get(tmr_type *tmr_x, uint32_t tmr_flag);
Function description	Get flag status
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_flag: Flag selection Refer to the “tmr_flag” description below for details.
Output parameter	NA
Return value	flag_status: indicates the status of flags Return SET or RESET
Required preconditions	NA
Called functions	NA

tmr_flag

This is used for flag selection, including:

TMR_OVF_FLAG:	Overflow interrupt flag
TMR_C1_FLAG:	Channel 1 interrupt flag
TMR_C2_FLAG:	Channel 2 interrupt flag
TMR_C3_FLAG:	Channel 3 interrupt flag
TMR_C4_FLAG:	Channel 4 interrupt flag
TMR_HALL_FLAG:	HALL interrupt flag
TMR_TRIGGER_FLAG:	Trigger interrupt flag
TMR_BRK_FLAG:	Break interrupt flag
TMR_C1_RECAPTURE_FLAG:	Channel 1 recapture flag
TMR_C2_RECAPTURE_FLAG:	Channel 2 recapture flag
TMR_C3_RECAPTURE_FLAG:	Channel 3 recapture flag
TMR_C4_RECAPTURE_FLAG:	Channel 4 recapture flag

Example:

```
if(tmr_flag_get(TMR1, TMR_OVF_FLAG) != RESET)
```

5.20.46 tmr_flag_clear function

The table below describes the function `tmr_flag_clear`.

Table 516. tmr_flag_clear function

Name	Description
Function name	<code>tmr_flag_clear</code>
Function prototype	<code>void tmr_flag_clear(tmr_type *tmr_x, uint32_t tmr_flag);</code>
Function description	Clear flag
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_flag</code> : flag selection Refer to tmr_flag for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_flag_clear(TMR1, TMR_OVF_FLAG);
```

5.20.47 tmr_event_sw_trigger function

The table below describes the function `tmr_event_sw_trigger`

Table 517. tmr_event_sw_trigger function

Name	Description
Function name	<code>tmr_event_sw_trigger</code>
Function prototype	<code>void tmr_event_sw_trigger(tmr_type *tmr_x, tmr_event_trigger_type tmr_event);</code>
Function description	Software triggers TMR events
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR6, TMR7, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	<code>tmr_event</code> : select a TMR event
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_event

Set TMR events triggered by software.

TMR_OVERFLOW_SWTRIG: Overflow event
TMR_C1_SWTRIG: Channel 1 event
TMR_C2_SWTRIG: Channel 2 event
TMR_C3_SWTRIG: Channel 3 event
TMR_C4_SWTRIG: Channel 4 event
TMR_HALL_SWTRIG: HALL event

TMR_TRIGGER_SWTRIG: Trigger event

TMR_BRK_SWTRIG: Break event

Example:

```
tmr_event_sw_trigger(TMR1, TMR_OVERFLOW_SWTRIG);
```

5.20.48 tmr_output_enable function

The table below describes the function tmr_output_enable

Table 518. tmr_output_enable function

Name	Description
Function name	tmr_output_enable
Function prototype	void tmr_output_enable(tmr_type *tmr_x, confirm_state new_state);
Function description	Enable or disable TMR output
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	new_state: TMR output status This parameter can be Enable (TRUE) or Disable (FALSE).
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_output_enable(TMR1, TRUE);
```

5.20.49 tmr_internal_clock_set function

The table below describes the function tmr_internal_clock_set.

Table 519. tmr_internal_clock_set function

Name	Description
Function name	tmr_internal_clock_set
Function prototype	void tmr_internal_clock_set(tmr_type *tmr_x);
Function description	Set TMR internal clock
Input parameter	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
tmr_internal_clock_set(TMR1);
```

5.20.50 tmr_output_channel_polarity_set function

The table below describes the function tmr_output_channel_polarity_set.

Table 520. tmr_output_channel_polarity_set function

Name	Description
Function name	tmr_output_channel_polarity_set
Function prototype	void tmr_output_channel_polarity_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_polarity_active_type oc_polarity);
Function description	Set TMR output channel polarity
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_channel: Timer channel
Input parameter 3	oc_polarity: output channel polarity
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1:	Timer channel 1
TMR_SELECT_CHANNEL_1C:	Complementary channel 1
TMR_SELECT_CHANNEL_2:	Timer channel 2
TMR_SELECT_CHANNEL_2C:	Complementary channel 2
TMR_SELECT_CHANNEL_3:	Timer channel 3
TMR_SELECT_CHANNEL_3C:	Complementary channel 3
TMR_SELECT_CHANNEL_4:	Timer channel 4

oc_polarity

Select TMR channel polarity.

TMR_POLARITY_ACTIVE_HIGH: Active high

TMR_POLARITY_ACTIVE_LOW: Active low

Example:

```
tmr_output_channel_polarity_set(TMR1, TMR_SELECT_CHANNEL_1, TMR_POLARITY_ACTIVE_HIGH);
```

5.20.51 tmr_external_clock_config function

The table below describes the function tmr_external_clock_config.

Table 521. tmr_external_clock_config function

Name	Description
Function name	tmr_external_clock_config
Function prototype	void tmr_external_clock_config(tmr_type *tmr_x, tmr_external_signal_divider_type es_divide, tmr_external_signal_polarity_type es_polarity, uint16_t es_filter);
Function description	Configure TMR external clock
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4
Input parameter 2	es_divide: external signal frequency division factor
Input parameter 3	es_polarity: external signal polarity
Input parameter 4	es_filter: external signal filter value, 0x00~0x0F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

es_divide

Set TMR external signal frequency division factor.

TMR_ES_FREQUENCY_DIV_1: Divided by 1

TMR_ES_FREQUENCY_DIV_2: Divided by 2

TMR_ES_FREQUENCY_DIV_4: Divided by 4

TMR_ES_FREQUENCY_DIV_8: Divided by 8

es_polarity

Select TMR external signal polarity.

TMR_ES_POLARITY_NON_INVERTED: High or rising edge

TMR_ES_POLARITY_INVERTED: Low or falling edge

Example:

```
tmr_external_clock_config(TMR1, TMR_ES_FREQUENCY_DIV_1, TMR_ES_POLARITY_INVERTED, 0x0F);
```


5.20.52 tmr_external_clock_mode1_config function

The table below describes the function tmr_external_clock_mode1_config.

Table 522. tmr_external_clock_mode1_config function

Name	Description
Function name	tmr_external_clock_mode1_config
Function prototype	void tmr_external_clock_mode1_config(tmr_type *tmr_x, tmr_external_signal_divider_type es_divide, tmr_external_signal_polarity_type es_polarity, uint16_t es_filter);
Function description	Configure TMR external clock mode 1 (corresponding to external mode A in the reference manual)
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9
Input parameter 2	es_divide: external signal frequency division factor
Input parameter 3	es_polarity: external signal polarity
Input parameter 4	es_filter: external signal filter value, 0x00~0x0F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

es_divide

Set TMR external signal frequency division factor, refer to [es_divide](#) for details.

es_polarity

Set TMR external signal polarity, refer to [es_polarity](#) for details.

Example:

```
tmr_external_clock_mode1_config(TMR1, TMR_ES_FREQUENCY_DIV_1, TMR_ES_POLARITY_INVERTED, 0x0F);
```

5.20.53 tmr_external_clock_mode2_config function

The table below describes the function `tmr_external_clock_mode2_config`.

Table 523. tmr_external_clock_mode2_config function

Name	Description
Function name	<code>tmr_external_clock_mode2_config</code>
Function prototype	<code>void tmr_external_clock_mode2_config(tmr_type *tmr_x, tmr_external_signal_divider_type es_divide, tmr_external_signal_polarity_type es_polarity, uint16_t es_filter);</code>
Function description	Configure TMR external clock mode 2 (corresponding to external mode B in the reference manual)
Input parameter 1	<code>tmr_x</code> : indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4
Input parameter 2	<code>es_divide</code> : external signal frequency division factor
Input parameter 3	<code>es_polarity</code> : external signal polarity
Input parameter 4	<code>es_filter</code> : external signal filter value, 0x00~0x0F
Output parameter	NA
Return value	NA
Input parameter 2	<code>es_divide</code> : external signal frequency division factor
Input parameter 3	<code>es_polarity</code> : external signal polarity

es_divide

Set TMR external signal frequency division factor, refer to [es_divide](#) for details.

es_polarity

Set TMR external signal polarity, refer to [es_polarity](#) for details.

Example:

```
tmr_external_clock_mode2_config(TMR1, TMR_ES_FREQUENCY_DIV_1, TMR_ES_POLARITY_INVERTED, 0x0F);
```

5.20.54 tmr_encoder_mode_config function

The table below describes the function tmr_encoder_mode_config.

Table 524. tmr_encoder_mode_config function

Name	Description
Function name	tmr_encoder_mode_config
Function prototype	void tmr_encoder_mode_config(tmr_type *tmr_x, tmr_encoder_mode_type encoder_mode, tmr_input_polarity_type ic1_polarity, tmr_input_polarity_type ic2_polarity);
Function description	Configure TMR encoder mode
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4
Input parameter 2	encoder_mode: encoder mode
Input parameter 3	ic1_polarity: input channel 1 polarity
Input parameter 4	ic2_polarity: input channel 2 polarity
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

encoder_mode

Select a TMR encoder mode.

TMR_ENCODER_MODE_A: Encoder mode A

TMR_ENCODER_MODE_B: Encoder mode B

TMR_ENCODER_MODE_C: Encoder mode C

ic1_polarity

Select TMR input channel 1 polarity.

TMR_INPUT_RISING_EDGE: Rising edge

TMR_INPUT_FALLING_EDGE: Falling edge

TMR_INPUT_BOTH_EDGE: Both edges (Rising edge and Falling edge)

ic2_polarity

Select TMR input channel 2 polarity.

TMR_INPUT_RISING_EDGE: Rising edge

TMR_INPUT_FALLING_EDGE: Falling edge

TMR_INPUT_BOTH_EDGE: Both edges (Rising edge and Falling edge)

Example:

```
tmr_encoder_mode_config(TMR1, TMR_ENCODER_MODE_A, TMR_INPUT_RISING_EDGE,
TMR_INPUT_RISING_EDGE);
```

5.20.55 tmr_force_output_set function

The table below describes the function tmr_force_output_set.

Table 525. tmr_force_output_set function

Name	Description
Function name	tmr_force_output_set
Function prototype	void tmr_force_output_set(tmr_type *tmr_x, tmr_channel_select_type tmr_channel, tmr_force_output_type force_output);
Function description	Set TMR forced output
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	tmr_channel: timer channel
Input parameter 3	force_output: forced output level
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_channel

Select a TMR channel.

TMR_SELECT_CHANNEL_1: Timer channel 1

TMR_SELECT_CHANNEL_2: Timer channel 2

TMR_SELECT_CHANNEL_3: Timer channel 3

TMR_SELECT_CHANNEL_4: Timer channel 4

TMR_SELECT_CHANNEL_5: Timer channel 5

force_output

Forced output level of output channels.

TMR_FORCE_OUTPUT_HIGH: CxORAW forced high

TMR_FORCE_OUTPUT_LOW: CxORAW forced low

Example:

```
tmr_force_output_set(TMR1, TMR_SELECT_CHANNEL_1, TMR_FORCE_OUTPUT_HIGH);
```

5.20.56 tmr_dma_control_config function

The table below describes the function tmr_dma_control_config.

Table 526. tmr_dma_control_config function

Name	Description
Function name	tmr_dma_control_config
Function prototype	void tmr_dma_control_config(tmr_type *tmr_x, tmr_dma_transfer_length_type dma_length, tmr_dma_address_type dma_base_address);
Function description	Configure TMR DMA control
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR2, TMR3, TMR4, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	dma_length: DMA transfer length
Input parameter 3	dma_base_address: DMA transfer offset address
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

dma_length

Set DAM transfer bytes, including:

TMR_DMA_TRANSFER_1BYTE: 1 byte
 TMR_DMA_TRANSFER_2BYTES: 2 bytes
 TMR_DMA_TRANSFER_3BYTES: 3 bytes

...

TMR_DMA_TRANSFER_17BYTES: 17 bytes
 TMR_DMA_TRANSFER_18BYTES: 18 bytes

dma_base_address

Set DMA transfer offset address, starting from TMR control register 1, including:

TMR_CTRL1_ADDRESS
 TMR_CTRL2_ADDRESS
 TMR_STCTRL_ADDRESS
 TMR_IDEN_ADDRESS
 TMR_ISTS_ADDRESS
 TMR_SWEVT_ADDRESS
 TMR_CM1_ADDRESS
 TMR_CM2_ADDRESS
 TMR_CCTRL_ADDRESS
 TMR_CVAL_ADDRESS
 TMR_DIV_ADDRESS
 TMR_PR_ADDRESS
 TMR_RPR_ADDRESS
 TMR_C1DT_ADDRESS
 TMR_C2DT_ADDRESS
 TMR_C3DT_ADDRESS
 TMR_C4DT_ADDRESS
 TMR_BRK_ADDRESS

TMR_DMACTRL_ADDRESS

Example:

```
tmr_dma_control_config(TMR1, TMR_DMA_TRANSFER_8BYTES, TMR_CTRL1_ADDRESS);
```

5.20.57 tmr_brkdt_config function

The table below describes the function tmr_brkdt_config.

Table 527. tmr_brkdt_config function

Name	Description
Function name	tmr_brkdt_config
Function prototype	void tmr_brkdt_config(tmr_type *tmr_x, tmr_brkdt_config_type *brkdt_struct);
Function description	Configure TMR break mode and dead time
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR1, TMR9, TMR10, TMR11, TMR13, TMR14
Input parameter 2	brkdt_struct: tmr_brkdt_config_type pointer
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

tmr_brkdt_config_type structure

The tmr_brkdt_config_type is defined in the at32f402_405_tmr.h:

typedef struct

```
{
    uint8_t          brk_filter_value;;
    uint8_t          deadtime;
    tmr_brk_polarity_type brk_polarity;
    tmr_wp_level_type wp_level;
    confirm_state     auto_output_enable;
    confirm_state     fcsoen_state;
    confirm_state     fcsodis_state;
    confirm_state     brk_enable;
} tmr_brkdt_config_type;
```

brk_filter_value

Set break input filter value, ranging from 0x00 to 0x0F

deadtime

Set dead time, between 0x00~0xFF

brk_polarity

Select break input polarity

TMR_BRK_INPUT_ACTIVE_LOW: Active low

TMR_BRK_INPUT_ACTIVE_HIGH: Active high

wp_level

Set write protection level.

TMR_WP_OFF: Write protection OFF

TMR_WP_LEVEL_3:

Level 3 write protection, protecting the bits below:

- TMRx_BRK: DTC, BRKEN, BRKV and AOEN
- TMRx_CTRL2: CxIOS and CxCIOS

TMR_WP_LEVEL_2:

Level 2 write protection, protecting the bits below in addition to level-3 protected bits:

- TMRx_CCTRL: CxP and CxCP
- TMRx_BRK: FCSODIS and FCSOEN

TMR_WP_LEVEL_1:

Level 1 write protection, protecting the bits below in addition to level-2 protected bits:

- TMRx_CMx: CxOCTRL and CxOBEN

auto_output_enable

Enable auto output, Enable (TRUE) or disable (FALSE)

fcsoen_state

Indicates the frozen status when main output is ON. It is used to configure the status of complementary output channels when timer is OFF and output is enabled (OEN=1).

FALSE: Disable CxOUT/CxCOUT output

TRUE: Enable CxOUT/CxCOUT output, inactive level

fcsodis_state

Indicates the frozen status when main output is OFF. It is used to configure the status of complementary output channels when timer is OFF and output is disabled (OEN=0).

FALSE: Disable CxOUT/CxCOUT output

TRUE: Enable CxOUT/CxCOUT output, idle level

brk_enable

Enable break feature, Enable (TRUE) or disable (FALSE).

Example

```
tmr_brkdt_config_type tmr_brkdt_config_struct;
tmr_brkdt_config_struct.brk_enable = TRUE;
tmr_brkdt_config_struct.auto_output_enable = TRUE;
tmr_brkdt_config_struct.deadtime = 0;
tmr_brkdt_config_struct.fcsodis_state = TRUE;
tmr_brkdt_config_struct.fcsoen_state = TRUE;
tmr_brkdt_config_struct.brk_polarity = TMR_BRK_INPUT_ACTIVE_HIGH;
tmr_brkdt_config_struct.wp_level = TMR_WP_OFF;
tmr_brkdt_config_struct.brk_filter_value = 0;
tmr_brkdt_config(TMR1, &tmr_brkdt_config_struct);
```

5.20.58 tmr_iremap_config function

The table below describes the function tmr_iremap_config.

Table 528. tmr_iremap_config function

Name	Description
Function name	tmr_iremap_config
Function prototype	void tmr_iremap_config(tmr_type *tmr_x, tmr_input_remap_type input_remap);
Function description	Set TMR internal remapping
Input parameter 1	tmr_x: indicates the selected TMR peripheral, it can be TMR2, TMR14
Input parameter 2	input_remap: TMR input channel remap to be configured
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

input_remap

Set TMR2 internal trigger 1 remapping and TMR5 channel 4 input remapping

TMR14_GPIO: TMR14 is connected to GPIO

TMR14_ERTCCLK: TMR14 is connected to internal ERTC clock

TMR14_HEXT_DIV32: TMR14 is connected to HEXT/32

TMR14_CLKOUT: TMR14 is connected to CLKOUT

TMR2_OTG_FS_SOF: TMR2 internal selection 3 is connected to OTG_FS_SOF

TMR2_OTG_HS_SOF: TMR2 internal selection 3 is connected to OTG_HS_SOF

Example

```
tmr_iremap_config(TMR14, TMR14_GPIO);
```


5.21 Universal synchronous/asynchronous receiver/transmitter (USART)

The USART register structure `usart_type` is defined in the “at32f402_405_usart.h”:

```
/**
 * @brief type define usart register all
 */
typedef struct
{
    ...
} usart_type;
```

The table below gives a list of the USART registers

Table 529. Summary of USART registers

Register	Description
sts	Status register
dt	Data register
baudr	Baud rate register
ctrl1	Control register 1
ctrl2	Control register 2
ctrl3	Control register 3
gdiv	Guard time and divider Control register 1

The table below gives a list of USART library functions.

Table 530. Summary of USART library functions

Function name	Description
<code>usart_reset</code>	Reset USART peripheral registers
<code>usart_init</code>	Set baud rate, data bits and stop bits.
<code>usart_parity_selection_config</code>	Parity selection
<code>usart_enable</code>	Enable USART peripherals
<code>usart_transmitter_enable</code>	Enable USART transmitter
<code>usart_receiver_enable</code>	Enable USART receiver
<code>usart_clock_config</code>	Set clock polarity and phases for synchronization
<code>usart_clock_enable</code>	Set clock output for synchronization
<code>usart_interrupt_enable</code>	Enable interrupts
<code>usart_dma_transmitter_enable</code>	Enable DMA transmitter
<code>usart_dma_receiver_enable</code>	Enable DMA receiver
<code>usart_wakeup_id_set</code>	Set wakeup ID
<code>usart_wakeup_mode_set</code>	Set wakeup mode
<code>usart_receiver_mute_enable</code>	Enable receiver mute mode
<code>usart_break_bit_num_set</code>	Set break frame length
<code>usart_lin_mode_enable</code>	Enable LIN mode
<code>usart_data_transmit</code>	Data transmit

usart_data_receive	Data receive
usart_break_send	Send break frame
usart_smartcard_guard_time_set	Set smartcard guard time
usart_irda_smartcard_division_set	Set infrared and smartcard division
usart_smartcard_mode_enable	Enable smartcard mode
usart_smartcard_nack_set	Enable smartcard NACK
usart_single_line_halfduplex_select	Enable single-wire half-duplex mode
usart_irda_mode_enable	Enable infrared mode
usart_irda_low_power_enable	Enable infrared low-power mode
usart_hardware_flow_control_set	Enable hardware flow control
usart_flag_get	Get flag
usart_flag_clear	Clear flag
usart_rs485_delay_time_config	Set latency for starting or ending consecutive data transmission in RS 485
usart_transmit_receive_pin_swap	Swap transmit/receive pins
usart_id_bit_num_set	Set ID bit count
usart_de_polarity_set	DE signal polarity selection
usart_rs485_mode_enable	RS485 mode enable
usart_low_power_wakeup_set	Set low-power wakeup
usart_deep_sleep_mode_enable	Deepsleep mode enable
usart_msb_transmit_first_enable	Enable MSB-first transmission
usart_dt_polarity_reverse	Enable data polarity reverse mode
usart_transmit_pin_polarity_reverse	Enable data transmit pin polarity reverse mode
usart_receive_pin_polarity_reverse	Enable data receive pin polarity reverse mode
usart_receiver_timeout_detection_enable	Enable data receive timeout detection
usart_receiver_timeout_value_set	Set data receive timeout value

5.21.1 usart_reset function

The table below describes the function usart_reset.

Table 531. usart_reset function

Name	Description
Function name	usart_reset
Function prototype	void usart_reset(usart_type* usart_x);
Function description	Reset USART peripheral registers
Input parameter 1	usart_x: indicates the selected peripherals, it can be USART1, USART2, and USART3...
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	crm_periph_reset

Example:

```
/* reset usart1 */
usart_reset(USART1);
```

5.21.2 usart_init function

The table below describes the function usart_init.

Table 532. usart_init function

Name	Description
Function name	usart_init
Function prototype	void usart_init(usart_type* usart_x, uint32_t baud_rate, usart_data_bit_num_type data_bit, usart_stop_bit_num_type stop_bit);
Function description	Set baud rate, data bits and stop bits.
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	baud_rate: baud rate for serial interfaces
Input parameter 3	data_bit: data bit width for serial interfaces
Input parameter 4	stop_bit: stop bit width for serial interfaces
Output parameter	NA
Return value	NA
Required preconditions	This operation can be allowed only when external low-speed clock is disabled.
Called functions	NA

data_bit

Select data bit size for serial interface communication.

USART_DATA_8BITS: 8-bit

USART_DATA_9BITS: 9-bit

stop_bit

Select stop bit size for serial interface communication.

USART_STOP_1_BIT: 1 bit

USART_STOP_0_5_BIT: 0.5 bit

USART_STOP_2_BIT: 2 bit

USART_STOP_1_5_BIT: 1.5 bit

Example:

```
/* configure uart param */
usart_init(USART1, 115200, USART_DATA_8BITS, USART_STOP_1_BIT);
```

5.21.3 usart_parity_selection_config function

The table below describes the function usart_parity_selection_config.

Table 533. usart_parity_selection_config function

Name	Description
Function name	usart_parity_selection_config
Function prototype	void usart_parity_selection_config(usart_type* usart_x, usart_parity_selection_type parity);
Function description	Parity selection
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	Parity: parity mode for serial interface communication
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

parity

Select parity mode for serial interface communication.

USART_PARITY_NONE: No parity

USART_PARITY_EVEN: Even

USART_PARITY_ODD: Odd

Example:

```
/* config usart even parity */
usart_parity_selection_config(USART1, USART_PARITY_EVEN);
```

5.21.4 usart_enable function

The table below describes the function usart_enable.

Table 534. usart_enable function

Name	Description
Function name	usart_enable
Function prototype	void usart_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	new_state: Enable (TRUE) and disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 */
usart_enable(USART1, TRUE);
```

5.21.5 usart_transmitter_enable function

The table below describes the function usart_transmitter_enable.

Table 535. usart_transmitter_enable function

Name	Description
Function name	usart_transmitter_enable
Function prototype	void usart_transmitter_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART transmitter
Input parameter 1	usart_x: indicates the selected peripheral it can be USART1, USART2, and USART3...
Input parameter 2	new_state: Enable (TRUE) and disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 transmitter */  
usart_transmitter_enable(USART1, TRUE);
```

5.21.6 usart_receiver_enable function

The table below describes the function usart_receiver_enable.

Table 536. usart_receiver_enable function

Name	Description
Function name	usart_receiver_enable
Function prototype	void usart_receiver_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART receiver
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	new_state: Enable (TRUE) and disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 receiver */  
usart_receiver_enable(USART1, TRUE);
```

5.21.7 usart_clock_config function

The table below describes the function usart_clock_config.

Table 537. usart_clock_config function

Name	Description
Function name	usart_clock_config
Function prototype	void usart_clock_config(usart_type* usart_x, usart_clock_polarity_type clk_pol, usart_clock_phase_type clk pha, usart_lbcg_type clk_lb);
Function description	Configure clock polarity and phase for synchronization feature
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	clk_pol: clock polarity for synchronization
Input parameter 3	clk pha: clock phase for synchronization
Input parameter 4	clk_lb: selects whether to output clock on the last bit (upper bit) of data sent through synchronization feature
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

clk_pol

Clock polarity selection.

USART_CLOCK_POLARITY_LOW: Low

USART_CLOCK_POLARITY_HIGH: High

clk pha

Clock phase selection.

USART_CLOCK_PHASE_1EDGE: 1st edge

USART_CLOCK_PHASE_2EDGE: 2nd edge

clk_lb

Select whether to output clock on the last bit of data.

USART_CLOCK_LAST_BIT_NONE: No clock output

USART_CLOCK_LAST_BIT_OUTPUT: Clock output

Example:

```
/* config synchronous mode */
usart_clock_config(USART1, USART_CLOCK_POLARITY_HIGH, USART_CLOCK_PHASE_2EDGE,
USART_CLOCK_LAST_BIT_OUTPUT);
```

5.21.8 usart_clock_enable function

The table below describes the function usart_clock_enable.

Table 538. usart_clock_enable function

Name	Description
Function name	usart_clock_enable
Function prototype	void usart_clock_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable clock output
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable clock */
usart_clock_enable(USART1, TRUE);
```

5.21.9 usart_interrupt_enable function

The table below describes the function usart_interrupt_enable.

Table 539. usart_interrupt_enable function

Name	Description
Function name	usart_interrupt_enable
Function prototype	void usart_interrupt_enable(usart_type* usart_x, uint32_t usart_int, confirm_state new_state);
Function description	Enable interrupts
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	usart_int: interrupt type
Input parameter 3	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

usart_int

Defines a peripheral interrupt.

USART_IDLE_INT: Bus idle
 USART_RDBF_INT: Receive data buffer full
 USART_TDC_INT: Transmit data complete
 USART_TDBE_INT: Transmit data buffer empty
 USART_PERR_INT: Parity error
 USART_BF_INT: Break frame receive

USART_ERR_INT: Error interrupt
 USART_CTSCF_INT: CTS (Clear To Send) change
 USART_CMD_INT: Byte match detection interrupt
 USART_RTOD_INT: Receiver timeout detection interrupt

Example:

```
/* enable usart1 transmit complete interrupt */
usart_interrupt_enable (USART1, USART_TDC_INT, TRUE);
```

5.21.10 usart_dma_transmitter_enable function

The table below describes the function usart_dma_transmitter_enable.

Table 540. usart_dma_transmitter_enable function

Name	Description
Function name	usart_dma_transmitter_enable
Function prototype	void usart_dma_transmitter_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable DMA transmitter
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable dma transmitter */
usart_dma_transmitter_enable (USART1, TRUE);
```

5.21.11 usart_dma_receiver_enable function

The table below describes the function usart_dma_receiver_enable.

Table 541. usart_dma_receiver_enable function

Name	Description
Function name	usart_dma_receiver_enable
Function prototype	void usart_dma_receiver_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable DMA receiver
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable dma receiver */  
usart_dma_receiver_enable (USART1, TRUE);
```

5.21.12 usart_wakeup_id_set function

The table below describes the function usart_wakeup_id_set.

Table 542. usart_wakeup_id_set function

Name	Description
Function name	usart_wakeup_id_set
Function prototype	void usart_wakeup_id_set(usart_type* usart_x, uint8_t usart_id);
Function description	Set wakeup ID
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3...
Input parameter 2	usart_id: wakeup ID
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config wakeup id */  
usart_wakeup_id_set (USART1, 0x88);
```

5.21.13 usart_wakeup_mode_set function

The table below describes the function usart_wakeup_mode_set.

Table 543. usart_wakeup_mode_set function

Name	Description
Function name	usart_wakeup_mode_set
Function prototype	void usart_wakeup_mode_set(usart_type* usart_x, usart_wakeup_mode_type wakeup_mode);
Function description	Set wakeup mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	wakeup_mode: wakeup mode
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

wakeup_mode

Set wakeup mode to wake up from silent state.

USART_WAKEUP_BY_IDLE_FRAME: Woke up by idle frame

USART_WAKEUP_BY_MATCHING_ID: Woke up by ID matching

Example:

```
/* config usart1 wakeup mode */  
usart_wakeup_mode_set (USART1, USART_WAKEUP_BY_MATCHING_ID);
```

5.21.14 usart_receiver_mute_enable function

The table below describes the function usart_receiver_mute_enable.

Table 544. usart_receiver_mute_enable function

Name	Description
Function name	usart_receiver_mute_enable
Function prototype	void usart_receiver_mute_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable USART receiver mute mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config receiver mute */  
usart_receiver_mute_enable (USART1, TRUE);
```

5.21.15 usart_break_bit_num_set function

The table below describes the function usart_break_bit_num_set.

Table 545. usart_break_bit_num_set function

Name	Description
Function name	usart_break_bit_num_set
Function prototype	void usart_break_bit_num_set(usart_type* usart_x, usart_break_bit_num_type break_bit);
Function description	Set USART break frame length
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	break_bit: break frame length type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

break_bit

Set break frame length.

USART_BREAK_10BITS: 10 bits

USART_BREAK_11BITS: 11 bits

Example:

```
/* config break frame length 10bits */
```

```
usart_break_bit_num_set (USART1, USART_BREAK_10BITS);
```

5.21.16 usart_lin_mode_enable function

The table below describes the function usart_lin_mode_enable.

Table 546. usart_lin_mode_enable function

Name	Description
Function name	usart_lin_mode_enable
Function prototype	void usart_lin_mode_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable LIN mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	new_state: Enable (TRUE) or disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable usart1 lin mode */  
usart_lin_mode_enable (USART1, TRUE);
```

5.21.17 usart_data_transmit function

The table below describes the function usart_data_transmit.

Table 547. usart_data_transmit function

Name	Description
Function name	usart_data_transmit
Function prototype	void usart_data_transmit(usart_type* usart_x, uint16_t data);
Function description	Transmit data
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	Data: data to send
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* transmit data */  
uint16_t data = 0x88;  
usart_data_transmit (USART1, data);
```

5.21.18 usart_data_receive function

The table below describes the function usart_data_receive.

Table 548. usart_data_receive function

Name	Description
Function name	usart_data_receive
Function prototype	uint16_t usart_data_receive(usart_type* usart_x);
Function description	Receives data
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	NA
Output parameter	NA
Return value	uint16_t: return the received data
Required preconditions	NA
Called functions	NA

Example:

```
/* receive data */  
uint16_t data = 0;  
data = usart_data_receive (USART1);
```

5.21.19 usart_break_send function

The table below describes the function usart_break_send.

Table 549. usart_break_send function

Name	Description
Function name	usart_break_send
Function prototype	void usart_break_send(usart_type* usart_x);
Function description	Sends break frame
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* send break frame */  
usart_break_send (USART1);
```

5.21.20 usart_smartcard_guard_time_set function

The table below describes the function usart_smartcard_guard_time_set.

Table 550. usart_smartcard_guard_time_set function

Name	Description
Function name	usart_smartcard_guard_time_set
Function prototype	void usart_smartcard_guard_time_set(usart_type* usart_x, uint8_t guard_time_val);
Function description	Set smartcard guard time
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2, and USART3
Input parameter 2	guard_time_val: guard time, 0x00~0xFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* usart guard time set to 2 bit */
usart_smartcard_guard_time_set(USART1, 0x2);
```

5.21.21 usart_irda_smartcard_division_set function

The table below describes the function usart_irda_smartcard_division_set.

Table 551. usart_irda_smartcard_division_set function

Name	Description
Function name	usart_irda_smartcard_division_set
Function prototype	void usart_irda_smartcard_division_set(usart_type* usart_x, uint8_t div_val);
Function description	Infrared and smartcard frequency division settings
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	div_val: division value
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* usart clock set to (apbclk / (2 * 20)) */
usart_irda_smartcard_division_set(USART1, 20);
```

5.21.22 usart_smartcard_mode_enable function

The table below describes the function usart_smartcard_mode_enable.

Table 552. usart_smartcard_mode_enable function

Name	Description
Function name	usart_smartcard_mode_enable
Function prototype	void usart_smartcard_mode_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable smartcode mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the smartcard mode */  
usart_smartcard_mode_enable(USART1, TRUE);
```

5.21.23 usart_smartcard_nack_set function

The table below describes the function usart_smartcard_nack_set.

Table 553. usart_smartcard_nack_set function

Name	Description
Function name	usart_smartcard_nack_set
Function prototype	void usart_smartcard_nack_set(usart_type* usart_x, confirm_state new_state);
Function description	Enable smartcard NACK
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable the nack transmission */  
usart_smartcard_nack_set(USART1, TRUE);
```

5.21.24 usart_single_line_halfduplex_select function

The table below describes the function usart_single_line_halfduplex_select.

Table 554. usart_single_line_halfduplex_select function

Name	Description
Function name	usart_single_line_halfduplex_select
Function prototype	void usart_single_line_halfduplex_select(usart_type* usart_x, confirm_state new_state);
Function description	Enable single-wire half-duplex mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable halfduplex */  
usart_single_line_halfduplex_select(USART1, TRUE);
```

5.21.25 usart_irda_mode_enable function

The table below describes the function usart_irda_mode_enable.

Table 555. usart_irda_mode_enable function

Name	Description
Function name	usart_irda_mode_enable
Function prototype	void usart_irda_mode_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable infrared mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable irda mode */  
usart_irda_mode_enable(USART1, TRUE);
```

5.21.26 usart_irda_low_power_enable function

The table below describes the function usart_irda_low_power_enable.

Table 556. usart_irda_low_power_enable function

Name	Description
Function name	usart_irda_low_power_enable
Function prototype	void usart_irda_low_power_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable infrared low-power mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), Disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable irda lowpower mode */
usart_irda_low_power_enable (USART1, TRUE);
```

5.21.27 usart_hardware_flow_control_set function

The table below describes the function usart_hardware_flow_control_set.

Table 557. usart_hardware_flow_control_set function

Name	Description
Function name	usart_hardware_flow_control_set
Function prototype	void usart_hardware_flow_control_set(usart_type* usart_x, usart_hardware_flow_control_type flow_state);
Function description	Set peripheral hardware flow control
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	flow_state: flow control type
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flow_state

USART_HARDWARE_FLOW_NONE: No hardware flow control

USART_HARDWARE_FLOW_RTS: RTS

USART_HARDWARE_FLOW_CTS: CTS

USART_HARDWARE_FLOW_RTS_CTS: RTS and CTS

Example:

```
/* hardware flow set none */
usart_hardware_flow_control_set (USART1, USART_HARDWARE_FLOW_NONE);
```


5.21.28 usart_flag_get function

The table below describes the function usart_flag_get.

Table 558. usart_flag_get function

Name	Description
Function name	usart_flag_get
Function prototype	flag_status usart_flag_get(usart_type* usart_x, uint32_t flag);
Function description	Get flag status
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	Flag: flag selection
Output parameter	NA
Return value	flag_status: SET or RESET
Required preconditions	NA
Called functions	NA

flag

USART_CMDF_FLAG:	Byte match detection flag
USART_RTODF_FLAG:	Receiver timeout detection flag
USART_CTSCF_FLAG:	CTS (Clear To Send) change flag
USART_BFF_FLAG:	Break frame receive flag
USART_TDBE_FLAG:	Transmit buffer empty flag
USART_TDC_FLAG:	Transmit complete flag
USART_RDBF_FLAG:	Receive data buffer full flag
USART_IDLEF_FLAG:	Idle frame flag
USART_ROERR_FLAG:	Receive overflow flag
USART_NERR_FLAG:	Noise error flag
USART_FERR_FLAG:	Frame error flag
USART_PERR_FLAG:	Parity error flag

Example:

```
/* wait data transmit complete flag */  
while(usart_flag_get (USART1, USART_TDC_FLAG) == RESET);
```

5.21.29 usart_interrupt_flag_get function

The table below describes the function `usart_interrupt_flag_get`.

Table 559. usart_interrupt_flag_get function

Name	Description
Function name	<code>usart_interrupt_flag_get</code>
Function prototype	<code>flag_status usart_interrupt_flag_get(usart_type* usart_x, uint32_t flag);</code>
Function description	Check whether the selected flag is set or not
Input parameter 1	<code>usart_x</code> : indicates the selected peripheral, it can be USART1, USART2...
Input parameter 2	<code>flag</code> : flag selection
Output parameter	NA
Return value	<code>flag_status</code> : flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

flag

USART_RTODF_FLAG:	Receiver timeout detection flag
USART_CMDF_FLAG:	Byte match detection flag
USART_CTSCF_FLAG:	CTS (Clear To Send) change flag
USART_BFF_FLAG:	Break frame receive flag
USART_TDBE_FLAG:	Transmit buffer empty flag
USART_TDC_FLAG:	Transmit complete flag
USART_RDBF_FLAG:	Receive data buffer full flag
USART_IDLEF_FLAG:	Idle frame flag
USART_ROERR_FLAG:	Receive overflow flag
USART_NERR_FLAG:	Noise error flag
USART_FERR_FLAG:	Frame error flag
USART_PERR_FLAG:	Parity error flag

Example

```
/* check received data flag */  
if(usart_interrupt_flag_get(USART1, USART_RDBF_FLAG) != RESET)  
{  
}
```

5.21.30 usart_flag_clear function

The table below describes the function usart_flag_clear.

Table 560. usart_flag_clear function

Name	Description
Function name	usart_flag_clear
Function prototype	void usart_flag_clear(usart_type* usart_x, uint32_t flag);
Function description	Clear flag
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	Flag: clear the selected flag
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

flag

USART_CMDF_FLAG: Byte match detection interrupt

USART_RTODF_FLAG: Receiver timeout detection interrupt

USART_CTSCF_FLAG: CTS (Clear To Send) change flag

USART_BFF_FLAG: Break frame receive flag

USART_TDC_FLAG: Transmit complete flag

USART_RDBF_FLAG: Receive data buffer full flag

Example:

```
/* clear data transmit complete flag */  
usart_flag_clear (USART1, USART_TDC_FLAG );
```

5.21.31 usart_rs485_delay_time_config function

The table below describes the function usart_rs485_delay_time_config.

Table 561. usart_flag_clear function

Name	Description
Function name	usart_rs485_delay_time_config
Function prototype	void usart_rs485_delay_time_config(usart_type* usart_x, uint8_t start_delay_time, uint8_t complete_delay_time);
Function description	Delay time for setting and clearing data valid signal for continuous data transmission in RS485 mode
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	start_delay_time: After the first data is written in continuous transmit mode, data valid signal is set and "start_delay_time" is inserted before sending data. The time unit is 1/16 baud rate period.
Input parameter 3	complete_delay_time: After the last data is sent in continuous transmit mode, the "complete_delay_time" is inserted before clearing data valid signal. The time unit is 1/16 baud rate period.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config rs485 delay time */
usart_rs485_delay_time_config(USART1, 2, 2);
```

5.21.32 usart_transmit_receive_pin_swap function

The table below describes the function usart_transmit_receive_pin_swap.

Table 562. usart_transmit_receive_pin_swap function

Name	Description
Function name	usart_transmit_receive_pin_swap
Function prototype	void usart_transmit_receive_pin_swap(usart_type* usart_x, confirm_state new_state);
Function description	Swap transmit/receive pins
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: enabled (TRUE) or disabled (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable tx/rx swap */
usart_transmit_receive_pin_swap (USART1, TRUE);
```

5.21.33 usart_id_bit_num_set function

The table below describes the function usart_id_bit_num_set.

Table 563. usart_transmit_receive_pin_swap function

Name	Description
Function name	usart_id_bit_num_set
Function prototype	void usart_id_bit_num_set(usart_type* usart_x, usart_identification_bit_num_type id_bit_num);
Function description	Set ID bit number
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	id_bit_num: ID bit count
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

id_bit_num

USART_ID_FIXED_4_BIT: 4-bit ID

USART_ID_RELATED_DATA_BIT: Current data bit -1

Example:

```
/* config ID bit width */
usart_id_bit_num_set (USART1, USART_ID_FIXED_4_BIT);
```

5.21.34 usart_de_polarity_reverse function

The table below describes the function usart_de_polarity_reverse.

Table 564. usart_de_polarity_reverse function

Name	Description
Function name	usart_de_polarity_reverse
Function prototype	void usart_de_polarity_reverse(usart_type* usart_x, confirm_state new_state);
Function description	DE signal polarity reverse
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: Enable (TRUE), disable (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* config DE polarity */
usart_de_polarity_reverse (USART1, ENABLE);
```

5.21.35 usart_rs485_mode_enable function

The table below describes the function usart_rs485_mode_enable.

Table 565. usart_rs485_mode_enable function

Name	Description
Function name	usart_rs485_mode_enable
Function prototype	void usart_rs485_mode_enable(usart_type* usart_x, confirm_state new_state);
Function description	RS485 mode enable
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: enabled (TRUE) or disabled (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable rs485 mode */
usart_rs485_mode_enable (USART1, TRUE);
```

5.21.36 usart_msb_transmit_first_enable function

The table below describes the function usart_msb_transmit_first_enable

Table 566. usart_msb_transmit_first_enable function

Name	Description
Function name	usart_msb_transmit_first_enable
Function prototype	void usart_msb_transmit_first_enable(usart_type* usart_x, confirm_state new_state);
Function description	Enable MSB-first transmit
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: enabled (TRUE) or disabled (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable msb first transmission */
usart_msb_transmit_first_enable (USART1, ENABLE);
```

5.21.37 usart_dt_polarity_reverse function

The table below describes the function usart_dt_polarity_reverse

Table 567. usart_dt_polarity_reverse function

Name	Description
Function name	usart_dt_polarity_reverse
Function prototype	void usart_dt_polarity_reverse(usart_type* usart_x, confirm_state new_state);
Function description	Reverse data polarity
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: enabled (TRUE) or disabled (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reverse data polarity */
usart_dt_polarity_reverse (USART1, ENABLE);
```

5.21.38 usart_transmit_pin_polarity_reverse function

The table below describes the function usart_dt_polarity_reverse

Table 568. usart_dt_polarity_reverse function

Name	Description
Function name	usart_transmit_pin_polarity_reverse
Function prototype	void usart_transmit_pin_polarity_reverse(usart_type* usart_x, confirm_state new_state);
Function description	Reverse data transmit pin polarity
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: enabled (TRUE) or disabled (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reverse transmit pin polarity */
usart_transmit_pin_polarity_reverse (USART1, ENABLE);
```

5.21.39 usart_receive_pin_polarity_reverse function

The table below describes the function usart_receive_pin_polarity_reverse

Table 569. usart_receive_pin_polarity_reverse function

Name	Description
Function name	usart_receive_pin_polarity_reverse
Function prototype	usart_receive_pin_polarity_reverse (usart_type* usart_x, confirm_state new_state);
Function description	Reverse data receive pin polarity
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: enabled (TRUE) or disabled (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* reverse receive pin polarity */
usart_receive_pin_polarity_reverse (USART1, ENABLE);
```

5.21.40 usart_receiver_timeout_detection_enable function

The table below describes the function usart_receiver_timeout_detection_enable

Table 570. usart_receiver_timeout_detection_enable function

Name	Description
Function name	usart_receiver_timeout_detection_enable
Function prototype	usart_receiver_timeout_detection_enable (usart_type* usart_x, confirm_state new_state);
Function description	Enable data receive timeout detection
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	new_state: enabled (TRUE) or disabled (FALSE)
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* enable receive timeout detection */
usart_receiver_timeout_detection_enable (USART1, ENABLE);
```


5.21.41 usart_receiver_timeout_value_set function

The table below describes the function usart_receiver_timeout_value_set

Table 571. usart_receiver_timeout_value_set function

Name	Description
Function name	usart_receiver_timeout_value_set
Function prototype	void usart_receiver_timeout_value_set(usart_type* usart_x, uint32_t time);
Function description	Set data receive timeout value
Input parameter 1	usart_x: indicates the selected peripheral, it can be USART1, USART2 and USART3
Input parameter 2	Time: Set data receive timeout threshold
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
/* configure receive timeout value */
usart_receiver_timeout_value_set (USART1, 32);
```

5.22 Watchdog timer (WDT)

The WDT register structure wdt_type is defined in the “at32f402_405_wdt.h”:

```
/**
 * @brief type define wdt register all
 */
typedef struct
{

} wdt_type;
```

The table below gives a list of the WDT registers.

Table 572. Summary of WDT registers

Register	Description
cmd	Command register
div	Divider register
rld	Reload register
sts	Status register
win	Window register

The table below gives a list of WDT library functions.

Table 573. Summary of WDT library functions

Function name	Description
wdt_enable	Enable watchdog
wdt_counter_reload	Reload counter
wdt_reload_value_set	Set reload value
wdt_divider_set	Set division value
wdt_register_write_enable	Unlock WDT_DIV and WDT_RLD register write protection
wdt_flag_get	Get flag
wdt_window_counter_set	Set window counter

5.22.1 wdt_enable function

The table below describes the function wdt_enable.

Table 574. wdt_enable function

Name	Description
Function name	wdt_enable
Function prototype	void wdt_enable(void);
Function description	Enable watchdog
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_enable();
```

5.22.2 wdt_counter_reload function

The table below describes the function wdt_counter_reload.

Table 575. wdt_counter_reload function

Name	Description
Function name	wdt_counter_reload
Function prototype	void wdt_counter_reload(void);
Function description	Reload counter
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_counter_reload();
```

5.22.3 wdt_reload_value_set function

The table below describes the function wdt_reload_value_set.

Table 576. wdt_reload_value_set function

Name	Description
Function name	wdt_reload_value_set
Function prototype	void wdt_reload_value_set(uint16_t reload_value);
Function description	Set reload value
Input parameter	reload_value: reload value, 0x000~0xFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_reload_value_set(0xFF);
```

5.22.4 wdt_divider_set function

The table below describes the function wdt_divider_set.

Table 577. wdt_divider_set function

Name	Description
Function name	wdt_divider_set
Function prototype	void wdt_divider_set(wdt_division_type division);
Function description	Set division value
Input parameter	Division: watchdog division value Refer to the “division” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

division

Select watchdog division value.

WDT_CLK_DIV_4: Divided by 4

WDT_CLK_DIV_8: Divided by 8

WDT_CLK_DIV_16: Divided by 16

WDT_CLK_DIV_32: Divided by 32

WDT_CLK_DIV_64: Divided by 64

WDT_CLK_DIV_128: Divided by 128

WDT_CLK_DIV_256: Divided by 256

Example:

```
wdt_divider_set(WDT_CLK_DIV_4);
```

5.22.5 wdt_register_write_enable function

The table below describes the function wdt_register_write_enable.

Table 578. wdt_register_write_enable function

Name	Description
Function name	wdt_register_write_enable
Function prototype	void wdt_register_write_enable(confirm_state new_state);
Function description	Unlock WDT_DIV and WDT_RLD write protection
Input parameter	new_state: unlock register write protection This parameter can be TRUE or FALSE.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_register_write_enable(TRUE);
```

5.22.6 wdt_flag_get function

The table below describes the function wdt_flag_get.

Table 579. wdt_flag_get function

Name	Description
Function name	wdt_flag_get
Function prototype	flag_status wdt_flag_get(uint16_t wdt_flag);
Function description	Get flag
Input parameter	Flag: flag selection Refer to the “flag” description below for details.
Output parameter	NA
Return value	flag_status: flag status This parameter can be SET or RESET.
Required preconditions	NA
Called functions	NA

flag

This is used for flag selection, including:

WDT_DIVF_UPDATE_FLAG: Division value update complete

WDT_RLDF_UPDATE_FLAG: Reload value update complete

WDT_WINF_UPDATE_FLAG: Window value update complete

Example:

```
wdt_flag_get(WDT_DIVF_UPDATE_FLAG);
```

5.22.7 wdt_window_counter_set function

The table below describes the function wdt_window_counter_set.

Table 580. wdt_window_counter_set function

Name	Description
Function name	wdt_window_counter_set
Function prototype	void wdt_window_counter_set(uint16_t window_cnt);
Function description	Set window counter
Input parameter	window_cnt: window value, from 0x000 to 0xFF
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wdt_window_counter_set(0x7FF);
```

5.23 Window watchdog timer (WWDT)

The WWDT register structure `wwdt_type` is defined in the “`at32f402_405_wwdt.h`”:

```
/**
 * @brief type define wwdt register all
 */
typedef struct
{

} wwdt_type;
```

The table below gives a list of the WWDT registers.

Table 581. Summary of WWDT registers

Register	Description
ctrl	Control register
cfg	Configuration register
sts	Status register

The table below gives a list of WWDT library functions.

Table 582. Summary of WWDT library functions

Function name	Description
<code>wwdt_reset</code>	Reset window watchdog registers
<code>wwdt_divider_set</code>	Set divider
<code>wwdt_flag_clear</code>	Clear reload counter interrupt flag
<code>wwdt_enable</code>	Enable WWDT
<code>wwdt_interrupt_enable</code>	Enable reload counter interrupt
<code>wwdt_flag_get</code>	Get flag
<code>wwdt_counter_set</code>	Set counter value
<code>wwdt_window_counter_set</code>	Set window value

5.23.1 wwdt_reset function

The table below describes the function wwdt_reset.

Table 583. wwdt_reset function

Name	Description
Function name	wwdt_reset
Function prototype	void wwdt_reset(void);
Function description	Reset window watchdog registers to their initial values.
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	void crm_periph_reset(crm_periph_reset_type value, confirm_state new_state);

Example:

```
wwdt_reset();
```

5.23.2 wwdt_divider_set function

The table below describes the function wwdt_divider_set.

Table 584. wwdt_divider_set function

Name	Description
Function name	wwdt_divider_set
Function prototype	void wwdt_divider_set(wwdt_division_type division);
Function description	Set divider
Input parameter	Division: WWDT division value Refer to the “division” description below for details.
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

division

Select WWDT division value.

WWDT_PCLK1_DIV_4096: Divided by 4096

WWDT_PCLK1_DIV_8192: Divided by 8192

WWDT_PCLK1_DIV_16384: Divided by 16384

WWDT_PCLK1_DIV_32768: Divided by 32768

Example:

```
wwdt_divider_set(WWDT_PCLK1_DIV_4096);
```


5.23.3 wwdt_enable function

The table below describes the function wwdt_enable.

Table 585. wwdt_enable function

Name	Description
Function name	wwdt_enable
Function prototype	void wwdt_enable(uint8_t wwdt_cnt);
Function description	Enable WWDT
Input parameter	wwdt_cnt: WWDT counter initial value, 0x40~0x7F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_enable(0x7F);
```

5.23.4 wwdt_interrupt_enable function

The table below 3 describes the function wwdt_interrupt_enable.

Table 586. wwdt_interrupt_enable function

Name	Description
Function name	wwdt_interrupt_enable
Function prototype	void wwdt_interrupt_enable(void);
Function description	Enable reload counter interrupt
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_interrupt_enable();
```

5.23.5 wwdt_counter_set function

The table below describes the function wwdt_counter_set.

Table 587. wwdt_counter_set function

Name	Description
Function name	wwdt_counter_set
Function prototype	void wwdt_counter_set(uint8_t wwdt_cnt);
Function description	Set counter value
Input parameter	wwdt_cnt: WWDT counter value, 0x40~0x7F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_counter_set(0x7F);
```

5.23.6 wwdt_window_counter_set function

The table below describes the function wwdt_window_counter_set.

Table 588. wwdt_window_counter_set function

Name	Description
Function name	wwdt_window_counter_set
Function prototype	void wwdt_window_counter_set(uint8_t window_cnt);
Function description	Set window counter value
Input parameter	wwdt_cnt: WWDT window value, 0x40~0x7F
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_window_counter_set(0x6F);
```

5.23.7 wwdt_flag_get function

The table below describes the function wwdt_flag_get.

Table 589. wwdt_flag_get function

Name	Description
Function name	wwdt_flag_get
Function prototype	flag_status wwdt_flag_get(void);
Function description	Get reload counter interrupt flag
Input parameter	NA
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_flag_get();
```

5.23.8 wwdt_interrupt_flag_get function

The table below describes the function wwdt_interrupt_flag_get.

Table 590. wwdt_interrupt_flag_get function

Name	Description
Function name	wwdt_interrupt_flag_get
Function prototype	flag_status wwdt_interrupt_flag_get(void);
Function description	Get reload counter interrupt flag and judge the corresponding interrupt enable bit
Input parameter 1	NA
Output parameter	NA
Return value	flag_status: flag status Return SET or RESET.
Required preconditions	NA
Called functions	NA

Example

```
wwdt_interrupt_flag_get();
```

5.23.9 wwdt_flag_clear function

The table below describes the function wwdt_flag_clear.

Table 591. wwdt_flag_clear function

Name	Description
Function name	wwdt_flag_clear
Function prototype	void wwdt_flag_clear(void);
Function description	Clear reload counter interrupt flag
Input parameter	NA
Output parameter	NA
Return value	NA
Required preconditions	NA
Called functions	NA

Example:

```
wwdt_flag_clear();
```

6 Precautions

6.1 Device model replacement

While replacing the device part number in an existing project or demo with another one, if necessary, it is necessary to check the macro definitions corresponding to the device defined in [Table 1](#) before replacement. The subsequent sections give a detailed description of how to replace a device in KEIL and IAR environments (Just taking the at32f403avgt7 as an example as other devices share similar operations).

There are two steps to get this happen:

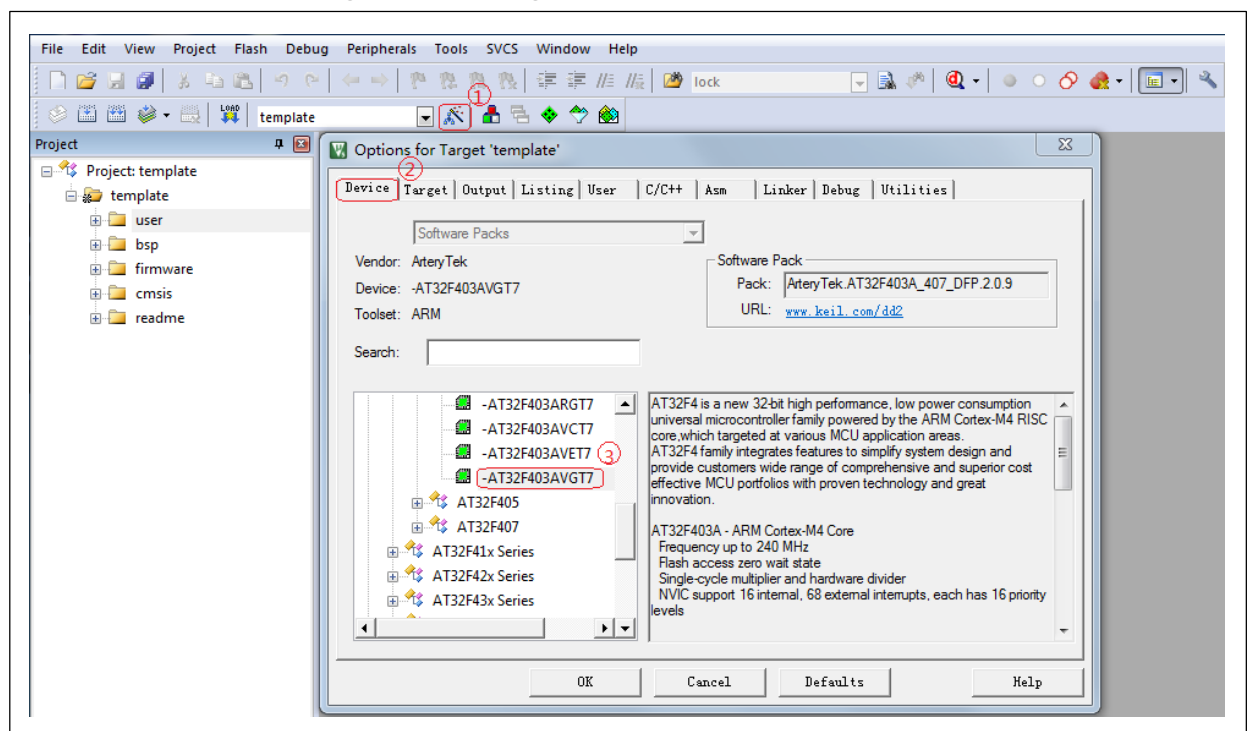
1. By changing device
2. By changing macro definition

6.1.1 KEIL environment

Follow the steps and illustration below for device replacement in Keil environment:

- ① Click on magic stick “Options for Target”
- ② Click on “Device”
- ③ Select the desired device part number

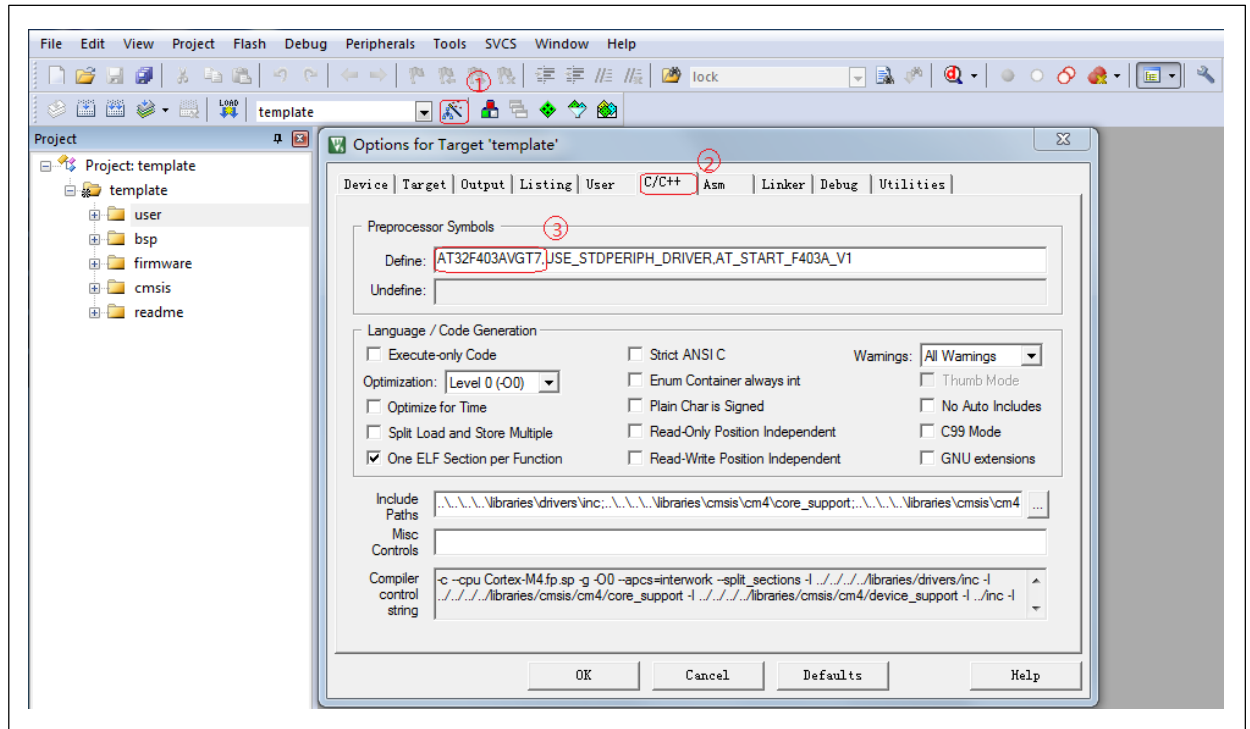
Figure 29. Change device part number in Keil



Follow the steps and illustration below to change macro definition.

- ① Click on magic stick “Options for Target”
- ② Click on “C/C++”
- ③ Delete the original macro definition in “Define” box, and write the desired one corresponding to the selected device part number based on [Table 1](#).

Figure 30. Change macro definition in Keil

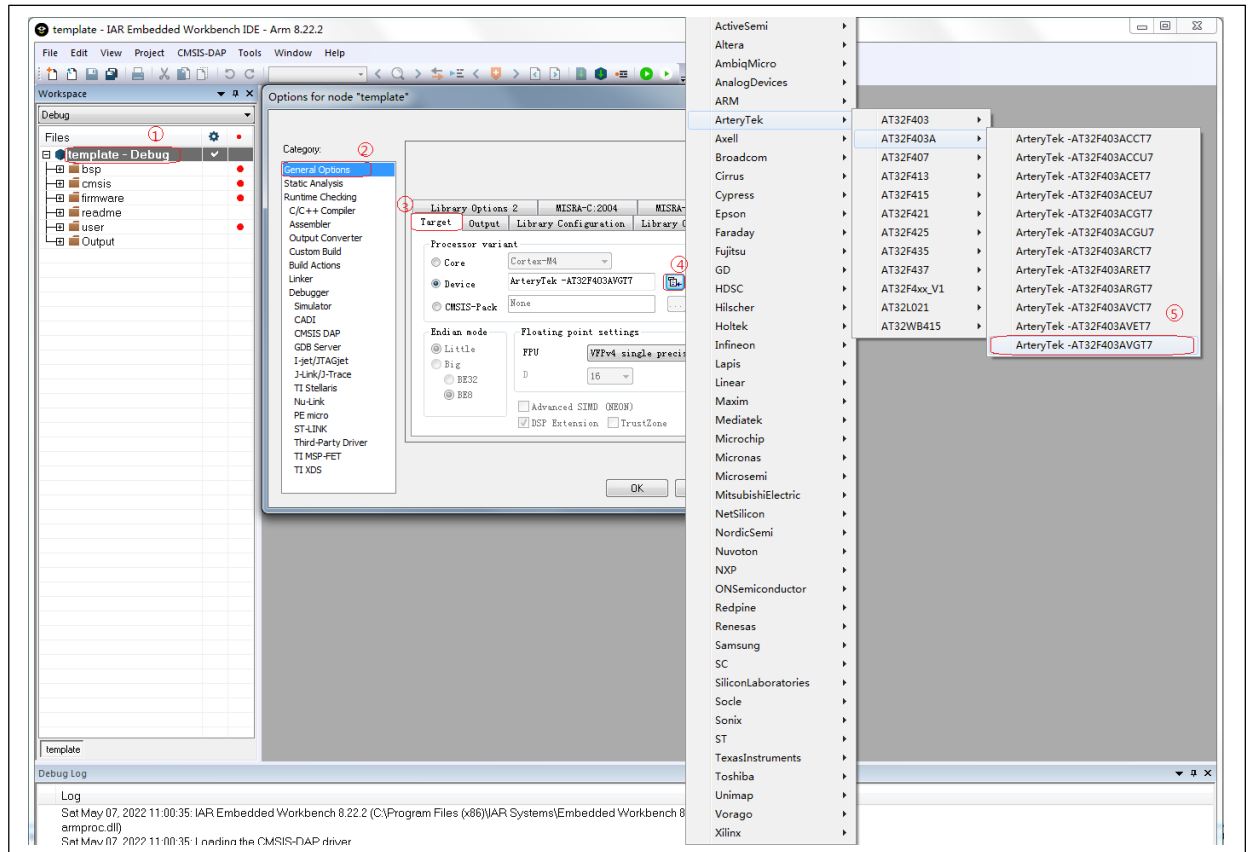


6.1.2 IAR environment

Follow the steps and illustration below for device replacement in IAR environment.

- ① Right click on the file name, and select "Options..."
- ② Select "General Options"
- ③ Select "Target"
- ④ Click on check box
- ⑤ Select the desired device part number.

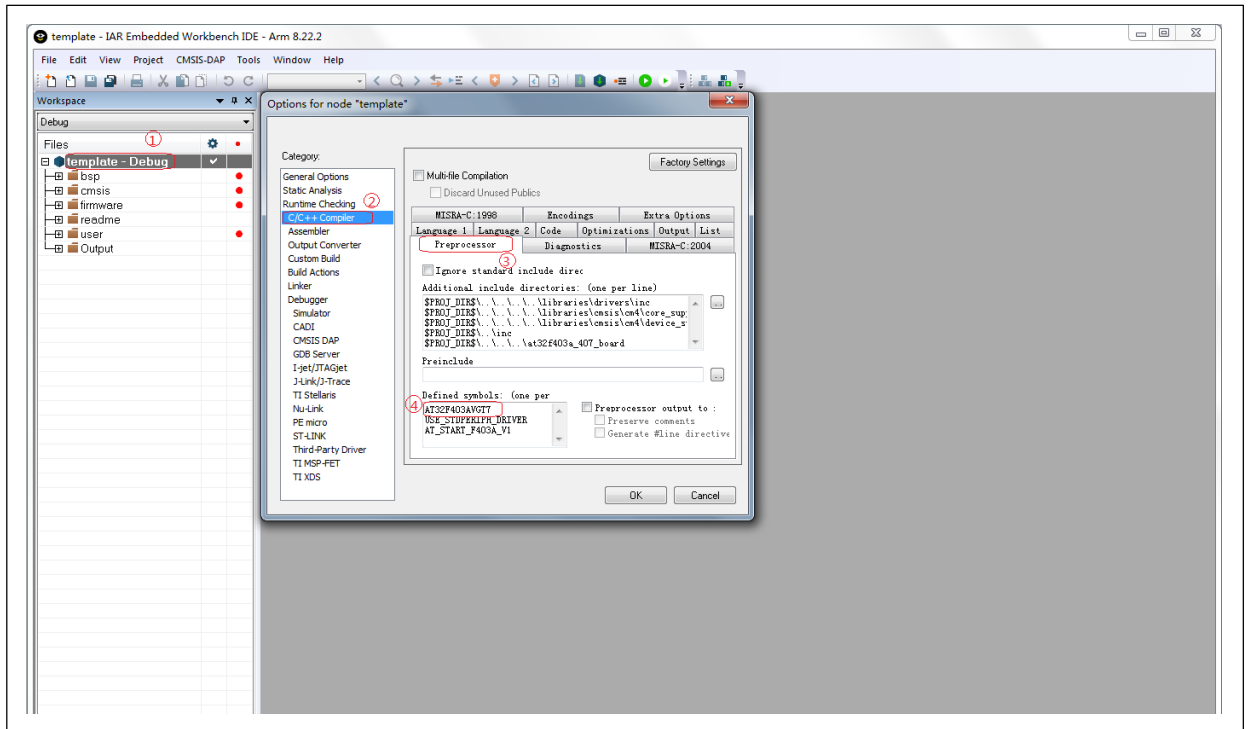
Figure 31. Change device part number in IAR



Follow the steps and illustration below to change macro definition in IAR environment.

- ① Right click on the file name, and select "Options..."
- ② Select "C/C++ Compiler"
- ③ Click on "Preprocessor"
- ④ Delete the original macro definition in "Defined symbols" column, and write the desired one corresponding to the selected device part number based on [Table 1](#).

Figure 32. Change macro definition in IAR



6.2 Unable to identify IC by JLink software in Keil

In special circumstances, the Keil project compiled by an engineer is unknown to the J-Link software even if it can be compiled by other engineers and identified by ICP software. For example, some warnings like below will be displayed.

Figure 33. Error warning 1

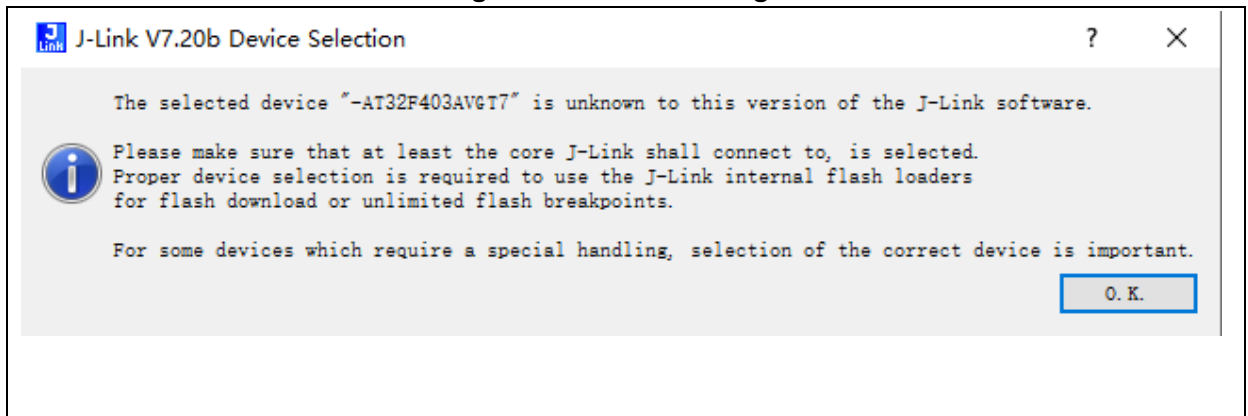


Figure 34. Error warning 2

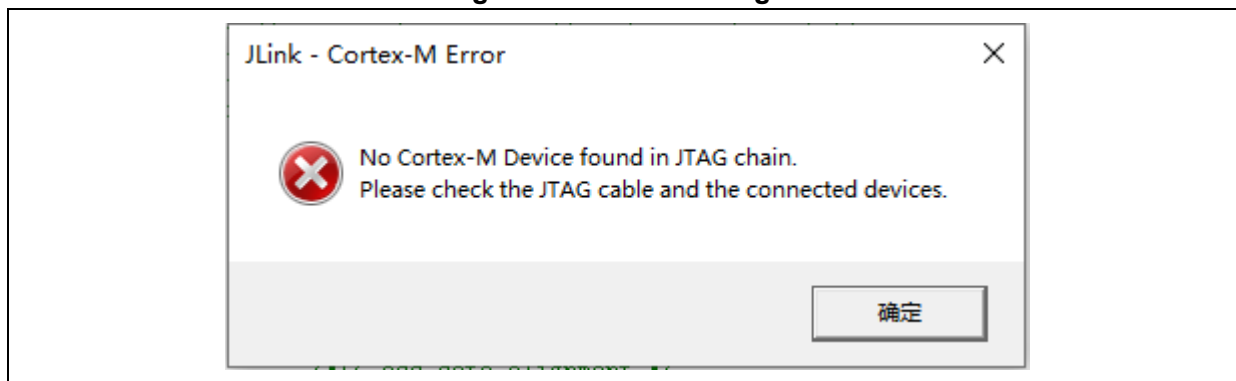
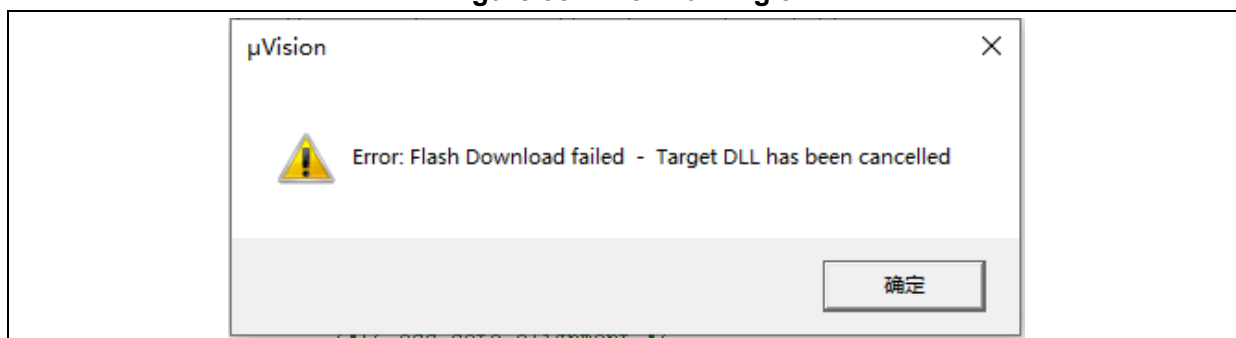


Figure 35. Error warning 3

**How to solve this problem?**

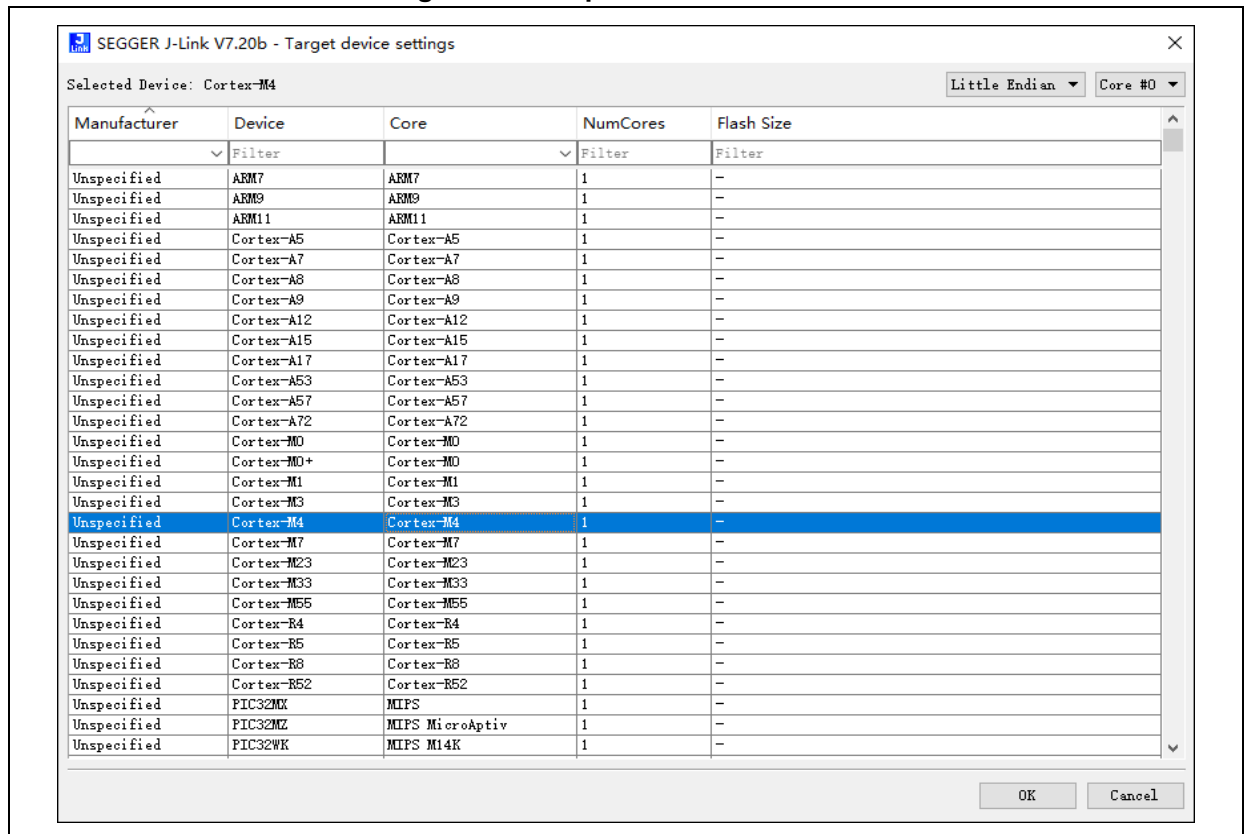
Step 1: Find "JLinkLog" and "JLinkSettings" files according to project path, and delete them.

Figure 36. JLinkLog and JLinkSettings

AT32F403A_407_Firmware_Library > project > at_start_f403a > examples > adc > combine_1				
名称	修改日期	类型	大小	
listings	2022/2/22 19:28	文件夹		
objects	2022/2/22 19:28	文件夹		
combine_mode_ordinary_simult.uvoptx	2022/2/22 19:28	UVOPTX 文件	12 KB	
combine_mode_ordinary_simult	2022/2/22 19:28	µVision5 Project	17 KB	
JLinkLog	2022/2/22 19:28	文本文档	7 KB	
JLinkSettings	2022/2/22 19:27	配置设置	1 KB	

Step 2: Click on magic wand, go to "Debug", select "Unspecified Cortex-M4"

Figure 37. Unspecified Cortex-M4



6.3 How to change HEXT crystal

All examples used in BSP implements frequency multiplication based on 8 MHz external high-speed crystal oscillator on the evaluation board. If a non-8 MHz external crystal is used in actual scenarios, it is necessary to modify clock configuration in BSP to allow for accurate and stable clock frequency.

Therefore, the “AT32_New_Clock_Configuration” tool is specially developed by Artery to generate the desired BSP system clock code file, including external clock source, frequency division factor, frequency multiplication factor, clock source selection and other parameters, marked in red in Figure 38. After the completion of parameter configuration, it is ready to generate code file, avoiding complicated operations involved in code modification.

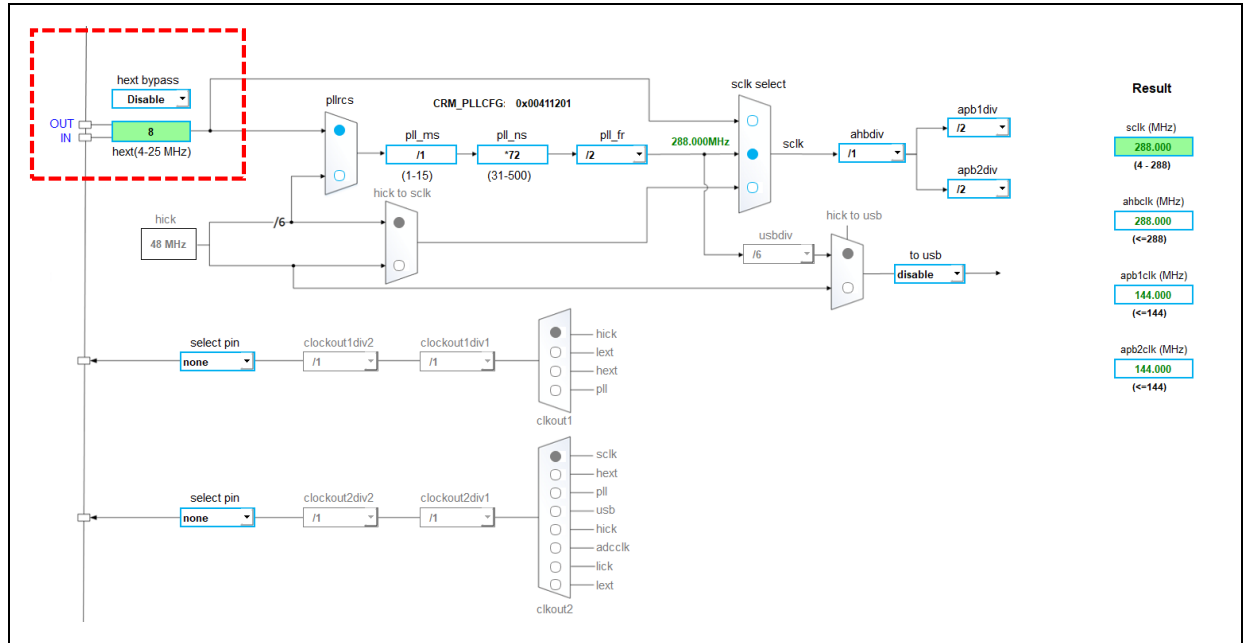
The users simply need to replace the original one in BSP demo with the newly generated clock code file (at32f4xx_clock.c/ at32f4xx_clock.h/ at32f4xx_conf.h), and call the function system_clock_config in main function.

Also, it is necessary to replace the macro definition HEXT_VALUE in the at32f4xx_conf.h. Taking the AT32F403A as an example, the HEXT_VALUE of the at32f403a_407_conf.h is defined as:

```
#define HEXT_VALUE ((uint32_t)8000000) /*!< value of the high speed external crystal in hz */
```

Figure 38 shows the window of AT32_New_Clock_Configuration tool.

Figure 38. AT32_New_Clock_Configuration window



For more information on the AT32_New_Clock_Configuration, please refer to the corresponding Application Note shown in Table 598, which are all available from the official website of Artery.

Table 592. Clock configuration guideline

Part number	Application note
AT32F403A/407 clock configuration	AN0082
AT32F435/437 clock configuration	AN0084
AT32F421 clock configuration	AN0116
AT32F415 clock configuration	AN0117
AT32F413 clock configuration	AN0118
AT32F425 clock configuration	AN0121
AT32F423 clock configuration	AN0158

7 Revision history

Table 593. Document revision history

Date	Revision	Revision note
2023.08.18	2.0.0	Initial release
2023.10.26	2.0.1	Added interrupt_flag_get function for each IP.
2024.03.01	2.0.2	Added "pwc_ldo_output_voltage_set" function and modified table names in Section 5.13 "Nested vectored interrupt controller (NVIC)".

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services, and ARTERY assumes no liability whatsoever relating to the choice, selection or use of the ARTERY products and services described herein

No license, express or implied, to any intellectual property rights is granted under this document. If any part of this document deals with any third party products or services, it shall not be deemed a license granted by ARTERY for the use of such third party products or services, or any intellectual property contained therein, or considered as a warranty regarding the use in any manner of such third party products or services or any intellectual property contained therein.

Unless otherwise specified in ARTERY's terms and conditions of sale, ARTERY provides no warranties, express or implied, regarding the use and/or sale of ARTERY products, including but not limited to any implied warranties of merchantability, fitness for a particular purpose (and their equivalents under the laws of any jurisdiction), or infringement on any patent, copyright or other intellectual property right.

Purchasers hereby agree that ARTERY's products are not designed or authorized for use in: (A) any application with special requirements of safety such as life support and active implantable device, or system with functional safety requirements; (B) any aircraft application; (C) any aerospace application or environment; (D) any weapon application, and/or (E) or other uses where the failure of the device or product could result in personal injury, death, property damage. Purchasers' unauthorized use of them in the aforementioned applications, even if with a written notice, is solely at purchasers' risk, and Purchasers are solely responsible for meeting all legal and regulatory requirements in such use.

Resale of ARTERY products with provisions different from the statements and/or technical characteristics stated in this document shall immediately void any warranty grant by ARTERY for ARTERY's products or services described herein and shall not create or expand any liability of ARTERY in any manner whatsoever.

© 2024Artery Technology -All rights reserved